



MATERIA

Matemáticas para Inteligencia Artificial

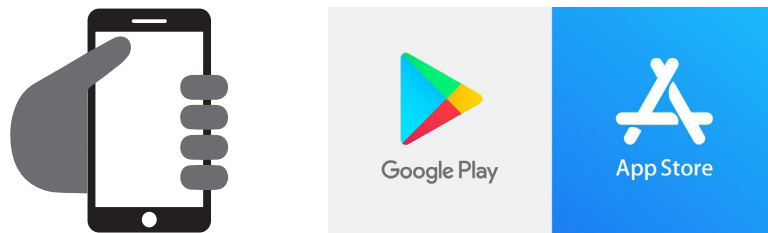
MIA · PRIMER CUATRIMESTRE

CÓDIGOS QR

Estimado Alumno:

Esta materia puede contener **Códigos QR** para agilizar el acceso a los contenidos audiovisuales que el docente ha desarrollado especialmente para Usted.

Para poder visualizar los videos utilice el lector de código QR que trae por defecto su teléfono o dispositivo móvil. En caso de no disponer de uno, puede descargarlo desde *Google Play* si su sistema operativo corresponde a Android o desde la *App Store* si utiliza IOS.



Una vez instalado el software en su dispositivo móvil:

- 1) Abra la aplicación
- 2) Apunte con la cámara de su dispositivo hacia los códigos QR que encuentre en el interior de su materia.



De esta manera estará visualizando los videos que el docente ha desarrollado y/o seleccionado especialmente para usted.

Índice

Bienvenida	4
Propósito	4
Objetivos	4
Competencias	5
Sistema de correlatividades	9
Contenidos mínimos	9
Contenidos	9
Bibliografía	10
Planificación	11
Metodología	14
Evaluación	15
Mapa conceptual	16
Módulos	
› Módulo 1	17
› Módulo 2	55
› Módulo 3	136
› Módulo 4	192
› Módulo 5	219

Importante

Lecturas básicas y complementarias disponibles desde plataforma.

Impresión total del documento 243 páginas

Bienvenida

Bienvenidos a la materia:

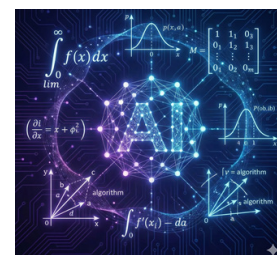
"Matemáticas para Inteligencia Artificial"

Los invitamos a ver el siguiente video del docente:



Propósito

El objetivo principal de la materia "Matemáticas para la Inteligencia Artificial" es proporcionarle a usted las bases matemáticas necesarias para el diseño, análisis y entendimiento de modelos de inteligencia artificial (IA). Este espacio curricular fomenta el pensamiento crítico y la habilidad analítica, a la vez que garantiza el dominio de herramientas matemáticas aplicadas a la IA, en concordancia con el perfil del egresado de la carrera.



Además, promueve una postura reflexiva y una fuerte responsabilidad ética en la utilización de la tecnología al incorporar habilidades socio-personales y tecnológicas que se hallan establecidas en el plan de estudios. La materia tiene un enfoque interdisciplinario y práctico en situaciones reales, que vincula los conocimientos teóricos con su implementación práctica. Esta integración resalta su importancia estratégica para la capacitación de profesionales que sean capaces de generar soluciones novedosas en el campo de la inteligencia artificial.

Objetivos

- › Utilizar operaciones con determinantes, matrices y vectores para cambiar y mostrar datos en espacios con más de tres dimensiones, que se emplean en modelos de inteligencia artificial.
- › Comprender nociones de derivadas, integrales y límites para examinar procesos y funciones de optimización que están presentes en los algoritmos de aprendizaje automático.
- › Emplear técnicas de estadística inferencial y de probabilidad para modelar la incertidumbre y tomar decisiones sustentadas en situaciones de análisis de datos con inteligencia artificial.
- › Aplicar técnicas de optimización y programación convexa para adaptar modelos y reducir funciones de costo en contextos de formación de sistemas inteligentes.
- › Utilizar algoritmos de búsqueda y estructuras de grafos para solucionar problemas relacionados con la representación, planificación y recorrido en sistemas inteligentes fundamentados en IA.

Competencias

Relación de la asignatura con las competencias de egreso

En la tabla siguiente se establece la relación de la asignatura con las competencias de egreso: Específicas, Genéricas Tecnológicas, Sociales, Políticas y Actitudinales de la carrera. Se incluyen las competencias de egreso a las que tributa su nivel de aporte (ninguno, bajo, medio y alto).

Competencias Genéricas Tecnológicas (CG)	Nivel
CG1. Identificación, formulación y resolución de problemas complejos mediante técnicas de ciencia de datos y desarrollo de sistemas de IA, empleando modelos matemáticos, algorítmicos y estadísticos para proponer soluciones basadas en datos.	alta
CG2. Concepción, diseño y desarrollo de proyectos de ciencia de datos y sistemas de inteligencia artificial, integrando metodologías de ingeniería, buenas prácticas de programación y principios éticos aplicados al uso de datos.	media
CG3. Gestión, planificación, ejecución y control de proyectos vinculados con la construcción, entrenamiento, validación y despliegue de modelos de IA y soluciones de ciencia de datos, asegurando eficiencia, trazabilidad y colaboración interdisciplinaria.	alta
CG4. Utilización de técnicas, lenguajes y herramientas especializadas de análisis de datos, aprendizaje automático y desarrollo de modelos de IA, seleccionando las más adecuadas según la naturaleza del problema y la disponibilidad de recursos.	media
CG5. Generación de desarrollos tecnológicos y/o innovaciones basadas en inteligencia artificial y ciencia de datos, proponiendo soluciones originales y escalables que respondan a necesidades reales de organizaciones y comunidades.	baja

Competencias Sociales, Políticas y Actitudinales (CSPA)	Nivel
CSPA6. Desempeño en equipos de trabajo.	alta
CSPA7. Comunicación efectiva.	media
CSPA8. Acción ética y responsable.	baja
CSPA9. Evaluación y actuación en relación con el impacto social de su actividad en el contexto global y local.	baja
CSPA10. Aprendizaje continuo.	media
CSPA11. Acción emprendedora	baja

Competencias Específicas (CE)	Nivel
CE 1.1 CD Especificación, proyección y desarrollo de modelos de ciencia de datos y sistemas basados en IA, considerando requerimientos técnicos, éticos y de disponibilidad de datos.	baja
CE 1.2 CD Especificación, proyección y desarrollo de infraestructuras y sistemas de comunicación de datos orientados a la adquisición, procesamiento y despliegue de modelos de IA.	baja
CE 1.3 IA Especificación, proyección y desarrollo de software para análisis, entrenamiento, validación y operación de modelos de inteligencia artificial, integrando algoritmos y estructuras de datos avanzadas.	media
CE 2.1 SI-IA Proyección y dirección de procesos vinculados a la seguridad de datos, resguardo de modelos y protección de información sensible empleada en sistemas de IA y ciencia de datos.	baja
CE 3.1 M-IA Establecimiento de métricas, normas y estándares de calidad para modelos analíticos, procesos de minería de datos y sistemas de inteligencia artificial, garantizando su confiabilidad y desempeño.	baja
CE 4.1 C-IA Certificación del funcionamiento, evaluación de riesgos, validación y condición de uso de modelos de IA, pipelines de ciencia de datos y sistemas de software asociados, incluyendo aspectos de transparencia, auditoría y robustez.	baja
CE 5.1 D-CD/IA Dirección y control de la implementación, operación y mantenimiento de soluciones de ciencia de datos y sistemas de inteligencia artificial, incluyendo infraestructura, monitoreo, mejora continua, seguridad y aseguramiento de calidad.	baja

Resultados de Aprendizaje (RA)

Resultado	Descripción
RA1	Manipular y transformar datos multivariados mediante operaciones con determinantes, matrices y vectores, para mostrar información en espacios de más de tres dimensiones aplicados a modelos de inteligencia artificial.
RA2	Analizar y aplicar conceptos de derivadas, integrales y límites para evaluar procesos y funciones de optimización presentes en algoritmos de aprendizaje automático.
RA3	Emplear técnicas de estadística inferencial y probabilidad para modelar la incertidumbre y tomar decisiones fundamentadas en contextos de análisis de datos con inteligencia artificial.

RA4	Implementar métodos de optimización y programación convexa para adaptar modelos y reducir funciones de costo en la formación y entrenamiento de sistemas inteligentes.
RA5	Resolver problemas utilizando algoritmos de búsqueda y estructuras de grafos, para diseñar y ejecutar estrategias de representación, planificación y recorrido en sistemas inteligentes aplicados a IA.

Relación de los RA y las competencias

En la tabla siguientes se indica el aporte de cada RA con la competencia de egreso: Específicas, Genéricas Tecnológicas, Sociales Políticas y Actitudinales.

Tabla 1: Competencias Genéricas Tecnológicas (CG)

Resultado de Aprendizaje (RA)	CG1	CG2	CG3	CG4	CG5
RA1. Manipular y transformar datos multivariados mediante operaciones con determinantes, matrices y vectores, para mostrar información en espacios de más de tres dimensiones aplicados a modelos de inteligencia artificial.	A	M	B	A	B
RA2. Analizar y aplicar conceptos de derivadas, integrales y límites para evaluar procesos y funciones de optimización presentes en algoritmos de aprendizaje automático.	A	A	M	A	M
RA3. Emplear técnicas de estadística inferencial y probabilidad para modelar la incertidumbre y tomar decisiones fundamentadas en contextos de análisis de datos con inteligencia artificial.	A	A	M	A	M
RA4. Implementar métodos de optimización y programación convexa para adaptar modelos y reducir funciones de costo en la formación y entrenamiento de sistemas inteligentes.	A	A	A	A	A
RA5. Resolver problemas utilizando algoritmos de búsqueda y estructuras de grafos, para diseñar y ejecutar estrategias de representación, planificación y recorrido en sistemas inteligentes aplicados a IA.	A	M	B	A	M

Tabla 2: Competencias Sociales, Políticas y Actitudinales (CSPA)

Resultado de Aprendizaje (RA)	CSPA6	CSPA7	CSPA8	CSPA9	CSPA10	CSPA11
RA1. Manipular y transformar datos multivariados mediante operaciones con determinantes, matrices y vectores.	–	B	–	–	M	–
RA2. Analizar y aplicar conceptos de derivadas, integrales y límites.	–	B	B	B	M	B
RA3. Emplear técnicas de estadística inferencial y probabilidad.	B	B	A	A	M	B
RA4. Implementar métodos de optimización y programación convexa.	B	B	M	B	A	M
RA5. Resolver problemas con algoritmos de búsqueda y grafos.	M	M	B	B	M	M

Tabla 3: Competencias Específicas (CE)

Resultado de Aprendizaje (RA)	CE1.1	CE1.2	CE1.3	CE2.1	CE3.1	CE4.1	CE5.1
RA1. Manipular y transformar datos multivariados mediante operaciones con determinantes, matrices y vectores.	A	B	M	B	M	B	B
RA2. Analizar y aplicar derivadas, integrales y límites para evaluar funciones de optimización.	A	B	M	B	A	M	M
RA3. Emplear estadística inferencial y probabilidad.	A	M	A	A	A	A	M
RA4. Implementar métodos de optimización y programación convexa.	A	M	A	M	A	A	A

RA5. Resolver problemas mediante grafos y algoritmos de búsqueda.	M	B	A	B	M	M	M
---	---	---	---	---	---	---	---

Sistema De Correlatividades

En la siguiente tabla se muestran los prerrequisitos y la correlativa posterior a este espacio curricular:

Cuatrimestre	Código	Asignatura	Abreviatura	Horas Totales	Anterior	Posterior
I	0101	Matemáticas para Inteligencia Artificial	MIA	68	--	Aprendizaje Automático AA - 0200
						Visión por Computadora VC - 0202
						Big Data y Herramientas de Análisis de Datos BD y HAD - 0203

Contenidos mínimos

- › Álgebra Lineal: Vectores, matrices, determinantes.
- › Cálculo diferencial e integral: Límites, derivadas, integrales y aplicaciones en IA.
- › Probabilidad y Estadística: Distribuciones, teorema de Bayes, inferencia estadística.
- › Optimización: Métodos de optimización, gradientes y programación convexa.
- › Teoría de grafos: Conceptos de grafos, algoritmos de recorrido y búsqueda.

Contenidos

Módulo I : Álgebra Lineal para la Representación y Transformación de Datos

- › Vectores, matrices, determinantes.

Módulo II : Cálculo Diferencial e Integral para el Análisis de Funciones en IA

- › Límites, derivadas, integrales y aplicaciones en IA.

Módulo III : Probabilidad y Estadística para la Toma de Decisiones Bajo Incertidumbre

- › Distribuciones, teorema de Bayes, inferencia estadística.
- › Tema 1: Variables aleatorias discretas y continuas
- › Tema 2: Distribuciones de probabilidad
- › Tema 3: Inferencia Estadística

Módulo IV : Optimización Matemática y Programación Convexa para Modelos de IA

- › Métodos de optimización, gradientes y programación convexa.
- › Tema 1: Introducción a la optimización matemática
- › Tema 2: Gradientes y técnicas basadas en derivadas
- › Tema 3: Programación convexa

Módulo V : Teoría de Grafos y Algoritmos de Recorrido y Búsqueda en Sistemas Inteligentes

- › Conceptos de grafos, algoritmos de recorrido y búsqueda.

Bibliografía

Obligatoria

Material didáctico de producción institucional diseñado y desarrollado especialmente para cada módulo

Complementaria

- › **Friedberg, S. H., Insel, A. J., & Spence, L. E. (2014).** Linear algebra. Pearson Education.
- › **Purcel, E. – Varberg, D. (2000).** Cálculo Diferencial Integral. Editorial Prentice Hall: Pearson Educación. México.
- › **Stewart, J. (2013).** Cálculo: Trascendentes Tempranas. Séptima Edición. Editorial Cengage Learning. México.
- › **Thomas, G. – Finney, R. (2010).** Cálculo una variable. Volumen I. Doceava Edición. Editorial Addison Wesley. México.
- › **Zill, D. (1987).** Cálculo con Geometría Analítica. Grupo Editorial Iberoamérica. México.

Planificación

Actividades y Consignas de Aprendizaje	Módulo	Semana	Metodología	Agrupamiento	Carácter	Competencia
Actividad Integradora Módulo 1: Álgebra Lineal para IA	Módulo 1	1	Proyecto integrador con análisis de matrices y grafos	Grupal	Insumo para evaluación integradora	CG1, CG2, CG4, CG5, CSPA6, CSPA7, CSPA10, CSPA11, CE1.1, CE1.3, CE3.1, CE4.1, CE5.1
Actividad 2: Álgebra Lineal en Sistemas Inteligentes	Módulo 1	1	Trabajo aplicado con vectores y transformaciones lineales	Grupal	Insumo para evaluación integradora	CG1, CG2, CG4, CSPA10, CE1.1, CE1.3, CE3.1
Actividad 1: Lectura e interpretación de gráficas de funciones aplicadas a la IA	Módulo 2	2	Uso de código Python y análisis de funciones	Individual	Opcional	CG1, CG2, CG3, CG4, CG5, CSPA10, CE1.1, CE1.3, CE3.1, CE4.1, CE5.1
Actividad 2: Funciones, crecimiento y continuidad en modelos de IA (informe co-construido con IA)	Módulo 2	3	Investigación y co-construcción de informe con asistentes de IA	Individual	Opcional	CG1, CG2, CG3, CG4, CG5, CSPA10, CE1.1, CE1.3, CE3.1, CE4.1, CE5.1
Actividad 3: Cálculo de límites	Módulo 2	4	Ejercicios prácticos de cálculo de límites	Individual	Insumo para evaluación integradora	CG1, CG2, CG3, CG4, CG5, CSPA10, CE1.1, CE1.3, CE3.1, CE4.1, CE5.1
Actividad 4: Solo cálculo de derivadas	Módulo 2	4	Ejercicios prácticos de derivadas	Individual	Insumo para evaluación integradora	CG1, CG2, CG3, CG4, CG5, CSPA10, CE1.1, CE1.3, CE3.1, CE4.1, CE5.1

Actividad 5: Solo cálculo de integrales	Módulo 2	4	Ejercicios prácticos de integrales	Individual	Insumo para evaluación integradora	CG1, CG2, CG3, CG4, CG5, CSPA10, CE1.1, CE1.3, CE3.1, CE4.1, CE5.1
Actividad 6: Estudio de una función de pérdida cuadrática y desarrollo de una herramienta en Python	Módulo 2	5	Proyecto de análisis simbólico y programación en Python	Grupal	Insumo para evaluación integradora	CG1, CG2, CG3, CG4, CG5, CSPA8, CSPA10, CSPA11, CE1.1, CE1.2, CE1.3, CE2.1, CE3.1, CE4.1, CE5.1
Evaluación Integradora – Parte 1						
Actividad 1: Muestreo y visualización de distribuciones	Módulo 3	6	Práctica con Python: muestreo, histogramas y análisis de distribuciones	Individual	Opcional	CG1, CG2, CG3, CG4, CG5, CSPA8, CSPA9, CSPA10, CE1.1, CE1.2, CE1.3, CE2.1, CE3.1, CE4.1, CE5.1
Actividad 2: Problemas en DataSecure	Módulo 3	7	Análisis estadístico e informe con inferencia y pruebas de hipótesis	Grupal	Insumo para evaluación integradora	CG1, CG2, CG3, CG4, CG5, CSPA6, CSPA7, CSPA8, CSPA9, CSPA10, CE1.1, CE1.2, CE1.3, CE2.1, CE3.1, CE4.1, CE5.1
Actividad 1: Problema 1 - Minimización de $h(x)$	Módulo 4	8	Ejercicio práctico con derivadas y gráfico en Python	Individual	Opcional	CG1, CG2, CG3, CG4, CG5, CSPA8, CSPA10, CSPA11, CE1.1, CE1.2, CE1.3, CE2.1, CE3.1, CE4.1, CE5.1

Actividad 2: Análisis guiado del video sobre Descenso por Gradiente	Módulo 4	9	Estudio de video y desarrollo de ejercicio de gradiente y reflexión	Individual	Opcional	CG1, CG2, CG3, CG4, CG5, CSPA8, CSPA10, CSPA11, CE1.1, CE1.2, CE1.3, CE2.1, CE3.1, CE4.1, CE5.1
Actividad 3: Análisis práctico de regresión lineal regularizada	Módulo 4	10 -11-12	Proyecto de programación en Python y análisis de programación convexa	Grupal	Insumo para evaluación integradora	CG1, CG2, CG3, CG4, CG5, CSPA6, CSPA7, CSPA8, CSPA10, CSPA11, CE1.1, CE1.2, CE1.3, CE2.1, CE3.1, CE4.1, CE5.1
Actividad 1: Estructuras, recorridos y accesibilidad: un análisis completo de un grafo aplicado a IA	Módulo 5	13 -14	Trabajo aplicado de grafos con construcciones de matrices, BFS, DFS y cierre transitivo	Grupal	Insumo para evaluación integradora	CG1, CG2, CG4, CG5, CSPA6, CSPA7, CSPA10, CSPA11, CE1.1, CE1.3, CE3.1, CE4.1, CE5.1
Evaluación Integradora – Parte 2						

Metodología

A) Estrategias de enseñanza: Este espacio curricular fue diseñado según lo previsto en el Sistema Institucional de Educación a Distancia (SIED) de la UBP (Resolución MECCYT 197/2019), las características de los destinatarios, las necesidades de esta disciplina, competencias que se espera promover en esta asignatura y el perfil del egresado.

En relación con lo anterior, para llevar a cabo la enseñanza se cuenta con:

- › Contenidos y actividades de aprendizaje desarrollados en la plataforma virtual miUBP.
- › Material bibliográfico especialmente seleccionado.
- › Clases en video, en las que el docente expone contenidos que considera relevantes y que pueden ser visualizadas en cualquier momento.
- › Tutorías virtuales que implican diferentes espacios de interacción.

B) Materiales curriculares: Durante el cursado, se fomenta el estudio de los contenidos (presentados en diferentes soportes y lenguajes) que componen el programa, y la resolución de las actividades de aprendizaje. De esta forma, se introduce al alumno en los conceptos que se deben dominar en todo lo que respecta a Matemática para la Inteligencia Artificial. También, se invita a leer bibliografía que enriquece y profundiza dicho desarrollo. Por otra parte, se propone la resolución de actividades de aprendizaje tanto de carácter optativo como obligatorio que se encuentran debidamente identificadas en la plataforma.

C) Modalidad de agrupamiento: Los alumnos pueden optar por agruparse libremente para estudiar la materia y resolver las actividades de aprendizaje planteadas, de forma individual o grupal. En este sentido, en esta asignatura se propone:

- a) Trabajo individual, de modo que el alumno pueda desenvolverse en forma autónoma, reconociendo sus fortalezas y debilidades, a la vez que puede desarrollar sus propias estrategias de aprendizaje.
- b) Trabajo colaborativo, de modo que se favorezca el desarrollo de las competencias del trabajo en equipo para resolver eficientemente las futuras situaciones a las cuales puede enfrentarse como profesional.

D) Interacción: La comunicación con el docente tutor se realiza a través de los diferentes medios disponibles en la plataforma miUBP.

E) Formación Práctica: En lo que se refiere a la formación práctica se hace foco en la resolución de problemas que implique el uso de los contenidos estudiados en la asignatura contextualizados en situaciones que tengan que ver con la carrera.

Evaluación

La metodología de evaluación propuesta se presenta conforme al Reglamento Académico General de la UBP.

Cada instrumento de evaluación contiene sus criterios de evaluación explicitados y el puntaje otorgado a cada consigna.

En cuanto a los criterios de acreditación, la escala cuantitativa utilizada para evaluar es del 1 al 100 y el puntaje mínimo que se requiere para la aprobación es de 50.

Regularidad

Para alcanzar la condición de alumno regular usted deberá aprobar la Evaluación Integradora dentro de los plazos previstos para el cursado.

Si reprueba esta instancia podrá reelaborar la evaluación según los requerimientos del docente.

Examen Final

Deberá rendir, en condición de regular, una evaluación final que se realizará a través de un examen escrito integrador, presencial, en el periodo que disponga la Universidad

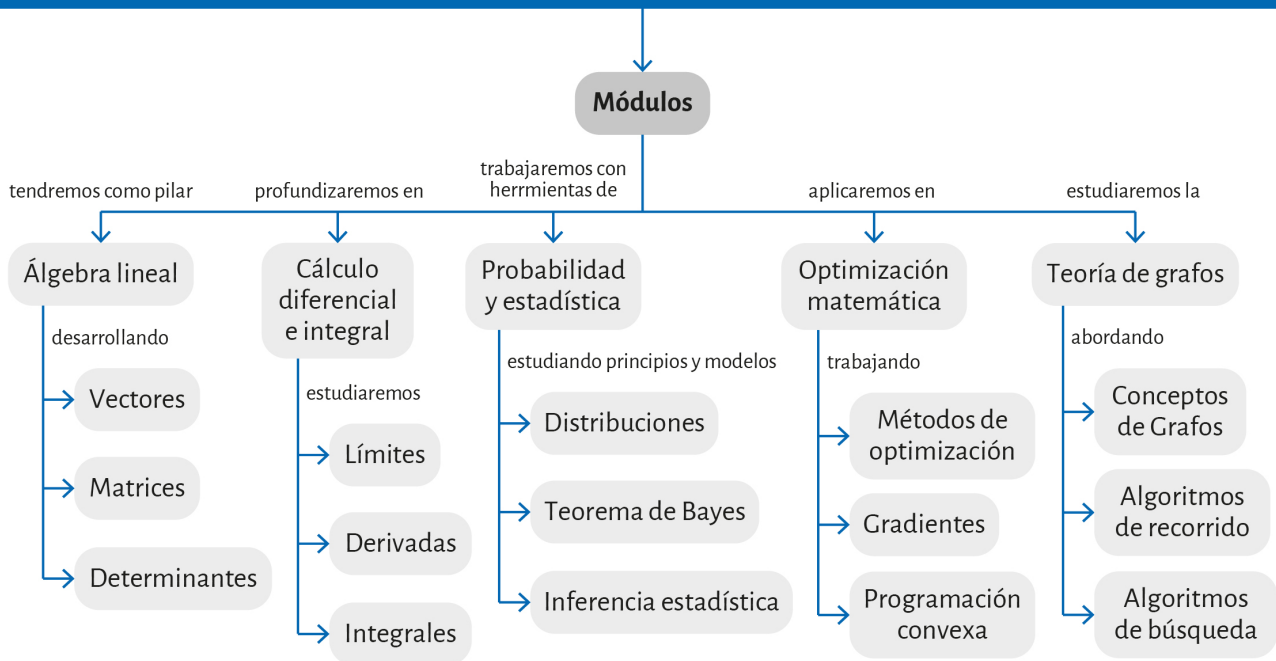
Mapa conceptual



A continuación, lo/a invitamos a escuchar el recorrido por el mapa conceptual de la materia propuesto por el docente.



Matemáticas para Inteligencia Artificial



MÓDULO 1

Microobjetivos

- › Analizar las estructuras vectoriales y matriciales fundamentales para la representación y transformación de datos en IA.
- › Determinar la independencia lineal, el rango y los autovalores relevantes para evaluar la preservación de información en modelos de IA.
- › Aplicar propiedades de matrices especiales y transformaciones lineales para poder interpretar y optimizar algoritmos utilizados en el campo de la IA.



VIDEO DOCENTE

Contenidos

- **Álgebra lineal para la representación y transformación de datos**

¡Bienvenidos/as al módulo inicial de Matemáticas para Inteligencia Artificial!

El álgebra lineal constituye uno de los pilares fundamentales sobre los cuales se estructura la Inteligencia Artificial moderna. La representación numérica de datos, el funcionamiento de los modelos de aprendizaje automático, las transformaciones de características, la reducción de dimensionalidad y las operaciones internas de las redes neuronales se sustentan en conceptos estrictamente lineales. En consecuencia, dominar estas herramientas no solo permite comprender el comportamiento de los algoritmos, sino también evaluar su estabilidad, interpretar sus resultados y optimizar su desempeño.

En este primer módulo, nos introduciremos a una revisión rigurosa de los elementos esenciales del álgebra lineal aplicados al modelado y análisis de sistemas inteligentes. Abordaremos con profundidad los vectores como unidades de representación, las matrices como operadores lineales que transforman datos y los determinantes como medidas críticas para evaluar la invertibilidad y el comportamiento geométrico de dichas transformaciones. Junto con explicaciones detalladas, ejemplos vinculados a IA y ejercicios de carácter analítico, el módulo le proporcionará, como estudiante profesional, una base robusta para comprender las estructuras matemáticas que subyacen a técnicas como redes neuronales, embeddings, PCA, métodos espectrales y modelos probabilísticos avanzados.

Comencemos.

1. Vectores

1.1 ¿Qué es un vector? Interpretaciones en IA



Énfasis

Un vector es una entidad matemática formada por una lista ordenada de números reales. Es mucho más que un conjunto de valores: es la unidad básica de representación en inteligencia artificial.

Un vector puede representar:

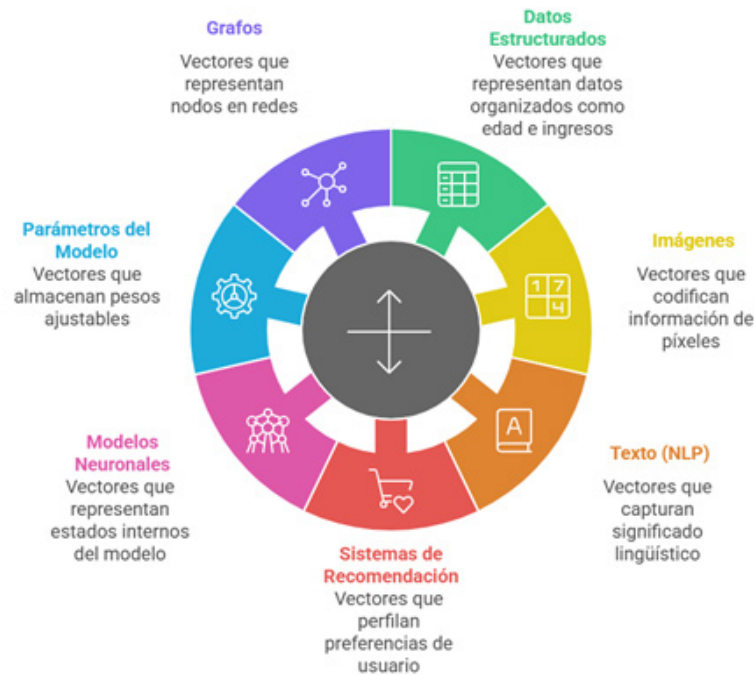


Ilustración 1 - Vectores - Representaciones - Generado con Napkin - Información Propia

Formalmente, un vector en $\mathbb{R}^n$ tiene la forma:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$



Énfasis

Cada componente representa una característica, dimensión, atributo o grado de libertad.

2. Espacios vectoriales: la "geometría" donde vive la IA



Énfasis

Un conjunto de vectores con operaciones de suma y multiplicación por escalares forma un espacio vectorial.

Por ejemplo:

Ejemplo clásico (en $\mathbb{R}^2$):

- El plano euclidiano. Los pares ordenados de números reales, como (3, 5) o (-1, 4), pueden sumarse y multiplicarse por números reales, y el resultado siempre es otro par en el plano.

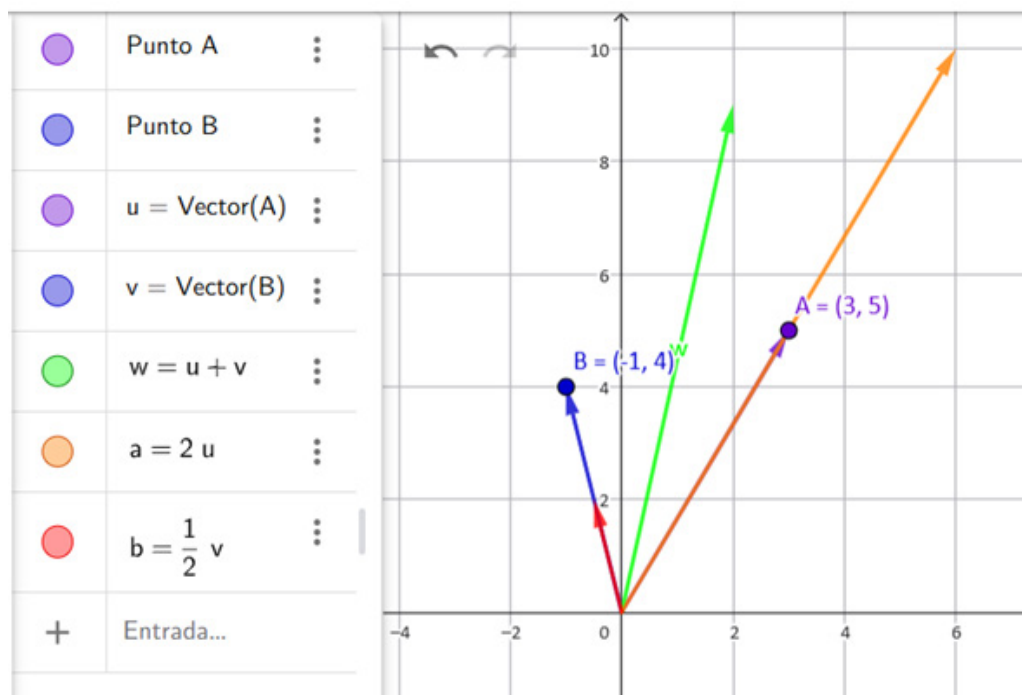


Ilustración 2 - Ejemplo R2 - Geogebra - Propio

› **Vectores en imágenes digitales:**

- › Cada píxel de una imagen en escala de grises puede verse como un valor en un vector, y toda la imagen como un vector en $\mathbb{R}^n$. Sumando imágenes o multiplicándolas por un número, seguimos obteniendo imágenes válidas.

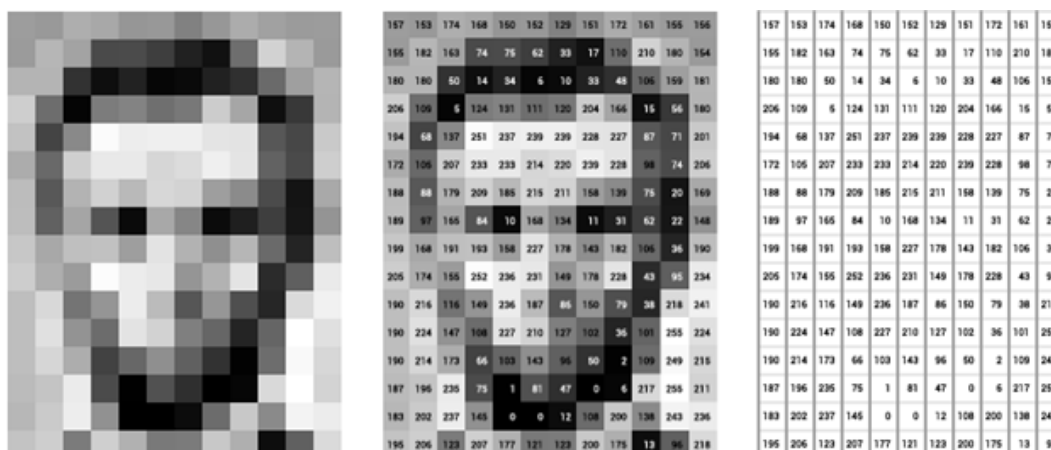


Ilustración 3 - Valores de escala de Grises - Fuente: <https://tinyurl.com/25mtanz4>



Para más información sobre el tema de vectores en imágenes digitales, le recomendamos leer el siguiente post: <https://tinyurl.com/3x9tbxuc>

› **Secuencias de audio:**

- › Un segmento de audio digitalizado puede representarse como un vector, donde cada componente es una muestra. Si sumamos dos grabaciones (vectores) o multiplicamos el volumen por un escalar, el resultado sigue siendo un vector válido en ese espacio.

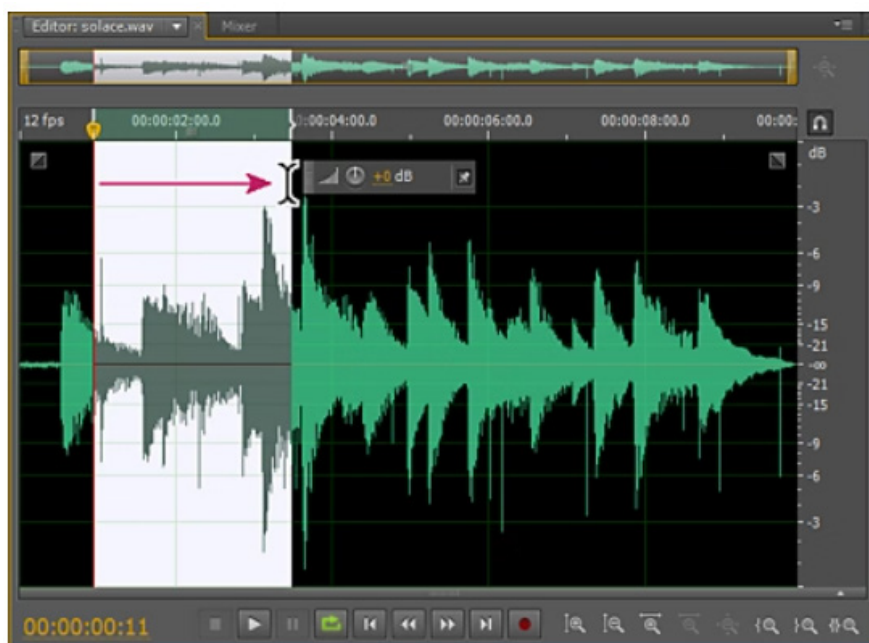


Ilustración 4 - Segmento de Audio - Fuente: <https://tinyurl.com/mrezfj4k>

› **Embeddings en IA:**

- › En el procesamiento de lenguaje natural, cada palabra puede representarse como un vector de características. El conjunto de todos estos vectores, al combinarse mediante suma y multiplicación por escalares, forma un espacio vectorial que permite cálculos y transformaciones significativas en IA.



Ilustración 5 - El procesamiento del lenguaje natural combina lingüística computacional y modelado con IA para interpretar datos de habla y texto. - Fuente: <https://tinyurl.com/43n77b5n>



Para continuar profundizando sobre cómo funciona el procesamiento del lenguaje natural, se le recomienda acceder al siguiente enlace: <https://tinyurl.com/y4sr8aub>

Estos ejemplos muestran cómo los espacios vectoriales aparecen en contextos cotidianos y en aplicaciones concretas de IA, proporcionando una estructura matemática flexible para representar y manipular información diversa.

La IA utiliza espacios vectoriales porque permiten:

- › Definir **distancias** (para clustering, kNN, métricas de similitud) para comparar objetos y establecer relaciones de proximidad en espacios de alta dimensión, facilitando tareas como la agrupación automática de datos, la clasificación basada en vecinos más cercanos y la evaluación de cuán parecidos son distintos elementos entre sí. Estas capacidades, apoyadas en la estructura de los espacios vectoriales, permiten que los algoritmos de IA operen con precisión y eficiencia al analizar grandes volúmenes de información compleja, contribuyendo así al diseño y mejora de modelos predictivos robustos.

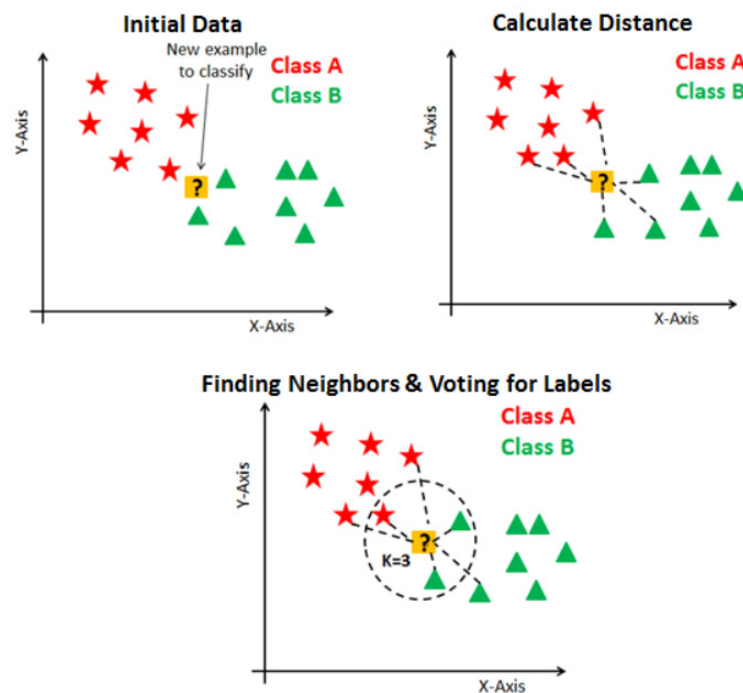


Ilustración 6 - KNN - Fuente: <https://tinyurl.com/anj8upr8>

Además de definir distancias entre objetos, los espacios vectoriales permiten calcular ángulos entre vectores, lo que resulta esencial para evaluar la similitud coseno y otras métricas de proximidad en IA.



Ilustración 7 - Paralelismo GPS - Fuente: <https://tinyurl.com/5n79rs32>

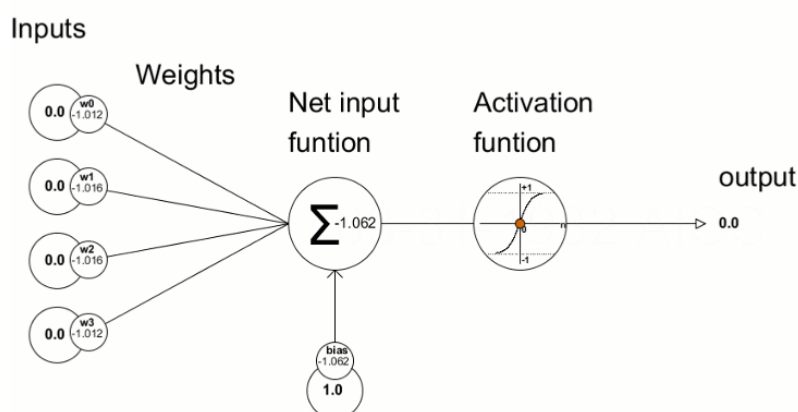


Lectura
complementaria

En este punto, se recomienda que ingrese al siguiente enlace: <https://tinyurl.com/rrh5dacb>

Esta capacidad se integra naturalmente en el flujo de trabajo de los modelos, ya que posibilita la comparación cuantitativa y geométrica de representaciones de datos, contribuyendo a la agrupación eficiente y a la detección de patrones relevantes.

Aplicar transformaciones lineales (propagación hacia adelante en redes neuronales), realizar descomposiciones como PCA y SVD, y representar la información en distintas bases mediante análisis espectral son ejemplos adicionales de cómo el álgebra lineal sustenta los procesos fundamentales de la inteligencia artificial.



Animación 1 - En la animación se muestra la composición del proceso de agregación y de la función de activación. - Fuente: <https://tinyurl.com/4hxmz2ee>

Todo esto refuerza la importancia de comprender las estructuras y operaciones matemáticas para diseñar sistemas robustos y optimizados, capaces de interpretar y manipular grandes volúmenes de datos complejos en distintos contextos.

A modo de síntesis, observe y lea el siguiente diagrama:

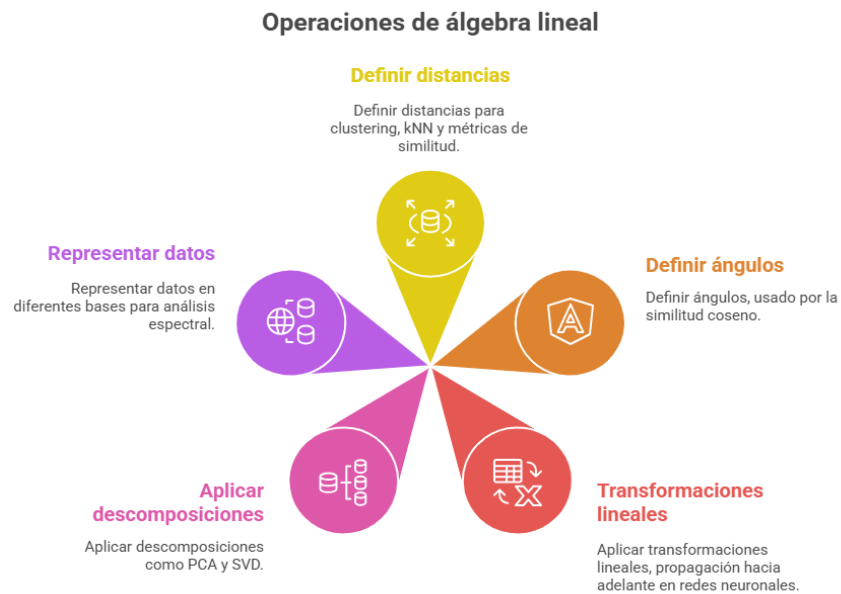


Ilustración 8 - Utilidad del Álgebra Lineal –Elaborado con Napkin - Información Propia

3. Matrices

3.1 ¿Qué es una matriz y por qué es esencial en IA?

Una matriz es un arreglo rectangular de números que representa:

- › un operador lineal,
- › una transformación geométrica,
- › una capa de una red neuronal,
- › una estructura de datos (imágenes, señales, secuencias),
- › relaciones entre variables (matriz de covarianza),
- › conexiones entre nodos (matriz de adyacencia de un grafo).

La matriz es el **objeto matemático central de la IA moderna**. Todas las operaciones profundas de aprendizaje automático (forward pass, backpropagation, atención, convolución, SVD, PCA, normalización, kernels) dependen del álgebra matricial.



Énfasis

Cada vez que un modelo de IA aprende, lo hace a través de matrices.

Esto es, con las herramientas que conocemos del Álgebra Lineal se puede paralelizar las operaciones sobre el vector de entrada y hacer el cálculo simultáneamente sobre un conjunto de datos de entrada haciendo uso del producto de matrices.

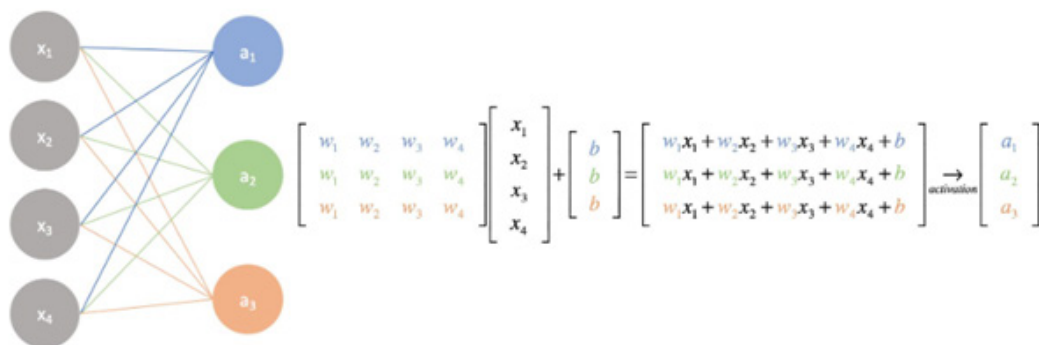


Ilustración 9 - IA aprende con matrices - Fuente: <https://tinyurl.com/38mtr6sx>



Le sugerimos la lectura del siguiente artículo, el cual aborda las redes neuronales artificiales desde su fundamento matemático y muestra cómo las matrices son fundamentales para representar las conexiones y pesos entre capas de neuronas:

<https://tinyurl.com/38mtr6sx>

Formalmente, una matriz $A \in \mathbb{R}^{m \times n}$ es:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

3.2. Matrices como transformaciones lineales

Toda matriz A define una transformación:

$$T_A : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad T_A(\mathbf{x}) = A\mathbf{x}.$$



Ilustración 10 - Transformaciones y Matrices—Elaboración propia—Herramienta: Napkin

- › **Interpretación geométrica:** La matriz describe *cómo* cambia el espacio. De hecho, al aplicar una matriz a un conjunto de vectores, se pueden visualizar rotaciones, escalados, reflexiones o incluso deformaciones más complejas dentro del espacio multidimensional.

Rotaciones Planas

Rotación (Rx)

$$\begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{z} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & -\sin \varphi \\ 0 & \sin \varphi & \cos \varphi \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Rotación (Ry)

$$\begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{z} \end{bmatrix} = \begin{bmatrix} \cos \varphi & 0 & \sin \varphi \\ 0 & 1 & 0 \\ -\sin \varphi & 0 & \cos \varphi \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Rotación (Rz)

$$\begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{z} \end{bmatrix} = \begin{bmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

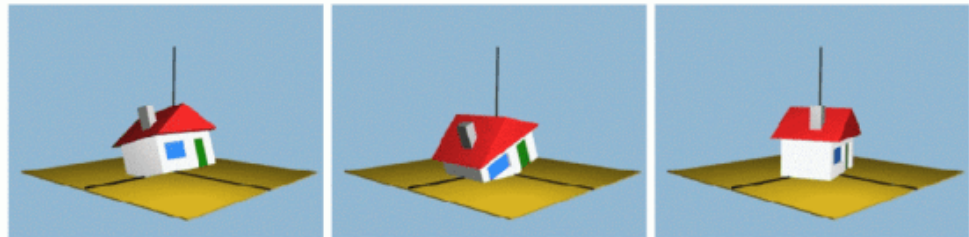


Ilustración 11 - Matriz de Rotación - Fuente: <https://tinyurl.com/2kt328a5>

- › **Interpretación en IA:** Una capa densa en una red neuronal hace exactamente esto:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}.$$

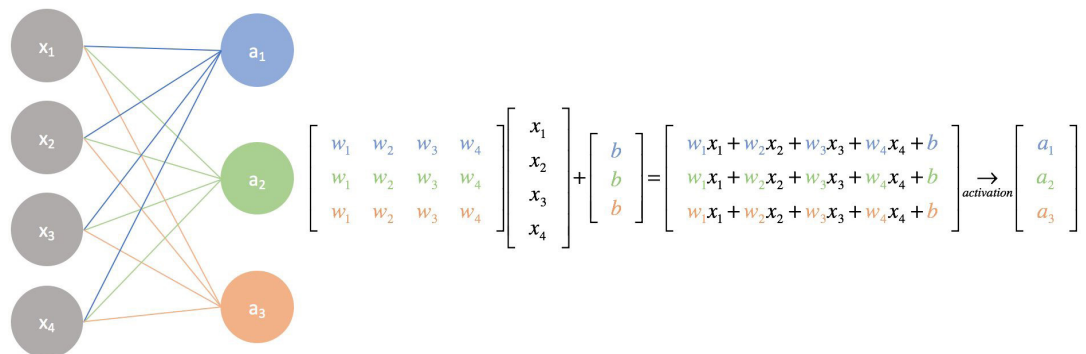


Ilustración 12 - Red Neuronal - Capa densa - Fuente: <https://tinyurl.com/hmrvm76w>



Énfasis

La red neuronal aprende cambiando los valores de la matriz **W**.

3.3. Multiplicación matriz–vector y matriz–matriz



Énfasis

La multiplicación matriz–vector y matriz–matriz constituye el mecanismo mediante el cual se implementan las transformaciones lineales en el contexto del aprendizaje automático.

Aplicar una matriz a un vector equivale a calcular una combinación lineal de las componentes del vector original, generando un nuevo vector que refleja la acción geométrica definida por la matriz.

De manera análoga, multiplicar dos matrices permite encadenar transformaciones, sintetizando operaciones sucesivas en una sola expresión compacta.

Este procedimiento es fundamental, tanto para la propagación de señales en redes neuronales como para la composición de funciones más complejas en modelos avanzados de IA, y resulta imprescindible para analizar el impacto acumulativo de múltiples capas o etapas del procesamiento de datos.

3.3.1. Multiplicación matriz–vector

Cuando se multiplica una matriz por un vector, el resultado es un nuevo vector cuyas componentes reflejan la acción geométrica de la matriz sobre el espacio original, permitiendo modelar desde simples escalados hasta rotaciones o deformaciones complejas.

Supóngase la siguiente matriz de pesos de una *capa densa*¹ en una red neuronal:

$$W = \begin{bmatrix} 0.2 & -0.4 & 1.0 \\ 0.5 & 0.3 & -0.2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix}$$

Esta capa tiene:

- 3 entradas (coinciden con la longitud del vector $\mathbf{x}$),
- 2 neuronas (coinciden con las 2 filas de la matriz W).

Para calcular, haremos:

$$W\mathbf{x} = \begin{bmatrix} 0.2(2) + (-0.4)(1) + 1.0(-1) \\ 0.5(2) + 0.3(1) + (-0.2)(-1) \end{bmatrix}$$

Resolviendo cada componente:

1. Primera neurona:	$0.2 \cdot 2 = 0.4, \quad -0.4 \cdot 1 = -0.4, \quad 1.0 \cdot (-1) = -1$ $0.4 - 0.4 - 1 = -1$
2. Segunda neurona:	$0.5 \cdot 2 = 1, \quad 0.3 \cdot 1 = 0.3, \quad -0.2 \cdot (-1) = 0.2$ $1 + 0.3 + 0.2 = 1.5$
3. Resultado final:	$W\mathbf{x} = \begin{bmatrix} -1 \\ 1.5 \end{bmatrix}$

¹ Una capa densa (también llamada capa totalmente conectada o fully connected layer) es un tipo de capa en una red neuronal artificial en la cual cada neurona recibe todas las entradas provenientes de la capa anterior.

Desde el punto de vista de la matemática formal, se puede decir que la operación realizada se basa en la definición formal de multiplicación matriz–vector, donde:

$A = [a_{ij}] \in \mathbb{R}^{m \times n}$ y $\mathbf{x} \in \mathbb{R}^n$, entonces:

$$A\mathbf{x} = \begin{bmatrix} a_{11}x_1 + \cdots + a_{1n}x_n \\ \vdots \\ a_{m1}x_1 + \cdots + a_{mn}x_n \end{bmatrix}$$

Esta operación es una combinación lineal de las columnas de la matriz, donde los coeficientes provienen de las componentes de $\mathbf{x}$.

En términos teóricos, esto corresponde a aplicar una transformación lineal $T_A(\mathbf{x}) = A\mathbf{x}$.

› ¿Qué pasa con la IA?

Este cálculo es exactamente el que realiza una neurona antes de aplicar su función de activación. En una red neuronal:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

- › Cada fila de W corresponde a los pesos de una neurona.
- › Cada multiplicación $w_{i1}x_1 + \cdots + w_{in}x_n$ es un producto matriz–vector.
- › El resultado $\mathbf{z}$ es el vector de *activaciones pre-no lineales*.

3.3.2. Multiplicación matriz–matriz

En el contexto de redes neuronales y álgebra lineal aplicada a IA, la multiplicación matriz–matriz surge cuando se requiere procesar múltiples vectores de entrada de manera simultánea.

Matemáticamente, esta operación implica combinar dos matrices, donde cada columna de la primera matriz representa un conjunto de entradas, y la segunda matriz contiene los pesos de las neuronas.

El resultado es una nueva matriz en la que cada columna corresponde a las activaciones pre-no lineales obtenidas para cada entrada del lote, extendiendo así el cálculo de la multiplicación matriz–vector a un escenario más general y eficiente.

Supóngase que se tienen dos matrices:

- Una matriz A que representa una **capa densa** con 2 neuronas y 3 entradas:

$$A = \begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & -1 \end{bmatrix} \quad (A \in \mathbb{R}^{2 \times 3})$$

- Una matriz B que contiene **tres vectores de entrada** (por ejemplo, tres ejemplos de un *batch*):

$$B = \begin{bmatrix} 2 & 1 & 0 \\ 1 & -1 & 3 \\ 0 & 2 & 1 \end{bmatrix} \quad (B \in \mathbb{R}^{3 \times 3})$$

El producto AB produce una matriz $C \in \mathbb{R}^{2 \times 3}$:

$$C = AB$$

Cada elemento de C se obtiene multiplicando la fila correspondiente de A por la columna correspondiente de B .

Quedando:

$$C = AB = \begin{bmatrix} 1 & 6 & -1 \\ 3 & -5 & 8 \end{bmatrix}$$

Desde el punto de vista de la matemática, diremos que, la multiplicación matriz–matriz se define formalmente como:

$$(AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj},$$

donde:

- › la matriz $A \in \mathbb{R}^{m \times n}$,
 - › la matriz $B \in \mathbb{R}^{n \times p}$,
 - › el resultado $AB \in \mathbb{R}^{m \times p}$.
- *Aplicación directa en Inteligencia Artificial*

Este ejemplo tiene tres interpretaciones relevantes, que serán revisadas y ampliadas en otras materias del ciclo.

1. Redes neuronales profundas	2. Transformers: atención	3. Visión por computadora
Cuando se procesa un batch de ejemplos, la red realiza: $Z=WX$ donde: › W: matriz de pesos (como A), › X: matriz con las entradas del batch (como B), › Z: salidas lineales de la capa. El cálculo AB es exactamente el cálculo interno de una capa densa sobre múltiples ejemplos.	En el mecanismo de atención se compute: $\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$ Aquí se realizan dos productos matriz–matriz seguidos. Este ejemplo permite visualizar cómo funciona la operación QK^T.	Transformaciones geométricas como rotaciones, escalados y traslaciones se componen multiplicando matrices. El ejemplo ilustra la composición de operaciones lineales en visión.

3.4. Rango, Núcleo e Imagen

3.4.1. Rango

El rango de una matriz es el número máximo de columnas (o filas) linealmente independientes. También puede interpretarse como la dimensión del espacio generado por sus columnas (o filas), es decir, el número de direcciones diferentes en las que la matriz puede “extenderse”.



Sea A una matriz de tamaño $m \times n$. El *rango* de A , denotado como $\text{rango}(A)$, es: El número de vectores columna (o fila) linealmente independientes de A . Equivale a la dimensión del subespacio columna (o fila) de A .

El rango indica cuánta **información** se conserva en una transformación lineal representada por una matriz:

- › Si una matriz tiene *rango máximo*, no colapsa el espacio: no hay pérdida de dimensión.
- › Si el rango es *menor* que el número de columnas (o filas), la transformación *comprime* el espacio y se pierde información: varias entradas del dominio pueden llevar al mismo punto en la imagen.

Ejemplos

Ejemplo 1: Matriz con rango máximo

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- › Tiene 2 columnas linealmente independientes.
- › $\text{rango}(A) = 2$
- › La transformación asociada no altera la dimensión: es una identidad en $\mathbb{R}^2$.

Ejemplo 2: Matriz con rango bajo

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

- › La segunda columna es múltiplo de la primera ($C_2 = 2 \cdot C_1$).
- › Las columnas no son linealmente independientes.
- › $\text{rango}(B) = 1$
- › La transformación comprime $\mathbb{R}^2$ a una línea: hay pérdida de información.

Antes de continuar, algunas observaciones importantes:

- › El rango de una matriz siempre es menor o igual que el mínimo entre el número de filas y de columnas: $\text{rango}(A) \leq \min(m, n)$
- › Para una matriz cuadrada $n \times n$, si el rango es n , entonces:
 - › Es invertible.
 - › Su determinante es distinto de cero.

Veamos un ejemplo sencillo de Rango aplicado a IA:

Considere la siguiente matriz de pesos de una capa densa con 2 neuronas y 3 entradas:

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$$

Observe que:

La segunda fila es exactamente el doble de la primera:

$$(2, 4, 6) = 2 \cdot (1, 2, 3)$$

Esto significa que las dos filas no aportan información distinta: una de ellas es una copia escalada de la otra.

En este caso, las filas son linealmente dependientes → solo hay una dirección independiente.

Por lo tanto: $\text{rango}(W) = 1$

- **Interpretación en IA**

Esta matriz representa una capa con dos neuronas, pero debido a la dependencia entre las filas:

- › Ambas neuronas aprenden exactamente la misma combinación de características.
- › En términos prácticos: la segunda neurona no aporta nada nuevo.

Esto implica:

- › Pérdida de capacidad representativa.
- › Red involuntariamente “más pequeña” de lo esperado.
- › Redundancia → mal aprovechamiento de parámetros.

En otras palabras, aunque la capa tiene dos neuronas, en realidad solo está aprendiendo una dirección útil en el espacio de características. La red se comporta como si tuviera una sola neurona en esa capa.

Ahora bien, revisemos el siguiente ejemplo aplicado a visión por computadora:

Considere un píxel de una imagen en RGB representado por un vector:

$$x = \begin{bmatrix} 120 \\ 200 \\ 100 \end{bmatrix}$$

Supongamos que queremos aplicar una transformación lineal para generar un nuevo espacio de color (por ejemplo, para extracción de características):

$$W = \begin{bmatrix} 1 & 1 & -1 \\ 0 & 1 & 1 \end{bmatrix}$$

Cálculo del rango

$$W = \begin{bmatrix} 1 & 1 & -1 \\ 0 & 1 & 1 \end{bmatrix}$$

Las dos filas no son proporcionales, por lo que son linealmente independientes.

$$\text{rango}(W) = 2$$

- **Interpretación en visión**

Aunque la imagen tiene 3 canales (RGB), esta transformación genera solo dos canales nuevos que capturan:

- › combinación de colores,
- › contrastes,
- › diferencias entre intensidades.

Una matriz de rango 2 implica que:

- › la imagen se proyecta a un espacio de dimensión menor (de 3 a 2),
- › pero se retienen dos direcciones independientes de variación,
- › la pérdida de información es mínima.

Esta idea es la base de:

- › reducción de dimensionalidad,
- › transformaciones de color,
- › extracción de componentes principales en imágenes.

3.4.2. Núcleo (kernel)

El núcleo de una matriz $A \in \mathbb{R}^{m \times n}$, también llamado espacio nulo, es el conjunto de todos los vectores $\mathbf{x} \in \mathbb{R}^n$ que son enviados al vector cero por la transformación lineal $A\mathbf{x}$:

$$\ker(A) = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0}\}$$



Énfasis

El núcleo de una transformación lineal $T: V \rightarrow W$, denotada como $\ker(T)$, es el subconjunto de vectores de V que son mapeados al vector nulo de W :

$$\ker(T) = \{\mathbf{v} \in V : T(\mathbf{v}) = \mathbf{0}\}$$

Cuando T está representada por una matriz A , se traduce como:

$$\ker(A) = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0}\}$$

- **Interpretación geométrica**

La noción de núcleo tiene una interpretación geométrica muy clara: revela qué partes del espacio de entrada son invisibles para la transformación.

A continuación, resumimos las ideas clave que permiten entender su significado espacial:

- › El núcleo contiene todos los vectores que se “colapsan” a cero bajo la transformación.
- › Describe la información que se pierde: si $\mathbf{x} \in \ker(A)$, entonces la transformación no puede distinguir entre $\mathbf{x}$ y el vector nulo.
- › Si $\ker(A) = \{\mathbf{0}\}$, la transformación es inyectiva: no pierde información.
- › Si $\dim(\ker(A)) > 0$, hay direcciones enteras del espacio que son destruidas (aplastadas a cero).

Ejemplo 1: Núcleo trivial	Ejemplo 2: Núcleo no trivial
$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \Rightarrow A\mathbf{x} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$ › Solución: solo $\mathbf{x}=\mathbf{0}$ cumple la ecuación. › $\ker(A)=\{\mathbf{0}\}$ › No se pierde información, entonces, transformación es inyectiva.	$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ Buscamos los vectores $\mathbf{x}=(1, 2)$ que cumplen: $\begin{cases} x_1 + 2x_2 = 0 \\ 2x_1 + 4x_2 = 0 \end{cases} \Rightarrow x_1 = -2x_2$ Entonces: $\ker(B) = \text{span}\left\{\begin{bmatrix} -2 \\ 1 \end{bmatrix}\right\}$ › Dimensión del núcleo: 1. › El espacio $\mathbb{R}^2$ se colapsa en una recta → pérdida de información.

- **Aplicación en IA (Machine Learning)**

En redes neuronales o modelos lineales:

- › Si $\ker(A) \neq \{\mathbf{0}\}$, existen vectores distintos que son transformados en el mismo resultado: el modelo no puede distinguir entre ellos.
- › Esto puede llevar a:
 - › Ambigüedad en la clasificación.
 - › Pérdida de capacidad predictiva.
 - › Colinealidad entre características (problema frecuente en regresión).

3.4.3. Imagen

La imagen de una matriz $A \in \mathbb{R}^{m \times n}$, también conocida como espacio columna o imagen de la transformación lineal asociada, está definida como:

$$\text{Im}(A) = \{\mathbf{y} \in \mathbb{R}^m : \exists \mathbf{x} \in \mathbb{R}^n \text{ tal que } A\mathbf{x} = \mathbf{y}\}$$

Es decir, es el conjunto de todos los vectores que pueden obtenerse como resultado de aplicar la transformación A a algún vector del dominio.



Sea $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$, definida por $T(\mathbf{x}) = A\mathbf{x}$. La imagen de T es:

- › $\text{Im}(T) = \{T(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^n\}$

- **Interpretación geométrica**

Desde el punto de vista geométrico, la imagen de una transformación lineal nos dice hacia dónde se proyecta el espacio de entrada.

Estas ideas ayudan a visualizar qué ocurre con los vectores al ser transformados:

- › La imagen representa todo el espacio alcanzado por la transformación.
- › Es un subespacio de $\mathbb{R}^m$.
- › Determina qué dimensiones están presentes en el resultado de la transformación.
- › La dimensión de la imagen es igual al rango de la matriz:

$$\dim(\text{Im}(A)) = \text{rango}(A)$$

Veamos algunos ejemplos:

Ejemplo 1 - Imagen completa	Ejemplo 2: Imagen como subespacio
$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \Rightarrow \text{Im}(A) = \mathbb{R}^2$ › La matriz identidad no comprime el espacio. › Cualquier vector de $\mathbb{R}^2$ es imagen de otro vector de $\mathbb{R}^2$	$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \Rightarrow \text{Im}(B) = \text{span}\left\{\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right\}$ › La segunda columna es combinación lineal de la primera. › La imagen es una recta en $\mathbb{R}^2$ › La transformación colapsa el plano a una dimensión.

3.5. Matrices especiales relevantes en IA

Las matrices “especiales” poseen propiedades estructurales que permiten diseñar, analizar y optimizar modelos de Inteligencia Artificial de forma más eficiente y estable. Conocer sus diferencias permite interpretar datos, entender el comportamiento de funciones de costo, analizar redes neuronales y aplicar métodos espectrales.

3.5.1. Matrices simétricas

Una matriz cuadrada $A \in \mathbb{R}^{n \times n}$ se llama simétrica si coincide con su traspuesta:

$A = A^T$ Equivalente a decir que sus entradas cumplen

$$a_{ij} = a_{ji} \text{ para todo } i, j.$$



Énfasis

Es decir, la matriz es “espejada” respecto a la diagonal principal.

Ejemplo 1	Ejemplo 2
$A = \begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix}, A^T = \begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix} \Rightarrow A = A^T.$	$B = \begin{bmatrix} 1 & 0 & 4 \\ 0 & 2 & -1 \\ 4 & -1 & 5 \end{bmatrix}$

Aquí se cumple $b_{12} = b_{21} = 0$, $b_{13} = b_{31} = 4$, $b_{23} = b_{32} = -1$, por lo que B es simétrica.



En una matriz simétrica basta con especificar los elementos por encima de la diagonal; el resto queda determinado por simetría.



Ilustración 13 - Matrices simétrica e IA - Esquema elaborado con Napkin - Información propia

En IA, cuando una matriz es simétrica, se sabe que su análisis será estable, interpretativo, geométrico y basado en direcciones ortogonales.

3.5.2. Matrices definidas positivas

Una matriz simétrica A es definida positiva si:

$$x^T A x > 0 \forall x \neq 0.$$

Estas matrices representan funciones convexas, energía o distancia generalizada. Tienen autovalores estrictamente positivos. Por ejemplo:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$$

Para cualquier: $x \neq 0$

$$x^T A x = 2x_1^2 + 5x_2^2 > 0.$$

Importancia de la Definición Positiva en IA

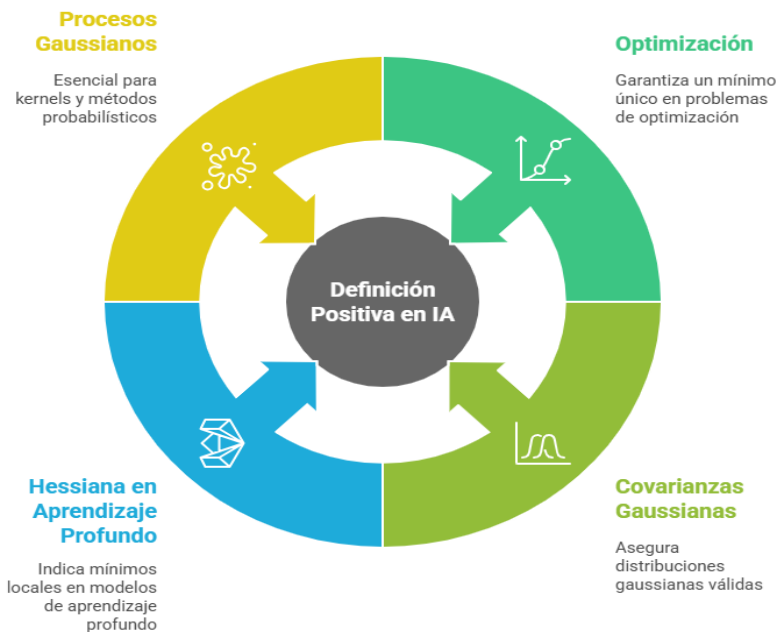


Ilustración 14 - Matrices Positivas e IA — Esquema elaborado con Napkin - Información propia

Saber reconocer estas matrices permite saber *cuándo una función es convexa, cuándo un modelo es estable y cuándo un método tiene solución única*.

3.5.3 Teorema espectral

- **Enunciado**

Toda matriz simétrica real puede descomponerse como:

$$A = Q\Lambda Q^T,$$

donde:

- › Q : matriz ortogonal (autovectores ortogonales),
- › Λ : matriz diagonal con autovalores reales.

Este teorema permite “descomponer” la acción de la matriz en direcciones independientes, cada una amplificada por un autovalor.

Fundamentos de la IA



Ilustración 15 - Teorema espectral e IA - Construido con Napkin - Información propia

Veamos que sucede en el ejemplo:

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

Su descomposición espectral genera:

- › autovalores: 4 y 2
- › autovectores ortogonales

Aplicaciones del Teorema de Descomposición Espectral



Ilustración 16 - Teorema de Descomposición Espectral e IA - Esquema elaborado con Napkin - Información propia

Sin esta herramienta, sería imposible interpretar cómo se distribuyen los datos o cómo evolucionan las funciones de costo.

3.5.4. Matrices ortogonales

Una matriz $Q \in \mathbb{R}^{n \times n}$ es ortogonal si:

$$Q^T Q = I.$$

Sus columnas son vectores unitarios y mutuamente perpendiculares. Por lo tanto:

- › preservan longitudes,
- › preservan ángulos,
- › no distorsionan el espacio.

Ejemplo

$$Q = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} Q^T Q = I.$$

Es una reflexión/rotación del plano.



Ilustración 17 - Importancia de la Ortogonalidad en IA- Esquema elaborado con Napkin - Información propia

En IA, una matriz ortogonal garantiza que la información no se deforma.

3.5.5. Matrices dispersas (sparse)

Una matriz es dispersa si la mayoría de sus entradas son cero. Se utiliza para representar estructuras donde las conexiones son escasas:



Ilustración 18 - Matrices dispersas - Esquema elaborado con Napkin - Información propia

Ejemplo

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

La mayoría son ceros → matriz dispersa.

Importancia de la distinción en IA

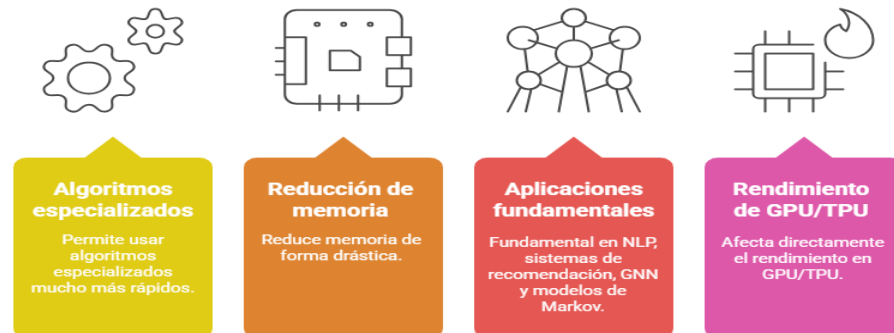


Ilustración 19 - Matriz Dispersa e IA - Esquema elaborado con Napkin - Información propia

En IA, la dispersión permite escalar modelos a millones de parámetros sin costos prohibitivos.

A continuación, dejamos a disposición lecturas y recursos que le serán de utilidad:



Lecturas generales sobre Álgebra Lineal y IA	Mathematics for Machine Learning – Deisenroth, Faisal & Ong (2020)
Estas obras cubren los fundamentos con orientación moderna:	PDF gratuito (sitio oficial): https://tinyurl.com/4vpsretu Capítulos clave: › Cap. 2 (Álgebra Lineal) › Cap. 6 (Métodos Numéricos) › Cap. 7 (Análisis Espectral)
Lecturas específicas por tipo de matriz	
Matrices Simétricas	Libros › Spectral Graph Theory – Fan Chung (1997) › Matrix Analysis – Horn & Johnson (2012) (texto clásico para teoría espectral) Papers › Ng, Jordan & Weiss (2002) — Spectral Clustering: Analysis and Algorithm › Belkin & Niyogi (2003) — Laplacian Eigenmaps for Dimensionality Reduction



Recursos



Lectura
complementaria

Recursos interactivos

- › Visualización de PCA y autovectores:
<https://tinyurl.com/4f9yb5tc>
- › Tutorial visual sobre autovalores/autovectores
(3Blue1Brown): <https://tinyurl.com/mttydc5r>

Matrices Definidas Positivas

(convexidad, optimización, modelos gaussianos)

Libros

- › Convex Optimization – Boyd & Vandenberghe (2004)
- › PDF gratuito: <https://web.stanford.edu/~boyd/cvxbook/>

Capítulos relevantes:

- › Cap. 2 (Matrices definidas positivas)
- › Cap. 4 (Funciones cuadráticas)

Papers

- › Rasmussen & Williams (2006) — Gaussian Processes for Machine Learning
- › PDF gratuito: <http://www.gaussianprocess.org/gpml/>

Recursos interactivos

- › Playground de Gaussian Processes : <https://tinyurl.com/4yc276tb>

Teorema Espectral y métodos espectrales

Libros

- › Applied Linear Algebra – Olver & Shakiban (2020)
- › Matrix Computations – Golub & Van Loan (clásico para descomposiciones)

Papers

- › Shi & Malik (2000) — Normalized Cuts and Image Segmentation
- › Von Luxburg (2007) — Tutorial on Spectral Clustering



Recursos

Recursos interactivos

Explicación visual del espectro y PCA (3Blue1Brown)

Matrices Ortogonales

(rotaciones, estabilidad numérica, preservación de norma)



Libros

- › Numerical Linear Algebra – Trefethen & Bau (1997) Excelente para entender por qué las matrices ortogonales son numéricamente estables.



Recursos interactivos

Rotaciones en 2D/3D (GeoGebra): <https://tinyurl.com/3xs6eyt9>

Matrices Dispersas (Sparse)

(NLP, grafos, sistemas de recomendación, deep learning eficiente)



Libros

- › Foundations of Data Science – Blum, Hopcroft, Kannan (2015) Cap. sobre matrices dispersas y grafos.

Documentación técnica (muy útil para IA práctica)

- › documentación SciPy sobre scipy.sparse <https://tinyurl.com/47wxu9a3>
- › PyTorch sparse tensors <https://pytorch.org/docs/stable/sparse.html>

Papers

- › Kipf & Welling (2017) — Graph Convolutional Networks
- › Hamilton (2020) — Graph Representation Learning



Recursos interactivos

- › Visualizador de matrices dispersas: <https://sparse.tamu.edu>
- › Playground de grafos (Stanford SNAP): <https://tinyurl.com/2d2mj3vv>



En este punto se encuentra en condiciones de realizar la **Actividad 1: Álgebra Lineal para IA**.

• Determinantes

El determinante es un escalar asociado a una matriz cuadrada que describe:

- › La capacidad de la matriz para ser invertible.
- › El cambio de volumen producido por la transformación lineal.
- › La existencia de soluciones únicas en sistemas lineales.

Veamos cómo realizar el cálculo de Determinantes 2x2

Para una matriz 2×2 :

$$\det(A) = ad - bc$$

- **Importancia en IA**

El determinante es fundamental para:

- › Verificar si una transformación es invertible (claves en normalización, PCA).
- › Detectar singularidades en modelos.
- › Analizar estabilidad numérica en métodos de optimización.

Aquí puede ver un ejemplo aplicado en IA

Considérese la matriz de covarianzas de un conjunto de datos bidimensional:

$$\Sigma = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$$

$$\det(\Sigma) = 4(3) - 2(2) = 8$$

Como el determinante es positivo y distinto de cero, la matriz es **definida positiva**, condición requerida para:

- › Modelos Gaussianos.
- › Métodos de optimización convexa.
- › PCA y análisis multivariado.

Ahora bien, no siempre las matrices son 2x2 y eso hace que existan otros métodos para poder calcular la determinante.

- **Regla de Sarrus (matrices 3x3)**

Este método es una forma rápida de aplicar la expansión por cofactores en el caso 3x3.

Funciona *porque* el determinante de 3x3 se puede escribir como suma de productos de 3 elementos, con signos + y – alternados.

Considere la matriz:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$$

- › **Paso 1: repetir las dos primeras columnas a la derecha**

$$\begin{bmatrix} 1 & 2 & 3 & 1 & 2 \\ 0 & 1 & 4 & 0 & 1 \\ 5 & 6 & 0 & 5 & 6 \end{bmatrix}$$

- › Paso 2: multiplicar las diagonales “hacia abajo” y sumarlas

Diagonales principales:

$$\begin{aligned} 1 \cdot 1 \cdot 0 &= 0 \\ 2 \cdot 4 \cdot 5 &= 40 \\ 3 \cdot 0 \cdot 6 &= 0 \end{aligned}$$

Suma de esas diagonales:

$$S_{\downarrow} = 0 + 40 + 0 = 40$$

- › Paso 3: multiplicar las diagonales “hacia arriba” y sumarlas

Diagonales secundarias:

$$\begin{aligned} 5 \cdot 1 \cdot 3 &= 15 \\ 6 \cdot 4 \cdot 1 &= 24 \\ 0 \cdot 0 \cdot 2 &= 0 \end{aligned}$$

Suma:

$$S_{\uparrow} = 15 + 24 + 0 = 39$$

- › Paso 4: restar

$$\det(A) = S_{\downarrow} - S_{\uparrow} = 40 - 39 = 1$$

- **Expansión por cofactores (para $n \times n$)**

Este método es general: sirve para cualquier tamaño $n \times n$.

La idea es:

Fijar una fila o columna, y escribir el determinante como suma de entradas de esa fila/columna multiplicadas por los determinantes de las submatrices que quedan al eliminar su fila y columna, con signos + y – alternados.

- **Fórmula general**

Sea $A = (a_{ij})$ una matriz $n \times n$. El determinante se puede calcular, por ejemplo, expandiendo por la primera fila:

$$\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(M_{1j})$$

donde M_{1j} es la submatriz que resulta de eliminar la fila 1 y la columna j .

Los factores $(-1)^{1+j}$ dan la alternancia de signos.

› *Ejemplo 3×3 con la misma matriz*

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$$

Expansión por la primera fila:

$$\det(A) = (+) 1 \cdot \det \begin{bmatrix} 1 & 4 \\ 6 & 0 \end{bmatrix} (-) 2 \cdot \det \begin{bmatrix} 0 & 4 \\ 5 & 0 \end{bmatrix} (+) 3 \cdot \det \begin{bmatrix} 0 & 1 \\ 5 & 6 \end{bmatrix}$$

Calculemos cada 2×2:

$$1. \det \begin{bmatrix} 1 & 4 \\ 6 & 0 \end{bmatrix} = 1 \cdot 0 - 4 \cdot 6 = -24$$

$$2. \det \begin{bmatrix} 0 & 4 \\ 5 & 0 \end{bmatrix} = 0 \cdot 0 - 4 \cdot 5 = -20$$

$$3. \det \begin{bmatrix} 0 & 1 \\ 5 & 6 \end{bmatrix} = 0 \cdot 6 - 1 \cdot 5 = -5$$

› *Sustituimos:*

$$\text{› Primer término: } +1 \cdot (-24) = -24$$

$$\text{› Segundo término: } -2 \cdot (-20) = +40$$

$$\text{› Tercer término: } +3 \cdot (-5) = -15$$

Sumando:

$$\det(A) = -24 + 40 - 15 = 1$$

Coincide con Sarrus, como debe suceder.

› *Para n×n*

› El mismo procedimiento se repite “recursivamente”: cada $\det(M_{ij})$ es un determinante de tamaño $n-1$, que se vuelve a expandir por cofactores, etc.

› En la práctica, para n grande, esto es costoso (crece factorialmente).

› Por eso, en matrices grandes se usa casi siempre el método de reducción por filas o algoritmos más eficientes (LU, etc.).

› *Reducción por filas (Gauss) y producto de la diagonal*

Este método se basa en la idea de que es más fácil calcular el determinante de una matriz triangular (superior o inferior), porque en ese caso:

El determinante de una matriz triangular es el producto de sus elementos en la diagonal principal. Luego se transforma la matriz original en una triangular, usando operaciones de fila que conocemos cómo afectan el determinante.

› *Reglas clave*

Al aplicar operaciones elementales por filas:

1. Intercambiar dos filas $\rightarrow$ multiplica el determinante por -1 .
2. Multiplicar una fila por un escalar $k \rightarrow$ multiplica el determinante por k .
3. Sumar a una fila un múltiplo de otra fila $\rightarrow$ no cambia el determinante.

Estas propiedades se derivan de la definición del determinante como función multilineal y alternante, pero para usar el método basta con recordarlas.

A continuación, un ejemplo (3x3) por reducción a triangular

Considere la matriz:

$$B = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 2 & 4 \end{bmatrix}$$

Queremos llevarla a forma triangular superior usando solo operaciones del tipo “fila $\leftarrow$ fila – (número) * otra fila”, que no cambian el determinante. Así, el determinante final será simplemente el producto de la diagonal.

› *Paso 1: usar la primera fila como pivote*

Pivot en (1,1) = 2.

Queremos hacer cero el elemento de la posición (2,1), que vale 1.

Factor para restar: $\frac{1}{2}$

Nueva fila 2:

$$F_2 \leftarrow F_2 - \frac{1}{2}F_1$$

Cálculo:

- Fila 1: [2, 1, 0]
- Fila 2 original: [1, 3, 1]

$$F_2 = \left[1 - \frac{1}{2} \cdot 2, \quad 3 - \frac{1}{2} \cdot 1, \quad 1 - \frac{1}{2} \cdot 0\right] = [0, 3 - 0.5, 1] = [0, 2.5, 1]$$

Fila 3 queda igual: [0, 2, 4]

Nueva matriz:

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 2.5 & 1 \\ 0 & 2 & 4 \end{bmatrix}$$

El determinante sigue siendo el mismo, porque solo sumamos a una fila un múltiplo de otra.

► *Paso 2: usar el pivote de la segunda fila*

Ahora el pivote en (2,2) es 2.5 (que es $\frac{5}{2}$).

Queremos hacer cero el elemento (3,2), que vale 2.

Factor:

$$\frac{2}{2.5} = 0.8 = \frac{4}{5}$$

Nueva fila 3:

$$F_3 \leftarrow F_3 - 0.8 F_2$$

Cálculo:

- Fila 3 original: $[0, 2, 4]$
- Fila 2: $[0, 2.5, 1]$

$$F_3 = [0, 2 - 0.8 \cdot 2.5, 4 - 0.8 \cdot 1] = [0, 2 - 2, 4 - 0.8] = [0, 0, 3.2]$$

La matriz queda:

$$B' = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2.5 & 1 \\ 0 & 0 & 3.2 \end{bmatrix}$$

Y nuevamente, el determinante no cambió, porque solo se aplicó operación tipo 3 (*sumar múltiplos de filas*).

► *Paso 3: producto de la diagonal*

Como ahora B' es triangular superior:

$$\det(B') = 2 \cdot 2.5 \cdot 3.2$$

Cálculo:

- $2 \cdot 2.5 = 5$
- $5 \cdot 3.2 = 16$

$$\Rightarrow \det(B) = 16$$

No hace falta ajustar signos ni multiplicadores porque no intercambiamos filas ni multiplicamos filas por escalares, solo restamos múltiplos.

La tabla que sigue muestra los pro y contras de cada método:

Método	Ventajas	Desventajas	Notas
Sarrus (3×3)	Rápido, visual	Solo sirve para 3×3	Forma compacta de la expansión por cofactores
Cofactores ($n \times n$)	Método general, muy bueno para demostraciones y matrices pequeñas	Costoso para matrices grandes	
Reducción por filas ($n \times n$)	Método práctico para matrices grandes		Base de cómo lo implementan los programas de cálculo (NumPy, MATLAB, etc.)

Tabla 1 - Cálculo de determinantes –Elaboración propia



Actividad

Ahora, usted se encuentra en condiciones de realizar la **Actividad 2: Álgebra Lineal en Sistemas Inteligentes**.

La comprensión profunda del álgebra lineal permite acceder al funcionamiento interno de los algoritmos de inteligencia artificial y proporciona las herramientas matemáticas necesarias para analizar, optimizar y justificar decisiones técnicas en entornos de aprendizaje automático. Los conceptos de vectores, matrices y determinantes —aunque fundamentales— poseen una riqueza estructural que se manifiesta en aplicaciones críticas: la propagación lineal en redes neuronales, la interpretación geométrica de representaciones vectoriales, la estabilidad numérica de los modelos, la reducción de dimensionalidad y la formulación de modelos probabilísticos multivariados.

A lo largo del módulo hemos podido señalar que el álgebra lineal no es únicamente un recurso operativo, sino una estructura conceptual que otorga coherencia teórica a los métodos modernos de IA. Su dominio permite identificar relaciones entre variables, evaluar la calidad de las transformaciones aplicadas, diagnosticar problemas de singularidad, comprender el papel de los subespacios vectoriales y anticipar el rendimiento o las limitaciones de los algoritmos. De esta manera, las nociones desarrolladas en este módulo constituyen un punto de partida indispensable para avanzar hacia contenidos más complejos del curso, tales como cálculo aplicado, optimización y técnicas estadísticas, que se apoyan directamente en los principios establecidos aquí.

MÓDULO 1 | GLOSARIO

Autovalor: Cantidad que describe cuánto estira o comprime una transformación en una dirección específica.

Autovector: Dirección en la que una matriz estira o comprime sin cambiar la orientación. Crucial en PCA, análisis espectral y optimización.

Combinación lineal: Resultado de sumar vectores ponderados por escalares. Describe todas las representaciones posibles en un modelo lineal.

Descomposición Espectral: Representación de una matriz simétrica como $A=Q Q^T$. Es la base de PCA, análisis de grafos y métodos espectrales.

Determinante: Valor asociado a matrices cuadradas que indica volumen, invertibilidad y estabilidad numérica. En IA se utiliza en modelos generativos y normalizing flows.

Espacio vectorial: Conjunto de vectores que permite realizar suma y multiplicación por escalares. Es el “espacio” donde viven los datos en IA.

Imagen de una matriz: Subespacio formado por todas las salidas posibles de la transformación lineal. En IA refleja la capacidad representativa de un modelo.

Independencia lineal: Conjunto de vectores donde ninguno puede escribirse como combinación de los otros. En IA indica ausencia de redundancia.

Matriz de Covarianza: Matriz simétrica que describe la relación entre variables. Sus autovalores y autovectores permiten extraer componentes principales.

Matriz Definida Positiva: Matriz que produce valores positivos para cualquier $x \neq 0$. Indica convexidad y estabilidad en optimización y modelos gaussianos.

Matriz Dispersa (Sparse): Matriz con mayoría de valores cero. Común en NLP, grafos y sistemas recomendadores; permite cálculos eficientes.

Matriz Invertible: Matriz con determinante no nulo. Garantiza que la transformación no pierde información.

Matriz Ortogonal: Matriz cuyas columnas son ortonormales ($Q^T Q=I$). Preserva longitudes y ángulos. Se usa en PCA y en inicialización de redes.

Matriz Simétrica: Matriz igual a su traspuesta. Tiene autovalores reales y autovectores ortogonales. Fundamental en covarianzas y Laplacianos de grafos.

Matriz: Arreglo rectangular de números que representa transformaciones lineales, parámetros de redes neuronales o relaciones entre variables.

Norma: Medida de la longitud o magnitud de un vector. Se usa para normalización, distancias y regularización.

Nulidad (Kernel): Cantidad de dimensiones que la matriz destruye al aplicar la transformación. Representa pérdida de información.

PCA (Principal Component Analysis): Método para reducir dimensionalidad usando autovectores de la matriz de covarianza.

Producto Punto: Operación que relaciona dos vectores y permite medir similitud. Fundamental en redes neuronales y mecanismos de atención.

Rango: Número de columnas (o filas) linealmente independientes de una matriz. Indica cuánta información preserva una transformación.

Subespacio: Conjunto formado por todas las combinaciones lineales de ciertos vectores. Representa el “universo de soluciones” que puede generar un modelo.

Transformación Lineal: Operación que preserva suma y multiplicación por escalares. En redes neuronales corresponde al cálculo $Wx+b$.

Vector: Entidad matemática formada por una lista ordenada de números. Representa datos, características, embeddings o estados en modelos de IA.

MÓDULO 1 | ACTIVIDADES



ACTIVIDAD N° 1: Álgebra Lineal para IA

Usted trabaja en un pequeño equipo de Inteligencia Artificial que necesita verificar si los datos y las matrices utilizadas en los modelos son adecuadas, no tienen redundancias y no generan pérdida de información.

Para esto, debe aplicar los conceptos fundamentales del módulo.

Datos para trabajar

1. Mini-dataset (3 muestras $\times$ 4 variables)

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \\ 1 & 1 & 0 & 2 \end{bmatrix}$$

2. Matriz de pesos de una capa densa

$$W = \begin{bmatrix} 1 & -1 & 2 & 0 \\ 2 & -2 & 4 & 0 \end{bmatrix}$$

3. Matriz de covarianza

$$\Sigma = \begin{bmatrix} 5 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 4 \end{bmatrix}$$

4. Matriz dispersa de un grafo pequeño

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Tareas para realizar:

1. Vectores y redundancia en los datos:
 - › Analice si las tres filas del dataset X son independientes.
 - › Explique brevemente por qué la dependencia o independencia afecta a los modelos de IA.
2. Transformación lineal y rango de la matriz W
 - › Determine si W tiene rango máximo o no.
 - › Indique si la capa densa asociada a W pierde información.

3. Matriz de covarianza y autovalores

- › Verifique que es simétrica.
- › Utilice una herramienta (*GeoGebra*, *WolframAlpha*, etc.) para calcular sus autovalores.
- › Explique qué representa el autovalor mayor en términos de “dirección principal”.

4. Matrices dispersas y grafos

- › Indique qué nodos están más conectados en el grafo representado por A.
- › Explique en dos o tres líneas por qué las matrices dispersas ayudan a mejorar el rendimiento en IA.

5. Reflexión final

- › En 5–7 líneas, explique cómo el análisis de rango, autovalores y dispersidad ayuda a comprender mejor el comportamiento de los modelos de IA.



Recursos

Le sugerimos utilizar los siguientes recursos:

- › GeoGebra Matrix Calculator: <https://www.geogebra.org/matrix>
- › 3Blue1Brown – Álgebra Lineal (videos): <https://www.3blue1brown.com/>
- › PCA interactivo (visualizador): <https://setosa.io/ev/principal-component-analysis/>



Actividad

ACTIVIDAD N° 2:

Álgebra Lineal en Sistemas Inteligentes

En esta actividad:

- › Aplicaremos operaciones fundamentales con vectores y matrices en un escenario realista.
- › Interpretaremos el significado de una transformación lineal dentro de un modelo de IA.
- › Analizaremos la invertibilidad y estabilidad de una matriz mediante su determinante.
- › Relacionaremos resultados algebraicos con decisiones de modelado.

Una empresa de análisis predictivo está desarrollando un modelo de clasificación para identificar patrones en un conjunto de datos de dos características. Cada instancia se representará mediante vectores, y las relaciones entre las características deberán analizarse mediante operaciones matriciales. Además, se estudiará la estabilidad del modelo evaluando determinantes y transformaciones lineales asociadas.

Como parte del equipo de análisis, usted debe realizar un trabajo exploratorio con las matrices y vectores provistos, justificando sus procedimientos y explicando la relevancia de cada operación dentro del contexto de la Inteligencia Artificial.

- › *Parte 1: Representación vectorial y análisis preliminar*

Se cuenta con tres instancias de datos representadas por los siguientes vectores en $\mathbb{R}^2$

$$x^{(1)} = (2,1), x^{(2)} = (1,3), x^{(3)} = (4,2).$$

- Describa cómo interpretaría cada vector dentro de un contexto de IA (por ejemplo: entradas, características, estados, etc.).
- Determine si existe relación lineal entre estos vectores. En caso afirmativo, analice qué implicaría para un modelo lineal.
-



Guíe su análisis usando el concepto de combinación e independencia lineales.

Recomendaciones

- › **Parte 2: Transformación lineal aplicada al modelo**

Se propone una transformación lineal representada por la matriz:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

Esta transformación se aplicará a cada vector del conjunto de datos.

- Interprete, conceptualmente, qué significa aplicar la transformación Aa cada instancia.
- Aplique la transformación a cada vector y discuta qué cambios se producen en el espacio de características.
- Explique qué tipo de efectos puede generar una transformación lineal en el rendimiento de un modelo de clasificación.



Use ideas de geometría lineal: *rotaciones, estiramientos, compresiones, mezcla de características*.

Recomendaciones

- › **Parte 3: Determinante e invertibilidad**

Para evaluar la estabilidad del modelo, se necesita verificar si la matriz de transformación Aes invertible.

- Calcule el determinante de Autilizando uno de los métodos estudiados (Sarrus, cofactores o reducción por filas).
- Evalúe si Aes invertible y explique por qué.
- Según su resultado, analice si la transformación preserva información o si existe pérdida de dimensionalidad.
- Interprete qué implicaciones tendría una matriz no invertible en:
 - › la reconstrucción de datos,
 - › la separación de clases,
 - › la estabilidad de un modelo.

› *Parte 4: Matriz de covarianza y variabilidad del sistema*

A partir de los vectores transformados por A , construya la matriz de datos correspondiente y:

- a) Calcule la matriz de covarianza.
- b) Analice si las dos características tienen correlación lineal.
- c) Explique cómo la correlación puede afectar al modelo.



Recomendaciones

No se requiere cálculo de autovalores, pero puede mencionarse su papel si el estudiante lo considera pertinente.

› *Parte 5: Reflexión final aplicada a IA - FORO*

Redacte una reflexión integradora breve (8–10 líneas) respondiendo:

- › ¿Cómo ayuda el álgebra lineal a entender el funcionamiento interno de los modelos?
- › ¿Por qué el determinante es una herramienta relevante para evaluar transformaciones?
- › ¿Cómo se relacionan estos conceptos con la capacidad de un modelo para generalizar?

Indicaciones para la entrega

- › La actividad es grupal.
- › Puede utilizar cuadernos Jupyter, GeoGebra o MATLAB Online para visualizar operaciones (*no para obtener las respuestas directamente*).
- › Cite las fuentes si utiliza bibliografía complementaria.
- › Si uso un asistente inteligente, consigne los diálogos que mantuvo y las decisiones que tomó en función de las respuestas asistidas.

MÓDULO 2

Microobjetivos

- › Analizar funciones reales para identificar dominio, imagen, continuidad y comportamiento gráfico en el contexto del estudio de funciones utilizadas en modelos de inteligencia artificial.
- › Aplicar límites y derivadas para describir crecimiento, decrecimiento, puntos críticos y extremos con la finalidad de interpretar el comportamiento local de funciones de pérdida en el entrenamiento de modelos.
- › Calcular integrales indefinidas y definidas para interpretar áreas, acumulación y promedios en funciones de pérdida media y evaluación de modelos de IA.
- › Diseñar estudios completos de funciones de pérdida sencillas (por ejemplo, pérdida cuadrática) para fundamentar decisiones de optimización y ajuste de parámetros en problemas básicos de aprendizaje supervisado.



VIDEO DOCENTE

Contenidos

- **Cálculo Diferencial e Integral para el Análisis de Funciones en IA**

Antes de iniciar con este módulo, y a manera introductoria, vamos a repasar los conceptos y cálculos más importantes, que servirán de base para la temática específica del módulo.

Necesitamos conocer sobre:

- › *Continuidad* para evitar saltos bruscos de la pérdida al mover parámetros.
- › *Derivabilidad* y así poder calcular gradientes (derivadas parciales) respecto de cada parámetro.
- › *Comportamiento controlado en el infinito* para prevenir que la función no “explote” numéricamente en las regiones donde el optimizador se mueve.

Entonces, bajo esas premisas, comenzamos.



Fuente: Elaboración propia con IA (Gemini)

- **Repaso general del cálculo**

1. ¿Qué es una función?

Una función es una relación que asigna a cada elemento de un conjunto (dominio) un único valor en otro conjunto (codominio). Se denota como:

$$f: X \rightarrow Y \text{ o } y = f(x)$$

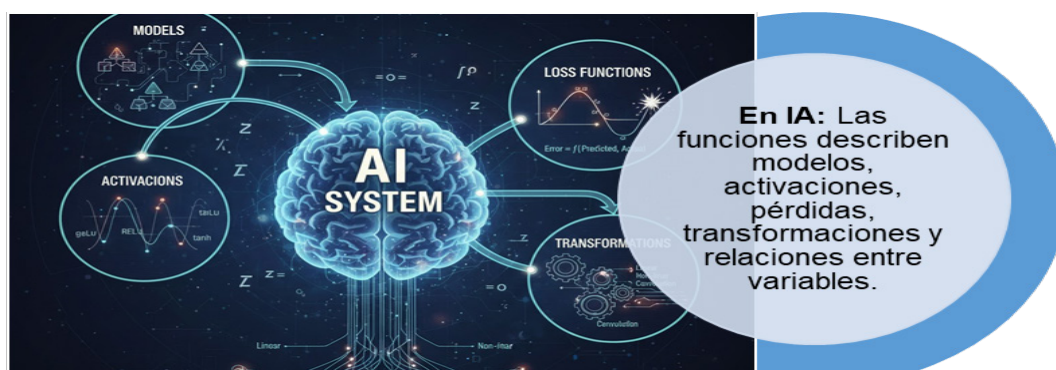
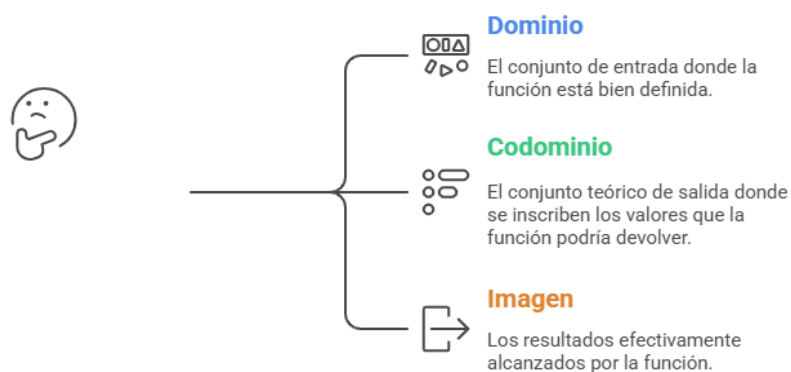


Ilustración 2 - Función e IA - Fuente de la Imagen: <https://tinyurl.com/bdenp6yj>

2. Dominio, codominio e imagen

Antes de avanzar en el terreno del estudio de función, del cálculo y además de relacionarlo con la IA, debemos tener en claro conceptos muy importantes: ¿De dónde extraigo datos? ¿Cuáles puedo usar? ¿Si los uso, que resultado puedo llegar a tener?, y es ahí donde aparecen estos conjuntos: Dominio, Codominio e Imagen.

¿Cómo entender los conceptos de función?



Esquema 1 - Conceptos Base de Estudio de Función—Elaboración propia con Napkin

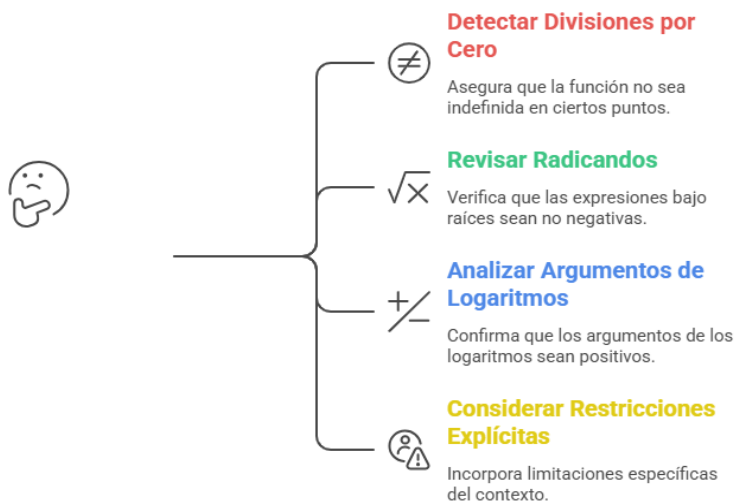
- **Dominio**



El dominio de una función es el conjunto de elementos para los cuales la expresión que la define es matemáticamente válida.

Formalmente, si $f: A \subseteq \mathbb{R} \rightarrow \mathbb{R}$, entonces A es el dominio, y contiene exactamente todos los valores x para los cuales $f(x)$ puede calcularse sin ambigüedades.

¿Cómo identificar el dominio en la práctica técnica?



Esquema 2 - Restricciones del Dominio - Elaboración propia con Napkin

Ejemplos técnicos:

- › $f(x) = \frac{1}{x}$: el dominio excluye a 0 $\rightarrow A = \mathbb{R} \setminus \{0\}$.
- › $f(x) = \ln(x)$: sólo admite valores positivos $\rightarrow A = (0, \infty)$.
- › $f(x) = \sqrt{4 - x^2}$: requiere $4 - x^2 \geq 0 \rightarrow A = [-2, 2]$.
- › $f(x) = e^{-x}$: no presenta restricciones $\rightarrow A = \mathbb{R}$.

- **Codominio**



El codominio es el conjunto en el cual la función toma valores. No indica únicamente qué valores produce efectivamente la función, sino en qué conjunto conceptual se sitúan sus resultados.

Si $f: A \subseteq \mathbb{R} \rightarrow B \subseteq \mathbb{R}$, el conjunto B es el codominio.

En muchos contextos técnicos, sobre todo en IA, el codominio se elige en función del tipo de salida esperada:

- › probabilidades ($[0,1]$),
- › valores de activación (por ejemplo $(0,1)$ para sigmoide),
- › valores sin restricción ($\mathbb{R}$).

Ejemplos:

Para $f(x) = \ln(x)$, el codominio natural es $\mathbb{R}$.

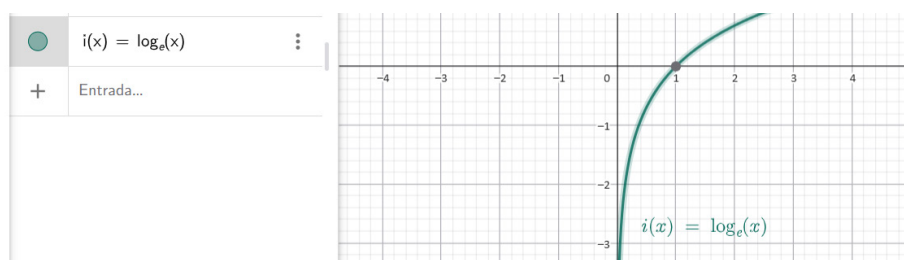


Ilustración 3 - $f(x) = \ln(x)$ - Geogebra – Elaboración propia

Para una función de activación sigmoide, el codominio es $(0,1)$.

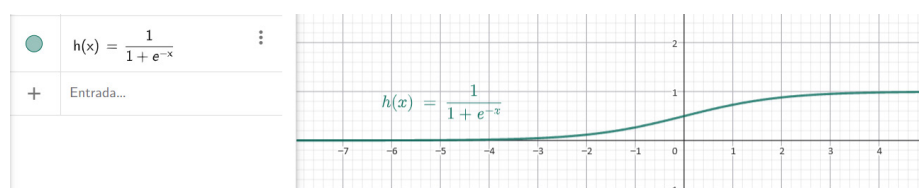


Ilustración 4 - Sigmoide – Geogebra – Elaboración propia

En ReLU, $f(x) = \max(0, x)$, el codominio es $[0, \infty)$.

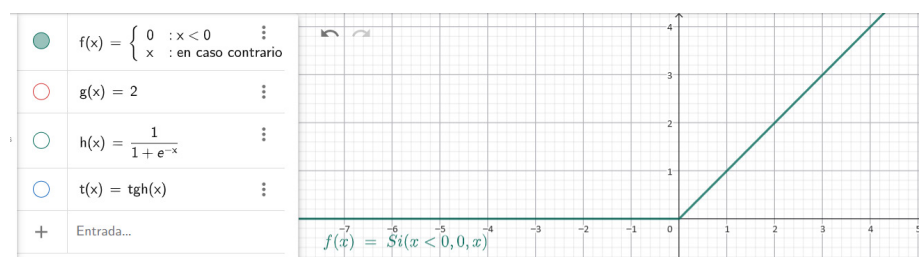


Ilustración 5 - ReLU – Geogebra - Elaboración propia

• Imagen

Es útil recordar un tercer concepto:

- › La imagen es el conjunto de valores que la función efectivamente produce.
- › Siempre es un subconjunto del codominio.

Por ejemplo, si definimos $f: \mathbb{R} \rightarrow \mathbb{R}$ por $f(x) = e^{-x}$, el codominio es $\mathbb{R}$, pero la imagen real es $(0, \infty)$.



Las funciones en la Inteligencia Artificial son los puentes fundamentales que transforman un dominio, como un espacio de características, en resultados interpretables como probabilidades, errores o salidas predichas, permitiendo así que los sistemas de IA comprendan y actúen sobre los datos.

Ilustración 6 - Elaboración propia basada en conceptos de Aprendizaje Automático y Redes Neuronales

• Gráfica de una función

Introducción

Seguimos realizando el repaso. Ahora, es turno de las gráficas de función.



La gráfica de una función es una representación visual de la relación entre valores de entrada (variable independiente) y valores de salida (variable dependiente).

La forma de la gráfica permite comprender patrones de comportamiento que son fundamentales para el análisis matemático y para interpretar modelos de inteligencia artificial.

Esta representación gráfica permite identificar:

- › crecimiento o decrecimiento
- › máximos y mínimos
- › comportamiento en extremos
- › suavidad o discontinuidades
- › zonas donde el gradiente se hace muy pequeño (problema del vanishing gradient)



En IA, analizar la forma de la función de pérdida es fundamental para entender cómo y por qué un modelo aprende.

¿Qué muestra una gráfica?

En su forma más simple, una gráfica en el plano muestra puntos del tipo:

$$(x, f(x))$$

Esto permite visualizar:

- › La tendencia (creciente, decreciente, constante)
- › La curvatura
- › Cambios bruscos o suavidad
- › Máximos y mínimos
- › Puntos críticos
- › Comportamiento asintótico
- › Rangos de valores posibles

Todo esto es clave para entender el comportamiento de funciones usadas en IA.

- **Interpretación geométrica**

Una función puede verse como una curva que describe cómo varía una magnitud respecto de otra.

¿Cómo interpretar una curva en un gráfico?

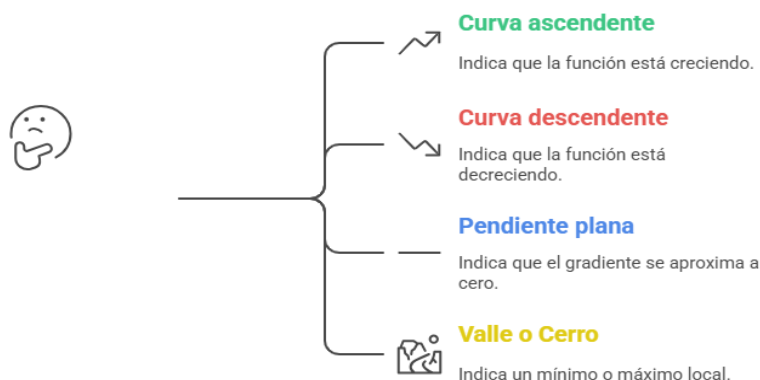


Ilustración 7 - Interpretación geométrica – Elaboración propia con Napkin

Esto se vincula directamente con los problemas de optimización, donde buscamos encontrar el mínimo de una función de pérdida.

- **Comportamientos relevantes en IA**

Veamos el caso de la función que muestra el Gráfico 1: Curva de función de pérdida con mínimo global.

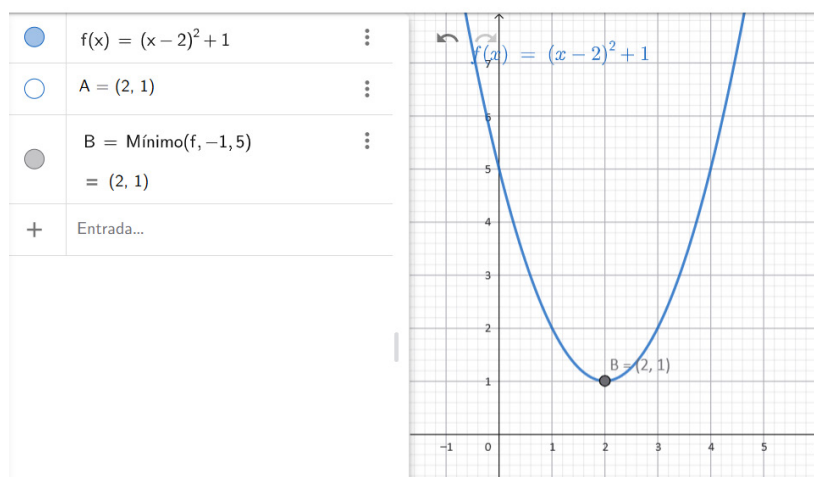


Ilustración 8 - Gráfico 1 – Curva de función de pérdida con mínimo global – Elaboración propia

- **Descripción verbal del gráfico:**

- › Supongamos un sistema de ejes cartesianos:
 - › Eje horizontal: “parámetro del modelo” (w).
 - › Eje vertical: “valor de la función de pérdida” ($L(w)$).
- › La curva tiene forma de “U” suave, abierta hacia arriba (como una parábola).
- › En el centro de la “U” hay un punto marcado en la parte más baja: el mínimo global.
- › A la izquierda y a la derecha del mínimo, la curva sube de manera simétrica.
- › **Qué se explica con este gráfico:**

1. “La función de pérdida es más alta cuando el modelo está mal ajustado”

Traducido:

- › Cuando el punto en la curva está alto, significa que el modelo se está equivocando mucho (predice mal).
- › Cuando el punto está bien abajo, cerca del mínimo de la “U”, significa que sus predicciones son mucho mejores.

¿Cómo está rindiendo el modelo de IA?

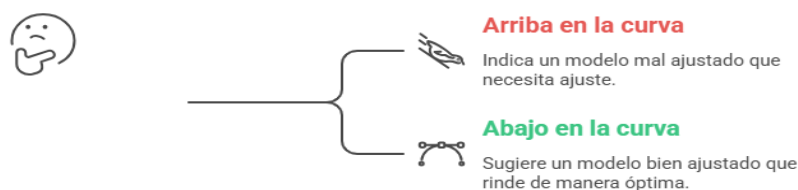


Ilustración 9 – Crecimiento en IA – Elaboración propia con Napkin

2. “El objetivo del algoritmo de entrenamiento es moverse sobre la curva hasta el mínimo”

Acá entra la idea de caminar sobre la curva:

- › Imagine que el modelo empieza con un valor cualquiera de w , por ejemplo $w = -3$.
- › Ese punto está en uno de los “brazos” de la U, relativamente alto $\rightarrow$ hay mucho error.
- › El algoritmo de entrenamiento lo que hace es ir ajustando w , paso a paso, para ir bajando por la curva.

La idea de descenso por gradiente es:

¿En qué dirección debo moverme en la curva?

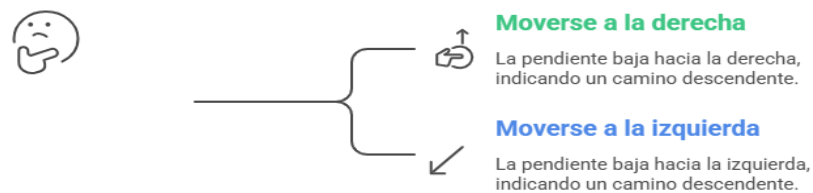


Ilustración 10—Gradiente—Propio - Elaboración propia con Napkin

Eso es literalmente lo que hace el entrenamiento: ir probando nuevos valores de los parámetros para reducir la pérdida.

3. “La pendiente indica hacia dónde debe moverse el parámetro para reducir el error”

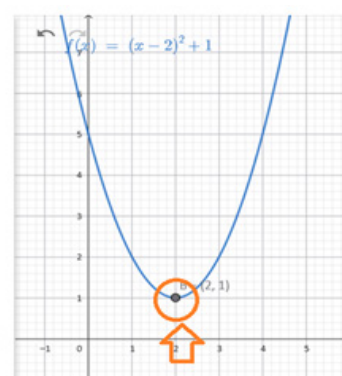
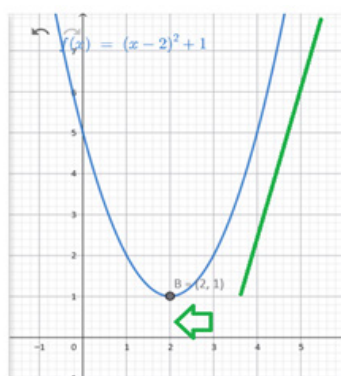
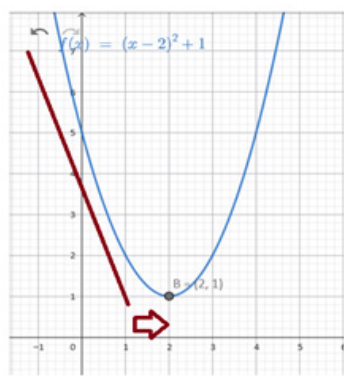
Esta es la parte clave para conectar con derivadas y gradientes¹:

- › La pendiente en un punto de la curva nos dice si la función está subiendo o bajando en ese lugar y en qué dirección.

¹ Tema que se desarrollará más adelante dentro de este módulo.

Imagine tres escenarios:

Escenario A: Está en el brazo izquierdo de la U	Escenario B: Está en el brazo derecho de la U	Escenario C: Está en el punto más bajo
La curva está bajando hacia la derecha:	La curva está bajando hacia la izquierda:	La pendiente allí es cero:
- La pendiente es negativa (la recta tangente se inclina hacia abajo cuando mirás hacia la derecha). - Eso significa: "Si avanzo hacia la derecha (aumento w), el valor de la pérdida va a bajar". Entonces el algoritmo decide: aumentar w.	- La pendiente es positiva. - Eso significa: "Si avanzo hacia la izquierda (disminuyo w), el valor de la pérdida va a bajar". Entonces el algoritmo decide: disminuir w.	- No sube ni a la izquierda ni a la derecha. - El algoritmo interpreta: "Llegué a un punto donde moverme un poco hacia un lado o hacia el otro ya no mejora el error". - Eso es un mínimo (idealmente, el mínimo global).



¿Cuáles son los elementos clave de una gráfica para IA?

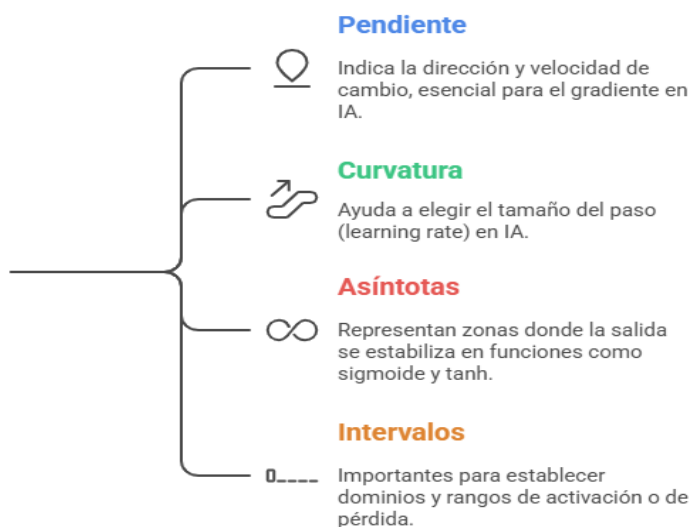


Ilustración 11 – Elementos Clave – Elaboración propia con Napkin



Actividad

Estamos en condiciones de hacer la Actividad 1: Lectura e interpretación de gráficas de funciones aplicadas a la Inteligencia Artificial. Le sugerimos resolver la misma antes de continuar con la lectura del módulo.

- ***Tipos de funciones relevantes en Inteligencia Artificial***

En el contexto de la Inteligencia Artificial, las funciones se utilizan para modelar relaciones entre variables, transformar representaciones, calcular probabilidades o definir criterios de optimización (funciones de pérdida). A continuación, se presentan los tipos de funciones más utilizados, con su formulación matemática y su aplicación típica en modelos de IA.

- ***Funciones lineales y afines***

Una función lineal en una variable real se define como:

$$f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = ax,$$

donde $a \in \mathbb{R}$ es una constante.

Más generalmente, en dimensión n :

$$f: \mathbb{R}^n \rightarrow \mathbb{R}, f(\mathbf{x}) = \mathbf{w}^T \mathbf{x},$$

donde $\mathbf{x} \in \mathbb{R}^n$ y $\mathbf{w} \in \mathbb{R}^n$.

Una función afín añade un término independiente:

$$f(x) = ax + b, \text{ o bien } f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b.$$

En regresión lineal univariada:

$$\hat{y} = wx + b,$$

donde $\hat{y}$ es la predicción del modelo para una entrada escalar x .

Aplicaciones de la transformación afín en IA

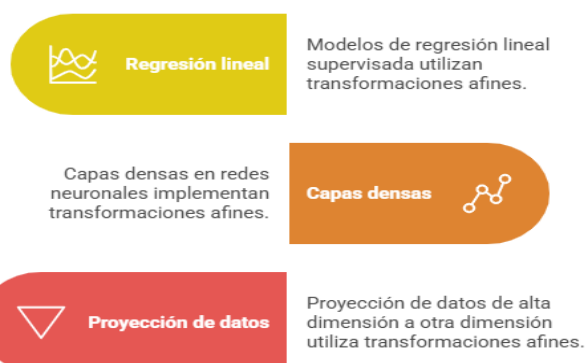
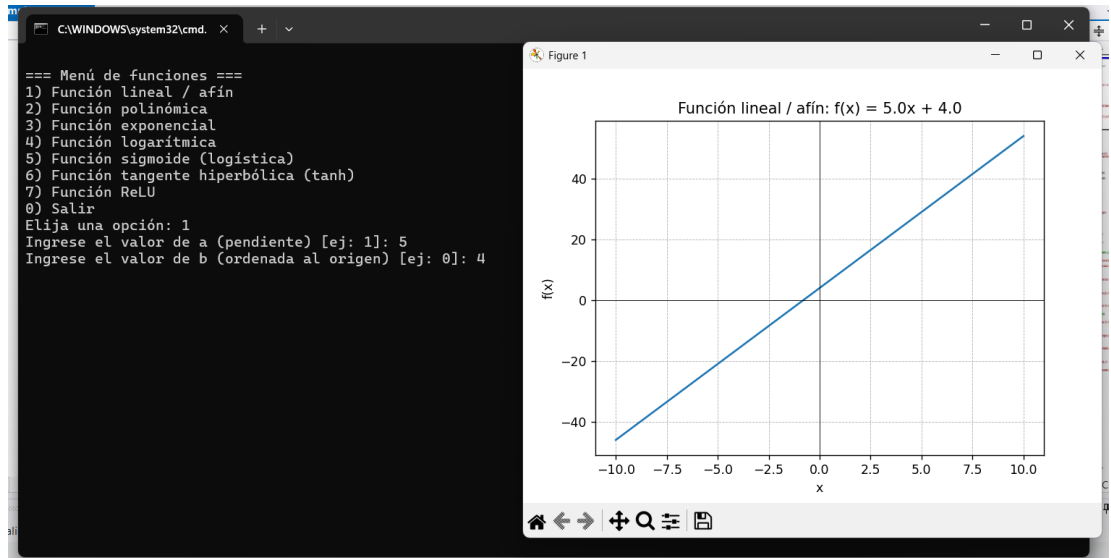


Ilustración 12 – Función Afín e IA – Elaboración propia con Napkin



Python es un gran aliado a la hora de trabajar con funciones y gráficas. La imagen que puede ver a continuación es el resultado de un desarrollo simple que permite graficar los ejemplos del módulo.

Dejamos aquí el enlace hacia el código: <https://tinyurl.com/yeyvth5n> ;Recuerde descargarlo y correrlo en forma local!



- *Funciones polinómicas*

Una función polinómica de grado n se define como:

$$p: \mathbb{R} \rightarrow \mathbb{R}, p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0,$$

con $a_i \in \mathbb{R}$ y $a_n \neq 0$.

Ejemplo

$$p(x) = 2x^2 - 3x + 1.$$

Uso de las Funciones polinómicas en IA

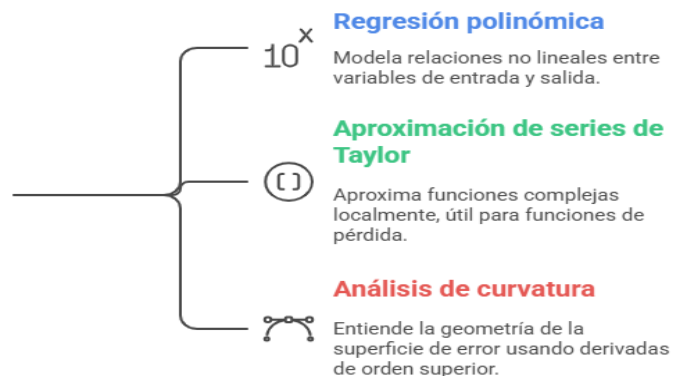
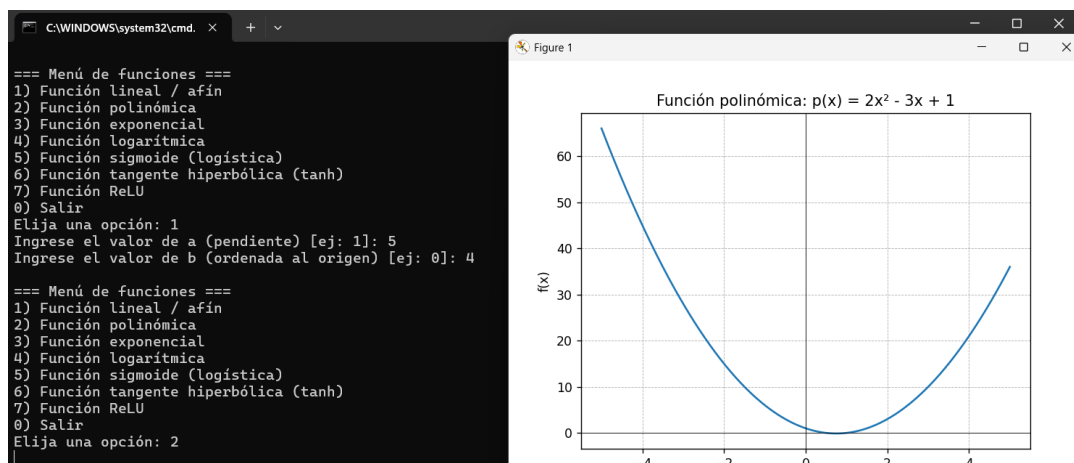


Ilustración 13 – Funciones Polinómicas – Elaboración propia con Napki



Dejamos aquí el enlace hacia el código: <https://tinyurl.com/yeyvth5n> ;No olvide descargarlo y correrlo en forma local!



- *Funciones exponenciales y logarítmicas*

- a) *Función exponencial*

La función exponencial real se define como:

$$f: \mathbb{R} \rightarrow \mathbb{R}_{>0}, f(x) = e^x,$$

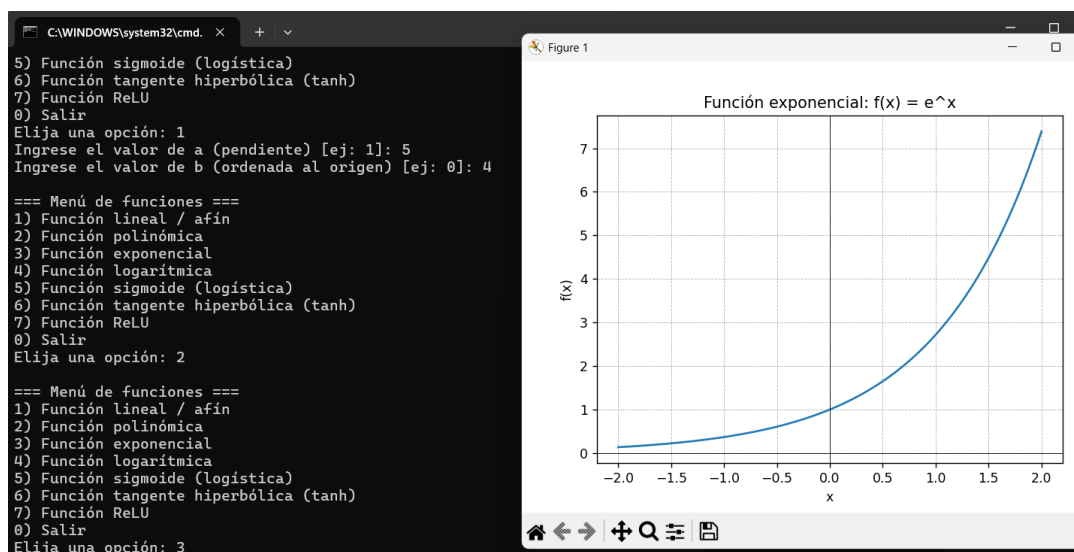
donde e es la base del logaritmo natural.

Propiedades relevantes:

- › $e^x > 0$ para todo $x \in \mathbb{R}$.
- › $e^{x+y} = e^x \cdot e^y$.



A continuación, puede ingresar al enlace del código: <https://tinyurl.com/bd8nshx3>



b) Función logarítmica

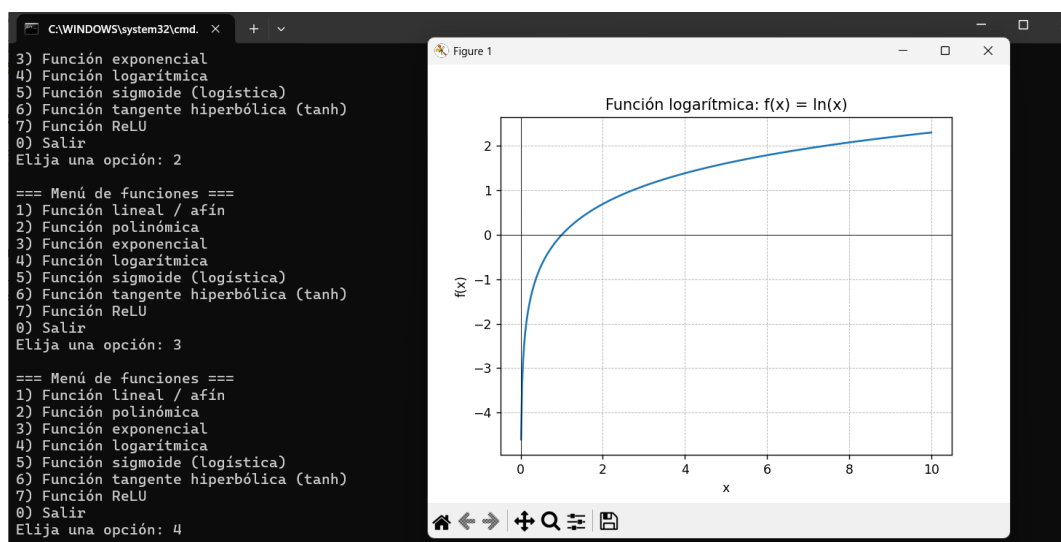
La función logaritmo natural es la inversa de la exponencial:

$$\ln: \mathbb{R}_{>0} \rightarrow \mathbb{R}, y = \ln(x) \Leftrightarrow x = e^y.$$



Código

Dejamos el enlace al código a continuación: <https://tinyurl.com/yeyvth5n> Como siempre, recuerde descargar y correrlo en forma local.



¿Cómo se utilizan las funciones exponenciales y logarítmicas en IA?

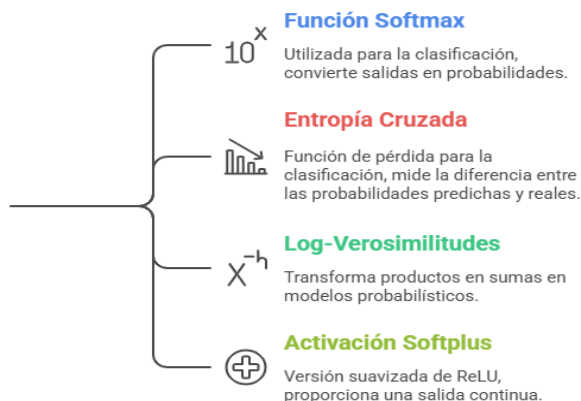


Ilustración 14 – Funciones exponenciales y Logarítmicas – Elaboración propia con Napkin

• Funciones sigmoides

Las funciones sigmoides son funciones suaves, acotadas y monótonamente crecientes, con forma característica de “S”. Son ampliamente utilizadas como funciones de activación.

A. Función logística (sigmoid clásica)

$$\sigma: \mathbb{R} \rightarrow (0,1), \sigma(x) = \frac{1}{1 + e^{-x}}.$$

Propiedades:

- › $\lim_{x \rightarrow -\infty} \sigma(x) = 0$, $\lim_{x \rightarrow +\infty} \sigma(x) = 1$.
- › Es continua y derivable en todo $\mathbb{R}$.

¿Cómo usar la función logística en IA?



Función de activación

$$10^x$$

Se utiliza en neuronas para clasificación binaria, interpretando como una probabilidad.

Capa de salida



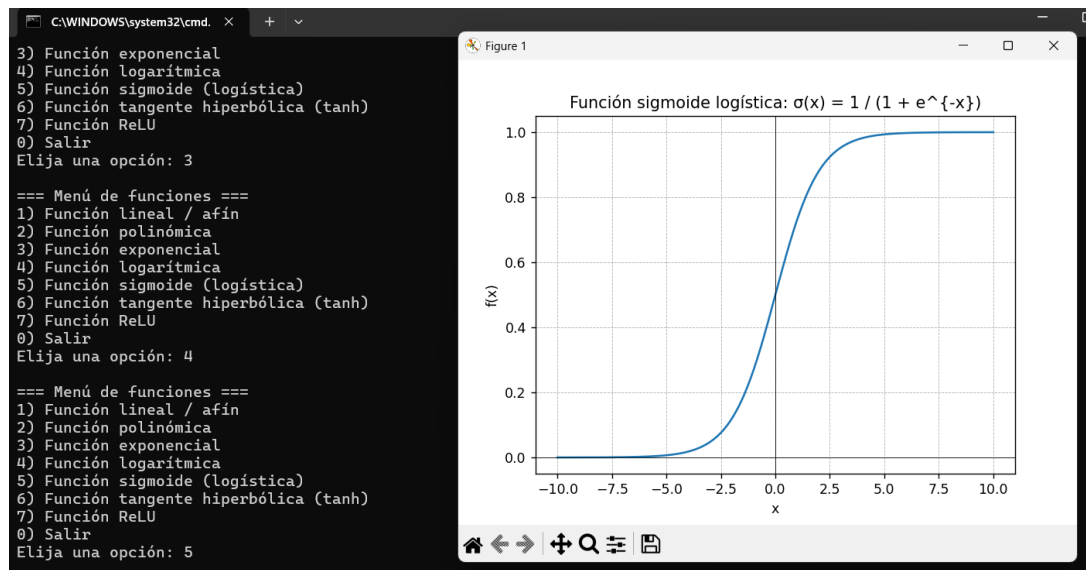
Se utiliza en modelos de clasificación binaria como la regresión logística.

Ilustración 15 – Función Logística – Elaboración propia con Napkin



Código

A continuación, puede ingresar al enlace del código: <https://tinyurl.com/yeyvth5n>



B. Función tangente hiperbólica

$$\tanh: \mathbb{R} \rightarrow (-1,1), \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Propiedades:

- › Simétrica respecto del origen: $\tanh(-x) = -\tanh(x)$.
- › Rango más “centrado” en torno a 0, lo que favorece ciertas dinámicas de aprendizaje.

¿Dónde usar la función tangente hiperbólica en IA?



Capas ocultas

Utilizar como función de activación en redes neuronales clásicas para mejorar el rendimiento.

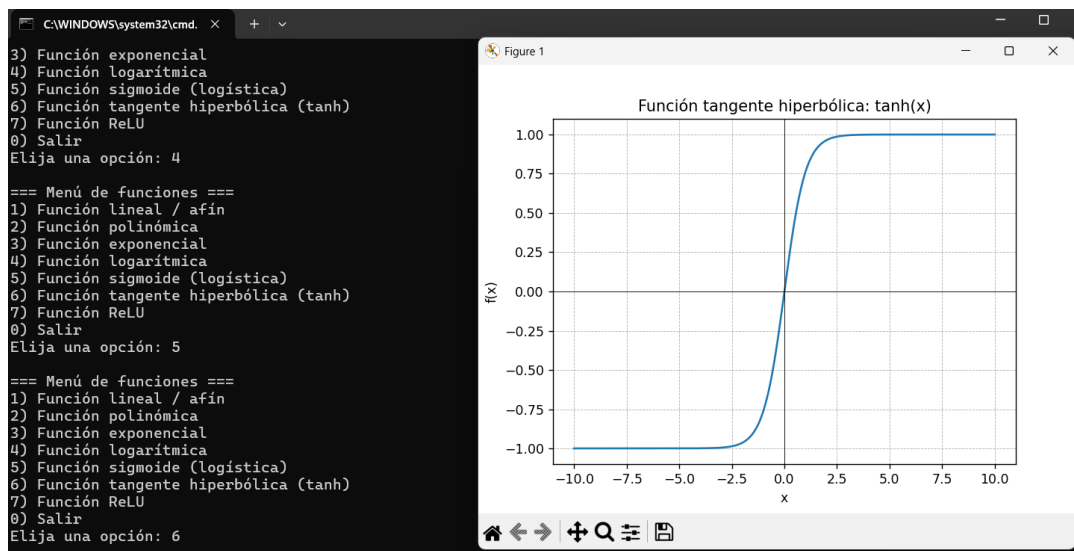
Arquitecturas recurrentes

Implementar como función de activación interna en celdas LSTM para manejar datos secuenciales.

Ilustración 16—Función Tangente Hiperbólica—Elaboración propia con Napkin



A continuación, puede ingresar al enlace del código: <https://tinyurl.com/bd8nshx3> Recuerde descargar y correrlo en forma local.



- *Funciones definidas a trozos: ReLU y variantes*

Una función definida a trozos se expresa mediante distintas fórmulas en diferentes subdominios.

La Rectified Linear Unit (ReLU) se define como:

$$\text{ReLU}: \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}, \text{ReLU}(x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases}$$

Propiedades:

- › Es continua en todo $\mathbb{R}$.
- › Es lineal para $x \geq 0$ y constante para $x < 0$.
- › No es derivable en $x = 0$, pero se trabaja con una derivada “convencional” a efectos del algoritmo (por ejemplo, eligiendo un valor subgradiente).

Variantes frecuentes:

- *Leaky ReLU*:

$$\text{LeakyReLU}(x) = \begin{cases} \alpha x, & x < 0, \\ x, & x \geq 0, \end{cases} \text{ con } \alpha \in (0,1) \text{ pequeño.}$$

Parametric ReLU (PReLU), donde α es un parámetro aprendible.

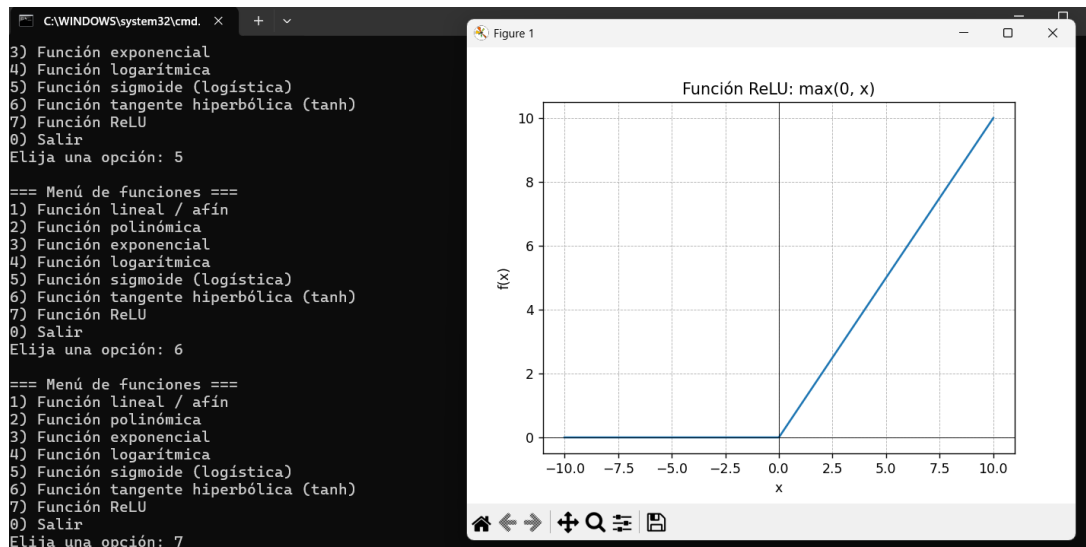


Ilustración 17– Funciones de Activación –Elaboración propia con Napkin



A continuación, puede ingresar al enlace del código: <https://tinyurl.com/yeyvth5n>

Código



- *Función softmax*

La función softmax transforma un vector de valores reales en un vector de probabilidades.

Sea $\mathbf{z} = (z_1, z_2, \dots, z_K) \in \mathbb{R}^K$. Entonces:

$$\text{softmax}: \mathbb{R}^K \rightarrow (0,1)^K, \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, i = 1, \dots, K.$$

Propiedades:

- › $\text{softmax}(\mathbf{z})_i > 0$ para todo i .
- › $\sum_{i=1}^K \text{softmax}(\mathbf{z})_i = 1$: define una distribución de probabilidad discreta.

¿Cómo usar la función softmax en modelos de clasificación multiclase?

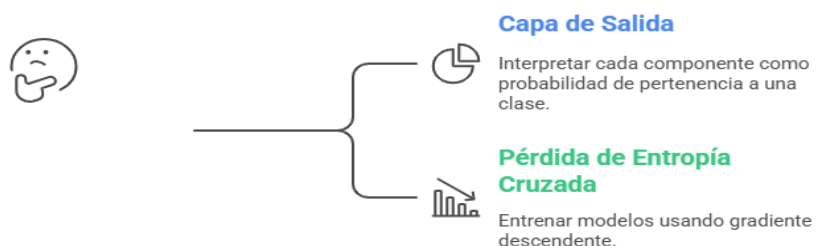


Ilustración 18 – Función Softmax – Elaboración propia con Napkin

- *Funciones de varias variables*

Muchos modelos de IA dependen de múltiples variables (parámetros y características). Formalmente, se trabaja con funciones:

$$f: \mathbb{R}^n \rightarrow \mathbb{R}$$

o

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^m$$

Veamos algunos ejemplos:

1. Función de costo (loss) en aprendizaje supervisado:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\mathbf{w}}(\mathbf{x}_i), y_i),$$

donde $\mathbf{w} \in \mathbb{R}^n$ son los parámetros del modelo, $\mathbf{x}_i$ son las entradas y y_i las salidas verdaderas.

2. Modelo lineal multivariable:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{x} \in \mathbb{R}^n.$$

¿Cómo se utilizan las funciones multivariables en IA?

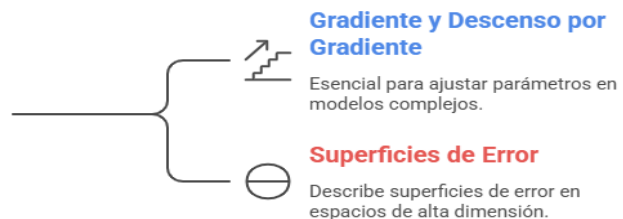


Ilustración 19 – Funciones Multivariables – Elaboración propia con Napkin

- *Crecimiento, decrecimiento y extremos de una función*

Otro de los temas que debemos recordar al momento de trabajar con funciones es la manera en que se comportan en todo su trayecto o parte de él y es allí donde aparece la necesidad de analizar el crecimiento y decrecimiento funcional.

- › *Crecimiento y decrecimiento (monotonía)*

Sea $f: D \subset \mathbb{R} \rightarrow \mathbb{R}$ una función real.

- › f es creciente en un intervalo $I \subset D$ si:

$$\forall x_1, x_2 \in I, x_1 < x_2 \Rightarrow f(x_1) \leq f(x_2).$$

- › f es estrictamente creciente en I si:

$$\forall x_1, x_2 \in I, x_1 < x_2 \Rightarrow f(x_1) < f(x_2).$$

- › f es decreciente en un intervalo $I \subset D$ si:

$$\forall x_1, x_2 \in I, x_1 < x_2 \Rightarrow f(x_1) \geq f(x_2).$$

- › f es estrictamente decreciente en I si:

$$\forall x_1, x_2 \in I, x_1 < x_2 \Rightarrow f(x_1) > f(x_2).$$

¿La función es creciente o decreciente?

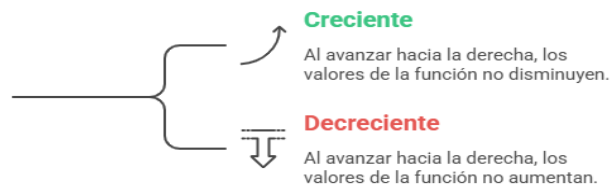


Ilustración 20—Crecimiento—Elaboración propia con Napkin

- *Extremos locales y globales*

Sea $f: D \rightarrow \mathbb{R}$.

- › Un punto $x_0 \in D$ es un **máximo local** si existe un intervalo alrededor de x_0 tal que, para todos los puntos x de ese intervalo,

$$f(x) \leq f(x_0).$$

- › Un punto $x_0 \in D$ es un **mínimo local** si existe un intervalo alrededor de x_0 tal que, para todos los puntos x de ese intervalo,

$$f(x) \geq f(x_0).$$



Es decir, en un extremo local, el valor de la función en x_0 es el “más alto” o el “más bajo” comparado con los valores cercanos.

- › Un **máximo global** es un punto $x_0 \in D$ tal que:

$$\forall x \in D, f(x) \leq f(x_0).$$

- › Un **mínimo global** es un punto $x_0 \in D$ tal que:

$$\forall x \in D, f(x) \geq f(x_0).$$



En problemas de optimización (como en IA), los **mínimos globales** suelen representar la mejor solución posible según la función que se está evaluando (por ejemplo, una función de error).

Veamos algunos ejemplos:

Ejemplo 1: Función parabólica simple

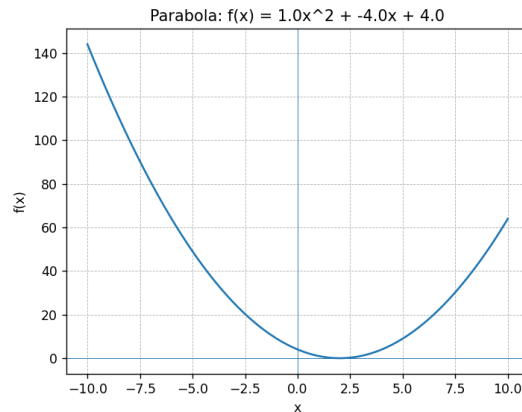


Ilustración 21 – Parábola – Elaboración propia

Considérese:

$$f(w) = (w-2)^2.$$

Podemos analizar su comportamiento comparando valores (observe la siguiente tabla):

Si tomamos valores <i>menores que 2</i> (por ejemplo, $w=1, 0, -1$), los valores de $f(w)$ son: › $f(1)=1$ › $f(0)=4$ › $f(-1)=9$ A medida que nos alejamos de 2 hacia la izquierda, los valores de la función aumentan. En la región a la izquierda de 2, si nos movemos de izquierda a derecha, los valores disminuyen: la función es decreciente.	Si tomamos valores <i>mayores que 2</i> (por ejemplo, $w=3, 4, 5$): › $f(3)=1$ › $f(4)=4$ › $f(5)=9$ Ahora, al avanzar hacia la derecha, los valores aumentan: la función es creciente a la derecha de 2.
---	---

› En $w = 2$, $f(2) = 0$ es el *valor más bajo* que la función alcanza en todo $\mathbb{R}$.

Por lo tanto, $w = 2$ es un *mínimo global*.



En un modelo IA, si w representa un parámetro de un modelo y $f(w)$ una medida de error, este análisis muestra que el “mejor” valor de w (el que produce menor error) es 2, y que a ambos lados el error crece.

Ejemplo 2: Función ReLU

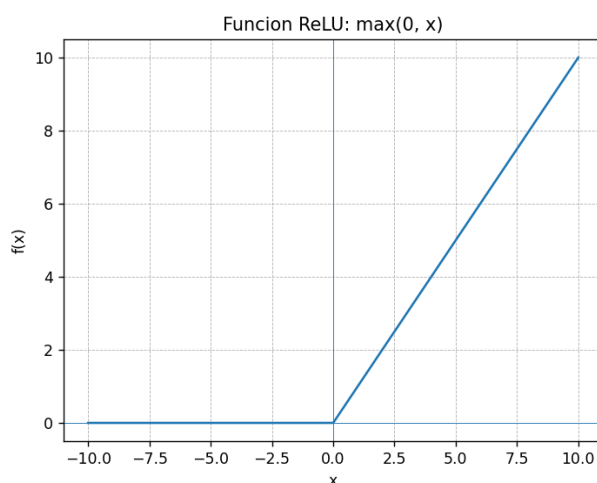


Ilustración 22 – ReLU – Elaboración propia

La función ReLU se define como:

$$\text{ReLU}(x) = \begin{cases} 0 & \text{si } x < 0, \\ x & \text{si } x \geq 0. \end{cases}$$

Podemos describir:

- › En $x < 0$, la función vale siempre 0: *es constante*.
- › En $x \geq 0$, la función coincide con la recta $f(x)=x$: si tomamos 0,1,2,3, los valores son 0,1,2,3: aquí la función es *estrictamente creciente*.



Énfasis

La función no tiene un mínimo global en todo $\mathbb{R}$ si consideramos que continúa creciendo hacia $+\infty$; pero en la zona $x \leq 0$, el valor 0 se mantiene constante y es el valor más bajo que toma.

En el caso de trabajar en redes neuronales, ReLU se utiliza porque mantiene salida *cero* para valores de entrada negativos y crece linealmente para valores positivos, permitiendo que la salida aumente al aumentar la entrada. Esto influye directamente en cómo la red “*activa*” o “*apaga*” neuronas según la región del espacio de entrada.

Ejemplo 3: Función sigmoide (comportamiento cualitativo)

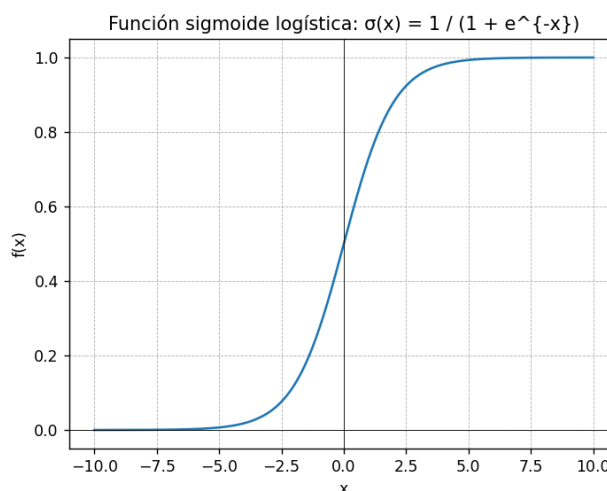


Ilustración 23—Sigmoide—Elaboración propia

La función sigmoide estándar:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Observando sus valores numéricos:

- › Para valores muy negativos ($x=-5, -10$), el resultado es muy cercano a 0.
- › Para valores intermedios ($x=0$), la función toma un valor aproximadamente 0,5.
- › Para valores grandes ($x=5, 10$), el resultado es muy cercano a 1.

Lo importante aquí:

- › A medida que x aumenta, los valores de $\sigma(x)$ siempre aumentan: la función es estrictamente creciente en todo su dominio.
- › Está acotada entre 0 y 1, por lo que no hay máximos o mínimos globales en el sentido de “punto aislado”, pero sí se reconocen valores mínimos y máximos en el límite (que formalmente se tratarán más adelante con límites).



En clasificación binaria, la sigmoide se interpreta como una función que transforma cualquier valor real en un “*grado de pertenencia*” entre 0 y 1. Es decir, los valores de entrada más grandes producen salidas más cercanas a 1 y los valores de entrada más pequeños producen salidas cercanas a 0.

El hecho de que la función sea creciente garantiza que el orden se conserva: si un ejemplo es “más positivo” según el modelo, su salida será mayor.

Ahora bien, para terminar este breve recorrido por los conceptos principales de funciones hablaremos de continuidad.

- *Continuidad de una función*

Sea $f: D \subset \mathbb{R} \rightarrow \mathbb{R}$. De manera intuitiva, decimos que una función es continua en un punto $a \in D$ cuando:

- › Al tomar valores de entrada suficientemente cercanos a a , los valores de la función resultan también tan cercanos como se desee al valor $f(a)$.

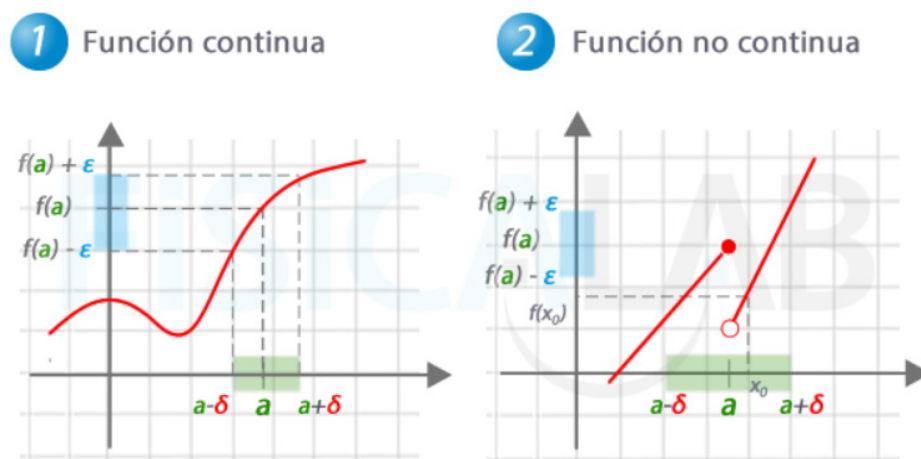


Ilustración 24 - Continuidad - No continuidad - Fuente: <https://tinyurl.com/5n8vunzx>



Actividad

Habiendo avanzado hasta aquí, usted ya está en condiciones de realizar la **Actividad 2**: Funciones, crecimiento y continuidad en modelos de IA: informe co-construido con asistentes de IA.

A partir de este punto comenzaremos a incorporar cálculos muy poderosos que tienen aplicaciones en prácticamente todos los campos de la ciencia y, por ende, no podían faltar en los modelos de IA. Estos son: Límites, Derivadas e Integrales.

Vamos a dar una mirada general de cada uno de estos temas, dado que nuestra materia tiene como objetivo estudiar las maneras en que estos cálculos aportan a los modelos; y no el abordaje de los cálculos en sí mismos.

- *Límites de funciones reales*

El concepto de límite formaliza la idea de “valor al que tiende” una función cuando la variable se aproxima a un punto dado. Es la base del cálculo diferencial e integral, y permite describir con precisión el comportamiento local de las funciones.

Trabajaremos en el marco de *funciones reales de variable real*:

$f: A \subset \mathbb{R} \rightarrow \mathbb{R}$ donde A es un subconjunto de $\mathbb{R}$.

- *Intuición del límite*

Intuitivamente, decimos que el límite de $f(x)$ cuando x se aproxima a a es L si:

- › cuando x toma valores suficientemente cercanos a a (sin necesidad de ser exactamente a),
- › los valores de $f(x)$ se acercan tanto como se quiera a L .

Se escribe:

$$\lim_{x \rightarrow a} f(x) = L$$

y se lee: “el límite de $f(x)$ cuando x tiende a a es L ”.

- *Definición formal ε - δ*

La definición clásica rigurosa del límite es la siguiente.

- *Definición: límite finito en un punto finito*

Sea $f: A \subset \mathbb{R} \rightarrow \mathbb{R}$, y sea a un punto de acumulación de A . Decimos que $\lim_{x \rightarrow a} f(x) = L$

sí, y sólo si, para todo $\varepsilon > 0$ existe $\delta > 0$ tal que:

$$0 < |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon,$$

para todo $x \in A$.

Comentarios importantes:

- › La condición $0 < |x - a|$ excluye el punto $x = a$: el límite describe el comportamiento alrededor de a , no necesariamente en a .
- › El parámetro ε controla “qué tan cerca” queremos que esté $f(x)$ de L .
- › El parámetro δ indica “qué tan cerca” debe estar x de a para lograr esa proximidad en la imagen.

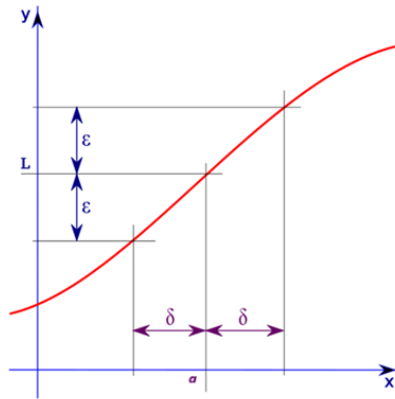


Ilustración 25 - Concepto de Límite - Fuente: <https://tinyurl.com/ybcrwsu4>

- Límites laterales

En muchos contextos interesa considerar el comportamiento de la función cuando la variable se aproxima a a sólo por la izquierda o sólo por la derecha.

Definición: Límite por la derecha.	Definición: Límite por la izquierda.
Decimos que el límite de $f(x)$ cuando x tiende a a por la derecha es L_2, y escribimos $\lim_{x \rightarrow a^+} f(x) = L_2,$ sí para todo $\varepsilon > 0$ existe $\delta > 0$ tal que: $0 < x - a < \delta \Rightarrow f(x) - L_2 < \varepsilon.$	Decimos que el límite de $f(x)$ cuando x tiende a a por la izquierda es L_1, y escribimos $\lim_{x \rightarrow a^-} f(x) = L_1,$ sí para todo $\varepsilon > 0$ existe $\delta > 0$ tal que: $0 < a - x < \delta \Rightarrow f(x) - L_1 < \varepsilon.$
Ilustración 26 – Límite por la derecha https://tinyurl.com/4jxkk3ar	Ilustración 27 – Límite por la izquierda https://tinyurl.com/4jxkk3ar

- **Proposición: existencia de límite y límites laterales**



El límite $\lim_{x \rightarrow a} f(x)$ existe y es igual a L si, y sólo si,

$$\lim_{x \rightarrow a^-} f(x) = L$$

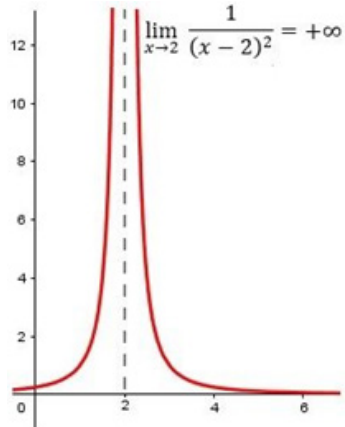
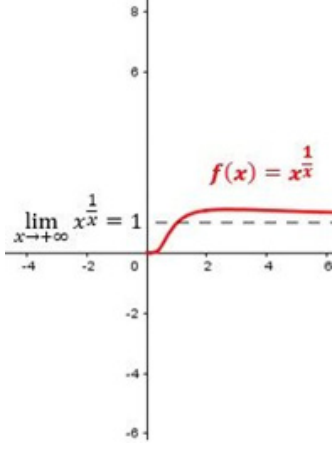
y

$$\lim_{x \rightarrow a^+} f(x) = L.$$

Es decir, para que exista el límite en un punto, los dos límites laterales deben existir y coincidir.

- **Límites infinitos y límites al infinito**

El concepto de límite se extiende a situaciones donde la función “crece sin cota” o donde la variable tiende a infinito.

Definición: Límite infinito en un punto	Definición: Límite al infinito
Decimos que $\lim_{x \rightarrow a} f(x) = +\infty$ si para todo número real M existe $\delta > 0$ tal que: $0 < x - a < \delta \Rightarrow f(x) > M.$ Análogamente, $\lim_{x \rightarrow a} f(x) = -\infty$ si para todo número real N existe $\delta > 0$ tal que: $0 < x - a < \delta \Rightarrow f(x) < N.$ En estos casos se dice que la función presenta una asíntota vertical en $x = a$.	Decimos que $\lim_{x \rightarrow +\infty} f(x) = L$ si para todo $\varepsilon > 0$ existe un número real K tal que: $x > K \Rightarrow f(x) - L < \varepsilon.$ De manera análoga, $\lim_{x \rightarrow -\infty} f(x) = L$ si para todo $\varepsilon > 0$ existe un número real K tal que: $x < K \Rightarrow f(x) - L < \varepsilon.$ Cuando $\lim_{x \rightarrow \pm\infty} f(x) = L$, se dice que la recta $y = L$ es una asíntota horizontal de la función.
Veamos el siguiente gráfico:  Ilustración 28 - Se puede comprobar si damos valores a la x cada vez más cercanos a 2, tanto acercándonos por su izquierda como por su derecha, como se ve en el siguiente cuadro, el límite tiende a $+\infty$ - Fuente: https://tinyurl.com/4jxkk3ar	Veamos el siguiente gráfico:  Ilustración 29 - Se puede comprobar si damos valores a la x cada vez más cercanos a $+\infty$. Como se ve en el siguiente cuadro, el límite tiende a 1— Fuente: https://tinyurl.com/3x8uwmff

- *Teoremas básicos sobre límites*

A continuación, presentamos resultados clásicos que permiten **calcular límites** a partir de límites conocidos.

Sea $f, g: A \subset \mathbb{R} \rightarrow \mathbb{R}$ y sea a un punto de acumulación de A . Suponemos que existen los límites finitos:

$$\lim_{x \rightarrow a} f(x) = L, \lim_{x \rightarrow a} g(x) = M.$$

Entonces, se verifican las siguientes propiedades:

1. **Unicidad del límite:** Si el límite de $f(x)$ cuando $x \rightarrow a$ existe, entonces es único. No puede haber dos valores distintos $L_1 \neq L_2$ tales que ambos satisfagan la definición de límite.
2. **Linealidad:** Para cualquier $\alpha, \beta \in \mathbb{R}$, $\lim_{x \rightarrow a} [\alpha f(x) + \beta g(x)] = \alpha L + \beta M$.
3. **Producto:** $\lim_{x \rightarrow a} [f(x) \cdot g(x)] = L \cdot M$.
4. **Cociente:** Si además $M \neq 0$, entonces $\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = \frac{L}{M}$, siempre que $g(x) \neq 0$ en un entorno de a .
5. **Límite de la composición (bajo condiciones adecuadas)** Si $\lim_{x \rightarrow a} f(x) = L$ y h es continua en L , entonces: $\lim_{x \rightarrow a} h(f(x)) = h(L)$.
6. **Carácter local del límite** El valor del límite $\lim_{x \rightarrow a} f(x)$ depende únicamente del comportamiento de f en un entorno de a . Modificaciones de f en puntos alejados de dicho entorno no alteran el límite.

- *Relación entre límite y continuidad*

Definición

Sea $f: A \subset \mathbb{R} \rightarrow \mathbb{R}$ y sea $a \in A$. Decimos que f es continua en a si: $\lim_{x \rightarrow a} f(x) = f(a)$.

Es decir, f es continua en a si el valor del límite de la función cuando x se aproxima a a coincide con el valor de la función en el propio punto.

De aquí se deduce que:

- › el concepto de límite es más general (puede existir el límite, aunque la función no esté definida en a),
- › la continuidad es una propiedad que combina límite y valor de la función en el punto.

Llegó el momento de ver ejemplos, donde se aplican los temas tratados:

- **Ejemplo 1: Límite de un polinomio (cálculo directo)**

Calcular: $\lim_{x \rightarrow 2} (3x^2 - 5x + 1)$.

Los polinomios son funciones continuas en $\mathbb{R}$, por lo tanto:

$$\lim_{x \rightarrow 2} (3x^2 - 5x + 1) = 3 \cdot 2^2 - 5 \cdot 2 + 1 = 3 \cdot 4 - 10 + 1 = 12 - 10 + 1 = 3.$$

Conclusión: $\boxed{\lim_{x \rightarrow 2} (3x^2 - 5x + 1) = 3.}$



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 1 - límites.m4a

También disponible desde:

<https://tinyurl.com/ypbu2uy9>



- **Ejemplo 2: Límite con indeterminación 0/0 y factorización**

Calcular: $\lim_{x \rightarrow 1} \frac{x^2 - 1}{x - 1}$.

Si reemplazamos directamente $x=1$:

$$\frac{1^2 - 1}{1 - 1} = \frac{0}{0},$$

forma indeterminada. No podemos concluir así.

1. *Factorizamos el numerador:*

$$x^2 - 1 = (x - 1)(x + 1).$$

2. *Para $x \neq 1$,*

$$\frac{x^2 - 1}{x - 1} = \frac{(x - 1)(x + 1)}{x - 1} = x + 1.$$

En un entorno de $x=1$, salvo el punto $x=1$, la función coincide con $x+1$.

3. *Usamos el carácter local del límite:*

$$\lim_{x \rightarrow 1} \frac{x^2 - 1}{x - 1} = \lim_{x \rightarrow 1} (x + 1) = 1 + 1 = 2.$$

Conclusión:

$$\boxed{\lim_{x \rightarrow 1} \frac{x^2 - 1}{x - 1} = 2.}$$



En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 2 - límites.m4a

También disponible desde:

<https://tinyurl.com/57ptv58d>



• Ejemplo 3: Límite con racionalización

Calcular: $\lim_{x \rightarrow 0} \frac{\sqrt{x+1} - 1}{x}$.

Reemplazar $x = 0$ da $\frac{0}{0}$ (indeterminación).

Racionalizamos:

1. Multiplicamos numerador y denominador por el conjugado:

$$\frac{\sqrt{x+1} - 1}{x} \cdot \frac{\sqrt{x+1} + 1}{\sqrt{x+1} + 1} = \frac{(x+1) - 1}{x[\sqrt{x+1} + 1]} = \frac{x}{x[\sqrt{x+1} + 1]}.$$

2. Para $x \neq 0$, simplificamos:

$$\frac{x}{x[\sqrt{x+1} + 1]} = \frac{1}{\sqrt{x+1} + 1}.$$

3. Calculamos el límite:

$$\lim_{x \rightarrow 0} \frac{\sqrt{x+1} - 1}{x} = \lim_{x \rightarrow 0} \frac{1}{\sqrt{x+1} + 1} = \frac{1}{\sqrt{0+1} + 1} = \frac{1}{1+1} = \frac{1}{2}.$$

Conclusión:

$$\boxed{\lim_{x \rightarrow 0} \frac{\sqrt{x+1} - 1}{x} = \frac{1}{2}}.$$



En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 3 - límites.m4a

También disponible desde:

<https://tinyurl.com/wdhmvntz>



• Ejemplo 4: Límite al infinito de una función racional

Calcular: $\lim_{x \rightarrow +\infty} \frac{2x^2 - 3x + 1}{x^2 + x}$.

► **Estrategia clásica:** dividir numerador y denominador por la mayor potencia de x presente (aquí x^2):

$$\frac{2x^2 - 3x + 1}{x^2 + x} = \frac{x^2(2 - \frac{3}{x} + \frac{1}{x^2})}{x^2(1 + \frac{1}{x})} = \frac{2 - \frac{3}{x} + \frac{1}{x^2}}{1 + \frac{1}{x}}.$$

Ahora tomamos el límite cuando $x \rightarrow +\infty$:

$$\frac{3}{x} \rightarrow 0,$$

$$\frac{1}{x^2} \rightarrow 0,$$

$$\frac{1}{x} \rightarrow 0.$$

Por continuidad de las operaciones elementales:

$$\lim_{x \rightarrow +\infty} \frac{2x^2 - 3x + 1}{x^2 + x} = \frac{2 - 0 + 0}{1 + 0} = \frac{2}{1} = 2.$$

Conclusión:

$$\lim_{x \rightarrow +\infty} \frac{2x^2 - 3x + 1}{x^2 + x} = 2.$$



En este podcast encontrará una explicación detallada de este ejemplo:
Ejemplo 4 - límites.m4a

También disponible desde:
<https://tinyurl.com/5b6y799u>



- **Ejemplo 5: Límites laterales en una función definida a trozos**

$$\text{Sea } f(x) = \begin{cases} x^2 & \text{si } x < 1, \\ 2x - 1 & \text{si } x \geq 1. \end{cases}$$

Calcular $\lim_{x \rightarrow 1} f(x)$.

1. Límite por la izquierda ($x \rightarrow 1^-$): usamos la rama x^2 :

$$\lim_{x \rightarrow 1^-} f(x) = \lim_{x \rightarrow 1^-} x^2 = 1^2 = 1.$$

2. Límite por la derecha ($x \rightarrow 1^+$): usamos la rama $2x - 1$:

$$\lim_{x \rightarrow 1^+} f(x) = \lim_{x \rightarrow 1^+} (2x - 1) = 2 \cdot 1 - 1 = 1.$$

3. Como los límites laterales existen y coinciden,

$$\lim_{x \rightarrow 1} f(x) = 1.$$

Además, $f(1) = 2 \cdot 1 - 1 = 1$, por lo que la función es continua en $x=1$, pero eso ya es parte de 0.6.

Conclusión:

$$\lim_{x \rightarrow 1} f(x) = 1.$$



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 5 - límites.m4a

También disponible desde:

<https://tinyurl.com/y9bv2mkj>



- **Ejemplo 6: Límite infinito: asíntota vertical**

Calcular: $\lim_{x \rightarrow 2} \frac{1}{x-2}$.

Observamos el comportamiento al aproximar x a 2:

Si $x \rightarrow 2^+$, entonces $x - 2 > 0$ y muy pequeño $\Rightarrow \frac{1}{x-2} \rightarrow +\infty$.

Si $x \rightarrow 2^-$, entonces $x - 2 < 0$ y muy pequeño en valor absoluto $\Rightarrow \frac{1}{x-2} \rightarrow -\infty$.

Formalmente

$$\lim_{x \rightarrow 2^+} \frac{1}{x-2} = +\infty, \lim_{x \rightarrow 2^-} \frac{1}{x-2} = -\infty.$$

Como los límites laterales no coinciden (ni son finitos), el límite bilateral no existe en $\mathbb{R}$. Sin embargo, en el sentido de límites infinitos decimos que la función tiene una asíntota vertical en $x=2$.



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 6 - límites.m4a

También disponible desde:

<https://tinyurl.com/ymsrst4f>



Actividad

Ya puede realizar la **Actividad 3: Cálculo de límites**. Lo/a invitamos a resolverla antes de continuar con el estudio del módulo.



Recursos

Le recomendamos también navegar por el contenido del siguiente enlace, allí se presentan múltiples ejemplos y casos para revisar: <https://tinyurl.com/4jxkk3ar>



Multimedia

Finalmente, para consolidar sus aprendizajes hasta aquí, se recomienda ver el siguiente video donde se presentan las explicaciones de los 6 ejemplos presentados: *Límites 6 Ejemplos Clave*

También disponible desde:

<https://tinyurl.com/5zhxfwxj>



Y ahora, la pregunta es:

¿Qué sucede cuando calculamos el límite del cociente incremental cuando dicho incremento tiende a cero?

Si sabía la respuesta ¡muy bien!, sino lo/a animamos a ver el tema que sigue a continuación.

- *Derivadas de funciones reales*

› *Idea intuitiva de derivada*

La derivada formaliza la idea de tasa de cambio instantánea de una función en un punto y, geoméricamente, la pendiente de la recta tangente a la gráfica en ese punto.

Dada una función real de variable real:

$$f: A \subset \mathbb{R} \rightarrow \mathbb{R},$$

y un punto $a \in A$, consideramos el cociente incremental:

$$\frac{f(a+h) - f(a)}{h},$$

que es la pendiente de la recta secante que pasa por los puntos $(a, f(a))$ y $(a+h, f(a+h))$



Énfasis

La derivada en a se define como el límite de este cociente cuando h se aproxima a cero, si dicho límite existe.

- *Definición formal de derivada en un punto*

Definición:

Sea $f: A \subset \mathbb{R} \rightarrow \mathbb{R}$ y sea a un punto de acumulación de A , con $a \in A$.

Decimos que f es derivable en a si existe el límite finito:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

A este número real se lo llama derivada de f en a .

Notas:

- › La derivada, si existe, es un número real único.
- › La expresión $\frac{f(a+h)-f(a)}{h}$
- › se interpreta como la tasa de variación promedio de f en el intervalo $[a, a+h]$

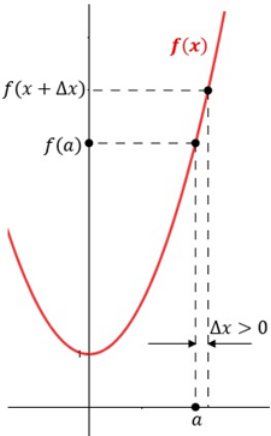
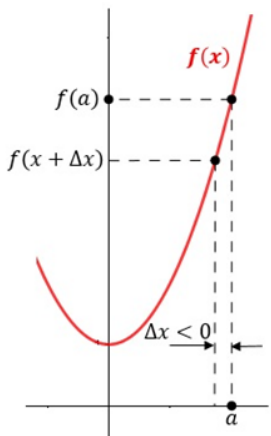


Énfasis

El límite describe la tasa de variación instantánea, cuando el intervalo se hace arbitrariamente pequeño.

- *Derivadas laterales*

Al igual que en los límites, es útil considerar derivadas por izquierda y por derecha.

Derivada por la derecha	Derivada por la izquierda
Sea $f: A \subset \mathbb{R} \rightarrow \mathbb{R}$ y sea $a \in A$. La derivada por la derecha de f en a es: $f'_+(a) = \lim_{h \rightarrow 0^+} \frac{f(a+h) - f(a)}{h},$ si este límite existe (y es finito).	Sea $f: A \subset \mathbb{R} \rightarrow \mathbb{R}$ y sea $a \in A$. La derivada por la izquierda de f en a es: $f'_-(a) = \lim_{h \rightarrow 0^-} \frac{f(a+h) - f(a)}{h},$ si este límite existe (y es finito).
 Ilustración 30 – Derivada por la Derecha Fuente: https://tinyurl.com/385ath7u	 Ilustración 31 – Derivada por la Izquierda Fuente: https://tinyurl.com/mrp5ewku



Proposición La función f es derivable en a si, y sólo si, existen las derivadas laterales en a y se cumple:

$$f'_+(a) = f'_-(a).$$

En ese caso:

$$f'(a) = f'_+(a) = f'_-(a).$$

Esto es especialmente relevante en funciones definidas a trozos, donde puede haber cambios de regla en ciertos puntos.

- Interpretación geométrica

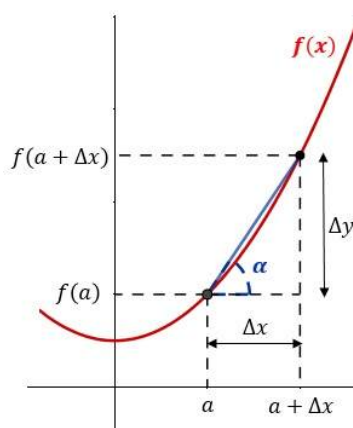


Ilustración 32 – Interpretación Geométrica de la derivada – Fuente: <https://tinyurl.com/yux3vv9n>

Sea f derivable en a . La recta tangente a la curva $y=f(x)$ en el punto $(a, f(a))$ es la recta que mejor aproxima localmente a la función en ese punto.

Su ecuación es:

$$y=f(a)+f'(a)(x-a).$$

De este modo:

- › $f'(a)$ es la pendiente de la recta tangente.
- › Si $f'(a)>0$, la función crece localmente en torno a a .
- › Si $f'(a)<0$, la función decrece localmente en torno a a .

Si $f'(a)=0$, el punto puede ser un extremo local, un punto de inflexión horizontal u otra situación, que se analiza con herramientas adicionales.

- Derivabilidad y continuidad

- › Teorema

Si f es derivable en a , entonces f es continua en a .

- › Demostración:

Si f es derivable en a , el cociente incremental tiende a un número finito:

$$\lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h} = f'(a).$$

De aquí se puede deducir que $f(a+h) \rightarrow f(a)$ cuando $h \rightarrow 0$, lo que equivale a decir que:

$$\lim_{x \rightarrow a} f(x) = f(a).$$

Es decir, f es continua en a .

- **Observación importante:** La recíproca no vale en general: que f sea continua en a no garantiza que f sea derivable en a .
- **Ejemplos típicos:** funciones con “picos” o ángulos, como $f(x) = |x|$ en $x = 0$, que es continua pero no derivable en ese punto.

Veamos el caso:

Sea $f(x) = |x|$. Calculamos en $x = 0$:

Derivada por la derecha	Derivada por la izquierda
$f'_+(0) = \lim_{h \rightarrow 0^+} \frac{f(0+h) - f(0)}{h} = \lim_{h \rightarrow 0^+} \frac{ h - 0}{h}$ $= \lim_{h \rightarrow 0^+} \frac{h}{h} = 1.$	$f'_-(0) = \lim_{h \rightarrow 0^-} \frac{f(0+h) - f(0)}{h} = \lim_{h \rightarrow 0^-} \frac{ h - 0}{h}$ $= \lim_{h \rightarrow 0^-} \frac{-h}{h} = -1.$
Por lo tanto: $f'_+(0)=1, f'_-(0)=-1$, al no coincidir, f no es derivable en $x=0$	

- **Reglas de derivación**

Sea $f, g: A \subset \mathbb{R} \rightarrow \mathbb{R}$ derivables en a , y sean $\alpha, \beta \in \mathbb{R}$.

Entonces:

1. **Derivada lineal (combinación lineal):**

$$(\alpha f + \beta g)'(a) = \alpha f'(a) + \beta g'(a).$$

2. **Regla del producto:**

$$(f \cdot g)'(a) = f'(a)g(a) + f(a)g'(a).$$

3. **Regla del cociente:** si $g(a) \neq 0$,

$$\left(\frac{f}{g}\right)'(a) = \frac{f'(a)g(a) - f(a)g'(a)}{[g(a)]^2}.$$

4. **Regla de la cadena:** Sean $f: A \rightarrow \mathbb{R}$ derivable en a y $g: B \rightarrow \mathbb{R}$ derivable en $f(a)$, con $f(A) \subset B$.

Entonces la composición $g \circ f$ es derivable en a y:

$$(g \circ f)'(a) = g'(f(a)) \cdot f'(a).$$



Estas reglas se demuestran a partir de la definición de derivada como límite, utilizando propiedades de los límites vistas anteriormente.

- *Derivada de funciones elementales*

En el contexto del módulo, será conveniente disponer de la tabla con las derivadas más usadas.

TABLA DE DERIVADAS			
FUNCIÓN SIMPLE		FUNCIÓN COMPUESTA	
$f(x)$	$f'(x)$	$z(x)$	$z'(x)$
a	0		
x	1		
$ x $	$\frac{x}{ x }$	$ f(x) $	$\frac{f(x)}{ f(x) } \cdot f'(x)$
x^n	nx^{n-1}	$[f(x)]^n$	$n[f(x)]^{n-1}f'(x)$
$\frac{1}{x}$	$-\frac{1}{x^2}$	$\frac{1}{f(x)}$	$-\frac{f'(x)}{f(x)^2}$
$\sqrt{x}$	$\frac{1}{2\sqrt{x}}$	$\sqrt{f(x)}$	$\frac{f'(x)}{2\sqrt{f(x)}}$
$\sqrt[n]{x}$	$\frac{1}{n\sqrt[n]{x^{n-1}}}$	$\sqrt[n]{f(x)}$	$\frac{f'(x)}{n\sqrt[n]{f(x)^{n-1}}}$
e^x	e^x	$e^{f(x)}$	$e^{f(x)}f'(x)$
a^x	$a^x \ln a$	$a^{f(x)}$	$a^{f(x)} \ln a \cdot f'(x)$
$\ln a$	$\frac{1}{x}$	$\ln f(x)$	$\frac{f'(x)}{f(x)}$
$\log_a x$	$\frac{1}{x \ln a} = \frac{\log_a e}{x}$	$\log_a f(x)$	$\frac{f'(x)}{f(x) \ln a} = \frac{f'(x) \log_a e}{f(x)}$
$\text{sen}(x)$	$\cos(x)$	$\text{sen}[f(x)]$	$f'(x) \cos[f(x)]$
$\cos(x)$	$-\text{sen}(x)$	$\cos[f(x)]$	$-f'(x) \text{sen}[f(x)]$
$\tan(x)$	$1 + \tan^2(x) = \frac{1}{\cos^2(x)}$	$\tan[f(x)]$	$f'(x) \{1 + \tan^2[f(x)]\} = \frac{f'(x)}{\cos^2[f(x)]}$
$\text{cosec}(x)$	$\frac{-\cos(x)}{\text{sen}^2(x)} = -\cot(x) \text{cosec}(x)$	$\text{cosec}[f(x)]$	$-\frac{f'(x) \cos[f(x)]}{\text{sen}^2[f(x)]}$
$\sec(x)$	$\frac{\text{sen}(x)}{\cos^2(x)} = \sec(x) \cdot \tan(x)$	$\sec[f(x)]$	$\frac{f'(x) \cdot \text{sen}[f(x)]}{\cos^2[f(x)]} = f'(x) \sec[f(x)] \tan[f(x)]$
$\cot(x)$	$-\frac{1}{\text{sen}^2(x)} = -[1 + \cot^2(x)]$	$\cot[f(x)]$	$-\frac{f(x)}{\text{sen}^2[f(x)]} = -f'(x) \{1 + \cot^2[f(x)]\}$
$\text{arc sen}(x)$	$\frac{1}{\sqrt{1-x^2}}$	$\text{arc sen}[f(x)]$	$\frac{f'(x)}{\sqrt{1-[f(x)]^2}}$
$\text{arc cos}(x)$	$-\frac{1}{\sqrt{1-x^2}}$	$\text{arc cos}[f(x)]$	$-\frac{f'(x)}{\sqrt{1-[f(x)]^2}}$
$\text{arctan}(x)$	$\frac{1}{1+x^2}$	$\text{arctan}[f(x)]$	$\frac{f'(x)}{1+[f(x)]^2}$
$f(x)$	$f'(x)$	$z(x)$	$z'(x)$
FUNCIÓN SIMPLE		FUNCIÓN COMPUESTA	
TABLA DE DERIVADAS			

Ilustración 33 - Tabla de derivadas - Fuente: <https://tinyurl.com/yynsxcma>

Y ahora el momento de los ejemplos, donde se pueden ver plasmados los aspectos más relevantes de derivadas:

- **Ejemplo 1: Derivada de un polinomio (vía definición en un punto)**

Calcular la derivada de $f(x)=x^2$ en un punto a usando la definición:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

Tenemos:

$$\frac{f(a+h) - f(a)}{h} = \frac{(a+h)^2 - a^2}{h} = \frac{a^2 + 2ah + h^2 - a^2}{h} = \frac{2ah + h^2}{h} = 2a + h.$$

Tomando el límite cuando $h \rightarrow 0$:

$$f'(a) = \lim_{h \rightarrow 0} (2a + h) = 2a.$$

Por lo tanto, para todo $x \in \mathbb{R}$, $f'(x) = 2x$.



Multimedia

En este podcast² encontrará una explicación detallada de este ejemplo: *Ejemplo 1 - der.m4a*

También disponible desde:
<https://tinyurl.com/b7drdtyc>



- **Ejemplo 2: Derivada de un polinomio general (regla de la potencia)**

$$\text{Sea } f(x) = 3x^3 - 5x + 2.$$

Como suma de potencias, podemos derivar término a término:

$$\frac{d}{dx}(3x^3) = 3 \cdot 3x^2 = 9x^2.$$

$$\frac{d}{dx}(-5x) = -5.$$

$$\frac{d}{dx}(2) = 0.$$

Entonces: $f'(x) = 9x^2 - 5$.



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo: *Ejemplo 2 - der.m4a*

También disponible desde:
<https://tinyurl.com/yff3mmy2>



²Todos los PODCAST que se encuentran en el material fueron generados en NoteBookLM usando como insumo las resoluciones de los ejemplos. Dichas resoluciones son de elaboración propia.

- **Ejemplo 3: Derivada de una función racional**

Sea $f(x) = \frac{x^2+1}{x-1}$, definida para $x \neq 1$.

Usamos la regla del cociente: si $f(x) = \frac{u(x)}{v(x)}$, entonces $f'(x) = \frac{u'(x)v(x) - u(x)v'(x)}{[v(x)]^2}$.

Aquí: $u(x) = x^2 + 1, u'(x) = 2x, v(x) = x - 1, v'(x) = 1$.

Por lo tanto: $f'(x) = \frac{(2x)(x-1) - (x^2+1)(1)}{(x-1)^2} = \frac{2x(x-1) - x^2 - 1}{(x-1)^2}$.

Desarrollamos el numerador: $2x(x-1) - x^2 - 1 = 2x^2 - 2x - x^2 - 1 = x^2 - 2x - 1$.

Así: $f'(x) = \frac{x^2 - 2x - 1}{(x-1)^2}, x \neq 1$.



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:
Ejemplo 3 - der.m4a

También disponible desde:
<https://tinyurl.com/82fv298n>



- **Ejemplo 4: Derivada de una composición (regla de la cadena)**

Sea $f(x) = e^{2x+1}$.

Podemos verla como composición: $f(x) = e^{g(x)}, g(x) = 2x + 1$.

Sabemos que: $\frac{d}{dx} [e^{g(x)}] = e^{g(x)} \cdot g'(x)$.

Como: $g'(x) = 2$, entonces: $f'(x) = e^{2x+1} \cdot 2 = 2e^{2x+1}$.



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:
Ejemplo 4 - der.m4a

También disponible desde:
<https://tinyurl.com/3s3cdbny>



- **Ejemplo 5: Derivada de ln de una función**

Sea $f(x) = \ln(x^2 + 1), x \in \mathbb{R}$.

Tomamos de nuevo: $g(x) = x^2 + 1$.

Sabemos que: $\frac{d}{dx} [\ln g(x)] = \frac{g'(x)}{g(x)}$.

Aquí: $g'(x) = 2x$.

Entonces: $f'(x) = \frac{2x}{x^2 + 1}$.

- **Ejemplo 6: Función definida a trozos y derivadas laterales**

Sea $f(x) = \begin{cases} x^2, & x < 0, \\ x, & x \geq 0. \end{cases}$

Queremos estudiar la derivada en $x=0$.

› Para: $x < 0$, $f(x) = x^2$, luego $f'(x) = 2x$.

Derivada por la izquierda en 0:

$$f'_-(0) = \lim_{h \rightarrow 0^-} \frac{f(0+h) - f(0)}{h} = \lim_{h \rightarrow 0^-} \frac{h^2 - 0}{h} = \lim_{h \rightarrow 0^-} h = 0.$$

› Para: $x > 0$, $f(x) = x$, luego $f'(x) = 1$.

Derivada por la derecha en 0:

$$f'_+(0) = \lim_{h \rightarrow 0^+} \frac{f(0+h) - f(0)}{h} = \lim_{h \rightarrow 0^+} \frac{h - 0}{h} = \lim_{h \rightarrow 0^+} 1 = 1.$$

Como $f'_-(0) \neq f'_+(0)$, las derivadas laterales no coinciden, por lo tanto f no es derivable en $x=0$, aunque sí es continua en ese punto (se puede verificar aparte).



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 6 - der.m4a

También disponible desde:

<https://tinyurl.com/3rz4hhea>



Recursos

Para complementar, le recomendamos explorar el contenido del enlace donde se presentan múltiples ejemplos y casos para revisar:

<https://tinyurl.com/mcj36wxu>



Multimedia

En el siguiente video podrá ver las explicaciones de los ejemplos anteriores: *Cálculo de Derivadas*³

También disponible desde:

<https://tinyurl.com/47w84c83>



Actividad

Ahora está Ud. en condiciones de hacer la **Actividad 4: Solo cálculo de derivadas**. ¡Adelante!

Vamos con otro interrogante:

› ¿Cómo calculamos el área bajo una curva?

Si la respuesta fue usando integrales, ¡Felicitaciones! A continuación, vamos a trabajar con el concepto de integrales y sus formas de cálculo.

³ Los videos de los ejemplos fueron generados con NoteBookLM, el insumo fue el conjunto de ejemplos. La resolución es de elaboración propia.

¿Qué tipo de integral se debe usar?

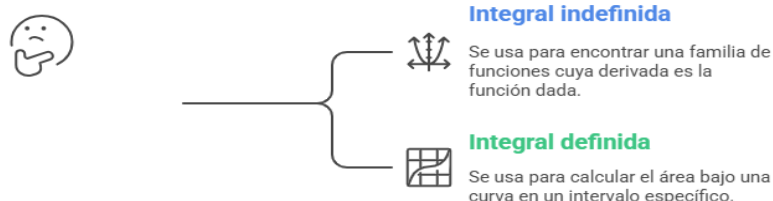


Ilustración 34 – Integrales – Elaboración propia con Napkin

- **Integrales de funciones reales**

En el contexto del cálculo clásico trabajaremos con funciones reales de variable real:

$$f: [a, b] \subset \mathbb{R} \rightarrow \mathbb{R}.$$

Hay dos nociones estrechamente relacionadas:

- › **Integral indefinida:** familia de primitivas.
- › **Integral definida:** integral de Riemann en un intervalo $[a, b]$.

El vínculo entre ambas se establece mediante el **Teorema Fundamental del Cálculo**.

- **Integral indefinida – Primitivas**

¿Qué es una primitiva?	¿Qué es una integral indefinida?
Definición Sea $I \subset \mathbb{R}$ un intervalo y $f: I \rightarrow \mathbb{R}$ una función dada. Se dice que $F: I \rightarrow \mathbb{R}$ es una primitiva (o antiderivada) de f en I si F es derivable en I y $F'(x) = f(x), \text{ para todo } x \in I.$ El conjunto de todas las primitivas de f en I se denota por: $\int f(x) dx.$ Si F es una primitiva de f, entonces todas las primitivas de f tienen la forma: $F(x) + C, C \in \mathbb{R},$ porque la diferencia de dos primitivas es una función constante.	Definición La integral indefinida de f se define como: $\int f(x) dx = F(x) + C,$ donde F es una primitiva de f.



Énfasis

La integral indefinida es, por tanto, una *operación inversa* a la derivación.

- Reglas básicas de integración indefinida

Si las primitivas existen, se verifican:

Linealidad	Potencias ($n \neq -1$)
$\int [\alpha f(x) + \beta g(x)] dx = \alpha \int f(x) dx + \beta \int g(x) dx, \alpha, \beta \in \mathbb{R}.$	$\int x^n dx = \frac{x^{n+1}}{n+1} + C.$
Función $1/x$	Exponencial base e
$\int \frac{1}{x} dx = \ln x + C.$	$\int e^x dx = e^x + C.$
1. Trigonómicas básicas (si se incluyen): $\cos \int x dx = \sin x + C,$ $\int \sin x dx = -\cos x + C.$	



Énfasis

Estas fórmulas se justifican comprobando que, al derivar el resultado, se recupera la función integrando.

- Integral definida (Riemann)

La integral definida de Riemann formaliza la idea de *área orientada bajo la curva* de una función.

Sea $f: [a, b] \rightarrow \mathbb{R}$ acotada.

- › Particiones y sumas de Riemann

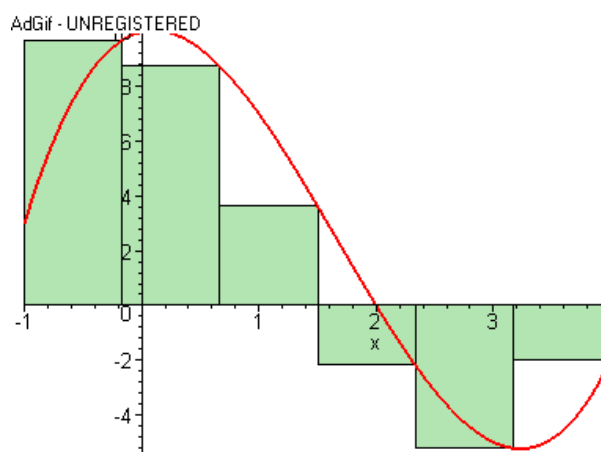


Ilustración 35 – Particiones de Riemann – Fuente: <https://tinyurl.com/3bawam3a>

› Una *partición* de $[a, b]$ es un conjunto finito de puntos:

$P = \{x_0, x_1, \dots, x_n\}$ tal que

$$a = x_0 < x_1 < \dots < x_n = b.$$

Para cada subintervalo $[x_{i-1}, x_i]$ se escoge un punto $t_i \in [x_{i-1}, x_i]$.

› La *suma de Riemann* de f asociada a P y a los puntos t_i es:

$$S(f, P, (t_i)) = \sum_{i=1}^n f(t_i) \Delta x_i, \text{ donde } \Delta x_i = x_i - x_{i-1}.$$

› La *norma* de la partición es:

$$\|P\| = \max_{1 \leq i \leq n} \Delta x_i.$$

- *Definición de la integral de Riemann*

Se dice que f es *integrable en el sentido de Riemann* en $[a, b]$ si existe un número real I tal que para todo $\varepsilon > 0$ existe $\delta > 0$ con la siguiente propiedad:

para toda partición P de $[a, b]$ y toda elección de puntos $t_i \in [x_{i-1}, x_i]$, si $\|P\| < \delta$, entonces:

$$|S(f, P, (t_i)) - I| < \varepsilon.$$

En ese caso, se escribe:

$$\int_a^b f(x) dx = I.$$

Intuitivamente, esto significa que todas las sumas de Riemann “finas” se aproximan al mismo número I , independientemente de los puntos t_i elegidos dentro de cada subintervalo.

- *Propiedades básicas de la integral definida*

Si f, g son Riemann-integrables en $[a, b]$ y $\alpha, \beta \in \mathbb{R}$, se cumplen:

<i>Linealidad</i>	$\int_a^b [\alpha f(x) + \beta g(x)] dx = \alpha \int_a^b f(x) dx + \beta \int_a^b g(x) dx$
<i>Aditividad respecto del intervalo</i>	Si c es un punto intermedio, $a < c < b$, entonces: $\int_a^b f(x) dx = \int_a^c f(x) dx + \int_c^b f(x) dx.$
<i>Cambio de orientación</i>	$\int_b^a f(x) dx = - \int_a^b f(x) dx.$

Monotonía	Si $f(x) \leq g(x)$ para todo $x \in [a, b]$, entonces: $\int_a^b f(x) dx \leq \int_a^b g(x) dx.$
Cota por acotación	Si $f(x) \leq M$ para todo $x \in [a, b]$, entonces: $\left \int_a^b f(x) dx \right \leq M(b - a)$



Estas propiedades se deducen de la definición de Riemann y del comportamiento de las sumas.

¿Qué propiedad de la integral definida se debe aplicar?

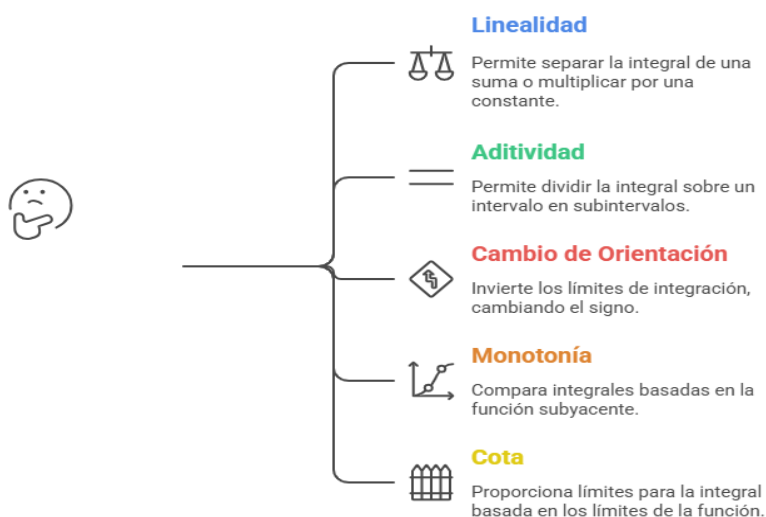


Ilustración 36—Propiedades de la Integral—Elaboración propia con Napkin

- **Criterio de integrabilidad: continuidad**

No toda función acotada es integrable en el sentido de Riemann, pero existe un resultado fundamental:

- **Teorema**

Si $f: [a, b] \rightarrow \mathbb{R}$ es continua, entonces es integrable en el sentido de Riemann en $[a, b]$.

Es decir, la continuidad en un intervalo cerrado y acotado garantiza la existencia de la integral de Riemann.

Más aún, incluso funciones con ciertas discontinuidades acotadas pueden ser integrables:

Si f es acotada en $[a, b]$ y tiene solo un número finito de discontinuidades, entonces también es Riemann-integrable en $[a, b]$.

- **Teorema Fundamental del Cálculo**

El Teorema Fundamental del Cálculo (TFC) establece el vínculo profundo entre derivadas e integrales.

Primera parte del TFC Teorema (TFC I).	Segunda parte del TFC Teorema (TFC II).
Sea $f:[a,b] \rightarrow \mathbb{R}$ una función continua. Definimos: $F(x) = \int_a^x f(t) dt, x \in [a, b].$ Entonces: 1. F es continua en $[a,b]$ y derivable en (a,b). 2. Para todo $x \in (a,b)$, $F'(x) = f(x).$ Es decir, la función definida como “área acumulada” de f desde a hasta x es una primitiva de f.	Sea $f:[a,b] \rightarrow \mathbb{R}$ una función continua y sea F una primitiva de f en $[a,b]$, es decir, $F'(x) = f(x) \text{ para todo } x \in [a, b].$ Entonces: $\int_a^b f(x) dx = F(b) - F(a).$ Este resultado permite calcular integrales definidas usando primitivas, sin recurrir explícitamente a sumas de Riemann. Conocida como Regla de Barrow.

- **Técnicas clásicas de integración**

En el marco formal, las siguientes técnicas se justifican a partir del TFC y reglas de derivación:

Cambio de variable o por sustitución	Integración por partes
Sea f continua y g derivable con derivada continua, con $g([a, \beta]) \subset [a, b]$. Entonces: $\int_{g(a)}^{g(\beta)} f(x) dx = \int_a^\beta f(g(t)) g'(t) dt.$ Esto permite transformar integrales complicadas en otras más manejables.	Deriva de la regla del producto de derivadas. Si u, v son funciones derivables con derivadas continuas en $[a, b]$, entonces: $\begin{aligned} \int_a^b u(x)v'(x) dx &= [u(x)v(x)]_a^b \\ &\quad - \int_a^b u'(x)v(x) dx. \end{aligned}$ En notación indefinida: $\int u(x)v'(x) dx = u(x)v(x) - \int u'(x)v(x) dx + C.$

¿Qué técnica de integración clásica debería usar?



Ilustración 37— Integración clásica—Elaboración propia con Napkin

Ahora, llegó el momento de los ejemplos de cálculo para cada uno de los casos analizados:

- **Ejemplo 1: Integral indefinida de un polinomio**

Calcular $\int (3x^2 - 4x + 5) dx$.

Usamos la regla de la potencia y la linealidad:

$$\int (3x^2 - 4x + 5) dx = 3 \int x^2 dx - 4 \int x dx + 5 \int 1 dx.$$

Sabemos que:

$$\int x^2 dx = \frac{x^3}{3} + C,$$

$$\int x dx = \frac{x^2}{2} + C,$$

$$\int 1 dx = x + C.$$

Por lo tanto:

$$\int (3x^2 - 4x + 5) dx = 3 \cdot \frac{x^3}{3} - 4 \cdot \frac{x^2}{2} + 5x + C = x^3 - 2x^2 + 5x + C.$$



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:
Ejemplo 1 - int.m4a

También disponible desde:
<https://tinyurl.com/4smhva7t>



- **Ejemplo 2: Integral definida de un polinomio (Regla de Barrow)**

Calcular $\int_0^2 (2x - 1) dx$.

Primero se halla una primitiva:

$$\int (2x - 1) dx = x^2 - x + C.$$

Aplicamos la regla de Barrow:

$$\int_0^2 (2x - 1) dx = [x^2 - x]_0^2 = (2^2 - 2) - (0^2 - 0) = (4 - 2) - 0 = 2.$$



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 2 - int.m4a

También disponible desde:

<https://tinyurl.com/t3jcdbbj>



- **Ejemplo 3: Integral indefinida con cambio de variable sencillo**

Calcular $\int 2x(x^2 + 1)^3 dx$.

Observamos la composición: aparece x^2+1 y su derivada es $2x$.

Tomamos el cambio de variable:

$$u = x^2 + 1 \Rightarrow du = 2x dx.$$

Entonces la integral se transforma en:

$$\int 2x(x^2 + 1)^3 dx = \int u^3 du.$$

Ahora integramos en u : $\int u^3 du = \frac{u^4}{4} + C$.

Volviendo a la variable original:

$$\int 2x(x^2 + 1)^3 dx = \frac{(x^2 + 1)^4}{4} + C.$$



Multimedia

En este podcast encontrará una explicación detallada de este ejemplo:

Ejemplo 3 - int.m4a

También disponible desde:

<https://tinyurl.com/2cz56c3p>



- **Ejemplo 4: Integral por partes (indefinida)**

Calcular: $\int xe^x dx$.

Usamos integración por partes: $\int u dv = uv - \int v du$.

Elegimos: $u = x \Rightarrow du = dx, dv = e^x dx \Rightarrow v = e^x$.

Entonces: $\int xe^x dx = xe^x - \int e^x dx = xe^x - e^x + C = e^x(x - 1) + C$.

- **Ejemplo 5: Integral definida con cambio de variable**

Calcular $\int_0^1 x\sqrt{x+1} \, dx.$

Tomamos: $u = x + 1 \Rightarrow du = dx, x = u - 1.$

Cambiamos los límites: Cuando $x = 0, u = 0 + 1 = 1.$

Cuando $x = 1, u = 1 + 1 = 2.$

La integral se transforma en:

$$\int_0^1 x\sqrt{x+1} \, dx = \int_1^2 (u-1)\sqrt{u} \, du.$$

Escribimos $\sqrt{u} = u^{1/2}:$

$$\int_1^2 (u-1)u^{1/2} \, du = \int_1^2 (u^{3/2} - u^{1/2}) \, du.$$

Integramos término a término:

$$\int u^{3/2} \, du = \frac{2}{5} u^{5/2}, \int u^{1/2} \, du = \frac{2}{3} u^{3/2}.$$

Por lo tanto:

$$\int_1^2 (u^{3/2} - u^{1/2}) \, du = \left[\frac{2}{5} u^{5/2} - \frac{2}{3} u^{3/2} \right]_1^2.$$

Evaluamos en $u=2$ y $u=1$:

$$= \left(\frac{2}{5} 2^{5/2} - \frac{2}{3} 2^{3/2} \right) - \left(\frac{2}{5} 1^{5/2} - \frac{2}{3} 1^{3/2} \right).$$

Se puede dejar en esta forma exacta; si se desea, se simplifica numéricamente.

- **Ejemplo 6: Integral definida de una función trigonométrica**

Calcular $\int_0^\pi \sin x \, dx.$

Sabemos que: $\int \sin x \, dx = -\cos x + C.$

Aplicamos Barrow:

$$\int_0^\pi \sin x \, dx = [-\cos x]_0^\pi = [-\cos(\pi)] - [-\cos(0)] = -(-1) - (-1) = 1 + 1 = 2.$$



Recursos

Una vez más, le recomendamos ver el contenido del enlace donde se presentan múltiples ejemplos y casos para revisar:

<https://tinyurl.com/3vas9bn8>



Multimedia

En este video podrá ver presentadas las explicaciones de los ejemplos trabajados: *Métodos de Integración*

También disponible desde:

<https://tinyurl.com/msz3dcsd>



Actividad

Ahora está Ud. en condiciones de hacer la última actividad del módulo. Se trata de la *Actividad 5: Solo cálculo de integrales*.

Llegó el momento de integrar todo lo visto estudiando una función de pérdida cuadrática puntual y una de pérdida esperada, ambas tienen estrecha relación con modelos de IA.

- *Pérdida cuadrática puntual como función real*

¿Qué es la *pérdida cuadrática puntual como función real*?

Es simplemente una función real de variable real que mide *cuánto se equivoca* una predicción escalar respecto de un valor objetivo.

Fijamos un valor real y (el “valor verdadero”) y definimos la función:

$\ell(z) = (z - y)^2, z \in \mathbb{R}$: es la salida del modelo (predicción).

y : es el valor real que queremos predecir.

$\ell(z)$: es el error (costo) asociado a esa predicción puntual.

Por eso se llama:

- › “*pérdida*”: porque mide cuánto “perdemos” o nos equivocamos.
- › “*cuadrática*”: porque el error se eleva al cuadrado.
- › “*puntual*”: porque está definida para un solo par (predicción, valor real); no promedia todavía sobre muchos datos.

Matemáticamente:

- › Es una función $\ell: \mathbb{R} \rightarrow [0, +\infty)$.
- › Para cada valor de z , devuelve un número ≥ 0 que indica el tamaño del error.

Luego, cuando pasamos a muchos datos, usamos integrales o sumas para construir la pérdida media (riesgo esperado), pero la unidad básica que se está promediando es justamente esta pérdida cuadrática puntual.

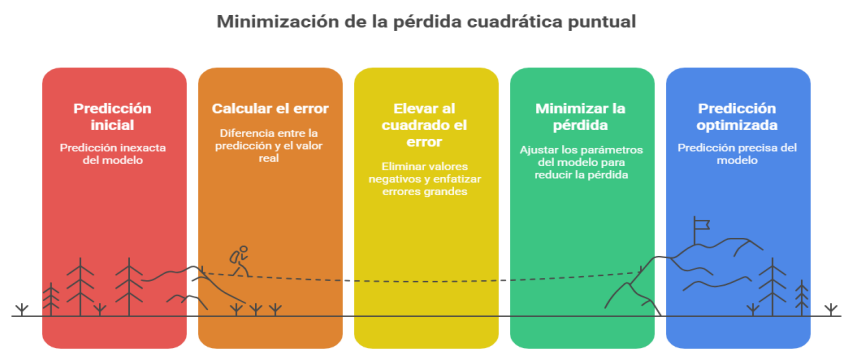
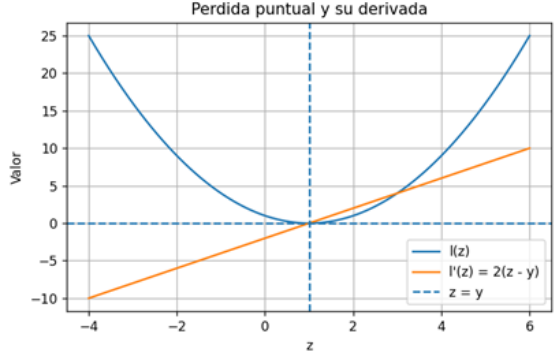


Ilustración 38 – Pérdida Cuadrática -- Elaboración propia con Napkin

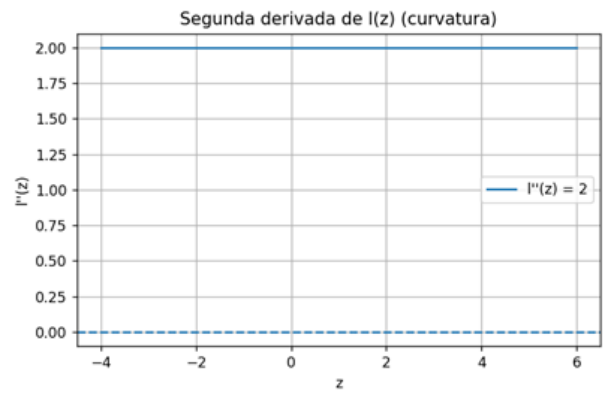
- Estudio de Función propiamente dicho
- › Espacio de Referencia: Dominio–Codominio–Imagen

Por lo tanto, tomemos una salida deseada fija y y consideremos la pérdida cuadrática como función del “output” del modelo z: $\ell(z) = (z - y)^2, z \in \mathbb{R}.$	Ilustración 39 - Propia En regresión, z sería la predicción del modelo y y el valor real.
Dominio	La expresión $(z-y)^2$ está definida para todo $z \in \mathbb{R}$. "Dom"(ℓ)= $\mathbb{R}$.
Codominio	Formalmente podemos tomar $\mathbb{R}$, pero la función solo toma valores ≥ 0 .
Imagen	Como un cuadrado nunca es negativo, $\ell(z) = (z - y)^2 \geq 0 \forall z \in \mathbb{R}$. Además, $\ell(z)=0$ si y solo si $z=y$. No hay cota superior: si $z \rightarrow \infty$, entonces $\ell(z) \rightarrow \infty$. Por tanto: "Im"($\ell$)=$[0, +\infty)$.

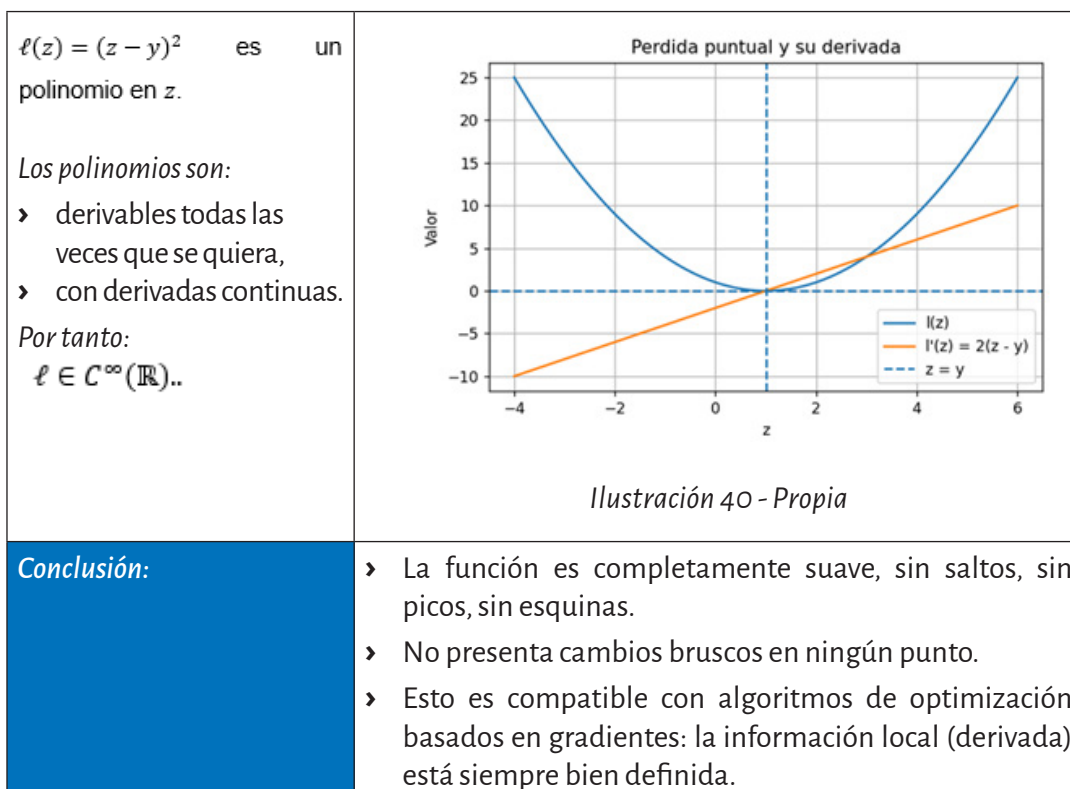
- *Tendencia: creciente, decreciente, constante*

Derivamos: $\ell(z) = (z - y)^2,$ $\ell'(z) = 2(z - y).$	 Ilustración 40 - Propia
Tendencia	› Si $z < y$, entonces $z - y < 0$ $\ell'(z) < 0$: la función es decreciente en $(-\infty, y)$. › Si $z > y$, entonces $z - y > 0$ $\ell'(z) > 0$: la función es creciente en $(y, +\infty)$. › En $z = y$, $\ell'(y) = 0$.
Conclusión:	› La curva tiene forma de “U”: decrece hasta $z = y$, luego crece. › No es constante en ningún intervalo.

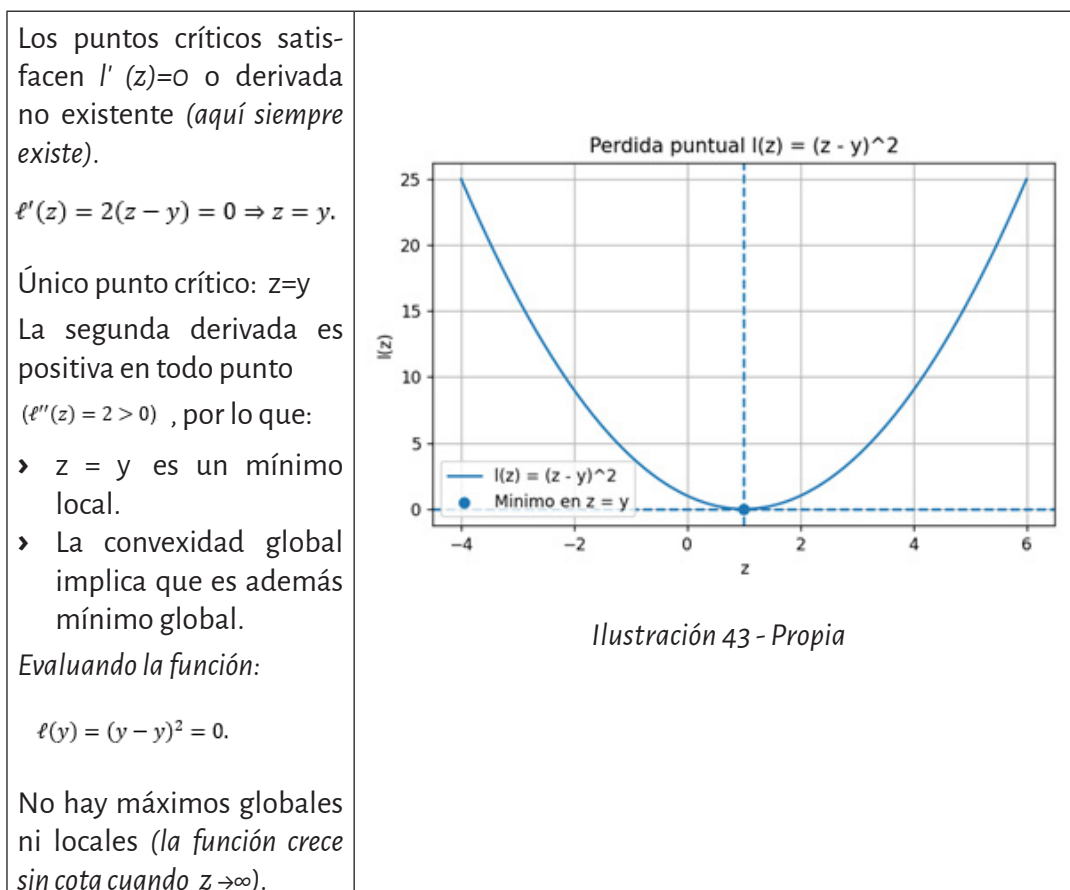
- *Curvatura*

Calculamos la segunda derivada: $\ell''(z) = \frac{d}{dz} [2(z - y)] = 2.$ $\ell''(z) = 2 > 0$ para todo $z \in \mathbb{R}$.	 Ilustración 41 - Propia
Conclusión:	› La función ℓ es estrictamente convexa en todo $\mathbb{R}$. › La curvatura es constante (misma “concavidad hacia arriba” en todo punto). › No hay cambios de concavidad, por lo tanto, no hay puntos de inflexión. › En IA Esta convexidad hace muy simple el paisaje de optimización en una dimensión: un único valle global

- *Cambios bruscos o suavidad*



- *Puntos críticos, máximos y mínimos*



- *Comportamiento asintótico*

Estudiamos los límites en $\pm\infty$:

$$\ell(z) = (z - y)^2.$$

- Cuando $z \rightarrow +\infty$, $(z - y)^2 \rightarrow +\infty$.
- Cuando $z \rightarrow -\infty$, $(z - y)^2 \rightarrow +\infty$.

No hay asíntotas horizontales ni oblicuas:

- › La función no se aproxima a una recta finita en los extremos.
- › Simplemente crece como un polinomio de grado 2.

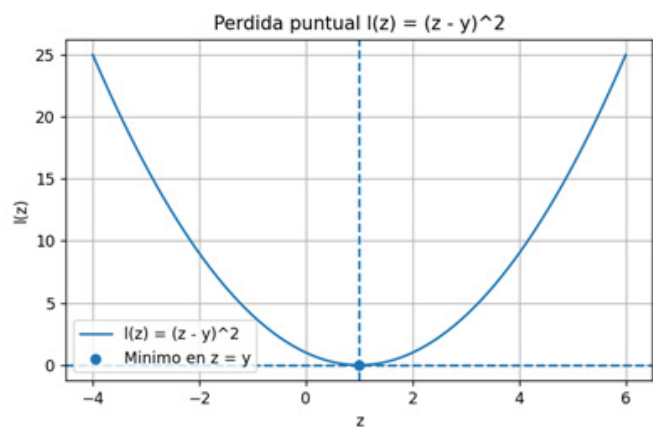


Ilustración 44 - Propia

- *Rangos de valores posibles*

Recogiendo lo anterior:

- › La función toma todos los valores reales mayores o iguales que 0.
- › El 0 se alcanza exactamente en $z=y$.
- › Para cualquier $M>0$, el conjunto $\{z: \ell(z) \leq M\}$ es un intervalo simétrico alrededor de y .

En resumen:

- › " $\text{Im}(\ell) = [0, +\infty)$,
- › con mínimo global alcanzado.

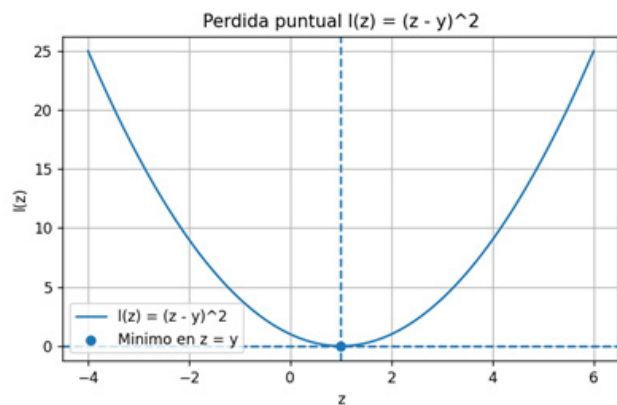


Ilustración 45 - Propia

- *Incorporando integrales: pérdida cuadrática media*

› ¿Qué es la pérdida cuadrática media?

Es una función de evaluación de un modelo que mide, en promedio, cuánto se alejan las predicciones de los valores reales, penalizando más fuerte los errores grandes porque los eleva al cuadrado.

Versión discreta (sobre un conjunto de datos)	Versión continua (con integrales)
Tienes datos (x_i, y_i), $i = 1, \dots, n$, y modelo $f_\theta(x)$ que predice $\hat{y}_i = f_\theta(x_i)$. La pérdida cuadrática media (MSE) es: $\text{MSE}(\theta) = \frac{1}{n} \sum_{i=1}^n (f_\theta(x_i) - y_i)^2.$ Cada término $(f_\theta(x_i) - y_i)^2$ es la pérdida cuadrática puntual. El promedio $\frac{1}{n} \sum$ es la media sobre todos los datos.	Si piensas los datos como realizaciones de una variable aleatoria (X, Y) con cierta distribución, la pérdida cuadrática media “ideal” (riesgo esperado) es: $\begin{aligned} L(\theta) &= \mathbb{E}[(f_\theta(X) - Y)^2] \\ &= \int (f_\theta(x) - y)^2 p_{X,Y}(x, y) dx dy. \end{aligned}$ En el ejemplo que construimos juntas antes, tomábamos algo aún más simple: En el ejemplo que construimos juntas antes, tomábamos algo aún más simple: $L(\theta) = \int_0^1 (\theta x - y)^2 dx,$ que es una pérdida cuadrática media sobre el intervalo $[0, 1]$ con distribución uniforme.

Entonces, si consideramos un modelo lineal muy simple en una dimensión:

$$\hat{y}(x) = \theta x,$$

y fijemos un valor objetivo $y \in \mathbb{R}$ para todos los x de un intervalo.

› La pérdida cuadrática puntual es:

$$\ell_\theta(x) = (\theta x - y)^2.$$

Supondremos que x está uniformemente distribuido en $[0, 1]$ (este es un caso simple, integrable a mano).

› La pérdida media en el intervalo $[0, 1]$ se define como:

$$L(\theta) = \int_0^1 (\theta x - y)^2 dx.$$

Aquí:

- › Para cada θ , $L(\theta)$ es un número real.
- › L es una función de la variable θ .

- Cálculo de la integral - pérdida media

Desarrollamos el cuadrado:

$$(\theta x - y)^2 = \theta^2 x^2 - 2\theta yx + y^2.$$

Por linealidad de la integral:

$$\begin{aligned} L(\theta) &= \theta^2 \int_0^1 x^2 dx \\ &\quad - 2\theta y \int_0^1 x dx + y^2 \int_0^1 1 dx. \end{aligned}$$

Calculamos cada integral:

- $\int_0^1 x^2 dx = \frac{1}{3}.$
- $\int_0^1 x dx = \frac{1}{2}.$
- $\int_0^1 dx = 1.$

Sustituyendo

$$\begin{aligned} L(\theta) &= \theta^2 \cdot \frac{1}{3} - 2\theta y \cdot \frac{1}{2} + y^2 \\ &\quad \cdot 1 \\ &= \frac{\theta^2}{3} - \theta y + y^2. \end{aligned}$$

Esta es ahora una función cuadrática de θ

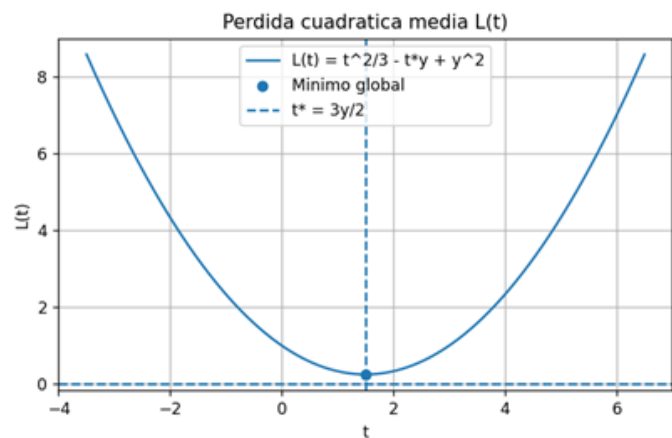


Ilustración 46 - Propia



De esta manera usamos integrales para pasar de la pérdida puntual a la pérdida media.

- Estudio de $L(\theta)$ como función real

Repetimos el análisis completo, ahora sobre la variable de parámetros de la función real obtenida en la conversión.

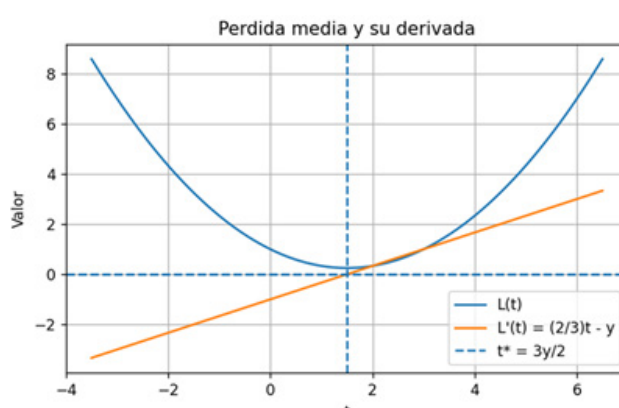


Ilustración 47 - Propia



De esta manera usamos integrales para pasar de la pérdida puntual a la pérdida media.

**Dominio Codominio
Imagen**

Dominio: $\mathbb{R}$. (cualquier valor real de θ es admisible).
 $L()$ es un polinomio cuadrático en θ , así que:

$L: \mathbb{R} \rightarrow \mathbb{R}$.

Como veremos, la imagen será también un intervalo $[L_{\min}, +\infty)$.

**Derivada, tendencia y
puntos críticos**

Derivamos:

$$L(\theta) = \frac{\theta^2}{3} - \theta y + y^2,$$

$$L'(\theta) = \frac{2\theta}{3} - y.$$

Punto crítico ($L'(\theta^*) = 0$):

$$\frac{2\theta}{3} - y = 0 \Rightarrow \theta^* = \frac{3}{2}y.$$

› Para $\theta < \theta^*$, $L'(\theta) < 0$: L decrece.

› Para $\theta > \theta^*$, $L'(\theta) > 0$: L crece.

Curvatura y convexidad

Segunda derivada:

$$L''(\theta) = \frac{2}{3} > 0, \forall \theta \in \mathbb{R}.$$

Luego:

L es **estrictamente convexa** en $\mathbb{R}$.

El punto crítico $\theta^* = \frac{3}{2}y$ es un **mínimo global único**.

Valor mínimo y rango de la función

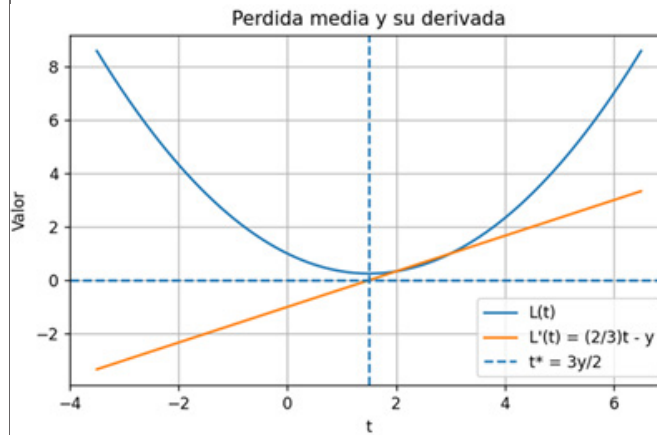


Ilustración 48 - propia

Evaluamos L

$$\theta^*: \theta^* = \frac{3}{2}y.$$

Sustituyendo:

$$\begin{aligned} L(\theta^*) &= \frac{\left(\frac{3}{2}y\right)^2}{3} - \left(\frac{3}{2}y\right)y + y^2 \\ &= \frac{9}{4}y^2 - \frac{3}{2}y^2 + y^2 \\ &= \frac{9}{12}y^2 - \frac{18}{12}y^2 + \frac{12}{12}y^2 \\ &= \frac{3}{12}y^2 \\ &= \frac{y^2}{4}. \end{aligned}$$

Por lo tanto:

› mínimo global:

$$L_{\min} = \frac{y^2}{4},$$

alcanzado en $\theta =$

θ^* .

Como el coeficiente de θ^2 es positivo ($\frac{1}{3} > 0$), se verifica que:

$$\lim_{\theta \rightarrow \pm\infty} L(\theta) = \left[\frac{y^2}{4}, +\infty \right).$$

Asintóticamente, cuando $\theta \rightarrow \pm\infty$, claramente $L(\theta) \sim$

$$\frac{\theta^2}{3} \rightarrow \infty.$$



Dejamos aquí el enlace hacia el código: <https://tinyurl.com/2pskmmjf> ¡Recuerde descargar y correrlo en forma local!

- *Lectura de este estudio de función en términos de modelos de IA*

En este ejemplo:

- › La integral $L(\theta) = \int_0^1 (\theta x - y)^2 dx$ representa la pérdida cuadrática media sobre entradas $x \in [0,1]$.
- › El cálculo de la integral nos da una función explícita de θ .
- › El análisis de $L(\theta)$ (derivada, convexidad, mínimo) muestra:
 - › existe un único parámetro óptimo θ^* que minimiza la pérdida media;
 - › la búsqueda de mínimo es un problema de optimización convexa unidimensional.

Este esquema es el núcleo de:

- › el ajuste por mínimos cuadrados en regresión lineal,
- › la idea de “minimizar la pérdida promedio” en aprendizaje supervisado,
- › el uso combinado de integrales (promedio continuo) + derivadas (gradientes) + convexidad.

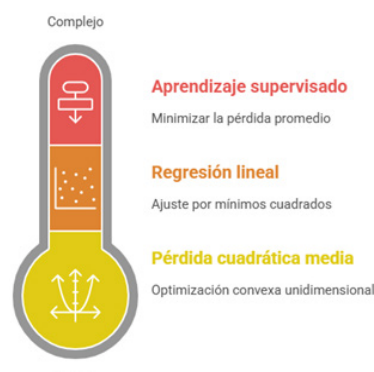


Ilustración 49- Aprendizaje – Propia - Napkin



Estamos en condiciones de hacer la **Actividad 6: Estudio de una función de pérdida cuadrática y desarrollo de una herramienta en Python**. La misma está planteada para ser realizada de forma grupal.

- *Introducción a funciones multivariantes y derivadas parciales*

Hasta ahora, el foco estuvo puesto en funciones de una variable real, del tipo

$$f: \mathbb{R} \rightarrow \mathbb{R}, y = f(x),$$

y estudiamos su dominio, límites, continuidad, derivadas, integrales y extremos. Este lenguaje es suficiente para describir, por ejemplo, una función de pérdida en una sola dimensión o la evolución de una magnitud en el tiempo.

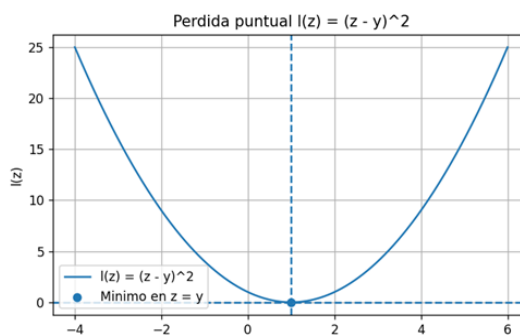


Ilustración 50 - Propia

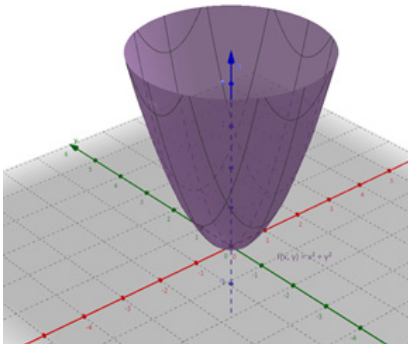
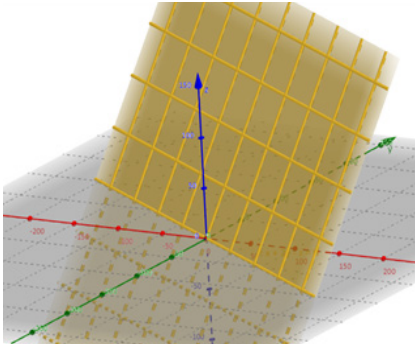
Sin embargo, en inteligencia artificial la mayoría de los modelos dependen de muchos parámetros y muchas variables al mismo tiempo.

Un modelo de regresión lineal simple ya tiene, al menos, una pendiente y una ordenada al origen; una red neuronal tiene cientos o miles de pesos.

Matemáticamente, esto nos lleva de manera natural al estudio de funciones multivariantes y de las derivadas parciales, que generalizan los conceptos vistos para una sola variable.

- **Funciones multivariantes**

Una función multivariable es una aplicación que toma varios números reales como entrada y devuelve un número real (u otro objeto) como salida.

Por ejemplo, en dos variables: $f: \mathbb{R}^2 \rightarrow \mathbb{R}, f(x, y) = x^2 + y^2,$	o en tres variables: $g: \mathbb{R}^3 \rightarrow \mathbb{R}, g(x, y, z) = x + 2y - z.$
 <i>Ilustración 51 - Propia - Geogebra</i>	 <i>Ilustración 52 - Propia - Geogebra</i>

- **Interpretaciones típicas:**

- › x, y pueden representar dos parámetros de un modelo de IA (por ejemplo, dos pesos).
- › $f(x, y)$ puede representar la pérdida del modelo al usar esos parámetros.
- › La gráfica de f ya no es una curva en el plano, sino una superficie en el espacio: a cada punto (x, y) se le asigna una altura $f(x, y)$.

En este contexto, muchos conceptos del caso unidimensional se generalizan:

- › El dominio de f es ahora un subconjunto de $\mathbb{R}^n$.
- › La imagen es el conjunto de valores reales que toma f .
- › La continuidad se define de forma análoga, pero ahora considerando cómo se comporta $f(\mathbf{x})$ cuando el vector $\mathbf{x}$ se aproxima a un punto desde cualquier dirección.

- **Derivadas parciales: idea básica**

En funciones de una variable, la derivada mide la tasa de cambio instantánea de la función respecto de su única variable.

En funciones de varias variables, nos interesa saber cómo cambia la función cuando solo una de las variables se modifica y las demás se mantienen fijas.

Aquí aparecen las derivadas parciales.

Sea $f(x, y)$ una función de dos variables. La derivada parcial respecto de x en un punto (x_0, y_0) se define (cuando existe) como el límite:

$$\frac{\partial f}{\partial x}(x_0, y_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h, y_0) - f(x_0, y_0)}{h},$$

es decir, derivamos como en una variable, pero dejando y fijo. De modo análogo, la derivada parcial respecto de y es:

$$\frac{\partial f}{\partial y}(x_0, y_0) = \lim_{h \rightarrow 0} \frac{f(x_0, y_0 + h) - f(x_0, y_0)}{h},$$

donde ahora x permanece fijo.

Ejemplo sencillo:

$$f(x, y) = x^2 + y^2.$$

Derivada parcial respecto de x :	Derivada parcial respecto de y :
x : $\frac{\partial f}{\partial x}(x, y) = 2x$,	y : $\frac{\partial f}{\partial y}(x, y) = 2y$
porque consideramos a y como constante.	porque consideramos a x como constante.

Cada derivada parcial describe cómo cambia la altura de la superficie $z = f(x, y)$ si nos movemos en una dirección específica: solo en el eje x o solo en el eje y .

- **Gradiente y dirección de máximo cambio**

Reunir todas las derivadas parciales de una función en un punto da lugar al gradiente:

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}(x, y), \frac{\partial f}{\partial y}(x, y) \right)$$

En dimensión mayor:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right).$$

El gradiente es un vector que apunta en la *dirección de máximo aumento* de la función y su módulo indica cuán rápido crece en esa dirección.

Siguiendo la función ejemplo $f(x, y) = x^2 + y^2$ tendremos que el *gradiente* de f es el vector de sus derivadas parciales:

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}(x, y) \quad \frac{\partial f}{\partial y}(x, y) \right)$$

1. *Derivada parcial respecto de x (considerando y constante):*

$$\frac{\partial f}{\partial x}(x, y) = \frac{\partial}{\partial x}(x^2 + y^2) = 2x.$$

2. *Derivada parcial respecto de y (considerando x constante):*

$$\frac{\partial f}{\partial y}(x, y) = \frac{\partial}{\partial y}(x^2 + y^2) = 2y.$$

Por lo tanto, el gradiente es: $\nabla f(x, y) = (2x, 2y)$

En IA, este concepto es central:

- los algoritmos de entrenamiento (como descenso por gradiente) actualizan los parámetros en la dirección opuesta al gradiente de la función de pérdida, para disminuirla;
- las funciones de pérdida de modelos reales dependen de muchos parámetros, por lo que el gradiente es un objeto multivariable formado por derivadas parciales.

- **Relación con extremos y convexidad**

En una variable, vimos que los puntos críticos son aquellos donde la derivada es cero o no existe, y que la segunda derivada ayuda a clasificar máximos, mínimos y puntos de inflexión.

En varias variables:

- los puntos críticos de f son los puntos donde todas las derivadas parciales se anulan simultáneamente:

$$\nabla f(\mathbf{x}_0) = \mathbf{0};$$

- la información sobre curvatura se generaliza mediante derivadas de orden superior (matriz Hessiana), que no trabajaremos en detalle, pero que se conecta conceptualmente con la idea de convexidad.

En el contexto de IA, muchas funciones de pérdida usadas en modelos simples son convexas en sus parámetros; esto garantiza que un punto donde el gradiente es cero corresponde a un mínimo global, lo que facilita el entrenamiento y el análisis teórico.



Para ampliar el tema de las funciones multivariantes y derivadas parciales, se recomienda navegar el sitio y ver los videos de Khan Academy como referencia de estudio más amplia.

Ejemplo con interpretación en IA

Consideremos una pérdida cuadrática media simplificada en dos parámetros, θ_1 y θ_2 :	$L(\theta_1, \theta_2) = \theta_1^2 + 2\theta_2^2.$
Derivadas parciales:	$\frac{\partial L}{\partial \theta_1} = 2\theta_1,$ $\frac{\partial L}{\partial \theta_2} = 4\theta_2.$
El gradiente es	$\nabla L(\theta_1, \theta_2) = (2\theta_1, 4\theta_2).$

El único punto crítico es $(\theta_1, \theta_2) = (0,0)$, donde el gradiente es cero. Allí la pérdida vale $L(0,0) = 0$, que es un mínimo global.

La finalización de este módulo no es un punto de llegada, sino un punto de partida. Hemos revisado y aplicado conceptos clásicos de la matemática —funciones, límites, continuidad, derivadas, integrales, estudio de curvas, optimización— mostrándolos no solo como herramientas abstractas, sino como el lenguaje que permite formular, analizar y mejorar modelos de inteligencia artificial.

Detrás de cada función de pérdida, de cada gradiente que guía el entrenamiento, de cada criterio de optimización y de cada modelo que “aprende”, hay ideas matemáticas que ustedes ya son capaces de reconocer y estudiar con rigor. Comprender cómo se comportan estas funciones, por qué convergen ciertos algoritmos y qué implican las decisiones de modelado no es solo teórico: es lo que diferencia a quien solo “usa” modelos de IA de quien puede diseñarlos, explicarlos y transformarlos.

Por eso, la invitación es a seguir investigando e indagando, a preguntarse siempre *¿qué función hay detrás de esto?*, *¿qué propiedades tiene?*, *¿cómo cambia si modifico...?*.

Aproveche los foros y actividades grupales que son espacios para compartir ideas, dudas, intuiciones, experimentos en Python y pequeños proyectos de modelado. Allí podemos contrastar miradas, revisar razonamientos, mejorar explicaciones y construir ejemplos que nos ayuden a todas las personas del grupo. La matemática se vuelve mucho más poderosa cuando se piensa y se discute en comunidad.



Estamos creando juntos una nueva manera de ver la matemática, abordándola como una herramienta viva para comprender y moldear la inteligencia artificial que nos rodea. Lo que han aprendido en este módulo es una base sólida; lo más interesante comienza cuando empiezan a usarla para hacerse preguntas nuevas.



Evaluación

En este momento, se encuentra Ud. en condiciones de realizar la primera parte de la evaluación integradora. ¡Éxitos!

- *Referencias y Recursos*

Apuntes

- › **González, F. J. P. (S/F). Cálculo Diferencial E Integral De Funciones De Una Variable.** Ugr.es. Recuperado el 21 de noviembre de 2025, de <https://tinyurl.com/jc67bva8>
- › **González, F. J. P. (s/f).** Apuntes de cálculo diferencial e integral de funciones de varias variables. Ugr.es. Recuperado el 21 de noviembre de 2025, de <https://tinyurl.com/4tk9sm6b>

Recursos recomendados

- › **Khan Academy – Cálculo diferencial (español)** <https://tinyurl.com/79k7rmtj>
- › **Khan Academy – Cálculo integral (español)** <https://tinyurl.com/3ruxbnep>
- › **Funciones de varias variables.** Derivadas parciales (OCW Univ. de Cantabria) <https://tinyurl.com/yc5kvbnz>

Matemáticas para IA / ampliación

- › **Las matemáticas de la inteligencia artificial – LibreTexts** (español) https://espanol.libretexts.org/Bookshelves/Matematicas/Las_matematicas_de_la_inteligencia_artificial

Cursos abiertos (en inglés)

- › **MIT OCW – Single Variable Calculus (18.01SC)** <https://tinyurl.com/3eu6yhfd>
- › **MIT OCW – Multivariable Calculus (18.02SC)** <https://tinyurl.com/57besc34>

Herramientas digitales

- › **GeoGebra – Calculadora Gráfica (web)** <https://www.geogebra.org/graphing?lang=es>
- › **Google Colab (Jupyter en la nube)** <https://colab.research.google.com/>

MÓDULO 2 | GLOSARIO

Asíntota: Recta a la que la gráfica de una función se aproxima cuando la variable tiende a un valor (finito o infinito). Puede ser horizontal, vertical u oblicua.

Codominio: Conjunto de posibles valores de salida de una función (puede ser más grande que el conjunto de valores que efectivamente toma la función).

Continuidad: Una función es continua en un punto a si $\lim_{x \rightarrow a} f(x) = f(a)$. Intuitivamente, se puede trazar su gráfica sin “levantar el lápiz” en ese punto.

Convexidad: Una función es convexa en un intervalo si la recta que une dos puntos de su gráfica queda siempre por encima o sobre la curva. En una variable, una condición suficiente es $f''(x) \geq 0$ en el intervalo. Es clave en optimización (unicidad de mínimos en muchos casos).

Derivabilidad: Propiedad de una función que admite derivada en un punto o en un intervalo. La derivabilidad en un punto implica continuidad en ese punto.

Derivada en un punto: Límite del cociente incremental que mide la tasa de cambio instantánea en un punto:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

Se interpreta como la pendiente de la recta tangente en $(a, f(a))$.

Derivada lateral: Derivada calculada usando solo valores a la derecha ($h \rightarrow 0^+$) o solo valores a la izquierda ($h \rightarrow 0^-$) de un punto. Si existen y coinciden, la función es derivable en ese punto.

Descenso por gradiente: Método iterativo de optimización en el que se actualizan los parámetros moviéndose en la dirección opuesta al gradiente de la función de pérdida para disminuir su valor.

Dominio: Conjunto de todos los valores de la variable independiente para los cuales una función está definida.

Función: Relación que asigna a cada elemento de un conjunto de entrada (dominio) un único valor en un conjunto de salida (codominio). Se escribe $f: X \rightarrow Y$ o $y = f(x)$.

Función de pérdida (loss function): Función que asigna un valor numérico (el “costo” o “error”) a la salida de un modelo de IA para un dato dado. Cuanto mayor es la pérdida, peor es el desempeño del modelo en ese ejemplo.

Gradiente (en dimensión 1): En una variable coincide con la derivada de la función de pérdida respecto del parámetro. Indica la dirección y magnitud del cambio más rápido de la función.

Gráfica de una función: Conjunto de puntos $(x, f(x))$ en el plano. Permite visualizar la forma de la función y analizar tendencias, extremos, continuidad, etc.

Imagen/Rango: Conjunto de valores que la función efectivamente toma al evaluar todos los puntos del dominio.

Integral definida (Riemann): Número real que representa el área “orientada” bajo la curva de una función $f(x)$ en un intervalo $[a, b]$:

$$\int_a^b f(x) \, dx.$$

Se define como el límite de sumas de Riemann cuando el tamaño de los subintervalos tiende a cero.

Integral indefinida / Primitiva: Familia de funciones F tales que $F'(x) = f(x)$. Se escribe:

$$\int f(x) \, dx = F(x) + C.$$

Es la operación inversa de derivar (hasta una constante).

Límite al infinito: Estudio del comportamiento de una función cuando la variable independiente crece o decrece sin cota, por ejemplo $x \rightarrow +\infty$ o $x \rightarrow -\infty$.

Límite de una función: Valor al que se acerca $f(x)$ cuando x se aproxima a un punto a , sin necesidad de que $x = a$. Formalmente, $\lim_{x \rightarrow a} f(x) = L$ si para todo $\varepsilon > 0$ existe $\delta > 0$ tal que $0 < |x - a| < \delta$ implica $|f(x) - L| < \varepsilon$.

Límite lateral: Límite de una función cuando la variable se aproxima a un punto desde la derecha ($x \rightarrow a^+$) o desde la izquierda ($x \rightarrow a^-$).

Máximo y mínimos globales: Valores extremos más altos o bajos que la función toma en todo su dominio (no solo en la vecindad de un punto).

Máximo local y mínimo local: Un máximo local es un punto donde la función toma un valor mayor (o igual) que en puntos cercanos. Un mínimo local es un punto donde la función toma un valor menor (o igual) que en puntos cercanos.

Modelo de IA / Modelo de aprendizaje: Función (o familia de funciones parametrizadas) que intenta aproximar una relación entre entradas y salidas a partir de datos. Suele escribirse $f_{\theta}(x)$, donde θ son parámetros.

Optimización: Proceso de encontrar valores de variables (por ejemplo, parámetros de un modelo) que minimizan o maximizan una función dada (por ejemplo, una función de pérdida).

Pérdida cuadrática media: Promedio (discreto o continuo) de las pérdidas cuadráticas puntuales. En versión continua sobre un intervalo:

$$L(\theta) = \int_a^b (\theta x - y_0)^2 \, dx.$$

Se usa en regresión y en muchos modelos de aprendizaje supervisado.

Pérdida cuadrática puntual: Función de pérdida de la forma $\ell(z) = (z - y)^2$,

donde z es la predicción y y el valor verdadero. Siempre es ≥ 0 y vale 0 solo cuando $z = y$.

Punto crítico: Punto del dominio donde la derivada es cero o no existe. Son candidatos a máximos, mínimos o puntos singulares.

Punto de discontinuidad: Punto donde una función no es continua (por salto, por agujero, por asíntota, etc.).

Punto de inflexión: Punto donde la función cambia de concavidad (de cóncava hacia arriba a cóncava hacia abajo, o viceversa).

Recta tangente: Recta que “toca” a la gráfica de una función en un punto y tiene la misma pendiente que la función en ese punto. Su pendiente es la derivada.

Segunda derivada: Derivada de la derivada. Proporciona información sobre la curvatura de la función:

$f''(x) > 0$: la función es cóncava hacia arriba (convexa).

$f''(x) < 0$: la función es cóncava hacia abajo.

Tasa de variación: Cambio promedio de una función en un intervalo $[a, b]$: $\frac{f(b) - f(a)}{b - a}$. La derivada es el límite de esta tasa cuando el intervalo se hace muy pequeño.

Teorema Fundamental del Cálculo: Vincula derivadas e integrales. En una formulación:

Si f es continua y F es una primitiva de f , entonces

$$\int_a^b f(x) \, dx = F(b) - F(a).$$

MÓDULO 2 | ACTIVIDADES



ACTIVIDAD N° 1:

Lectura e interpretación de gráficas de funciones aplicadas a la Inteligencia Artificial

Antes de empezar:

Estos ejercicios tienen la particularidad de combinar la matemática con la programación. Por ello, hemos generado un repositorio online para que descargue el código y pueda realizar los ejercicios:



Descargue el código Python desde <https://tinyurl.com/yeyvth5n> y ejecútelo en una IDE como Visual Studio Community para generar la gráfica.

› Usando Chat UBP

Esta materia tiene incorporado un asistente inteligente que le permitirá corroborar sus respuestas, consultar sus dudas o hacer parte de sus actividades si es que la consigna lo solicita.

› Consignas

Ejercicio 1: Función de pérdida tipo parábola (mínimo global)

A partir de la gráfica generada por el script de Python (función de pérdida)

$$L(w) = (w - 3)^2 + 2;$$

1. Explique con sus propias palabras qué indica la altura de la curva para un valor específico de w .
2. Analice la pérdida en $w = 0$ y en $w = 3$ a partir de la gráfica presentada. ¿En cuál de los dos casos es mayor? Explique las implicancias que esto tiene respecto a la calidad del modelo en cada situación.
3. Si el modelo estuviera actualmente en $w = 0$, usando solo la forma de la curva, ¿en qué dirección debería ajustarse w para reducir la pérdida? ¿hacia valores mayores o menores? Justifique.
4. Interprete el punto marcado como “mínimo” en la gráfica. ¿Qué información aporta respecto al proceso de entrenamiento?

Ejercicio 2: Funciones de activación: ReLU vs Sigmoide

A partir de la figura generada por el script en Python, en la cual se representan en un mismo gráfico las funciones de activación $\text{ReLU}(x)$ y $\text{sigmoide}(x)$, responda:

1. ¿Qué función anula es decir, lleva a 0 todos los valores negativos de la variable de entrada? Fundamente por qué este comportamiento puede resultar útil en una red neuronal.

2. ¿Cuál de las dos funciones considera más adecuada si se desea interpretar la salida como una probabilidad? Justifique su respuesta a partir de la forma de la gráfica.
3. Observando la curva correspondiente a la función sigmoide, ¿en qué intervalo de valores de x la función presenta un cambio más pronunciado? ¿Qué implicancias tiene esto para el tamaño del gradiente durante el entrenamiento, en términos de magnitud de la derivada?
4. A partir de la gráfica, explique por qué la función sigmoide puede ocasionar el problema de vanishing gradient cuando la entrada adquiere valores de gran magnitud en valor absoluto.

Ejercicio 3: Curva de pérdida vs épocas de entrenamiento

A partir de la gráfica generada por el script en Python, en la cual se representa la pérdida del modelo en función de las épocas de entrenamiento, responda:

1. Analice el comportamiento de la pérdida entre las épocas 1 y 10. ¿Cómo describiría la velocidad de aprendizaje del modelo en dicho intervalo?
2. Describa qué ocurre con la pérdida a partir de aproximadamente la época 30. Exponga al menos dos posibles interpretaciones de este comportamiento.
3. Suponga que, en otra ejecución del entrenamiento, la curva de pérdida comenzara a incrementarse luego de cierta cantidad de épocas. ¿Qué fenómeno conocido en el ámbito del aprendizaje automático podría estar produciéndose en este caso? Fundamente su respuesta.
4. En función de la forma de la curva observada, evalúe si resultase razonable prolongar el entrenamiento durante muchas más épocas. Justifique su respuesta.



ACTIVIDAD N° 2:

Funciones, crecimiento y continuidad en modelos de IA: informe co-construido con asistentes de IA

Esta actividad tiene el propósito de integrar los conceptos de función, crecimiento/decrecimiento y continuidad con aplicaciones en Inteligencia Artificial, mediante la co-construcción de un informe escrito apoyado en el diálogo crítico con distintos modelos de IA.

¿Cuál es el producto esperado?

Un informe escrito (máximo 8 páginas, sin contar anexos) que:

- › Explique con rigor los conceptos de función, crecimiento/decrecimiento y continuidad.
- › Relacione estos conceptos con aplicaciones concretas en IA (p. ej., funciones de activación, funciones de costo, comportamiento del modelo ante variaciones de entrada).

- › Sea el resultado de una co-construcción con diferentes asistentes de IA, cuya interacción se documenta en un anexo de diálogos.
- › Incluya una sección de fuentes utilizadas y una explicación explícita del criterio de selección de temas desarrollados en el informe.

Consignas generales

1. Trabajar de manera individual (o en grupos pequeños).
2. Elegir al menos 2 modelos de IA diferentes (por ejemplo: dos versiones de un mismo asistente, o plataformas distintas: ChatGPT, Copilot, otros).
3. Mantener con esos modelos diálogos orientados a:
 - › Clarificar conceptos matemáticos: función, crecimiento/decrecimiento, continuidad.
 - › Explorar ejemplos concretos en el contexto de IA:
 - › funciones de activación (ReLU, sigmoide, tanh, etc.),
 - › funciones de costo (sin entrar aún en su cálculo),
 - › comportamiento de salida de un modelo ante pequeñas variaciones en la entrada (estabilidad, sensibilidad).
4. Utilizar esos diálogos como insumo, no como producto final:
 - › el informe debe ser redactado, organizado y argumentado por usted.
 - › las respuestas de los modelos de IA se utilizan como materiales a analizar, integrar, contrastar y citar.
5. Adjuntar los diálogos completos (o fragmentos representativos) en un Anexo, indicando siempre:
 - › modelo utilizado,
 - › fecha,
 - › preguntas realizadas,
 - › fragmentos de respuesta relevantes.

Estructura sugerida del informe (máx. 8 páginas, sin anexos)

a. Portada

- › Título de la actividad.
- › Nombre del/la estudiante.
- › Materia: Matemática para la Inteligencia Artificial.
- › Docente.
- › Fecha.

b. Introducción (½–1 página)

- › Presentar el objetivo del informe.
- › Explicar brevemente que el trabajo se realizó en diálogo con modelos de IA y que dichos diálogos se incluyen en un anexo.
- › Justificar la relevancia de estudiar funciones, crecimiento y continuidad en el contexto de IA.

c. Marco conceptual (2–3 páginas)

Desarrollar, con redacción propia:

- › Concepto de función: dominio, codominio, imagen; ejemplos cualitativos vinculados con IA (funciones de salida, funciones de activación, etc.).
- › Concepto de crecimiento y decrecimiento:
 - › definición formal basada en comparación de valores,
 - › ejemplos numéricos y gráficos sencillos (sin derivadas).
- › Concepto de continuidad:
 - › idea intuitiva,
 - › tipos de discontinuidades a nivel cualitativo (salto, removible, esencial),
 - › ejemplos como ReLU y sigmoide, sin cálculo formal de límites.

Se espera que utilice definiciones trabajadas en el material, apoyadas en la bibliografía y en fragmentos de las explicaciones obtenidas en los diálogos con IA (*debidamente reelaboradas*).

d. Aplicaciones en Inteligencia Artificial (2–3 páginas)

Relacionar los conceptos anteriores con ejemplos concretos, por ejemplo:

- › Cómo se interpreta la función de activación como función matemática (dominio, codominio, comportamiento creciente o no).
- › Qué implicancias tiene que una función sea continua para la estabilidad de un modelo.
- › Ejemplos cualitativos de sensibilidad:
 - › pequeñas variaciones en la entrada y cambios en la salida (robustez),
 - › regiones “planas” vs. regiones de cambio rápido (sin introducir todavía derivadas).

- › Comparación cualitativa de dos funciones usadas en IA (por ejemplo, ReLU vs. sigmoide) desde el punto de vista de crecimiento/continuidad.

e. Reflexión crítica sobre la co-construcción con IA (1–2 páginas)

Incluya una reflexión propia sobre:

- › Cómo se utilizaron los modelos de IA para aprender y organizar los conceptos.
- › Qué aportes positivos tuvieron (por ejemplo, ejemplos, analogías, aclaración de definiciones).
- › Qué limitaciones o inconsistencias se encontraron (si las hubo).
- › Qué criterio se aplicó para aceptar, modificar o descartar respuestas de los modelos en el informe final.

f. Conclusiones (½–1 página)

- › Recapitular las ideas principales sobre funciones, crecimiento y continuidad en IA.
- › Señalar qué aprendizajes matemáticos y meta–cognitivos (sobre el uso de IA como herramienta) se consideran más relevantes.

g. Referencias bibliográficas

- › Incluir las fuentes teóricas utilizadas (libros, artículos, material de cátedra, recursos web confiables).
- › Citar también, en un ítem separado, los modelos de IA utilizados (nombre del modelo, versión si se conoce, plataforma).

h. Anexos

- › Transcripción o exportación de los diálogos mantenidos con los modelos de IA.
- › Cada diálogo debe identificarse claramente con:
 - › Modelo,
 - › Fecha,
 - › Pregunta inicial y repreguntas,
 - › Fragmentos de respuesta usados explícitamente en el informe (*pueden señalizarse*).

Acerca de las fuentes de información y criterio de selección de temas

El informe debe contener una breve sección explícita (puede integrarse en la Introducción o en las Referencias) donde el/la estudiante:

1. Enumere las fuentes de información utilizadas, por ejemplo:
 - › Apuntes o diapositivas de la cátedra.
 - › Libros de matemática y de IA (indicando autor/a, año, título).
 - › Artículos o recursos web académicos.
 - › Modelos de IA consultados (ChatGPT, Copilot, otros).
2. Explique el criterio de selección de los temas tratados, por ejemplo:
 - › Relevancia directa para la materia “Matemática para la IA”.
 - › Relación clara con los ejes: función, crecimiento/decrecimiento, continuidad.
 - › Importancia de determinadas funciones en IA (*ReLU*, *sigmoide*, *funciones de costo*).
 - › Interés personal vinculado con proyectos propios (por ejemplo, modelos de clasificación, redes neuronales, etc.).
3. Justifique la elección de ejemplos en IA, indicando por qué esos casos permiten ilustrar mejor:
 - › la idea de función como modelo,
 - › el comportamiento creciente/decreciente,
 - › la continuidad y su impacto en la estabilidad del sistema.



ACTIVIDAD N° 3: Cálculo de límites

Calcule los siguientes límites, justificando brevemente el procedimiento utilizado en cada caso (sustitución directa, factorización, racionalización, división por la mayor potencia, análisis de límites laterales, etc.).:

Parte 1: Sustitución directa y polinomios

1. $\lim_{x \rightarrow -1} (2x^2 + 3x - 5)$
2. $\lim_{x \rightarrow 3} (x^3 - 4x + 1)$

Parte 2: Indeterminaciones $0/0$ y factorización

3. $\lim_{x \rightarrow 2} \frac{x^2 - 4}{x - 2}$
4. $\lim_{x \rightarrow 1} \frac{x^2 + x - 2}{x - 1}$
5. $\lim_{x \rightarrow -2} \frac{x^2 + 3x + 2}{x + 2}$

Parte 3: Racionalización

6. $\lim_{x \rightarrow 0} \frac{\sqrt{x+4} - 2}{x}$

$$7. \lim_{x \rightarrow 5} \frac{\sqrt{x} - \sqrt{5}}{x - 5}$$

Parte 4: Límites al infinito (funciones racionales)

$$8. \lim_{x \rightarrow +\infty} \frac{3x^2 + 1}{x^2 - x}$$

$$9. \lim_{x \rightarrow -\infty} \frac{5x^3 - 2}{x^2 + x}$$

$$10. \lim_{x \rightarrow +\infty} \frac{4x - 7}{2x + 1}$$

Parte 5: Límites laterales y funciones a trozos

Considere la función:

$$f(x) = \begin{cases} x + 1 & \text{si } x < 0, \\ x^2 & \text{si } x \geq 0. \end{cases}$$

$$11. \text{Calcule } \lim_{x \rightarrow 0^-} f(x).$$

$$12. \text{Calcule } \lim_{x \rightarrow 0^+} f(x).$$

$$13. \text{Determinar si } \lim_{x \rightarrow 0} f(x) \text{ existe. Justifique.}$$

Parte 6: Límites infinitos y asíntotas verticales

$$14. \lim_{x \rightarrow 1} \frac{1}{(x-1)^2}$$

$$15. \lim_{x \rightarrow 0^-} \frac{1}{x}, \lim_{x \rightarrow 0^+} \frac{1}{x}.$$

ACTIVIDAD 3 | Clave de corrección CC 1

$$1. 2(-1)^2 + 3(-1) - 5 = -6$$

$$2. 3^3 - 4 \cdot 3 + 1 = 10$$

$$3. 4$$

$$4. 3$$

$$5. 1$$

$$6. \frac{1}{4}$$

$$7. \frac{1}{2\sqrt{5}}$$

8. 3

9. 5

10. 2

11. 1

12. 0

13. El límite bilateral no existe (laterales distintos).

14. $+\infty$

15. $\lim_{x \rightarrow 0^-} \frac{1}{x} = -\infty$

$\lim_{x \rightarrow 0^+} \frac{1}{x} = +\infty$



ACTIVIDAD N° 4: Solo cálculo de derivadas

Calcule las derivadas de las siguientes funciones. Indique, cuando corresponda, el dominio donde la expresión de la derivada es válida.

Aclaración: No se piden interpretaciones gráficas ni aplicaciones: solo cálculo.

Parte 1: Derivadas básicas (polinomios y funciones simples)

$$f(x) = 4x^3 - 2x^2 + 7x - 5$$

$$g(x) = \frac{1}{x}, x \neq 0.$$

$$h(x) = \sqrt{x}, x > 0.$$

Parte 2: Productos y cocientes

$$f(x) = (x^2 + 1)(x - 3).$$

$$g(x) = \frac{3x-1}{x^2+1}.$$

$$h(x) = \frac{x^2}{x-2}, x \neq 2.$$

Parte 3 — Compuestas (regla de la cadena)

$$f(x) = (2x^3 - 1)^4.$$

$$g(x) = e^{5x^2}.$$

$$h(x) = \ln(3x + 2), x > -2/3.$$

Parte 4: Funciones trigonométricas

$$f(x) = \sin(2x).$$

$$g(x) = \cos(x^2).$$

$$h(x) = \sin x \cdot \cos x.$$

Parte 5: Funciones definidas a trozos y derivadas laterales

$$13. \quad f(x) = \begin{cases} x^2, & x < 1, \\ 2x - 1, & x \geq 1. \end{cases}$$

a) Calcule $f'(x)$ para $x < 1$ y para $x > 1$.

b) Calcule $f'_-(1)$ y $f'_+(1)$.

c) Determine si f es derivable en $x = 1$.

14.

$$g(x) = \begin{cases} -x, & x < 0, \\ x, & x \geq 0. \end{cases}$$

Repita el análisis anterior en $x = 0$.

ACTIVIDAD 4 | Clave de corrección CC 1

$$1. \quad f'(x) = 12x^2 - 4x + 7.$$

$$2. \quad g'(x) = -\frac{1}{x^2}.$$

$$3. \quad h'(x) = \frac{1}{2\sqrt{x}}.$$

$$4. \quad f'(x) = (2x)(x - 3) + (x^2 + 1) = 3x^2 - 6x + 1.$$

$$5. \quad g'(x) = \frac{3(x^2+1) - (3x-1)(2x)}{(x^2+1)^2}.$$

$$6. \quad h'(x) = \frac{2x(x-2) - x^2}{(x-2)^2} = \frac{x^2 - 4x}{(x-2)^2}.$$

$$7. \quad f'(x) = 4(2x^3 - 1)^3 \cdot 6x^2 = 24x^2(2x^3 - 1)^3.$$

$$8. \quad g'(x) = e^{5x^2} \cdot 10x.$$

$$9. \quad h'(x) = \frac{3}{3x+2}.$$

$$10. f'(x) = 2\cos(2x).$$

$$11. g'(x) = -\sin(x^2) \cdot 2x.$$

$$12. h'(x) = \cos^2 x - \sin^2 x.$$

13.

$$\triangleright \text{ Para } x < 1: f'(x) = 2x.$$

$$\triangleright \text{ Para } x > 1: f'(x) = 2.$$

$$\triangleright f'_-(1) = 2 \cdot 1 = 2, f'_+(1) = 2$$

$\Rightarrow$ derivable en 1 y $f'(1) = 2$.

14.

$$\triangleright \text{ Para } x < 0: g'(x) = -1.$$

$$\triangleright \text{ Para } x > 0: g'(x) = 1.$$

$$\triangleright g'_-(0) = -1, g'_+(0) = 1$$

$\Rightarrow$ no derivable en 0.



ACTIVIDAD N° 5: Solo cálculo de integrales

Calcule las integrales que se presentan a continuación. En las integrales definidas, utilice la regla de Barrow siempre que sea posible. No se requieren interpretaciones geométricas ni aplicaciones, solo el cálculo.

Parte 1: Integrales indefinidas básicas

$$1. \int (5x^3 - 2x + 1) dx$$

$$2. \int (3x^2 + \frac{1}{x}) dx, x > 0$$

$$3. \int (2e^x - 4) dx$$

$$4. \int (\cos x + 2\sin x) dx$$

Parte 2: Cambio de variable (indefinidas)

$$5. \int (x^2 + 1)^4 \cdot 2x dx$$

6. $\int \frac{2x}{x^2+4} dx$
7. $\int \frac{1}{\sqrt{1-x^2}} dx, |x| < 1$

Parte 3: Integración por partes (indefinidas)

8. $\int x \cos x dx$
9. $\int x \ln x dx, x > 0$

Parte 4: Integrales definidas (polinomios y racionales)

10. $\int_0^1 (3x^2 - 1) dx$
11. $\int_{-1}^2 (2x + 3) dx$
12. $\int_1^2 \frac{1}{x} dx$

Parte 5: Integrales definidas con cambio de variable o técnicas mixtas

13. $\int_0^1 4x(x^2 + 1) dx$
14. $\int_0^{\pi/2} \cos x dx$
15. $\int_0^1 \frac{2}{x^2+1} dx$

ACTIVIDAD 5 | Clave de corrección CC 1

1. $\frac{5}{4}x^4 - x^2 + x + C$
2. $x^3 + \ln x + C$
3. $2e^x - 4x + C$
4. $\sin x - 2\cos x + C$
5. $\frac{(x^2+1)^5}{5} + C$
6. $\ln(x^2 + 4) + C$
7. $\arcsin x + C$

$$8. x \sin x + \cos x + C$$

$$9. \frac{x^2}{2} \ln x - \frac{x^2}{4} + C$$

$$10. \int_0^1 (3x^2 - 1) dx = 0$$

$$11. \int_{-1}^2 (2x + 3) dx = 9$$

$$12. \int_1^2 \frac{1}{x} dx = \ln 2$$

$$13. \int_0^1 4x(x^2 + 1) dx = 3$$

$$14. \int_0^{\pi/2} \cos x dx = 1$$

$$15. \int_0^1 \frac{2}{x^2+1} dx = \frac{\pi}{2}$$



ACTIVIDAD N° 6:

Estudio de una función de pérdida cuadrática y desarrollo de una herramienta en Python

En esta actividad se trabajará con una función de pérdida cuadrática muy sencilla, asociada a un modelo de regresión lineal en una dimensión. El objetivo es que integre los conceptos de funciones, límites, continuidad, derivadas, integrales, extremos y convexidad, y luego diseñe un programa en Python que sirva como “asistente matemático” para este y otros casos similares.

Consideremos un modelo de regresión muy simple, donde la predicción es

$\hat{y}(x) = \theta x$, y queremos aproximar un valor objetivo fijo $y_0 \in \mathbb{R}$ en el intervalo de entradas $x \in [0,1]$.

La *pérdida cuadrática media* (promediada en $[0,1]$) para un parámetro θ se define como

$$L(\theta) = \int_0^1 (\theta x - y_0)^2 dx.$$

Para esta actividad, se fija $y_0 = 2$.

Por lo tanto, trabajaremos con la función:

$$L(\theta) = \int_0^1 (\theta x - 2)^2 dx.$$

Parte 1: Análisis simbólico y estudio de la función

1. Cálculo de la expresión cerrada de $L(\theta)$

- › Desarrolle el cuadrado $(\theta x - 2)^2$.
- › Calcule la integral $L(\theta) = \int_0^1 (\theta x - 2)^2 dx$ utilizando las reglas de integración aprendidas en el módulo.
- › Expresé el resultado final como un polinomio en θ .

2. Dominio, codominio e imagen

- › Determine el dominio de la función L .
- › Indique el codominio (como función desde $\mathbb{R}$ a $\mathbb{R}$).
- › Analice la imagen de L : valores que efectivamente toma. ¿Está acotada inferiormente? ¿Tiene cota superior?

3. Continuidad

Justifique, utilizando propiedades de funciones polinómicas o teoremas de continuidad, que $L(\theta)$ es continua en su dominio.

4. Límites y comportamiento asintótico

Calcule y analice:

$$\lim_{\theta \rightarrow +\infty} L(\theta), \lim_{\theta \rightarrow -\infty} L(\theta).$$

¿Qué concluye sobre el comportamiento de L para valores grandes (en módulo) de θ ? ¿Existen asíntotas horizontales u oblicuas?

5. Derivada y monotonía

- › Calcular la derivada $L'(\theta)$.
- › Hallar los puntos críticos (valores de θ donde $L'(\theta) = 0$).
- › Estudiar el signo de $L'(\theta)$ en los intervalos determinados por esos puntos críticos y concluir en qué intervalos L es creciente o decreciente.

6. Segunda derivada, curvatura y convexidad

- › Calcular la segunda derivada $L''(\theta)$.

- › Analizar el signo de $L''(\theta)$ y deducir si la función es cóncava, convexa o presenta cambios de concavidad.
- › Utilizar esta información para clasificar los puntos críticos encontrados: ¿corresponden a máximos, mínimos o puntos de inflexión?

7. Extremos y rango de valores posibles

- › Determine el (los) mínimo(s) global(es) de $L(\theta)$, si existen, y el valor mínimo de la función.
- › Discuta si existen máximos globales.
- › A partir de esto, describa el rango de valores posibles de la pérdida cuadrática media para este modelo.

8. Interpretación en términos de modelo

Interprete, en lenguaje propio, lo siguiente:

- › ¿Qué significa el valor de θ que minimiza $L(\theta)$ desde el punto de vista del modelo?
- › ¿Por qué es importante que $L(\theta)$ sea convexa para procesos de optimización basados en gradientes?

Parte 2: Desarrollo de un programa en Python como herramienta general

Se propone que desarrolle un programa en Python que funcione como una herramienta reutilizable para estudiar funciones de pérdida cuadrática similares a la anterior.

Requerimientos mínimos del programa

1. Entrada de parámetros por teclado

Permitir al usuario ingresar:

- › un valor objetivo y_0 (por defecto, $y_0 = 2$),
- › un intervalo para θ (por ejemplo, $[\theta_{\min}, \theta_{\max}]$),
- › opcionalmente, los límites del intervalo de integración en x (por defecto $[0,1]$).

2. Cálculo simbólico o numérico básico

Calcule la expresión de $L(\theta) = \int_a^b (\theta x - y_0)^2 dx$

de una de las siguientes maneras:

- › Opción A (quizás la que más recomendamos): usando directamente la fórmula analítica obtenida en la Parte 1.
- › Opción B: aproximando la integral mediante un método numérico sencillo (por ejemplo, regla del rectángulo o del trapecio) para cada valor de θ .

3. Cálculo de derivadas y mínimo

Implemente funciones en Python que, dadas las fórmulas de $L(\theta)$ y $L'(\theta)$, permitan:

- › evaluar $L(\theta)$ en distintos puntos del intervalo elegido;
- › evaluar $L'(\theta)$ y localizar aproximadamente el punto donde se anula (mínimo).

De manera opcional, incluya también el cálculo de $L''(\theta)$ para mostrar la convexidad.

4. Generación de gráficos

Use *matplotlib* para:

- › graficar la curva de $L(\theta)$ en el intervalo elegido;
- › marcar en el gráfico el punto donde la función alcanza su mínimo;
- › opcionalmente, graficar también $L'(\theta)$ para visualizar la monotonía.

5. Diseño “a medida del usuario”

- › El programa debe estar escrito de forma que sea fácil de modificar:
 - › la forma de la función de pérdida (por ejemplo, cambiar $(\theta x - y_0)^2$ por otra expresión);
 - › el intervalo de integración en x ;
 - › el intervalo de exploración en θ .
- › La idea es que el mismo código pueda servir para otros casos de estudio de funciones de pérdida o de funciones reales simples.

6. Documentación mínima en el código

Incluya comentarios breves explicando:

- › qué representa cada función de Python;
- › qué significan las variables principales;
- › qué relación hay entre el código y el análisis matemático de la Parte 1.

Entrega mínima:

- › Desarrollo teórico completo de la Parte 1, ordenado y justificado.
- › Archivo .py con el programa solicitado en la Parte 2.
- › Capturas de pantalla de los gráficos generados (o indicación clara de cómo generarlos).

Opcional: Extensiones

- › Permita que el programa reciba una lista de valores y_0 y compare las curvas $L(\theta)$ para distintos objetivos.
- › Incorpore un pequeño menú textual que permita elegir:
 - › solo cálculo numérico,
 - › solo gráficos,
 - › o ambos.

Deberá adaptar el programa para que, en lugar de un y_0 fijo, lea un pequeño conjunto de datos (x_i, y_i) y calcule la MSE discreta.

MÓDULO 3

Microobjetivos

- › Analizar distribuciones de probabilidad para modelar fenómenos aleatorios en el contexto de la toma de decisiones bajo incertidumbre en IA.
- › Aplicar el teorema de Bayes para actualizar creencias y estimar probabilidades posteriores en el contexto de problemas de clasificación y diagnóstico basados en datos.
- › Estimar parámetros mediante técnicas de inferencia estadística para fundamentar la construcción y evaluación de modelos matemáticos en el contexto de algoritmos de aprendizaje automático.
- › Evaluar la incertidumbre y variabilidad de modelos para determinar la confiabilidad de sus predicciones en el contexto del análisis estadístico aplicado a sistemas de IA.



VIDEO DOCENTE

Contenidos

- Probabilidad y estadística para la toma de decisiones bajo incertidumbre



Fuente: Elaboración propia con IA (Gemini)

La inteligencia artificial opera en entornos donde la información es incompleta, ruidosa o cambiante. Por ello, la capacidad de razonar bajo incertidumbre es un componente central en la construcción de modelos inteligentes. Este módulo introduce los fundamentos matemáticos que permiten cuantificar, modelar y gestionar esa incertidumbre mediante herramientas de probabilidad y estadística.

A lo largo de este recorrido se estudian las distribuciones de probabilidad como modelos para representar fenómenos aleatorios, el teorema de Bayes como mecanismo para actualizar creencias ante nueva evidencia, y los principios de inferencia estadística que permiten realizar conclusiones generales a partir de muestras.

Estos conceptos son pilares en múltiples áreas de la IA moderna: clasificación probabilística, filtrado de señales, análisis de datos, aprendizaje automático, visión por computadora, sistemas de recomendación y modelos predictivos.

El módulo propone un abordaje que integra el rigor matemático con aplicaciones prácticas, fortaleciendo sus capacidades para interpretar datos, tomar decisiones fundamentadas y comprender cómo los modelos de IA incorporan, procesan y responden a la incertidumbre. El dominio de estos principios no solo es imprescindible para el desarrollo técnico, sino también para ejercer un juicio crítico sobre la confiabilidad, los límites y el alcance de los sistemas inteligentes.

¡Comencemos!

» *Tema 1: Variables aleatorias discretas y continuas*

Las variables aleatorias son el puente entre la matemática y los datos reales. Se usan en IA para modelar fenómenos inciertos: sensores, señales, errores, clasificaciones, ruido, valores medidos, entre otras cosas.



¿Qué es una variable aleatoria?

Una variable aleatoria (VA) es una función que asigna un número a cada posible resultado de un experimento aleatorio.



Ilustración 2 - Ejemplos - VA - Elaboración propia con Napkin



Se lo puede pensar como un “valor numérico que no es fijo”, sino que depende del azar.

» *Tipos de variables aleatorias*

Tipo de variable aleatoria	Características	Ejemplos reales	Aplicaciones en IA
Discretas	Valores aislados, típicamente enteros	Cantidad de intentos de login (0, 1, 2, 3...); número de eventos detectados por un sensor de movimiento; cantidad de paquetes perdidos en una transmisión; número de veces que aparece una palabra en un texto	Conteo de palabras (Bag of Word); modelos de clasificación Naive Bayes (frecuencias); análisis de logs; procesos de Poisson (eventos en tiempo real)

<i>Continuas</i>	Infinitos valores dentro de un intervalo	intensidad de ruido en micrófonos; temperatura de un sensor; distancia estimada por un sensor ultrasónico; probabilidad predicha por una red neuronal	modelado de ruido (Normal, Exponencial); visión por computadora (niveles de iluminación); robótica (posición, velocidad, orientación)
------------------	--	---	---

Tabla 1 - VA Tipos - Elaboración propia

¿Por qué importa distinguirlas?

Porque cada tipo usa:

- › funciones distintas,
- › métodos distintos de cálculo,
- › distribuciones distintas.

Y esto cambia cómo se modela el problema en IA

Tipo de VA	Se usa...	Ejemplos en IA
<i>Discreta</i>	función de probabilidad ($P(X=x)$)	Conteo de palabras, aparición de eventos.
<i>Continua</i>	función de densidad ($f(x)$)	Ruido en sensores, valores reales

- Aplicación directa en IA
- › En Naive Bayes

Cada palabra es una variable aleatoria discreta (frecuencia).

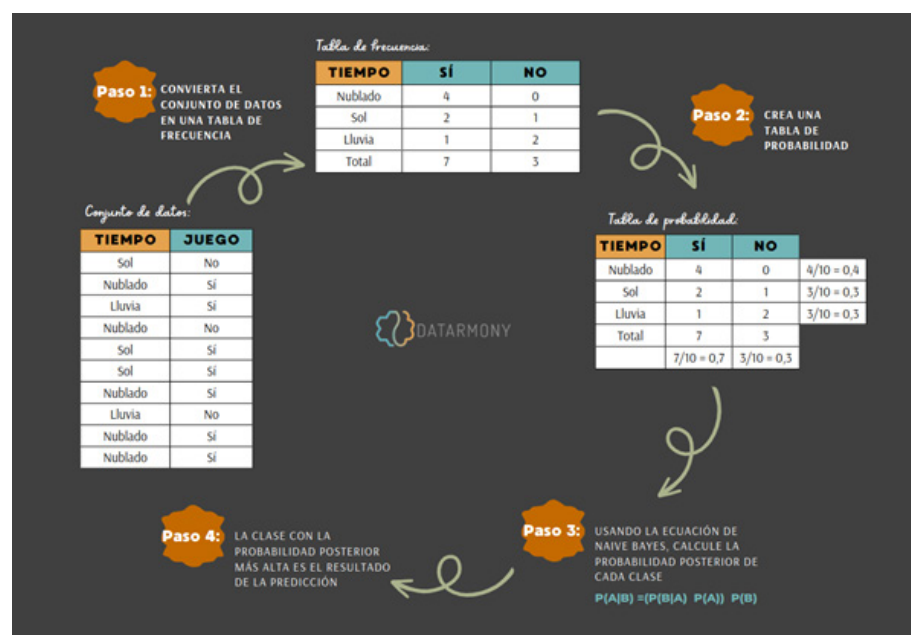


Ilustración 3 - Aplicación directa en IA - En Naive Bayes – Fuente: <https://tinyurl.com/4x7w849s>



Para seguir profundizando en el razonamiento completo le sugerimos leer la publicación del siguiente enlace: *Naive Bayes algoritmo, ejemplos, python | Significado y funcionamiento*

› *En visión por computadora*

Cada pixel puede modelarse como continua (iluminación) para detectar bordes, ruido y filtros.

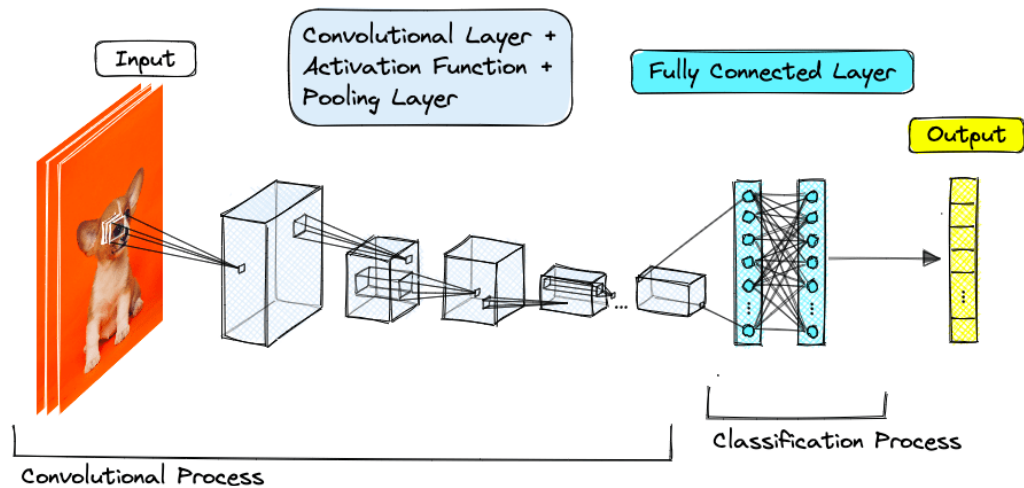


Ilustración 4 - Un diagrama de una red neuronal convolucional

Fuente: <https://tinyurl.com/4yf4p6ms>



Para para ahondar en este tema, recomendamos ver este video que explica de qué se trata la Visión por Computadora: *Tesis Doctoral sobre Visión Artificial*

También disponible desde:
<https://tinyurl.com/5n8j86mp>



Para seguir viendo el razonamiento completo se sugiere leer las publicaciones de los enlaces:

- › Visión artificial
- › Computer Vision: ¿Cómo las Máquinas Aprendieron a Ver?

• *En redes neuronales*

Los pesos y activaciones son variables continuas con distribuciones asociadas (*Normal para inicialización, por ejemplo*).

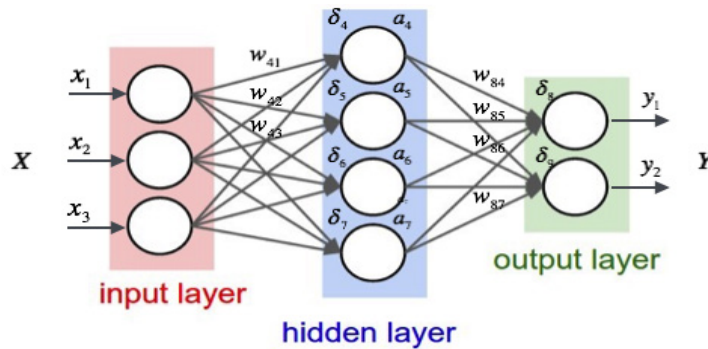


Ilustración 5 - "One gorgeous Neural Net" From <https://link.medium.com/7B6zUEsCoU>



Lectura
complementaria

Para ver el razonamiento completo, se sugiere leer: *Redes Neuronales: Un ejemplo práctico en R*

- En sensores robóticos

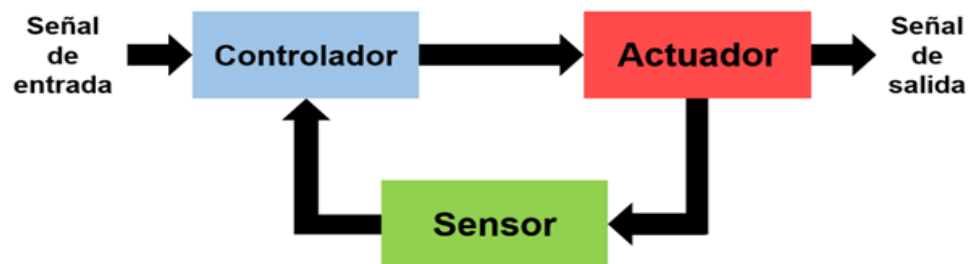


Ilustración 6 - Sensores - Fuente: <https://tinyurl.com/z6k65sxw>

- *Función de Probabilidad y Función de Densidad*

Este tema define cómo se comporta una variable aleatoria. Cualquier distribución, Normal, Bernoulli, Poisson, por listar las más conocidas, describen esta situación.

- *Función de Probabilidad (PMF)*

- › *Para variables aleatorias discretas*

Una función de probabilidad o PMF (Probability Mass Function) asigna a cada valor posible de una variable aleatoria discreta la probabilidad de que ese valor ocurra.

Se escribe como:

$$P(X = x)$$

Veamos un ejemplo:

- › X = cantidad de intentos fallidos de login en un servidor
- › Valores posibles: 0, 1, 2, 3, ...

La PMF te dice:

$$P(X = 0), P(X = 1), P(X = 2), \dots$$

Para que sea válida, se debe cumplir:

$$\sum_x P(X = x) = 1$$

Veamos un ejemplo:

Supongamos que un sistema registra:

- › 0 fallos → 50%
- › 1 fallo → 30%
- › 2 fallos → 15%
- › 3 fallos → 5%

La tabla nos quedaría de esta manera:

x	P(X=x)
0	0.50
1	0.30
2	0.15
3	0.05

Aplicaciones de PMF en IA



Naive Bayes

Frecuencia de palabras convertida en PMF para clasificación.

Distribución de Poisson utilizada en visión, colas y tráfico.

Modelos de conteo



Procesos de eventos discretos

Registros de logs y paquetes de red analizados como eventos.

Datos categóricos utilizados para entrenamiento de modelos.

Aprendizaje supervisado



Ilustración 7 - PMF en IA - Elaboración propia con Napkin

- *Función de Densidad (PDF)*
- › *Para variables aleatorias continuas*

Para variables continuas, no podemos hablar de “probabilidad exacta de un valor” porque:

$$P(X = x) = 0 \text{ (en cualquier VA continua)}$$

Por eso, usamos una función de densidad o PDF (Probability Density Function), escrita como: $f(x)$



Énfasis

Esta densidad describe qué tan probable es encontrar valores alrededor de un punto.

La probabilidad de que X esté entre a y b se calcula usando un área:

$$P(a < X < b) = \int_a^b f(x) \, dx$$

Veámoslo con un ejemplo. Supongamos que estamos analizando el valor del ruido de un sensor ultrasónico (mm):

- › La mayoría de los valores se concentran alrededor de la medida real
- › Valores alejados son menos frecuentes

Esto generalmente se modela con una Normal:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)}$$

Donde:

μ : valor central (promedio del sensor)

σ : nivel de ruido



Multimedia

Para seguir el razonamiento completo se sugiere ver el siguiente video: *Teoría De La Distribución Normal*

También disponible desde:
<https://tinyurl.com/y2bt7ue>



Importancia de la Distribución Normal en IA

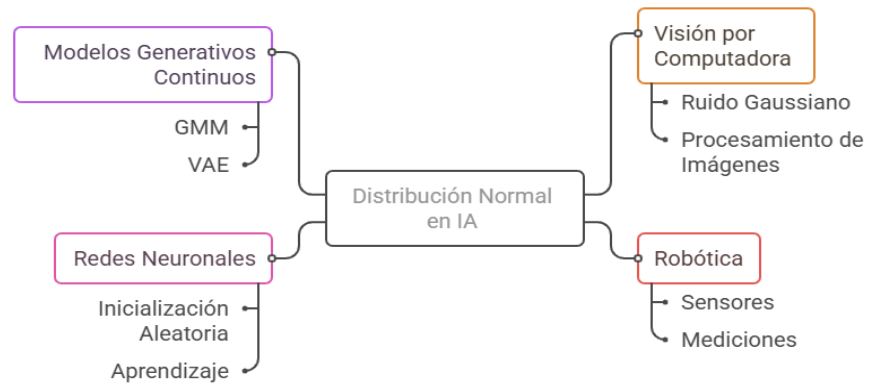


Ilustración 8 - PDF en IA – Elaboración propia con Napkin

Les dejamos a continuación una tabla resumen para distinguir rápidamente las diferencias entre ambas funciones:

Concepto	Discreta	Continua
Nombre	PMF	PDF
Notación	$(P(X=x))$	$(f(x))$
Probabilidad exacta	Válida	Siempre cero
Se obtiene probabilidad de un intervalo	Suma	Integral
Aparece en	Conteos; categorías	Mediciones reales; ruido



PMF da probabilidades; PDF da densidades.

En IA, tanto las PMFs como las PDFs tienen aplicación directa. Veamos la siguiente tabla:

Aplicación	Uso de PMF/PDF	Descripción
Modelos probabilísticos de clasificación	PMF o PDF	Cada característica puede ser una PMF o PDF según discreta o continua
Modelado de ruido en sensores	PDFs	PDFs describen cómo varía la medida real
Inicialización de redes neuronales	Normal o Uniforme → PDF	Pesos iniciales se extraen de una Normal o Uniforme
Supuestos distribucionales en ML	PDFs	Regresión logística y lineal parten de supuestos sobre PDFs
Métodos de Monte Carlo	PMFs o PDFs	Se generan valores desde PMFs o PDFs para simular escenarios

Ahora, nos dedicaremos a las **Distribuciones Fundamentales**, uno de los núcleos conceptuales centrales para la IA, porque estas distribuciones aparecen en cuestiones como:

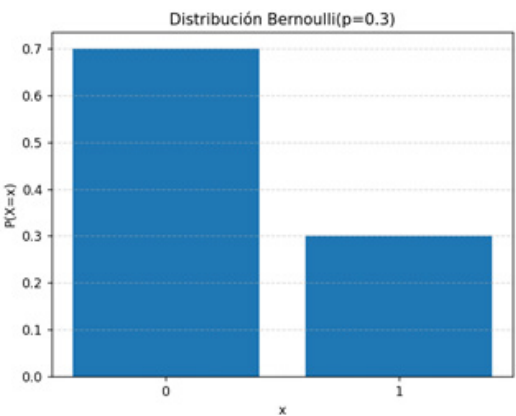
- › modelado de ruido,
- › clasificación probabilística,
- › detección de eventos,
- › estimación de parámetros,
- › inicialización de redes neuronales,
- › algoritmos de Monte Carlo,
- › filtrado bayesiano en robótica,
- › visión por computadora,
- › modelado de conteos y tiempos.

- *Distribuciones Fundamentales: Bernoulli, Binomial, Poisson, Uniforme, Normal, Exponencial*



Código

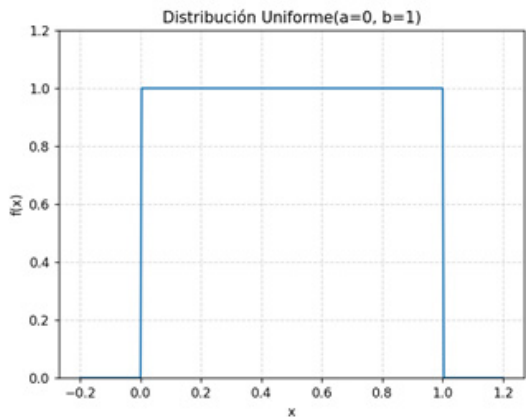
Para poder hacer las gráficas que representan a las distribuciones fundamentales se desarrolló un programa sencillo en Python que grafica la distribución que se le solicita. Aquí cuenta con el enlace de descarga: <https://tinyurl.com/479safvf>

Distribución Bernoulli <i>La más simple y la base de todas las demás</i>	 Gráfica 1 - Distribución Bernoulli(p) - Parámetro hardcoded: $p=0.3$ Fuente: https://tinyurl.com/479safvf	
Definición	Ejemplo	Aplicación en IA
Una variable aleatoria Bernoulli toma solo dos valores: › 1 → éxito › 0 → fracaso Con probabilidades: › $P(X=1)=p$, › $P(X=0)=1-p$	› El correo es spam (1) o no (0). › Un paquete llega dañado (1) o no (0). › Un sensor detecta movimiento (1) o no (0). › Resultado positivo/negativo en un modelo simple.	› Clasificación binaria básica. › Variables de presencia/ausencia en Naive Bayes. › Representación de activaciones discretas.

Distribución Binomial Suma de muchos Bernoulli	<table border="1"><caption>Data for Gráfica 2 - Distribución Binomial(n=10, p=0.5)</caption><thead><tr><th>k</th><th>P(X=k)</th></tr></thead><tbody><tr><td>0</td><td>0.001</td></tr><tr><td>1</td><td>0.011</td></tr><tr><td>2</td><td>0.044</td></tr><tr><td>3</td><td>0.105</td></tr><tr><td>4</td><td>0.205</td></tr><tr><td>5</td><td>0.246</td></tr><tr><td>6</td><td>0.205</td></tr><tr><td>7</td><td>0.105</td></tr><tr><td>8</td><td>0.044</td></tr><tr><td>9</td><td>0.011</td></tr><tr><td>10</td><td>0.001</td></tr></tbody></table>		k	P(X=k)	0	0.001	1	0.011	2	0.044	3	0.105	4	0.205	5	0.246	6	0.205	7	0.105	8	0.044	9	0.011	10	0.001
k	P(X=k)																									
0	0.001																									
1	0.011																									
2	0.044																									
3	0.105																									
4	0.205																									
5	0.246																									
6	0.205																									
7	0.105																									
8	0.044																									
9	0.011																									
10	0.001																									
Gráfica 2 - Distribución Binomial(n, p) - Parámetros hardcodedos: $n = 10, p = 0.5$ – Fuente: https://tinyurl.com/479safvf																										
Definición	Ejemplo	Aplicación en IA																								
Modelo de n ensayos independientes, cada uno con probabilidad p de éxito $X \sim \text{Bin}(n, p)$ $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$	› Cantidad de correos spam en un lote de n correos. › Cantidad de aciertos del detector de intrusos. › Cuántos paquetes llegaron correctamente en n transmisiones.	› Modelado de precisión/acierto en experimentos. › Evaluación de clasificadores (<i>true positives, false positives</i>). › Simulación de resultados de modelos probabilísticos																								

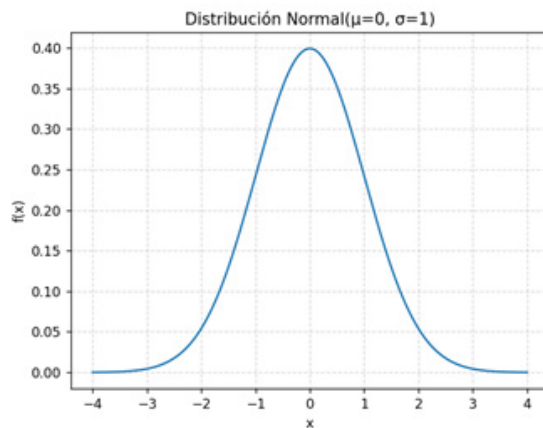
Distribución Poisson <i>–Conteo de eventos raros en un intervalo–</i>	Gráfica 3 - Distribución Poisson(λ) - Parámetro hardcodedo: $\lambda = 4$ – Fuente: https://tinyurl.com/479safvf
--	--

Definición	Ejemplo	Aplicación en IA
Usada para modelar eventos que ocurren con baja probabilidad, pero que pueden ocurrir varias veces como: $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$ Donde λ = tasa esperada de eventos.	› Número de accesos sospechosos por minuto en una API. › Cantidad de fallos de un sensor por hora. › Número de ocurrencias de una palabra rara por documento.	› Modelado de spam, ataques o eventos anómalos. › Conteo en minería de texto. › Procesos de llegada (redes, tráfico, logs).

Distribución Uniforme <i>Todos los valores tienen igual probabilidad</i>	 Gráfica 4 - Distribución Uniforme continua $U(a, b)$ - Parámetros hardcoded: $a = 0, b = 1$ – Fuente: https://tinyurl.com/479safvf	
Definición	Ejemplo	Aplicación en IA
Si X es uniforme en el intervalo $[a, b]$: $f(x) = \frac{1}{b - a}$	› Inicialización aleatoria de pesos en redes neuronales. › Selección aleatoria de píxeles o muestras. › Sampling básico en simulaciones.	› Métodos de Monte Carlo. › Perturbaciones aleatorias controladas. › Random search en optimización de hiperparámetros.

Distribución Normal

La más importante en IA para variables continuas

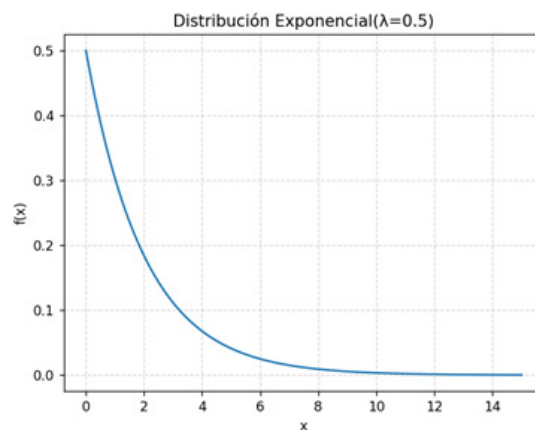


Gráfica 5 - Distribución Normal (μ, σ) - Parámetros hardcodedo: $\mu = 0, \sigma = 1$ – Fuente: <https://tinyurl.com/479safvf>

Definición	Ejemplo	Aplicación en IA
Si X es uniforme en el intervalo $[a, b]$: $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	› Ruido en sensores robóticos. › Niveles de iluminación en imágenes. › Error de predicción de modelos. › Activaciones y pesos iniciales en redes neuronales.	› Filtros de Kalman. › Modelos generativos (mixturas de gaussianas). › Estadística inferencial. › Regularización y inicialización de redes neuronales.

Distribución Exponencial

Tiempos entre eventos; memoria "sin recuerdo"



Gráfica 6 - Distribución Exponencial(λ) - Parámetro hardcodedo: $\lambda = 0.5$ – Fuente: <https://tinyurl.com/479safvf>

Definición	Ejemplo	Aplicación en IA
$f(x) = \lambda e^{-\lambda x}$	Modela tiempos entre eventos independientes, como: › tiempo entre dos accesos al sistema › tiempo entre fallos › tiempo hasta el próximo evento raro Propiedad clave - Sin memoria: El tiempo futuro no depende del tiempo ya transcurrido.	› Modelado de tiempos de respuesta. › Simulación de procesos estocásticos. › Algoritmos de espera y colas. › Procesos Poisson en visión y robótica.

A modo de resumen, le proporcionamos la siguiente tabla:

Distribución	Tipo	¿Cuándo se usa?	Ejemplo en IA
<i>Bernoulli</i>	Discreta	Éxito/fracaso	Spam / no spam
<i>Binomial</i>	Discreta	Conteo de éxitos	Métricas (TP, FP)
<i>Poisson</i>	Discreta	Eventos raros	Ataques, logs
<i>Uniforme</i>	Continua	Sin preferencia	Inicializar redes
<i>Normal</i>	Continua	Ruido, mediciones	Sensores, imágenes
<i>Exponencial</i>	Continua	Tiempo entre eventos	Tiempos de respuesta

Tabla 2 - Distribuciones – Elaboración propia



Estas seis distribuciones forman el lenguaje matemático básico para modelar ruido, eventos, mediciones, tiempos y conteos en IA.

- **Esperanza, Varianza y Momentos**

Estos tres conceptos describen *cómo se comporta una variable aleatoria*:

- › su valor típico (esperanza),
- › su dispersión (varianza),
- › y su forma (momentos).

Son esenciales para interpretar datos, calibrar modelos y entender algoritmos de IA.

• **Esperanza o Valor Esperado**

Definición	¿De qué trata?	Aplicación en IA
El valor esperado de una variable aleatoria es su promedio teórico. Discreta: $E[X] = \sum_x x P(X = x)$ Continua: $E[X] = \int_{-\infty}^{\infty} x f(x) dx$ Es el “centro de gravedad” de la distribución.	› Es lo que se esperaría ver en promedio si se pudiera repetir el experimento miles de veces. › Si los datos son un sensor, es la lectura típica. › Si es un clasificador, es la media de sus probabilidades	› Intensidad media de ruido en imágenes. › Promedio de pérdida (loss) en entrenamiento. › Valor esperado de activaciones en redes neuronales (para normalización). › Frecuencia media de aparición de palabras en texto.

• **Varianza**

Definición	¿De qué trata?	Aplicación en IA
La varianza mide cuánta variación hay alrededor de la esperanza. Discreta: $Var(X) = E[(X - \mu)^2]$ Continua: $Var(X) = \int (x - \mu)^2 f(x) dx$ La desviación estándar es: $\sigma = \sqrt{Var(X)}$	› Si los datos están muy dispersos entonces se tiene Alta varianza. › Baja varianza, cuando los datos están compactos, más previsibles. › En redes neuronales, una mala varianza en pesos puede bloquear el aprendizaje (exploding/vanishing gradients).	› Ruido del sensor = varianza del error. › Varianza en la distribución de pesos iniciales afecta estabilidad. › Evaluación de modelos: la varianza mide la consistencia del clasificador.

- **Momentos**

Definición	¿De qué trata?
Los momentos describen características más profundas de una distribución. › Momento 1 → <i>Esperanza</i> - “centro” › Momento 2 → <i>Varianza</i> - “dispersión” › Momento 3 → <i>Skewness</i> (asimetría) ¿la cola es más hacia la derecha o hacia la izquierda? › Momento 4 → <i>Kurtosis</i> (apuntamiento) ¿la distribución tiene colas pesadas (valores extremos frecuentes)?	› Los momentos permiten saber: › si los datos están sesgados (importante en aprendizaje supervisado), › si hay valores extremos que afectan entrenamiento, › si la distribución es apta para modelos lineales o no.

¿Por qué estos conceptos son esenciales en IA?

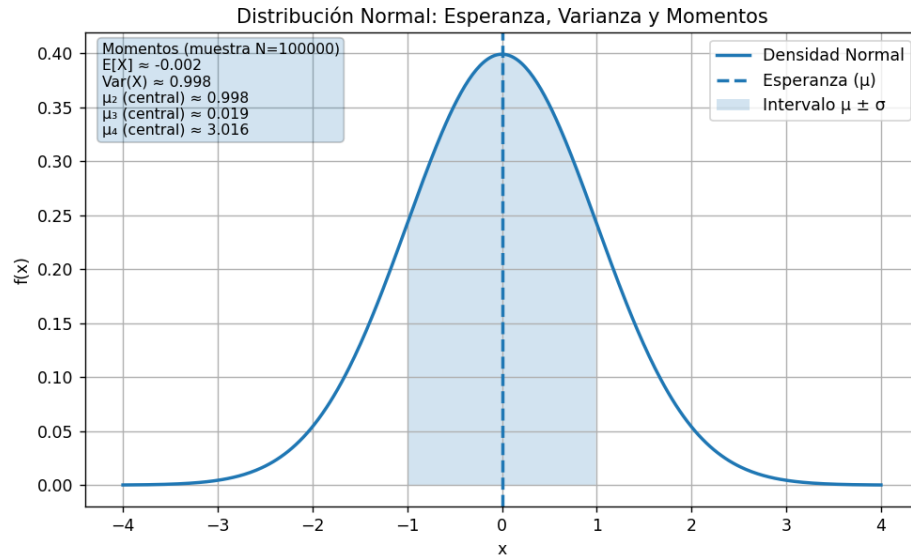
La siguiente tabla ilustra cómo los conceptos de media y varianza, fundamentales en estadística, se aplican y resultan esenciales en diferentes áreas de la inteligencia artificial. Se destacan ejemplos concretos en campos como redes neuronales, visión por computadora, clasificación probabilística, métodos Monte Carlo y modelos bayesianos, mostrando cómo estas métricas influyen en el funcionamiento, la confiabilidad y la interpretación de los resultados en cada contexto.

Área	Media	Varianza	Aplicación/Comentario
Redes neuronales	0	Controlada	Evita gradientes explosivos o nulos
Visión por computadora	Brillo medio = expectativa	Ruido se modela con varianza	
Probabilidades para clasificación	Esperanza de Bernoulli = p = probabilidad de éxito	Indica si un atributo es confiable	
Monte Carlo	Expectativa se estima simulando escenarios	Error depende de la varianza	
Modelos bayesianos	Cada posterior tiene su media	Cada posterior tiene su varianza	Indica el nivel de “certeza” del modelo

Tabla 3 Conceptos Básicos para IA – Elaboración propia



Para poder hacer las gráficas que representan las medidas en distribuciones fundamentales se desarrolló un programa sencillo en Python que grafica la distribución que se le solicita. Este es el enlace de descarga: <https://tinyurl.com/479safvf>



Gráfica 7—Medidas en la Normal—Propia - Fuente: <https://tinyurl.com/479safvf>



Énfasis

Para recordar: La esperanza dice dónde están los datos; la varianza dice qué tan dispersos están; los momentos dicen cómo es la forma completa de la distribución.

- *Muestreo y generación de datos a partir de distribuciones*

Generar datos desde distribuciones probabilísticas permite simular fenómenos reales, probar modelos, crear datasets artificiales, analizar el comportamiento de un algoritmo y entender procesos aleatorios.

Ahora bien: ¿Qué significa “muestrear” de una distribución?



Énfasis

Muestrear significa generar valores aleatorios siguiendo la forma definida por una distribución concreta (Normal, Poisson, Uniforme, etc.).

Aquí dejamos un ejemplo intuitivo para ilustrarlo:

- › Si muestreamos de una Normal centrada en 0, veremos valores cerca de 0, algunos alejados, muy pocos extremos.
- › Si muestreamos de una Uniforme entre 0 y 1, todos los valores serán igualmente probables.
- › Si muestreamos de una Poisson con $\lambda=3$, obtendremos conteos como 2,3,4 con más frecuencia.



Énfasis

Muestrear = crear datos que respetan el patrón probabilístico de la distribución.

- › *¿Por qué es tan importante en IA?*

Porque en IA todo lo que es aleatorio proviene de una distribución, por ejemplo:

- › Inicialización de pesos en redes neuronales
- › Ruido sintético para robustez

- › Simulación de sensores ruidosos (robótica)
- › Monte Carlo para estimar probabilidades
- › Generación de datos sintéticos para entrenar modelos
- › Validación estadística (bootstrapping, random sampling)
- › Selección aleatoria de muestras (data splitting)

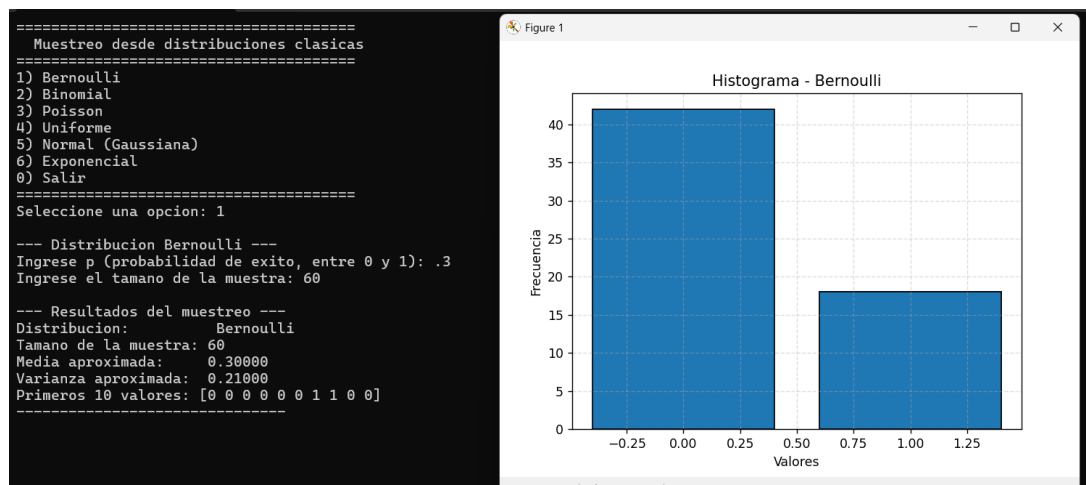
Los modelos probabilísticos necesitan simular datos para funcionar o evaluarse. Veamos algunos códigos en Python que nos permiten “muestrear”:



Código

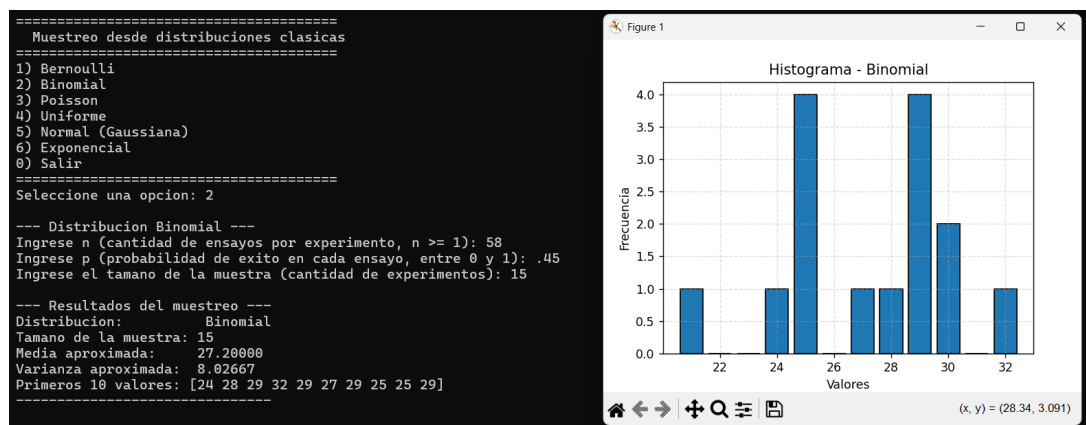
A continuación, el enlace de descarga: <https://tinyurl.com/479safvf>

- *Bernoulli (0 o 1)*
- › *Simular éxito/fracaso*



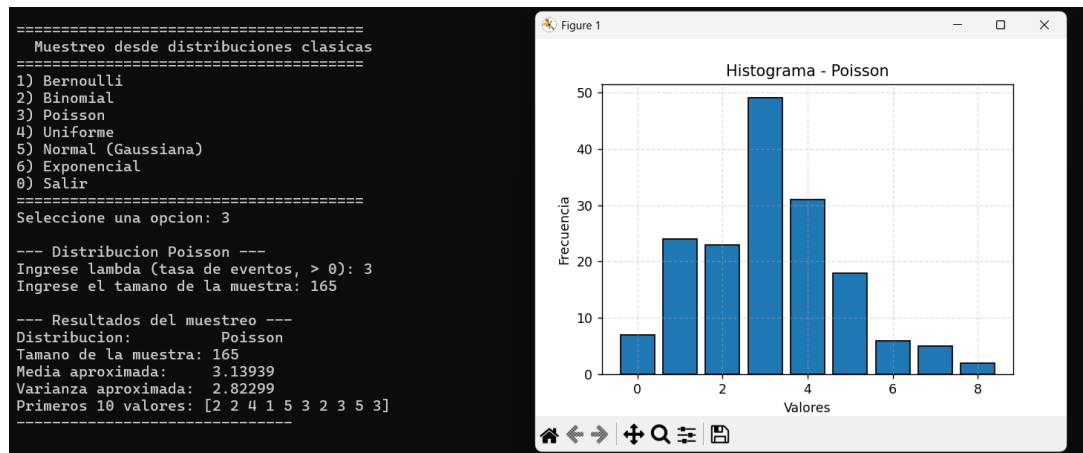
Gráfica 8 - Histograma - Muestreo Bernoulli - Elaboración propia

- *Binomial (conteo de éxitos)*
- › *Cantidad de éxitos en n ensayos:*



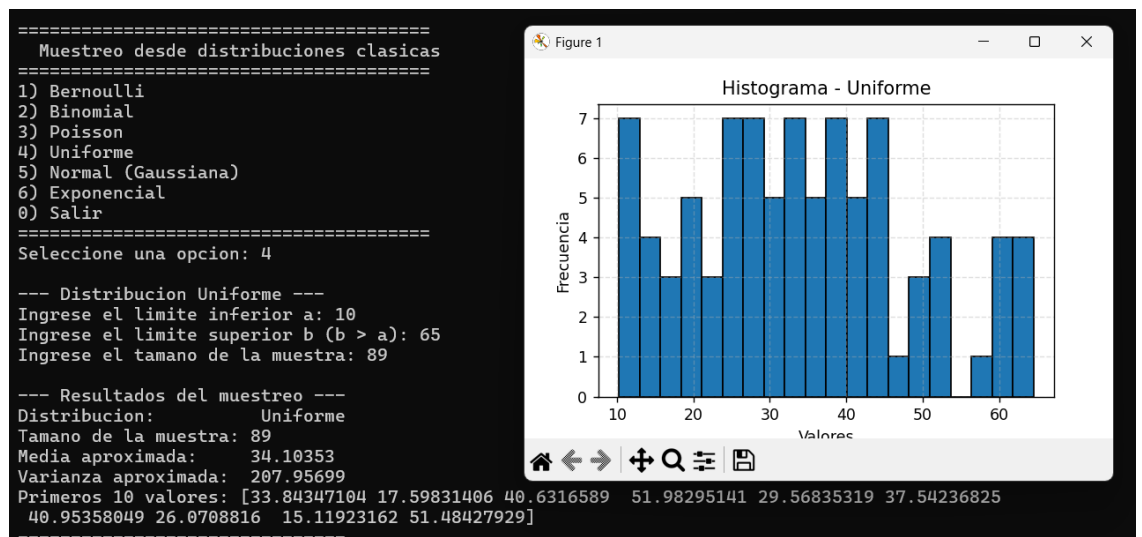
Gráfica 9 - Histograma - Muestreo Binomial - Elaboración propia

- *Poisson (eventos por intervalo)*



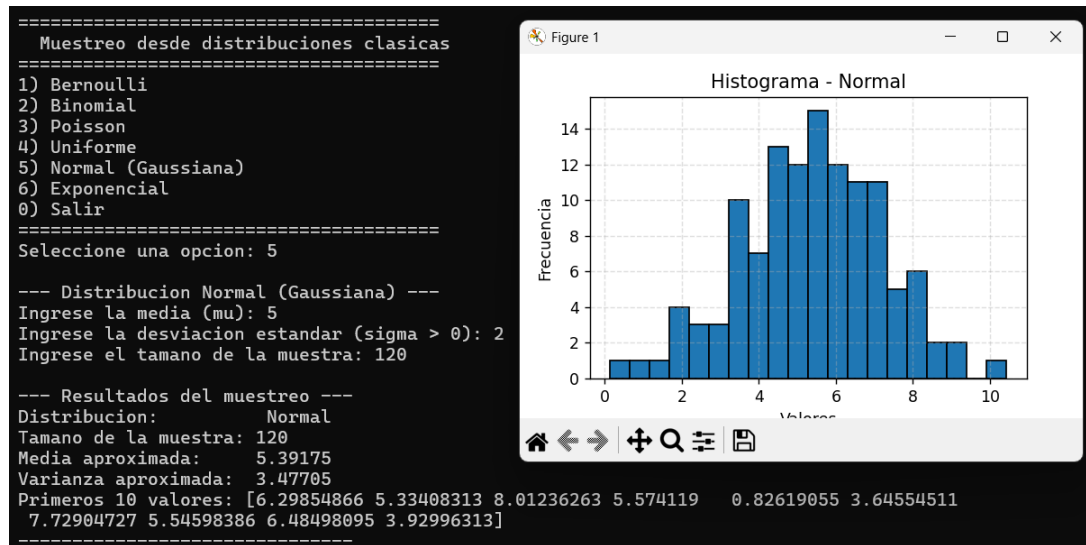
Gráfica 10 - Histograma - Muestreo Poisson – Elaboración propia

- *Uniforme (todos los valores igual probabilidad)*



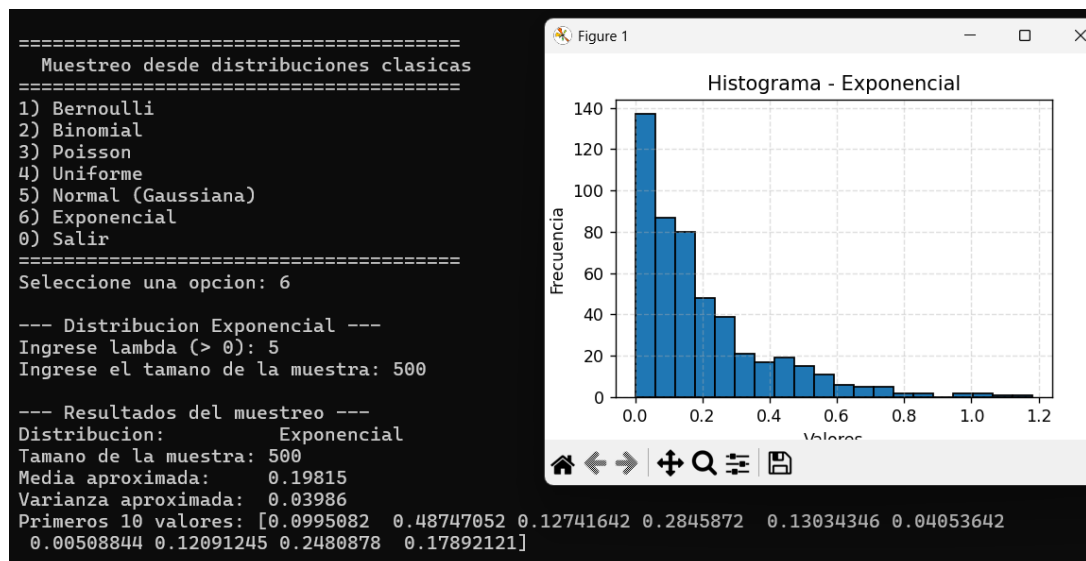
Gráfica 11 - Histograma - Muestreo Uniforme - Elaboración propia

- Normal (ruido en sensores)



Gráfica 12 - Histograma - Muestreo Normal - Elaboración propia

- Exponencial (tiempos entre eventos)



Gráfica 13 - Histograma - Muestreo Exponencial - Elaboración propia

Ahora nos haremos otra pregunta: ¿Qué significa “generar datos sintéticos”? se trata de crear datos falsos —pero realistas— usando distribuciones.

Algunos ejemplos:

- › Generar imágenes ruidosas para entrenar filtros.
- › Simular lecturas de un sensor cuando no se tiene hardware disponible.
- › Crear un dataset de pruebas para Naive Bayes.
- › Simular usuarios, interacciones o eventos de log.

Los datos sintéticos deben respetar la distribución real o una aproximación razonable.

› **Muestreo Monte Carlo** (La técnica más usada para simulación en IA)

La técnica de Monte Carlo es ampliamente utilizada en simulación dentro del campo de la inteligencia artificial y la estadística.

Su proceso fundamental se basa en tres pasos principales:

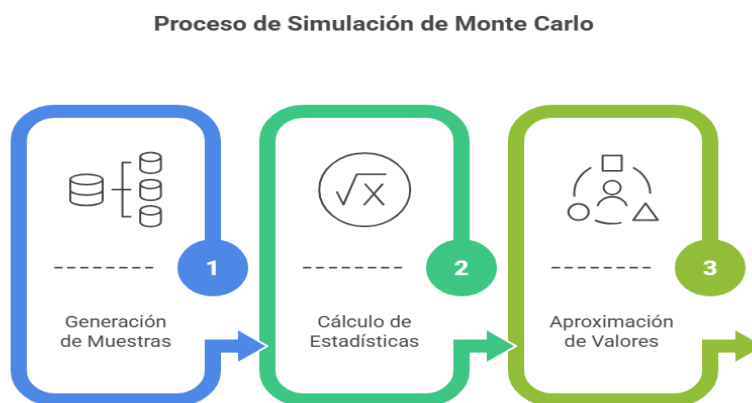


Ilustración 9- Pasos Monte Carlo - Elaboración propia con Napkin

1. **Generación de muestras:** Se crean miles de muestras a partir de una determinada distribución, representando posibles escenarios o resultados.
2. **Cálculo de estadísticas o probabilidades:** Sobre estas muestras generadas, se calculan estadísticas relevantes, probabilidades o resultados específicos que se desean analizar.
3. **Aproximación de valores:** Se utilizan los resultados obtenidos para aproximar valores que, por su complejidad, no pueden calcularse de manera exacta mediante métodos analíticos.

Algunos ejemplos conceptuales de este método son:

- › Para determinar la probabilidad de colisión de un robot bajo condiciones de incertidumbre, se simulan miles de posiciones posibles y se analiza la frecuencia de colisión.
- › Para estimar el error de un modelo, se generan nuevos datos sintéticos y se evalúa el desempeño del modelo sobre estos datos simulados.
- › Para calcular probabilidades complejas, se utilizan simulaciones para obtener aproximaciones mediante la repetición y análisis de los resultados obtenidos.



Énfasis

Muestrear distribuciones permite simular la realidad. Sin muestreo no hay Monte Carlo, no hay inicialización aleatoria y no hay modelos probabilísticos funcionales.



Llegando a este punto, y antes de continuar, lo/a invitamos a realizar la **Actividad 1: Muestreo y visualización de distribuciones**.

Tema 2: Distribuciones de probabilidad

- *Probabilidad condicional y Regla del Producto*

› Concepto básico

La probabilidad condicional mide la probabilidad de que ocurra un evento **A**, dado que sabemos que ocurrió un evento **B**. Se expresa como:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$



Énfasis

Es decir, qué tan probable es A considerando que B ya ocurrió.

- *Regla del producto*

A partir de la probabilidad condicional se define:

$$P(A \cap B) = P(B) P(A | B)$$

Esta expresión es clave porque permite descomponer problemas complejos en pasos probabilísticos simples.



Énfasis

Es el fundamento para todos los modelos bayesianos y para algoritmos como Naive Bayes.

- *Interpretación técnica para estudiantes de IA*

Para facilitar la comprensión de la probabilidad condicional en contextos informáticos, es útil establecer paralelos entre los conceptos matemáticos y las estructuras lógicas que usted como estudiante ya maneja en programación y análisis de datos. La siguiente enumeración traduce ideas fundamentales de la teoría de probabilidad a un lenguaje operativo, mostrando cómo estos conceptos se reflejan en decisiones, filtros y funciones que se utilizamos diariamente en el desarrollo de software, tales como:

- › “**A B**” representa la intersección de eventos, equivalente a pensar en condiciones combinadas en programación (**A & B**).
- › “**Condicionar**” equivale a incorporar *información parcial* (como filtrar datos en una base o aplicar un “if” que reduce el dominio).
- › La regla del producto establece un mecanismo para encadenar eventos, tal como encadenar funciones que dependen unas de otras.

- **Caso 1: Aplicación a la Ciberseguridad**

Veamos la siguiente situación en el campo de la ciberseguridad donde se detecta una actividad inusual (evento B). Queremos conocer la probabilidad de que sea un ataque real (evento A).

Supongamos:

- › $P(A) = 0.02 \Rightarrow 2\%$ de eventos son realmente ataques
- › $P(B | A) = 0.9 \Rightarrow$ si es ataque, 90% de las veces el detector lo marca
- › $P(B) = 0.1 \Rightarrow 10\%$ de todas las actividades activan la alerta

Usamos la regla del producto:

$$P(A \cap B) = P(B | A)P(A) = 0.9 \times 0.02 = 0.018$$

Interpretación: 1.8% de todas las actividades registradas corresponden a “ataque y alerta al mismo tiempo”.

- **Caso 2 – Aplicación en IA**

En un filtro bayesiano para robótica:

- › A = “el robot está realmente en la posición X”
- › B = “el sensor devuelve lectura cercana a X”

La regla del producto permite actualizar la ubicación probable del robot a partir de sus sensores, aun cuando haya ruido.



Lectura
complementaria

Para ver el razonamiento completo, se sugiere leer la publicación del enlace [The Bayes Filter for Robotic State Estimation](#)

- **Aplicaciones en IA**

1. **Detección de anomalías**

En muchos sistemas (redes, sensores, transacciones), se busca identificar comportamientos que *no siguen el patrón habitual*. La probabilidad condicional permite estimar si una observación es poco probable dado el comportamiento normal del sistema.

Por ejemplo: “Si el usuario inicia sesión desde un país no usual, ¿cuál es la probabilidad de que sea un acceso legítimo?”

2. **Modelos de recomendación basados en co-ocurrencias**

Los recomendadores simples trabajan analizando qué elementos aparecen juntos con frecuencia. La probabilidad condicional permite estimar la probabilidad de que un usuario quiera un ítem A dado que ya eligió el ítem B.

Por ejemplo: “Si una persona compró un teclado, ¿qué probabilidad hay de que también compre un mouse?”

3. Análisis de logs en ciberseguridad

Los sistemas de seguridad utilizan probabilidades condicionales para evaluar si cierta combinación de eventos en logs es sospechosa.

Otro ejemplo: “Si hay 5 intentos fallidos de acceso y luego un intento exitoso, ¿qué tan probable es que se trate de un ataque?”

4. Preprocesamiento bayesiano de señales

En robótica, domótica o visión por computadora, los sensores devuelven datos ruidosos. Se usan modelos probabilísticos para estimar la señal real condicionada a la lectura observada.

Ejemplo: “Dado el ruido del sensor, ¿cuál es la posición más probable del robot?”

5. Motores de decisión en agentes inteligentes

Un agente debe elegir acciones basadas en probabilidades condicionadas a su percepción del entorno.

Ejemplo: “Si el agente detecta un obstáculo a la derecha, ¿cuál es la probabilidad de que avanzar sea seguro?”

- *Independencia e independencia condicional*

Revisemos el concepto de **independencia**: Dos eventos **A** y **B** son independientes cuando el hecho de que ocurra uno **no altera** la probabilidad del otro.

Formalmente:

$$P(A \mid B) = P(A)$$

Esto implica que:

$$P(A \cap B) = P(A) P(B)$$

Si tuviéramos que analizarlo técnicamente sería similar a decir que dos variables no comparten información relevante o que las tablas en una base de datos no tengan campos en común. Muchas arquitecturas de IA suponen independencia para simplificar cálculos (lo veremos en Naive Bayes).

- *Independencia condicional*

Dos eventos A y B pueden no ser independientes en general, pero sí pueden ser independientes dado un tercer evento C.

Esto se expresa como:

$$P(A \mid B, C) = P(A \mid C)$$



Énfasis

Es decir: una vez que conocemos C, la información que aportaba B deja de ser relevante para predecir A.

Como interpretación técnica en términos computacionales, podemos señalar lo siguiente:

Imaginemos un sistema de seguridad de red:

- › A: “el paquete es malicioso”.
- › B: “el paquete llega desde el puerto 22”.
- › C: “el paquete pertenece a una sesión SSH autenticada”.

Ahora vamos paso a paso:

Sin conocer C	Conociendo C
Trabajamos con incertidumbre. Sabemos que: › Muchos ataques intentan entrar por puertos comunes (como 22). › Entonces B sí aporta información sobre A. $P(A \mid B) \neq P(A)$ Es decir: Si viene del puerto 22, cambia lo que creemos sobre A.	Si identificamos que la conexión SSH está › establecida, › autenticada, › y es parte de una sesión activa, entonces el hecho de que use el puerto 22 no aporta información adicional sobre si es malicioso. $P(A \mid B, C) = P(A \mid C)$ Explicación técnica: › SSH siempre usa el puerto 22, por diseño. › Entonces dado que sé que es SSH legítimo, el puerto no me dice nada nuevo sobre si hay un ataque. El conocimiento de C “absorbe” la explicación del uso del puerto.

- Aplicaciones en IA

1. Simplificación de modelos probabilísticos

Muchos modelos necesitan calcular probabilidades conjuntas de muchas variables. Sin independencia condicional, esto implica multiplicar y estimar una enorme cantidad de parámetros

Al asumir independencia condicional:

- › se reduce el número de variables que deben modelarse juntas,
- › los cálculos se vuelven más eficientes,
- › los modelos pueden entrenarse con menos datos.

Permite diseñar modelos que serían imposibles de entrenar con datos reales si hubiera que considerar todas las dependencias.

2. Construcción de redes bayesianas

Las redes bayesianas representan relaciones probabilísticas entre variables mediante un grafo dirigido.

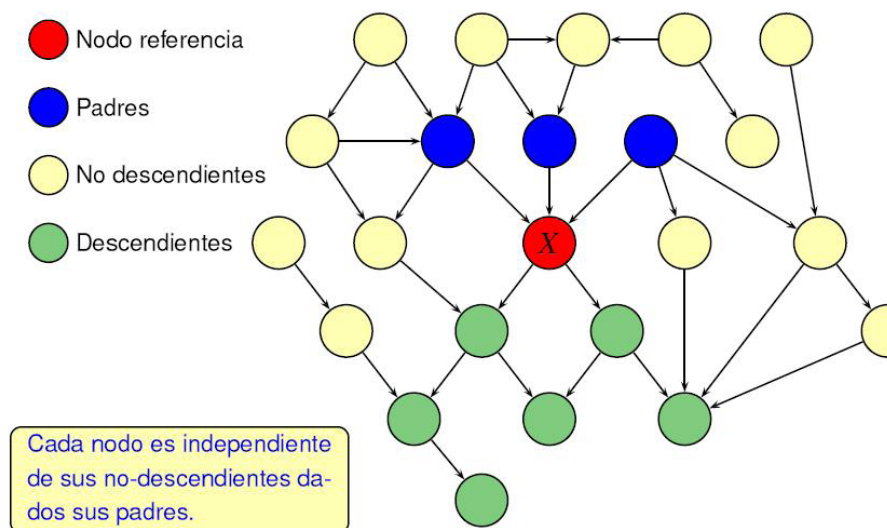


Ilustración 10 - Redes Bayesianas - Fuente: <https://tinyurl.com/46uhed57>

La independencia condicional es la regla estructural que permite:

- › decidir cuáles nodos se conectan,
- › determinar qué variables dependen de cuáles,
- › factorizar la distribución conjunta en partes pequeñas que sí pueden calcularse.

Idea central: si C “bloquea” la relación entre A y B, en la red se elimina esa arista.

Esto hace que el modelo sea más interpretable y computacionalmente viable.



Para profundizar sobre este tema, se recomienda realizar la lectura disponible en el siguiente link: <https://jpordonez.wordpress.com/2008/08/08/redes-bayesianas/>

3. Filtrado de señales en robótica

Los sensores en robótica tienen ruido. Los algoritmos como el filtro bayesiano, el filtro de Kalman o el filtro de partículas dependen de la independencia condicional para:

- › separar el error del sensor del estado real del robot,
- › actualizar la posición estimada sin tener que modelar dependencias innecesarias entre mediciones,
- › combinar lecturas de distintos sensores sin explotar combinaciones exponenciales.

Ejemplo: La lectura actual de un sensor es independiente de la lectura anterior **dado** el estado real del robot.

4. Clasificadores Naive Bayes

Naive Bayes funciona gracias a su hipótesis clave:

Las características son independientes entre sí condicionado a la clase.

Esto simplifica el cálculo de:

$$P(x_1, x_2, \dots, x_n \mid \text{Clase})$$

a una multiplicación de probabilidades simples:

$$P(x_1 \mid \text{Clase}) \cdot P(x_2 \mid \text{Clase}) \dots P(x_n \mid \text{Clase})$$

Gracias a la independencia condicional:

- › se puede entrenar con datasets pequeños,
- › se obtienen clasificadores rápidos y sorprendentemente eficaces,
- › es muy usado en spam detection, análisis de texto, clasificación básica.



Multimedia

Se recomienda ver este video que explica de forma muy sencilla el proceso spam detection: *Clasificador bayesiano ingenuo*

También disponible desde:

<https://tinyurl.com/5bbuswxs>



5. Modelos generativos con hipótesis independientes

En muchos modelos generativos se asume que ciertos rasgos son independientes para:

- › descomponer la generación en pasos simples,
- › permitir que cada variable se modele con su propia distribución,
- › evitar combinaciones explosivas.

Ejemplos donde esto ocurre:

- › modelos gráficos probabilísticos,
- › mixture models simples (p. ej., mezcla de gaussianas con covarianza diagonal),
- › variational autoencoders (VAE) al suponer un “espacio latente” con variables independientes.

Esto hace que se reduzca la complejidad del modelo sin perder capacidad de representación en muchos casos.

6. Reducción de complejidad en sistemas de recomendación

Los sistemas de recomendación que usan probabilidad condicional pueden suponer independencia entre ciertos factores para no modelar todas las interacciones posibles.

Esto:

- › reduce el espacio de parámetros,
- › acelera los cálculos de recomendación,
- › evita sobreajuste,
- › permite hacer recomendaciones en tiempo real.

Por ejemplo, el sistema puede asumir que las preferencias del usuario por categorías distintas son independientes dadas sus preferencias globales, dando como resultado que el sistema pueda predecir gustos sin tener que analizar todas las combinaciones posibles de ítems.

• Teorema de Bayes y Bayes Inverso

El Teorema de Bayes es una de las herramientas más poderosas de la matemática aplicada a la IA. Permite **actualizar probabilidades** cuando llega nueva información, y es la base matemática de clasificadores probabilísticos, filtros en robótica, sistemas de recomendación y modelos generativos.

• Teorema de Bayes

Dado dos eventos A y B:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

Se interpreta como:

- › $P(A)$: probabilidad inicial o priori.
- › $P(B | A)$: verosimilitud de observar B cuando A es cierto.
- › $P(A | B)$: probabilidad actualizada o posteriori.
- › $P(B)$: probabilidad total de B (normalización).

Bayes permite “invertir” la dirección lógica:

- › es fácil calcular $P(B | A)$ (por ejemplo: si un mail es spam, ¿cuál es la probabilidad de que tenga la palabra “gratis”?)
- › pero en IA necesitamos $P(A | B)$ (si aparece “gratis”, ¿cuál es la probabilidad de que sea spam?)



Énfasis

Bayes hace exactamente esa inversión. **Convierte verosimilitud en inferencia.**

- *Bayes Inverso*

¿Qué significa “Bayes inverso”?

Significa dar vuelta la dirección del razonamiento.

- › En la realidad, primero ocurre una *causa* → por ejemplo: “el correo es spam”.
- › Luego, vemos un *efecto* → por ejemplo: “el correo contiene la palabra GRATIS”.

Esto es lo que normalmente podemos calcular:

$P(\text{efecto} | \text{causa})$

Pero en IA y análisis de datos necesitamos lo contrario:

$P(\text{causa} | \text{efecto})$



Énfasis

Es decir: Dado lo que observamos, inferir qué lo produjo. Eso es Bayes inverso.

Supongamos que en un sistema de ciberseguridad ve un comportamiento extraño.

- › B: “Se detectó actividad sospechosa”.
- › A: “La actividad es un ataque”.



Énfasis

Lo que podemos medir directamente es:

$P(B | A)$

Si es un ataque, ¿cuán probable es que aparezca actividad sospechosa?

Pero lo que el sistema realmente necesita responder es:

$$P(A | B)$$

Dado que vi actividad sospechosa, ¿cuál es la probabilidad de que sea un ataque?

Por qué esto es importante?

Porque, en la práctica, tenemos acceso al efecto, no a la causa.

- › Vemos síntomas, no diagnósticos.
- › Vemos lecturas del sensor, no la posición real del robot.
- › Vemos palabras en un mail, no sabemos si es spam.
- › Vemos un clic, no sabemos si hay intención de compra.

Bayes permite reconstruir la causa más probable a partir del efecto observado.

Bayes reconstruye la causa a partir del efecto observado.

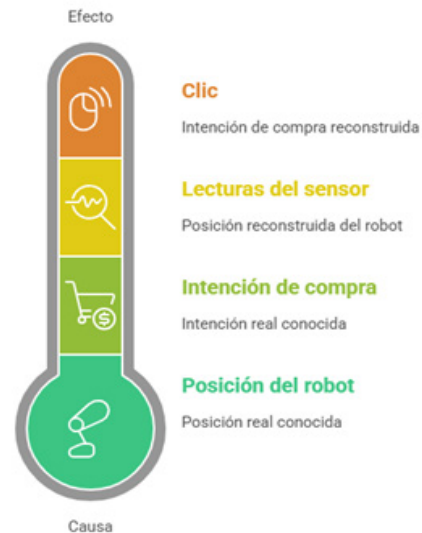


Ilustración 11 - Idea Bayesiana - Elaboración propia con Napkin

- **Concepto de “evidencia” ($P(B)$)**

- › ¿Qué es la evidencia?

La evidencia, también llamada probabilidad total del dato observado, es: $P(B)$ y representa qué tan probable es observar B, sin importar cuál sea la causa detrás.

En el teorema de Bayes:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

la evidencia $P(B)$ sirve para:

- › Ajustar la escala de probabilidades,
- › Hacer que los resultados finales sumen 1,
- › Comparar correctamente las probabilidades entre todas las causas posibles.



Énfasis

Sin $P(B)$, las probabilidades “a posteriori” quedarían sin normalizar.

› ¿De dónde sale $P(B)$?

En realidad, $P(B)$ es una suma ponderada sobre todas las causas posibles:

$$P(B) = \sum_i P(B | A_i) P(A_i)$$

Esto significa:

- › Tomas todas las posibles causas A_i .
- › Calculas qué tan probable es ver B si cada causa fuera cierta.
- › Las combinas según qué tan probable es cada causa en general.

Por ejemplo: Para saber qué tan común es ver la palabra “gratis” en correos, debe considerar:

- › correos spam
- › correos no spam

Cada uno contribuye una parte de la probabilidad total

› ¿Por qué muchas veces no calculamos la evidencia?

Porque cuando queremos saber qué causa es más probable, el denominador es idéntico para todas las clases.

Ejemplo con dos clases:

$$P(\text{Clase}_1 | B) = \frac{P(B | C_1)P(C_1)}{P(B)}$$

$$P(\text{Clase}_2 | B) = \frac{P(B | C_2)P(C_2)}{P(B)}$$

Como $P(B)$ es el mismo número, comparar clases NO requiere calcularlo. Lo único que cambia es el numerador.



Énfasis

Por eso, en Naive Bayes se usa:

$$P(A | B) \propto P(B | A)P(A)$$

El símbolo: $\propto$ significa “proporcional a”.

Aplicaciones de Bayes



Ilustración 12 - Aplicaciones Bayesianas - Elaboración propia con Napkin

- **Distribuciones a Posteriori**

La *distribución a posteriori* es una de las ideas centrales de la inferencia bayesiana y aparece cada vez que se aplica el teorema de Bayes. Es el “resultado final” después de combinar datos nuevos con lo que ya sabíamos.

- › ¿Qué es una *distribución a posteriori*?



Énfasis

La distribución a posteriori es: La probabilidad actualizada de una variable después de observar nuevos datos.

Se expresa como:

$$P(A | B) = \text{posterior}$$

Esto representa:

- › lo que creíamos antes (prior)
- › actualizado por
- › lo que observamos ahora (likelihood)

La forma completa es:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

¿Por qué se llama "a posteriori"?

Porque se calcula después ("post") de ver los datos.

- › **Prior** = antes de los datos
- › **Posterior** = después de los datos

Es el corazón de cómo "aprende" un modelo bayesiano. Veamos los siguientes casos:

Caso 1: Mail SPAM	Caso 2: Robótica
› $P(\text{Spam}) = 0.3 \rightarrow$ prior › $P(\text{"gratis"} \text{Spam}) = 0.6 \rightarrow$ likelihood › $P(\text{"gratis"}) = 0.2 \rightarrow$ evidencia $P(\text{Spam} \text{gratis}) = \frac{0.6 \cdot 0.3}{0.2} = 0.9$ <i>Interpretación:</i> Luego de ver la palabra "gratis", la probabilidad de que sea spam (posterior) sube al 90%. Esto ES la distribución posterior de "spam vs no-spam".	La localización en robótica usa: $P(\text{"posici"} \text{"o"} \text{"n actual"} \text{"lectura del sensor"})$ Donde: › el sensor da una lectura con ruido › la posterior combina: › el movimiento previo (prior) › la lectura del sensor (likelihood) Cada nuevo ciclo del robot vuelve a calcular una posterior nueva. El robot literalmente "se reposiciona" usando este cálculo.

¿Qué forma tiene una distribución posterior?

Puede ser:

- › discreta (lista de probabilidades)
- › continua (curva con densidad)

Un ejemplo continuo muy común: Si el prior es Normal y la verosimilitud también, la posterior también es Normal (caso conjugado). Esto es importante para IA porque simplifica cálculos y permite actualizar modelos en tiempo real (como en filtros de Kalman).

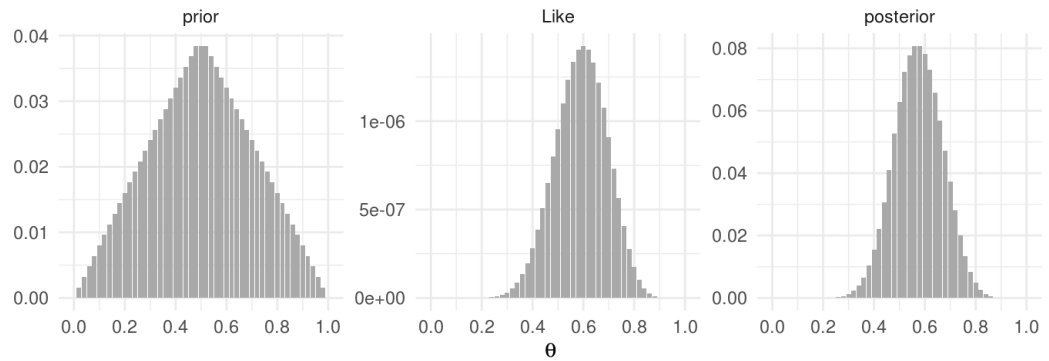


Ilustración 13- Distribución Posterior - Fuente: <https://tinyurl.com/uj9yfe6v>



Para profundizar en este tema, puede leer: *Regla de Bayes e inferencia bayesiana*. Allí encontrará cómo aplicar el teorema de Bayes en inferencia bayesiana.

Aplicaciones de la Probabilidad Posterior en IA

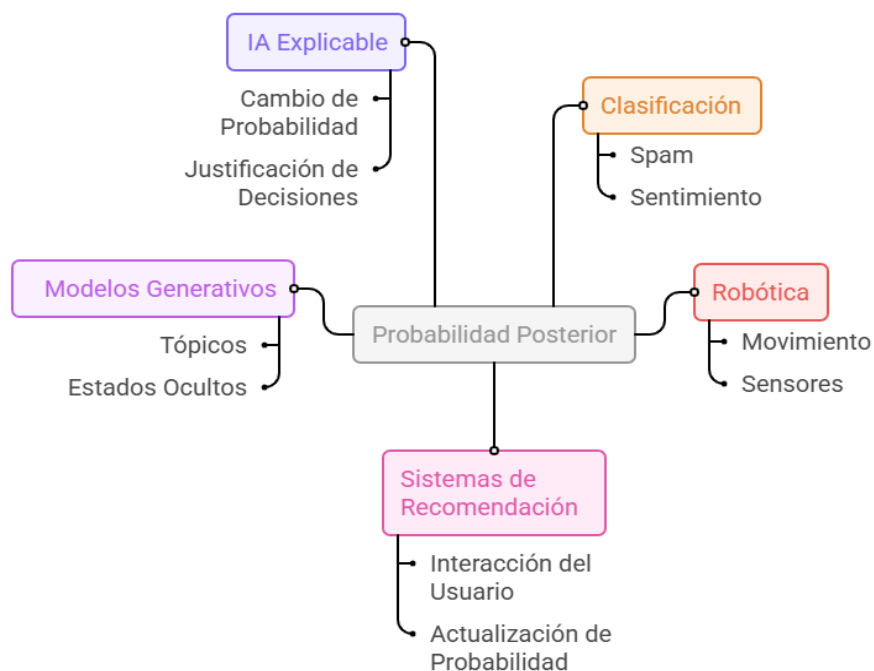


Ilustración 14- Aplicaciones en IA - Elaboración propia con Napkin

• Clasificador Naive Bayes

El clasificador Naive Bayes es un algoritmo de aprendizaje supervisado basado en el Teorema de Bayes, que asume que todas las características son independientes condicionadas a la clase. Aunque esta suposición no siempre es cierta, en la práctica funciona sorprendentemente bien.

¿Qué significa “naive”?

“Naive” indica que el modelo hace una hipótesis muy fuerte: Todas las variables del dataset son independientes entre sí, si conocemos la clase.

Por ejemplo: Si un correo es spam o no spam, entonces la presencia o ausencia de cada palabra se considera independiente.

En la realidad no es totalmente cierto, pero esta simplificación:

- › reduce la complejidad,
- › acelera los cálculos,
- › permite trabajar con pocos datos,
- › y da buenos resultados en clasificación de texto, recomendación, detección de anomalías y más.

- **Fórmula general del clasificador**

Queremos predecir la clase C dado un conjunto de características $X = (x_1, x_2, \dots, x_n)$.

Aplicando Bayes:

$$P(C | X) \propto P(C) \prod_{i=1}^n P(x_i | C)$$

El símbolo “ $\propto$ ” significa *proporcional a*, porque el denominador (la evidencia) es igual para todas las clases.

- › $P(C)$ → qué tan frecuente es cada clase en general (“prior”)
- › $P(x | C)$ → qué tan compatible es cada característica con la clase
- › Multiplicamos todo
- › La clase con mayor resultado gana

Por ejemplo: CASO - anti-spam:

Características del correo:

- › contiene “gratis”
- › contiene “clic aquí”
- › contiene “urgente”

Entonces calculamos:

Para SPAM:	$P(\text{spam}) \times P(\text{gratis spam}) \times P(\text{click spam}) \times P(\text{urgente spam})$
Para NO SPAM:	$P(\text{no_spam}) \times P(\text{gratis no_spam}) \times P(\text{click no_spam}) \times P(\text{urgente no_spam})$



El que dé más alto → es la clase final.

Ventajas del Modelo de Naive Bayes

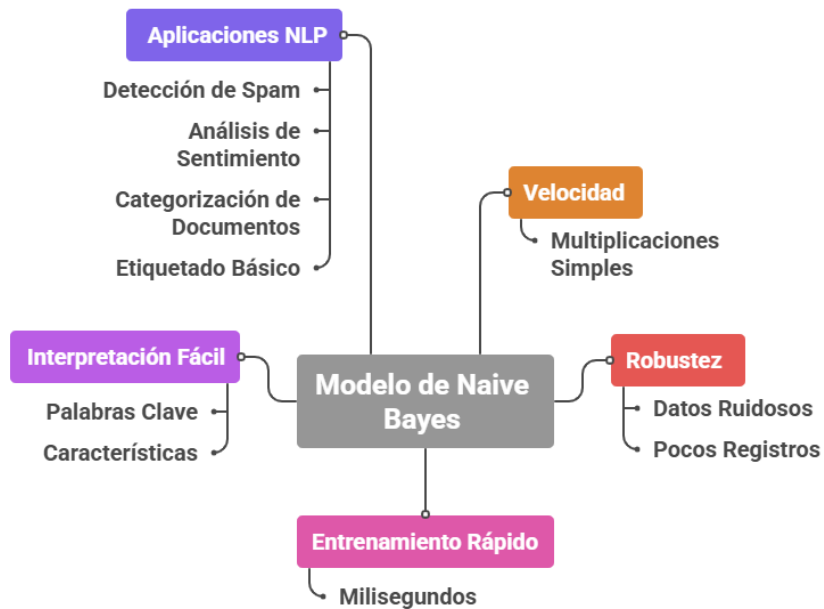


Ilustración 15 - Ventajas - Elaboración propia con Napkin

Limitaciones del Modelo de Bayes

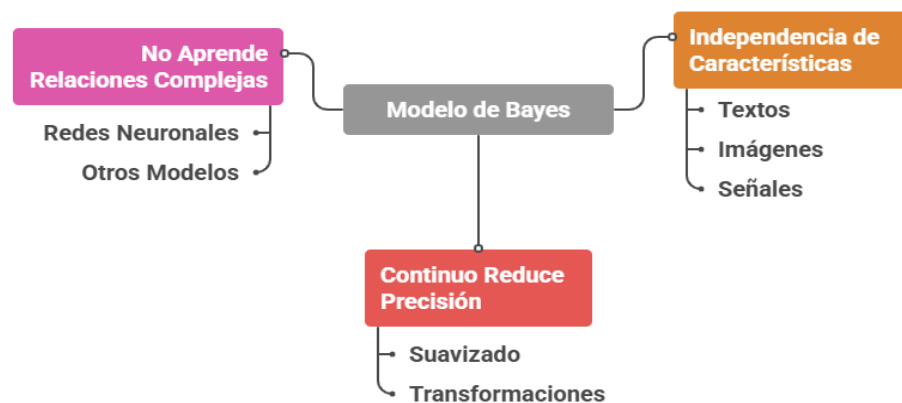


Ilustración 16 - Desventajas - Elaboración propia con Napkin

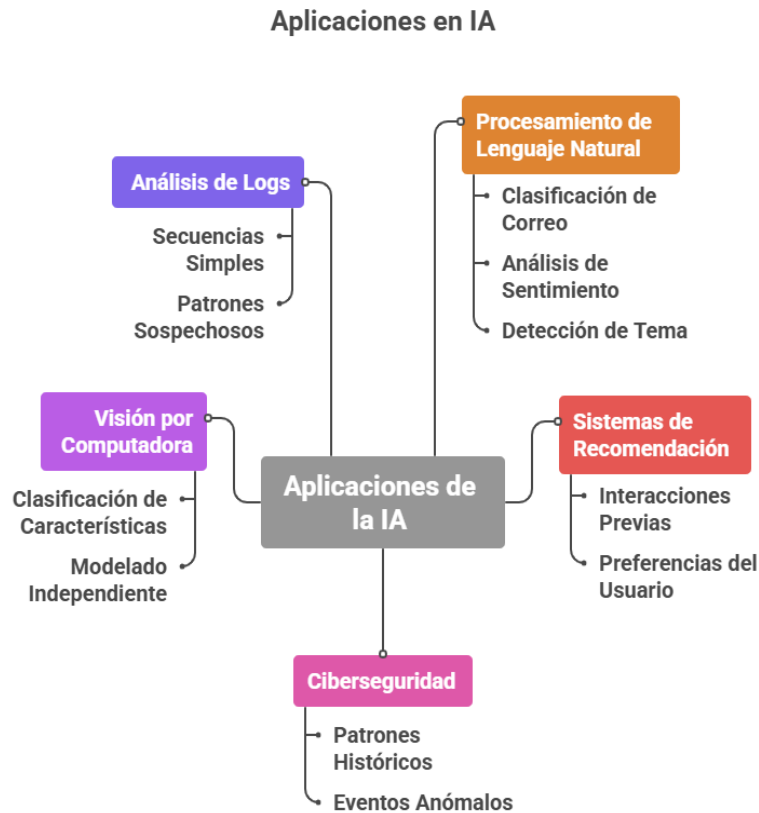


Ilustración 17- Aplicaciones en IA -Elaboración propia con Napkin

- **Modelos generativos bayesianos simples**

Un modelo generativo es un modelo matemático capaz de describir cómo se generan los datos, especificando la relación probabilística entre:

- › las clases o estados ocultos, y
- › las observaciones o variables visibles.

En contraste con los modelos discriminativos (que solo separan clases), los modelos generativos pueden simular, reconstruir y predecir datos.

¿Qué significa “generar datos”?	¿Por qué es “bayesiano”?
Un modelo generativo especifica: 1. Cómo se eligen las causas (<i>clases, estados, categorías</i>). 2. Cómo esas causas generan las observaciones. Formalmente: › Primero se elige una clase según $P(C)$. › Luego se generan las características X según $P(X C)$. Esto produce la distribución conjunta: $P(C,X)=P(C)P(X C)$	Porque utiliza el Teorema de Bayes para invertir el proceso: De: C "lase" "Datos" Hacia: "Datos" "Clase" Esto permite clasificar y generar, todo dentro del mismo modelo.

- **Modelos generativos**

Naive Bayes	Mezcla de Gaussianas (GMM)
Aunque se use como clasificador, Naive Bayes es un modelo generativo porque: › Define $P(C)$. › Define $P(x_i C)$. › Puede simular ejemplos nuevos generando palabras o características según la clase. Ejemplo generativo: 1. Elegir clase “spam”. 2. Elegir palabras según $P(\text{"palabra"} \text{spam})$. 3. Armar un correo simulado con esas palabras. Es simple, pero sirve para entender el concepto	Un caso muy usado en IA clásica: › Cada clase se modela con una distribución Normal (Gaussiana). › Combinar varias normales permite modelar datos complejos. El modelo: $P(X) = \sum_{k=1}^K \pi_k \mathcal{N}(X \mu_k, \Sigma_k)$ Donde: › π_k: peso o probabilidad de la componente k › μ_k: media › Σ_k: covarianza ¿Qué puede hacer un GMM? › Clasificar (<i>descubrir qué “componente” generó el dato</i>). › Generar datos nuevos similares a los originales. › Modelar grupos sin usar etiquetas (<i>aprendizaje no supervisado</i>).

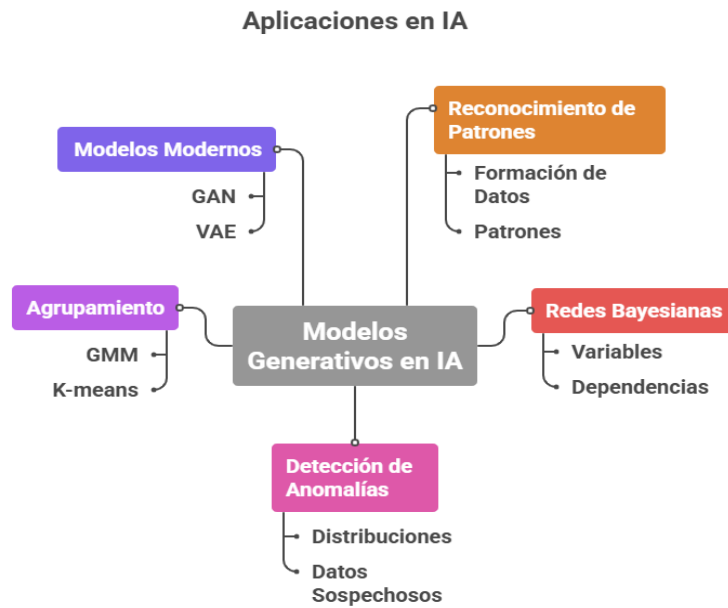


Ilustración 18 - Modelos Bayesiano y GMM en IA - Elaboración propia con Napkin

• **Tema 3: Inferencia Estadística**



La inferencia estadística es el conjunto de herramientas que permiten sacar conclusiones generales a partir de un conjunto limitado de datos (una muestra).

¿Por qué es esencial la inferencia estadística en IA?

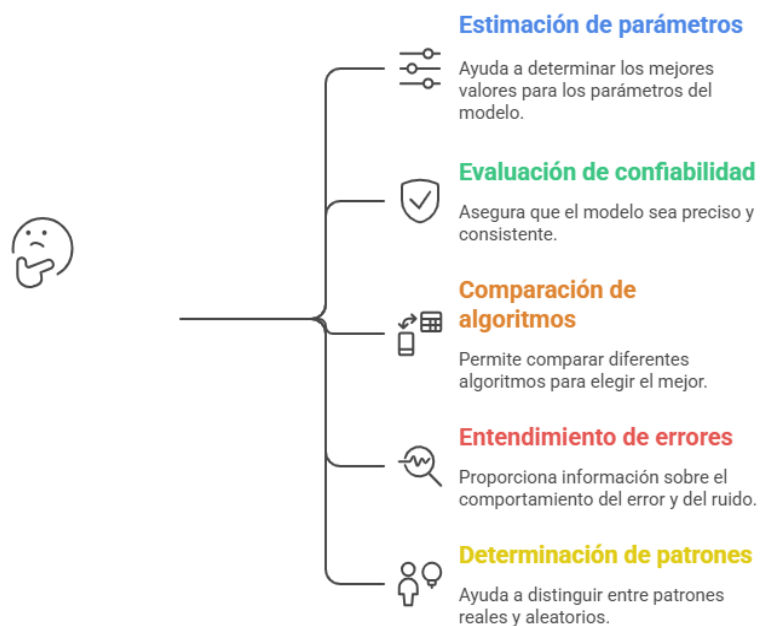


Ilustración 19- Inferencia Estadística e IA - Elaboración propia con Napkin

- **Conceptos fundamentales**

Para poder analizar datos y construir modelos confiables en inteligencia artificial, necesitamos comprender cómo se relacionan las observaciones que tenemos a disposición con la realidad que intentamos describir.

La inferencia estadística parte de una idea central: los datos que observamos son solo una parte de una población mucho más grande, y a partir de esa porción limitada debemos extraer conclusiones generales.

En este contexto, conceptos como *población*, *muestra*, *parámetros* y *estadísticos* se vuelven fundamentales. Los parámetros representan las características verdaderas de la población —pero no los conocemos—, mientras que los estadísticos son valores que calculamos a partir de la muestra —y que usamos como aproximaciones.

Esta distinción es clave en IA, porque los algoritmos, decisiones y estimaciones que hacemos siempre se construyen a partir de datasets, no de la totalidad de casos posibles.

Comprender esta relación entre lo observado y lo desconocido es el primer paso para desarrollar modelos matemáticos que puedan generalizar, predecir y tomar decisiones bajo incertidumbre.

Veamos entonces de que se trata cada uno de estos conceptos.

- **Población y muestra**

<i>Término</i>	<i>Definición</i>	<i>Ejemplos</i>
<i>Población</i>	Conjunto total sobre el cual queremos sacar conclusiones	Todos los usuarios, todos los sensores, todos los emails, etc.
<i>Muestra</i>	Subconjunto observado	Usuarios encuestados, señales captadas, emails procesados



Énfasis

En IA, casi nunca tenemos la población completa: siempre trabajamos con muestras (*datasets*).

- **Parámetros y estadísticos**

<i>Tipo</i>	<i>Descripción</i>	<i>Ejemplos</i>
<i>Parámetros (desconocidos)</i>	Valores reales que describen a la población	Media verdadera, varianza verdadera, tasa real de spam
<i>Estadísticos (observados)</i>	Valores calculados sobre la muestra	Media muestral, varianza muestral, proporción observada



Énfasis

La inferencia busca estimar los parámetros verdaderos usando estadísticos.

- **Estimación**

La estimación es una herramienta en la estadística y el análisis de datos, que permite inferir características desconocidas de una población a partir de información obtenida en muestras.

A través de este proceso, se busca aproximar valores verdaderos, como medias o proporciones, utilizando cálculos realizados sobre los datos observados.

Esta metodología ayuda a la toma de decisiones informadas en diversos campos, desde la investigación científica hasta la inteligencia artificial.

- **Estimación puntual**

La estimación puntual es un método de inferencia estadística que consiste en calcular un único valor numérico a partir de una muestra para aproximar un parámetro desconocido de la población.



Énfasis

Una estimación puntual es un valor calculado con los datos observados que intenta acercarse al valor real del parámetro que no conocemos.

Ejemplos típicos:

- › La media muestral $\bar{x}$ es una estimación puntual de la media poblacional μ .
- › La varianza muestral s^2 estima la varianza real σ^2 .
- › La proporción muestral $\hat{p}$ estima la proporción verdadera p .

- **Estimación por intervalos**

La estimación por intervalos consiste en calcular un rango de valores dentro del cual es probable que se encuentre el parámetro real de la población.



Énfasis

Un intervalo de confianza es un rango numérico construido a partir de la muestra que, con cierto nivel de confianza (por ejemplo 95%), contiene el valor verdadero del parámetro poblacional.

Ejemplos:

- › IC para la media
- › IC para proporciones
- › IC para diferencias entre modelos

Usos en IA

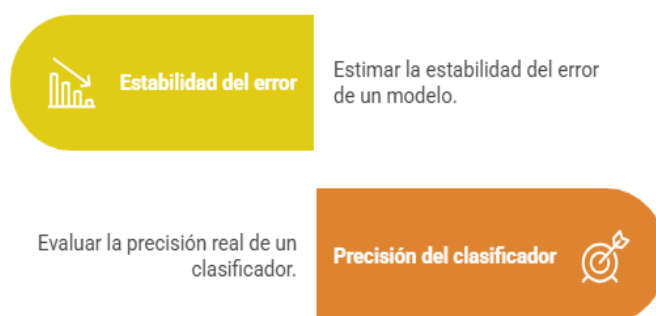


Ilustración 20- Estimación por intervalos en IA—Elaboración propia con Napkin

• Contrastes de hipótesis

El contraste de hipótesis permite decidir, con evidencia estadística, si una afirmación sobre un parámetro es razonable o debe descartarse.



Es un procedimiento que evalúa si los datos apoyan o no una afirmación inicial (hipótesis nula), comparando un estadístico de prueba con un valor crítico o un p-valor.

Incluye:

- › hipótesis nula H_0
- › hipótesis alternativa H_1
- › estadístico de prueba
- › región crítica o p-valor

Uso en IA



Ilustración 21- Contrastes de hipótesis en IA—Elaboración propia con Napkin

- **Errores Tipo I y Tipo II**



Error Tipo I (α) – Falso positivo	Error Tipo II (β) – Falso negativo
Definición clara:	Definición clara:
Error Tipo I es rechazar la hipótesis nula cuando realmente es verdadera.	Error Tipo II es no rechazar la hipótesis nula cuando la hipótesis alternativa es verdadera.
En otras palabras:	En otras palabras:
› ver una diferencia que no existe › detectar “mejora” en un modelo cuando no la hay	› no detectar una diferencia que sí existe
Importante en IA:	Importante en IA:
› evitar sobreestimar el rendimiento	› no detectar fallas reales en un modelo › no capturar mejoras verdaderas por falta de evidencia

- **Máxima verosimilitud (Maximum Likelihood)**

- › **Verosimilitud (Likelihood)**

La verosimilitud mide qué tan probable es observar los datos obtenidos bajo un valor específico del parámetro.



La verosimilitud es una función que indica qué tan compatibles son los datos observados con posibles valores del parámetro.

No calcula la probabilidad de los datos, sino qué valores del parámetro hacen más probable haber observado esa muestra.

Cuando hablamos de Máxima Verosimilitud decimos que:



Es un método que elige el valor del parámetro que hace máxima la verosimilitud, es decir, el valor que hace más probable haber observado los datos.

Ejemplos:

- › estimar p en Bernoulli $\rightarrow \hat{p}$ = proporción observada
- › estimar λ en Poisson $\rightarrow \hat{\lambda}$ = media muestral
- › estimar μ y σ en Normal $\rightarrow$ media y desviación muestral

Uso en IA

Modelos de lenguaje simples

Modelos de lenguaje simples en IA.



Entrenamiento de modelos

Entrenamiento de muchos modelos probabilísticos en IA.



Estimación en HMM

Estimación en Modelos Ocultos de Markov en IA.



Parámetro de Gaussianos

Parámetro de distribuciones Gaussianas en IA.



Ilustración 22 - Maximum Likelihood – Elaboración propia con Napkin

- **Inferencia bayesiana**

La inferencia clásica calcula estimadores y tests. La inferencia bayesiana calcula distribuciones posteriores.

$$\text{Posterior} \propto \text{Prior} \times \text{Verosimilitud}$$

Comparación de enfoques de inferencia estadística



Ilustración 23- Inferencia Clásica vs Inferencia Bayesiana - Elaboración propia con Napkin



La inferencia estadística permite pasar de muestras a conclusiones, y en IA esto significa transformar datos en decisiones confiables.

Para profundizar, se recomienda la lectura de:

- › Inferencia Estadística: Qué es y cómo funciona en el análisis de datos
- › Estadística inferencial: Qué es, importancia y ejemplos
- › <https://www.uv.es/ceaces/pdf/introinfer.pdf>

Ahora estamos en condiciones de seguir un ejemplo completo donde se integran todos lo tratado con respecto a la Inferencia Estadística.

Ejemplo completo: Inferencia Estadística

- **Tema: Estimación de la proporción real de correos spam en un sistema**

Este ejemplo integra:

- › población y muestra
- › parámetros y estadísticos
- › estimación puntual
- › intervalo de confianza
- › contraste de hipótesis
- › interpretación técnica para IA

1. Planteo del problema

Tenemos un sistema de correo que recibe miles de emails por día. Nos interesa estimar la proporción real de correos spam que llegan al servidor. Pero no podemos analizar todos los correos (sería la población completa). Tomamos una muestra aleatoria de correos para analizar.

2. Datos de la muestra

De una muestra de $n = 800$ correos, observamos: 240 son spam

Entonces la proporción muestral es:

$$\hat{p} = \frac{240}{800} = 0.30$$

3. Estimación puntual

Queremos estimar el parámetro desconocido: p = proporción real de spam en toda la población de correos

El mejor estimador puntual es: $\hat{p} = 0.30$



“Estimamos que el 30% de los correos que recibe el sistema son spam.”

4. Estimación por intervalos - Intervalo de confianza del 95%

Queremos calcular un rango donde es probable que esté el valor real de p .

La fórmula del intervalo de confianza para proporciones es:

$$IC = \hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

Para 95% de confianza $\rightarrow z = 1.96$

Cálculo:

1	2	3	4
Varianza muestral estimada:	Error estándar:	Margen:	Intervalo:
$\hat{p}(1 - \hat{p}) = 0.30 \times 0.70 = 0.21$	$\sqrt{\frac{0.21}{800}} = 0.0162$	$1.96 \times 0.0162 = 0.0317$	$IC = 0.30 \pm 0.0317$
Resultado:	$IC = (0.268, 0.3317)$ "La proporción real de spam está entre 26.8% y 33.2%, con 95% de confianza."		

5. Contraste de hipótesis ¿Cambió el nivel de spam?

Supongamos que el proveedor afirma que la tasa real de spam es:

$$p_0 = 0.25$$

Queremos evaluar si realmente es menor de lo que estamos observando.

1	2	3	4
Hipótesis:	Estadístico-z:	Cálculo:	Valor Crítico
$H_0: p = 0.25$ $H_1: p \neq 0.25$	$z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1 - p_0)}{n}}}$	$\sqrt{\frac{0.25 \times 0.75}{800}} = 0.0153$ $z = \frac{0.30 - 0.25}{0.0153} = 3.27$	Valor crítico (95%): $ z = 1.96$
Decisión:	Como $3.27 > 1.96$ Rechazamos H_0. Pues la proporción de spam es significativamente distinta de 0.25. "La evidencia sugiere que la proporción real de spam es mayor que 25%."		

6. Interpretación final. Si el análisis lo hiciera una IA

Lo que se sabe:

- › Estimación puntual = 0.30
- › Intervalo = (0.268, 0.332)
- › El contraste sugiere que el 25% es demasiado bajo

Por lo tanto:

El sistema recibe alrededor de un 30% de correos spam, con una variabilidad esperada de $\pm 3\%$, y este valor es significativamente mayor al 25% que se afirmaba previamente.

- › En términos de IA:
 - › Un clasificador anti-spam debe calibrarse para un entorno altamente desbalanceado (30% spam).
 - › Podría necesitar técnicas de:
 - › oversampling
 - › class weighting
 - › ajuste de umbrales de decisión
 - › La varianza baja indica que la estimación es estable.
 - › Puede ser necesario actualizar modelo y reglas si esta proporción cambia con el tiempo.



Multimedia

Usando como insumo el ejemplo anterior, generamos con NoteBookLM un video explicativo de la situación: *Detective de Spam Inferencia*

También disponible desde:
<https://tinyurl.com/5f4wbwa6>



› Otras técnicas

Oversampling (sobre-muestreo) es una técnica para equilibrar un dataset desbalanceado, especialmente utilizada en clasificación supervisada cuando una de las clases aparece mucho menos que la otra.

La siguiente tabla resume, sus características:

¿Para qué se usa?	¿Cómo se hace el oversampling?
› Para evitar que el modelo favorezca siempre la clase mayoritaria › Para mejorar la capacidad de detección de clases raras (<i>fallos, fraudes, spam, anomalías</i>) › Para que las métricas sean más equilibradas › Para que el modelo vea suficientes ejemplos de la clase minoritaria	1. Duplicación simple (<i>random oversampling</i>) Se copian instancias existentes de la clase minoritaria hasta equilibrar el dataset. <i>Ejemplo:</i> › Clase mayoritaria: 5000 ejemplos › Clase minoritaria: 500 ejemplos → Se duplican ejemplos minoritarios hasta llegar a 5000
	2. SMOTE (<i>Synthetic Minority Oversampling Technique</i>) En vez de duplicar, se generan ejemplos sintéticos nuevos combinando instancias reales. Es el método más usado en IA. <i>Funciona así:</i> › Toma un ejemplo de la clase minoritaria › Busca sus vecinos más cercanos › Genera nuevos puntos interpolando entre ellos



Si desea profundizar en este tema, le dejamos una lectura recomendada (*en inglés*): ***Oversampling and Undersampling, Explained: A Visual Guide with Mini 2D Dataset***

Class weighting (o ponderación de clases) es una técnica usada en aprendizaje automático para manejar datasets desbalanceados, al igual que el oversampling, pero con un enfoque diferente.

En otras palabras:

- › Si el modelo se equivoca con la clase poco frecuente → la penalización es más grande.
- › Si se equivoca con la clase muy común → la penalización es más chica.

El modelo aprende así que los errores en clases minoritarias “importan más”, obligándolo a prestarles atención.

La siguiente tabla resume sus características:

¿Por qué se usa?	¿Cómo funcionan los pesos?
› Porque muchos problemas reales tienen clases muy desbalanceadas: › fraudes (1% vs 99%) › fallas en sensores › spam vs no spam › diagnósticos médicos › anomalías de red › Si el modelo no considera pesos diferentes: › predecirá casi siempre la clase mayoritaria, › la accuracy será engañosamente alta, › la clase minoritaria será ignorada. › Con class weighting: › se evita el sesgo hacia la clase frecuente › se mejora el recall de la clase rara › no se alteran los datos (<i>a diferencia de oversampling</i>)	Si la clase minoritaria representa solo el 10% del dataset, podemos darle un peso de: $w_{\text{minorit.}} = \frac{1}{0.10} = 10$ Eso significa: › un error en la clase minoritaria vale 10 veces más que un error en la clase mayoritaria.



Para profundizar en este tema, se recomienda ver el video “Handling Imbalanced data using Class Weights Machine Learning Concepts” (inglés)

También disponible desde:
<https://tinyurl.com/yc62ab6u>



Llegado/a a este punto, está en condiciones de realizar la **Actividad 2: Problemas en Data-Secure**. La misma está planteada para realizarse de forma grupal.

¡Y llegamos al final del Módulo 3! Donde se abordaron los fundamentos de la probabilidad y la inferencia estadística, proporcionando un marco conceptual riguroso para el análisis y la interpretación de fenómenos aleatorios en el ámbito de la inteligencia artificial.

La incorporación del Teorema de Bayes y de los modelos generativos bayesianos permitió vincular formalmente la teoría probabilística con técnicas ampliamente utilizadas en IA, particularmente en clasificación, análisis de incertidumbre, filtrado, detección de anomalías y sistemas predictivos.

Asimismo, el estudio de métodos de muestreo y aproximación numérica introdujo herramientas fundamentales para la simulación de procesos, la evaluación empírica de modelos y la implementación de algoritmos basados en técnicas estocásticas. En conjunto, los contenidos de este módulo sientan las bases matemáticas necesarias para interpretar, diseñar y justificar modelos probabilísticos dentro de sistemas inteligentes, garantizando una comprensión integral de la incertidumbre, la variabilidad y la toma de decisiones informadas a partir de datos.

Sigamos adelante con el *Módulo 4*: Optimización Matemática y Programación Convexa para Modelos de IA.

MÓDULO 3 | GLOSARIO

Algoritmo de Monte Carlo: Método numérico basado en el muestreo aleatorio para aproximar valores, simular procesos estocásticos y estimar probabilidades o parámetros en sistemas donde el cálculo exacto resulta complejo o imposible.

Bayes (Teorema de): Principio que permite actualizar la probabilidad de un evento en función de nueva evidencia. Fundamenta la inferencia bayesiana y numerosos modelos probabilísticos utilizados en IA.

Class weighting: Técnica empleada en aprendizaje automático para asignar pesos diferenciales a las clases durante el entrenamiento, con el fin de manejar datasets desbalanceados sin modificar los datos originales.

Densidad de probabilidad (PDF): Función que describe cómo se distribuyen los valores de una variable aleatoria continua. La probabilidad de un intervalo se obtiene mediante la integral de la densidad.

Distribución Bernoulli: Distribución discreta para experimentos con dos posibles resultados (éxito/fracaso). Modela variables binarias ampliamente utilizadas en clasificación y detección.

Distribución Binomial: Distribución discreta que describe el número de éxitos en un conjunto de ensayos independientes con igual probabilidad. Útil en análisis de aciertos, errores y conteos.

Distribución Exponencial: Distribución continua que modela tiempos entre eventos independientes. Se utiliza en procesos de espera, análisis de fallos y modelado de tiempos de respuesta.

Distribución Normal (Gaussiana): Distribución continua simétrica alrededor de su media. Modela fenómenos de ruido, error de medición, variabilidad natural y numerosos procesos en IA.

Distribución Poisson: Distribución discreta que modela la cantidad de eventos raros que ocurren en un intervalo de tiempo. Se utiliza en detección de anomalías, tráfico de red y conteo de sucesos.

Distribución Uniforme: Distribución continua donde todos los valores dentro de un intervalo tienen igual probabilidad. Empleada en inicialización aleatoria, simulaciones y selección aleatoria.

Error Tipo I: Error que consiste en rechazar la hipótesis nula cuando es verdadera. Se interpreta como un falso positivo.

Error Tipo II: Error que consiste en no rechazar la hipótesis nula cuando es falsa. Se interpreta como un falso negativo.

Estimación por intervalos: Método inferencial que proporciona un rango de valores (*intervalo de confianza*) dentro del cual se encuentra un parámetro poblacional con un nivel determinado de confianza.

Estimación puntual: Valor único calculado a partir de una muestra que se utiliza para aproximar un parámetro poblacional desconocido.

Esperanza (valor esperado): Media teórica de una variable aleatoria, que indica el valor promedio que se espera observar en el largo plazo.

Función de probabilidad (PMF): Función que asigna probabilidades a cada valor posible de una variable aleatoria discreta.

Inferencia bayesiana: Enfoque inferencial que utiliza el Teorema de Bayes para actualizar creencias sobre parámetros o estados a partir de datos observados.

Inferencia estadística: Conjunto de métodos que permiten obtener conclusiones sobre una población utilizando información derivada de una muestra.

Intervalo de confianza: Rango calculado a partir de datos muestrales que contiene, con un nivel especificado (por ejemplo, 95%), al parámetro poblacional verdadero.

Likelihood (Verosimilitud): Función que mide qué tan probables son los datos observados bajo distintos valores del parámetro. Base del método de máxima verosimilitud.

Máxima Verosimilitud (MLE): Método de estimación que selecciona los valores de los parámetros que maximizan la verosimilitud de los datos observados.

Modelo generativo: Modelo probabilístico que describe cómo se generan los datos, especificando la distribución de las observaciones dado un conjunto de parámetros o clases.

Muestreo: Proceso de generar valores aleatorios a partir de una distribución probabilística. Fundamental para simulaciones, experimentación y análisis en IA.

Oversampling: Técnica que genera copias o ejemplos sintéticos de la clase minoritaria para equilibrar datasets desbalanceados en clasificación supervisada.

Parámetro: Valor desconocido que describe una característica de la población (media, varianza, proporción, etc.).

Población: Conjunto total de individuos o elementos sobre los cuales se desea realizar inferencia o tomar decisiones.

Posterior (Distribución a posteriori): Distribución actualizada del parámetro o estado luego de incorporar nueva evidencia mediante el Teorema de Bayes.

Prior (Distribución a priori): Distribución que representa el conocimiento o creencias iniciales sobre un parámetro antes de observar datos.

Sesgo (Skewness): Medida del grado de asimetría de una distribución respecto de su media.

Test de hipótesis: Procedimiento estadístico utilizado para evaluar afirmaciones sobre parámetros poblacionales mediante un estadístico de prueba y un nivel de significación.

Variable aleatoria: Variable cuyo valor depende del resultado de un fenómeno aleatorio. Puede ser discreta o continua.

Varianza: Medida de dispersión que indica cuánto se alejan los valores de una variable respecto a su media.

MÓDULO 3 | ACTIVIDADES



ACTIVIDAD N° 1:

Muestreo y visualización de distribuciones

Material de partida

Debe contar con:

- › Python 3 instalado.
- › Las librerías numpy y matplotlib.
- › El archivo .py con el programa de muestreo (al cual se le dio acceso en el material).

Consigna:

Parte A – Ejecución y exploración básica

1. Ejecutar el programa de muestreo en Python.
2. Probar, al menos una vez, cada una de las siguientes distribuciones:
 - › Bernoulli
 - › Binomial
 - › Poisson
 - › Uniforme
 - › Normal
 - › Exponencial
3. Para cada distribución:
 - › Elegir parámetros “razonables” (por ejemplo:
 - › Bernoulli: $p = 0.3$, $n = 1000$
 - › Binomial: $n = 10$, $p = 0.5$, muestra de 1000
 - › Poisson: $\lambda = 4$, muestra de 1000

- › Uniforme: $[0,1]$, muestra de 1000
- › Normal: $\mu = 0$, $\sigma = 1$, muestra de 1000
- › Exponencial: $\lambda = 0.5$, muestra de 1000)

› **Registrar:**

- › tamaño de la muestra
- › media aproximada
- › varianza aproximada

› **Capturar el histograma generado.**

Parte B – Análisis comparativo

4. Elaborar una tabla resumen con una fila por distribución y las siguientes columnas:

- › Nombre de la distribución
- › Tipo (discreta / continua)
- › Parámetros utilizados
- › Media empírica observada
- › Varianza empírica observada
- › Comentario sobre la forma del histograma (simétrica, sesgada, colas largas, etc.)

5. Para cada distribución, responder brevemente:

- a) ¿Qué tipo de fenómeno de IA podría modelar?

Ejemplos: ruido de sensor, conteo de eventos, tiempo entre accesos, aparición de palabras, etc.

- b) ¿La forma del histograma coincide con lo que esperabas teóricamente? ¿Por qué sí o por qué no?

Parte C – Aplicación a un caso de IA

6. Elegir uno de estos escenarios (o proponer uno similar):

- (1) Ruido en un sensor: pequeñas variaciones alrededor de un valor real.
- (2) Número de intentos fallidos de login por minuto en un sistema.
- (3) Tiempo entre dos accesos a una API.
- (4) Número de correos spam recibidos por hora.

6. Indicar:

- › Qué distribución usarías para modelar ese fenómeno.
- › Qué parámetros probarías.
- › Realizar el muestreo con el programa.
- › Incluir el histograma y un breve comentario justificando la elección de la distribución.

4. Entregables

Debe entregar:

1. Documento PDF o Word con:
 - › Tabla comparativa de las distribuciones.
 - › Capturas de pantalla de los histogramas (al menos uno por distribución).
 - › Respuestas a las preguntas de análisis.
 - › Desarrollo del caso de IA elegido (punto 6–7).
2. Archivo .py con el programa (sin modificaciones sustanciales, salvo pequeñas adaptaciones si fueran necesarias).



ACTIVIDAD N° 2: **Problemas en DataSecure**

Enunciado general:

La empresa DataSecure opera un sistema inteligente que monitorea eventos y clasifica actividades sospechosas. Para evaluar el comportamiento del sistema, se realiza un análisis estadístico sobre datos reales obtenidos durante una semana.

Se proporcionan siete conjuntos de datos (o uno elegido por el usted o por el equipo):

- (A) Cantidad de correos spam detectados por día
- (B) Lecturas de un sensor (ruido en milivoltios)
- (C) Tiempos entre solicitudes a una API (en segundos)
- (D) Cantidad de paquetes de red perdidos por segundo
- (E) Errores en detección de actividad anómala
- (F) Porcentaje de aciertos del clasificador día a día
- (G) (Opcional) Dataset elegido por el estudiante

Se deberá seleccionar uno de estos escenarios y realizar una inferencia estadística completa siguiendo los pasos detallados abajo.

Consigna:

Paso 1: Identificar población, muestra, parámetro y estadísticos

1. Definir qué representa la población en el problema.
2. Indicar cuál es la muestra disponible.
3. Especificar el parámetro que se desea estimar (media, proporción o tasa).
4. Calcular los estadísticos necesarios (media muestral, varianza, proporción, etc.).

Paso 2: Estimación puntual

1. Calcular la estimación puntual del parámetro elegido:
 - › Media muestral
 - › Varianza muestral
 - › Proporción muestral
 - › Tasa estimada (si corresponde)
2. Explicar brevemente qué representa ese valor en el contexto del problema.

Paso 3: Intervalo de confianza

1. Construir un intervalo de confianza del 95% para el parámetro.
2. Mostrar paso a paso:
 - › estadístico muestral
 - › error estándar
 - › valor crítico
 - › cálculo final
3. Interpretar el intervalo usando lenguaje técnico (*qué rango es plausible para la población*).

Paso 4: Contraste de hipótesis

1. Formular:
 - › Hipótesis nula H_0
 - › Hipótesis alternativa H_1
2. Elegir un valor de referencia (puede ser provisto o definido por el estudiante).
3. Calcular:
 - › estadístico z o t
 - › p -valor
4. Interpretar el resultado:
 - › ¿se rechaza H_0 ?
 - › ¿qué significa eso para el sistema?

Paso 5: Interpretación aplicada a IA

Redactar un análisis corto donde se conecte el resultado estadístico con un problema de IA real, por ejemplo:

- › calibración de un sistema anti-spam
- › confiabilidad de un sensor
- › estabilidad del tiempo entre accesos
- › robustez del clasificador a lo largo del tiempo
- › detección de eventos anómalos

Debe incluir:

- › si los datos son estables o no
- › si conviene ajustar parámetros
- › si el sistema requiere recalibración
- › si el modelo actual es confiable

Paso 6: Mini implementación en Python (opcional pero recomendado)

Implementar en Python:

- › la estimación puntual
- › el intervalo de confianza
- › el test de hipótesis

Entregables:

Se debe presentar:

1. Informe en PDF con:
 - › desarrollo completo de los 5 pasos
 - › cálculos intermedios
 - › interpretación técnica
 - › conclusiones aplicadas a IA
2. Código en Python (.py o Jupyter) si realiza el paso opcional.
3. Gráficos o tablas.

MÓDULO 4

Microobjetivos

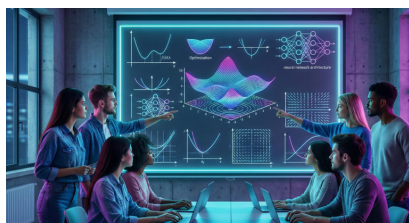
- › Analizar las funciones objetivo y sus propiedades básicas para comprender cómo influyen en la formulación y resolución de problemas de optimización en el entrenamiento de modelos de inteligencia artificial.
- › Aplicar métodos derivados del cálculo, como el gradiente, con el fin de identificar puntos críticos y evaluar su rol en la convergencia dentro de modelos supervisados y no supervisados.
- › Distinguir entre problemas convexos y no convexos para reconocer las implicancias en la existencia y unicidad de soluciones óptimas en algoritmos utilizados en machine learning.
- › Implementar técnicas de optimización lineal y no lineal para resolver instancias simples y fundamentar decisiones sobre métodos adecuados en contextos de modelado matemático aplicado a IA.



VIDEO DOCENTE

Contenidos

- **Optimización Matemática y Programación Convexa para Modelos de IA**



Fuente: Elaboración propia con IA (Gemini)

El Módulo 4 trata uno de los temas fundamentales de la inteligencia artificial: la optimización matemática como herramienta central para entrenar, ajustar y mejorar modelos.

Gran parte del aprendizaje automático desde la regresión y la clasificación hasta el deep learning se basa en resolver problemas de minimización que dependen de gradientes, funciones de pérdida y estructuras convexas. Este módulo introduce los métodos de optimización más utilizados, el análisis mediante gradientes y los principios de la programación convexa, proporcionando el marco teórico y práctico que permite entender cómo los modelos aprenden a partir de los datos.

A través de estos contenidos, se busca que usted conozca los mecanismos internos de los algoritmos de IA y desarrolle criterios para seleccionar, implementar y evaluar técnicas de optimización adecuadas para distintos escenarios. ¡Comencemos!

› Tema 1: Introducción a la optimización matemática

La optimización matemática es un tema importante en inteligencia artificial, dado que numerosos algoritmos de aprendizaje particularmente los modelos supervisados y los métodos de aprendizaje profundo se basan en la minimización sistemática de funciones que cuantifican el error, el costo o la adecuación del modelo a los datos. Aquí se introducen los conceptos que sustentan dichos procesos: la función objetivo, el dominio en el que se evalúa, las restricciones que delimitan el espacio de búsqueda y los criterios que permiten distinguir soluciones locales de soluciones globales y se revisan las estructuras básicas de los problemas lineales y no lineales, que conforman el punto de partida para la comprensión de los algoritmos de optimización utilizados en IA.

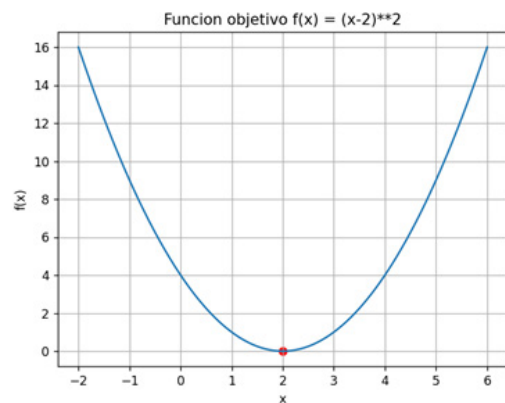
• Conceptos fundamentales

- › **Función objetivo:** Se trata de una aplicación matemática que asigna un valor cuantitativo al desempeño de una solución. En inteligencia artificial, esta función suele denominarse función de pérdida y representa el error cometido por un modelo respecto de los datos observados.

Consideremos la función:

$$f(x) = (x-2)^2,$$

que constituye un ejemplo elemental de una función convexa unidimensional. Su forma parabólica permite visualizar con claridad la tendencia a disminuir el valor de la función al acercarse al punto $x=2$, que constituye su mínimo global.



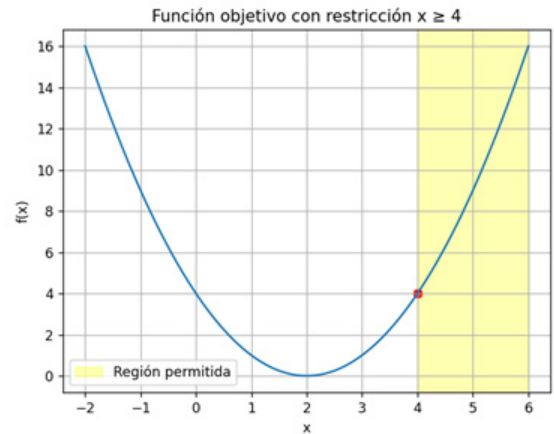
Gráfica 1 - Parábola - Elaboración Propia - Desarrollo Python - <https://tinyurl.com/yd4tk44x>

- › **Dominio:** El dominio establece el conjunto de valores permitidos para las variables. Determina la validez del problema, condiciona la existencia de soluciones y delimita la región donde los algoritmos pueden operar. En muchos modelos de IA, el dominio es todo $\mathbb{R}^n$; sin embargo, existen aplicaciones —por ejemplo, problemas de asignación o modelos con regularización— donde el dominio se restringe para garantizar estabilidad o viabilidad.
- › **Restricciones:** Las restricciones son condiciones que deben satisfacerse simultáneamente con la minimización de la función objetivo. Pueden expresarse como igualdades o desigualdades y, en problemas aplicados, representan limitaciones físicas, económicas o estructurales.

Un ejemplo elemental es:

$$x \geq 0.$$

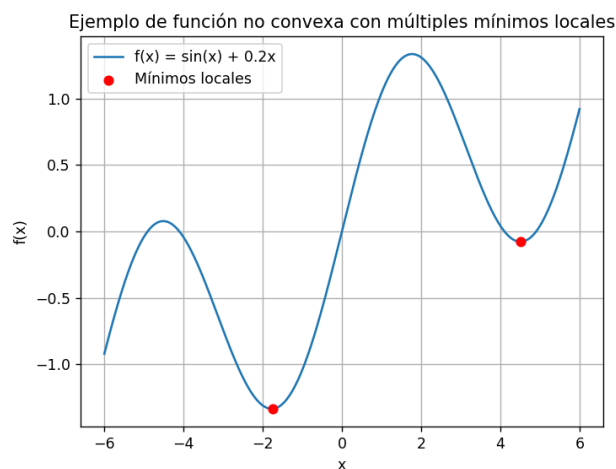
Si se combina con la función objetivo anterior, esta restricción no altera el mínimo. Sin embargo, si la condición impone que $x \geq 4$, entonces la solución óptima ya no será el mínimo natural de la función, sino el valor permitido que se encuentre más próximo al mismo, es decir, $x=4$.



Gráfica 2 – Restricción - Propia - Desarrollo Python
<https://tinyurl.com/yd4tk44x>

- **Óptimos locales y globales:** Una solución óptima global es aquella que minimiza (o maximiza) la función objetivo en todo el dominio permitido. Un óptimo local, en cambio, es una solución que minimiza la función solamente dentro de una vecindad cercana.

En numerosos modelos de aprendizaje profundo, la función de pérdida posee múltiples óptimos locales, lo que implica que la solución alcanzada depende de los valores iniciales y de las características del algoritmo utilizado.



Gráfica 3 – Función no Convexa - Elaboración Propia - Desarrollo Python
<https://tinyurl.com/yd4tk44x>

Veamos un ejemplo que permita dar cuenta del proceso: Considere que x representa un parámetro de un modelo de regresión (por ejemplo, la pendiente de una recta que intenta ajustar datos reales) y que la función

$$g(x) = x^2 - 6x + 10$$

modela, de manera simplificada, el valor del error cuadrático medio del modelo para cada elección de x . En este contexto, minimizar $g(x)$ equivale a encontrar el valor del parámetro que produce el menor error posible en el ajuste a los datos.

1. Cálculo de la derivada

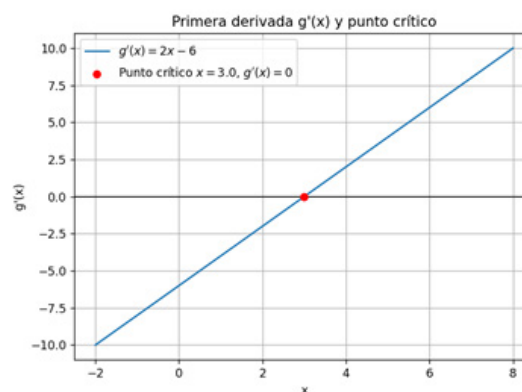
La derivada indica cómo cambia el error cuando se modifica ligeramente el parámetro:

$$g'(x) = 2x - 6.$$

2. Búsqueda de puntos críticos

Los puntos en los que la derivada se anula son candidatos a óptimos, porque allí el error deja de disminuir o aumentar:

$$2x - 6 = 0 \Rightarrow x = 3.$$



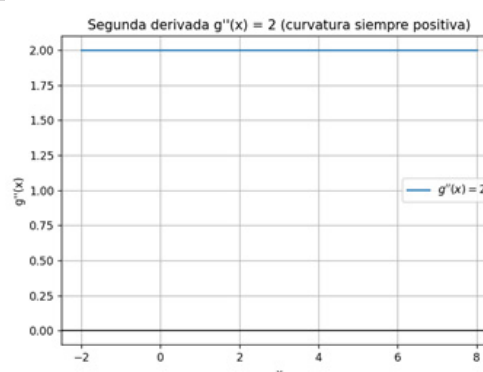
Gráfica 4 - paso 1 y 2 - Elaboración propia - Desarrollo Python - <https://tinyurl.com/yd4tk44x>

3. Verificación mediante la segunda derivada

Analizamos la curvatura de la función mediante la segunda derivada:

$$g''(x) = 2 > 0.$$

El valor positivo indica que, alrededor de $x=3$, la función es cóncava hacia arriba, por lo que en ese punto se alcanza un mínimo.



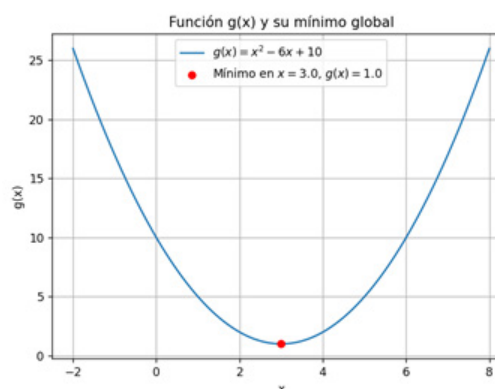
Gráfica 5 - paso 3 - Elaboración propia - Desarrollo Python - <https://tinyurl.com/yd4tk44x>

4. Evaluación de la función

Calculamos el valor del error en el punto óptimo:

$$\begin{aligned} g(3) &= 3^2 - 6 \cdot 3 + 10 \\ &= 9 - 18 + 10 \\ &= 1. \end{aligned}$$

Por lo tanto, el mínimo global de la función ocurre en $x=3$ y su valor es $g(3)=1$.



Gráfica 6 - paso 4 - Elaboración propia - Desarrollo Python - <https://tinyurl.com/yd4tk44x>

En términos del contexto planteado, esto significa que el modelo de regresión alcanza su mejor desempeño —es decir, el menor error cuadrático medio— cuando el parámetro considerado toma el valor $x = 3$. Este ejemplo ilustra cómo los procedimientos de optimización permiten seleccionar de manera sistemática los parámetros que hacen que un modelo de IA se ajuste adecuadamente a los datos.

```
Resumen del proceso de optimización:  
- Función analizada:  $g(x) = x^2 - 6x + 10$   
- Punto de mínimo:  $x = 3.00$   
- Valor mínimo de la función:  $g(3.00) = 1.00$   
- Primera derivada:  $g'(x) = 2x - 6 \rightarrow g'(3.00) = 0$   
- Segunda derivada:  $g''(x) = 2 > 0 \rightarrow$  mínimo global.
```

Captura 1 - Salida Consola - Desarrollo Python - <https://tinyurl.com/yd4tk44x>

Esto es aplicable para cualquier tipo de función teniendo en cuenta que, para que el modelo sea el mejor posible, debería existir un mínimo global.

Veamos este otro ejemplo: Supongamos que x representa un hiperparámetro de un modelo de inteligencia artificial (por ejemplo, la tasa de aprendizaje o un coeficiente de regularización) y que la función

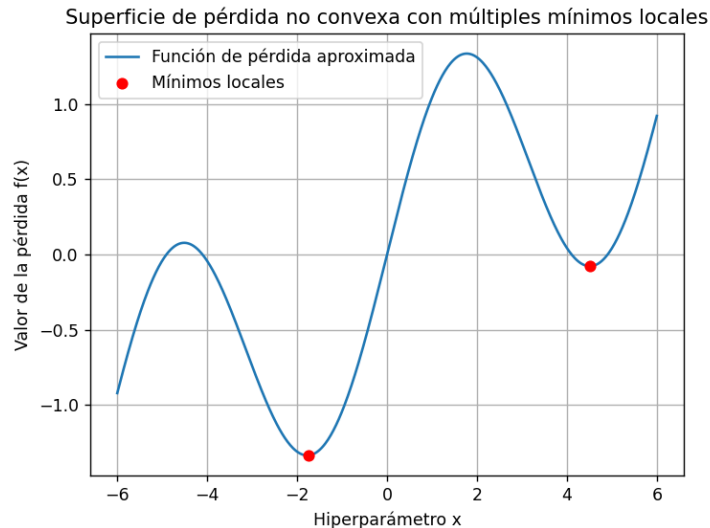
$$f(x) = \sin(x) + 0.2x$$

representa, de manera simplificada, el valor de la función de pérdida del modelo después de entrenarlo con ese hiperparámetro.

En este contexto:

- › Cada valor de x corresponde a “cómo configuramos” el modelo.
- › El valor $f(x)$ indica qué tan bien se desempeña: cuanto más bajo es $f(x)$, mejor es el ajuste del modelo a los datos.
- › Los mínimos locales son configuraciones “aceptables” que parecen buenas en un entorno cercano, pero que no necesariamente son las mejores posibles.
- › El mínimo global, si existe, sería la configuración que proporciona el mejor desempeño entre todas las consideradas.

En problemas reales (por ejemplo, redes neuronales profundas), la superficie de pérdida suele ser altamente no convexa, con muchos mínimos locales.



Gráfica 7—Pérdida no Convexa—Elaboración propia—<https://tinyurl.com/yd4tk44x>

Los puntos rojos ejemplifican justamente esta situación: muestran cómo varios valores de x (hiperparámetros) generan valles relativamente buenos, pero diferentes entre sí.

De esta manera, el gráfico deja de ser una simple curva matemática y pasa a interpretarse como una superficie de pérdida simplificada de un modelo de IA, donde se visualiza claramente el problema de los múltiples mínimos locales al entrenar modelos complejos.



Llegado a este punto, antes de continuar con el estudio del módulo, estamos en condiciones de realizar la **Actividad 1: Problema 1**.

- **Tema 2: Gradientes y técnicas basadas en derivadas**

En los modelos de IA, la mayoría de los parámetros se ajustan mediante procesos iterativos que dependen de la información que suministra la derivada: esta indica cómo cambia la función de costo ante pequeñas variaciones en los parámetros.

Comprender la dirección y la magnitud de ese cambio permite definir trayectorias eficientes hacia la minimización, especialmente en espacios de gran dimensión.

Aquí se ve la interpretación geométrica del gradiente, los métodos clásicos de descenso, y variantes adaptadas al trabajo con grandes volúmenes de datos, estableciendo así la base matemática que sustenta los algoritmos contemporáneos de entrenamiento.

¡Continuemos!

- **Gradiente como dirección de máxima variación**

Partimos de una función real $J(\theta)$ que mide el “costo” o “error” de un modelo al ajustarse a los datos. El parámetro θ puede ser un número (caso unidimensional) o un vector (caso multivariable).

Cuando la función depende de varias variables, por ejemplo

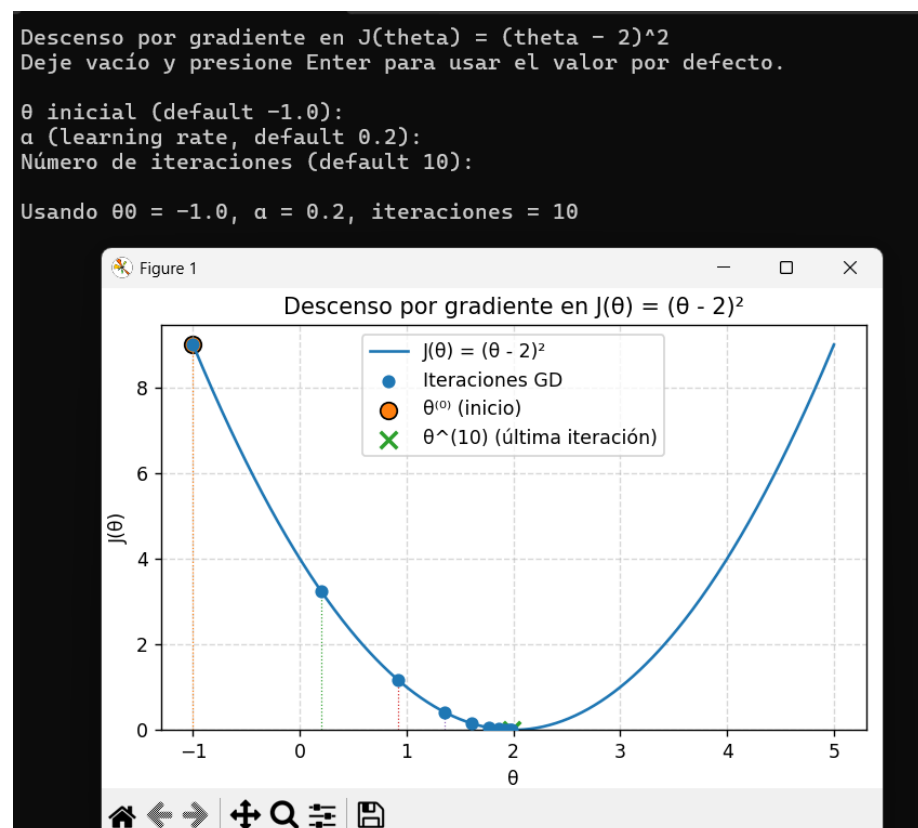
$$J(\theta_1, \theta_2, \dots, \theta_n),$$

el gradiente se define como el vector de derivadas parciales:

$$\nabla J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_n} \right).$$

Geométricamente, el gradiente indica la dirección en la que la función crece más rápido. Si queremos minimizar J , tiene sentido movernos en la dirección contraria: $-\nabla J$.

En términos de IA, J suele ser la función de pérdida (por ejemplo, el error cuadrático medio en una regresión, o la entropía cruzada en clasificación) y θ son los parámetros del modelo (pesos y sesgos).



Gráfica 8- La imagen que muestra una función de una variable $J(\theta)$ con forma de parábola y el punto actual $\theta^{(k)}$, junto con la pendiente $J'(\theta^{(k)})$ y la actualización hacia el mínimo.

Desarrollo propio



Puede descargar el ejemplo anterior desde <https://tinyurl.com/yd4tk44x>

- **Descenso por gradiente: idea básica**

El descenso por gradiente es un procedimiento iterativo para encontrar un mínimo de la función de costo. A partir de un valor inicial $\theta^{(0)}$, se repite:

$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla J(\theta^{(k)})$, donde $\alpha > 0$ es la tasa de aprendizaje (learning rate).

- › Si α es muy grande, el método puede “saltar” el mínimo y divergir.
- › Si α es demasiado pequeña, el método converge pero muy lentamente.

La elección de α es crítica en la práctica y se suele ajustar empíricamente o mediante técnicas automáticas.

Veamos un ejemplo:

Consideremos un problema de IA extremadamente simplificado. Supongamos que queremos ajustar un parámetro θ que representa la “intensidad” de una recomendación en una app, y sabemos (por análisis previo) que el error global del sistema se puede aproximar por:

$$J(\theta) = (\theta - 2)^2.$$

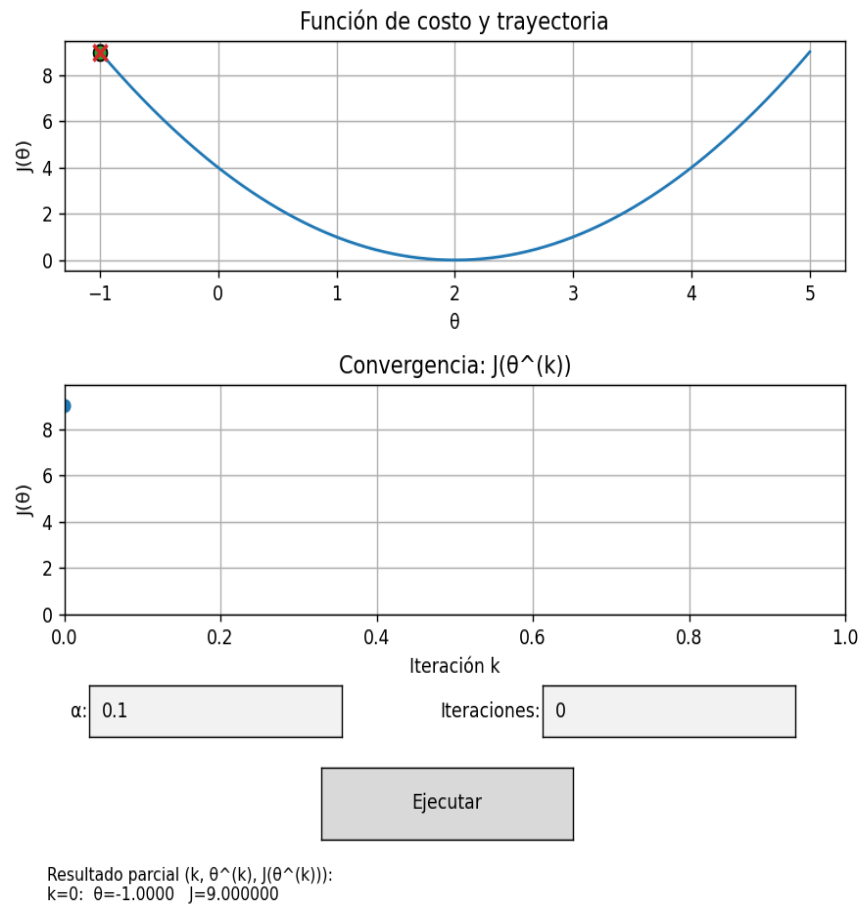
El mínimo evidente es $\theta^* = 2$, donde el error es cero. Sin embargo, imagine-mos que no lo conocemos de antemano y aplicamos descenso por gradiente.

1. Derivada de la función	$J'(\theta) = 2(\theta - 2)$
2. Elección de parámetros del método	› Tasa de aprendizaje: $\alpha = 0,1$. › Valor inicial: $\theta^{(0)} = 0$.
3. Iteraciones	
Iteración 0	$\theta^{(0)} = 0,$ $J'(\theta^{(0)}) = 2(0 - 2) = -4.$
	Actualización: $\theta^{(1)} = 0 - 0,1(-4) = 0,4.$
Iteración 1	$\theta^{(1)} = 0,4,$ $J'(\theta^{(1)}) = 2(0,4 - 2) = -3,2.$
	Actualización: $\theta^{(2)} = 0,4 - 0,1(-3,2) = 0,72.$
Iteración 2	$\theta^{(2)} = 0,72,$ $J'(\theta^{(2)}) = 2(0,72 - 2) = -2,56.$
	Actualización: $\theta^{(3)} = 0,72 - 0,1(-2,56) = 0,976.$

A medida que avanzan las iteraciones, $\theta^{(k)}$ se aproxima a 2 y el valor de $J(\theta^{(k)})$ disminuye. Este mismo mecanismo, con funciones de costo mucho más complejas, es el que se utiliza para entrenar redes neuronales y otros modelos de IA.

Veamos esta evolución gráficamente:

► Iteración 0

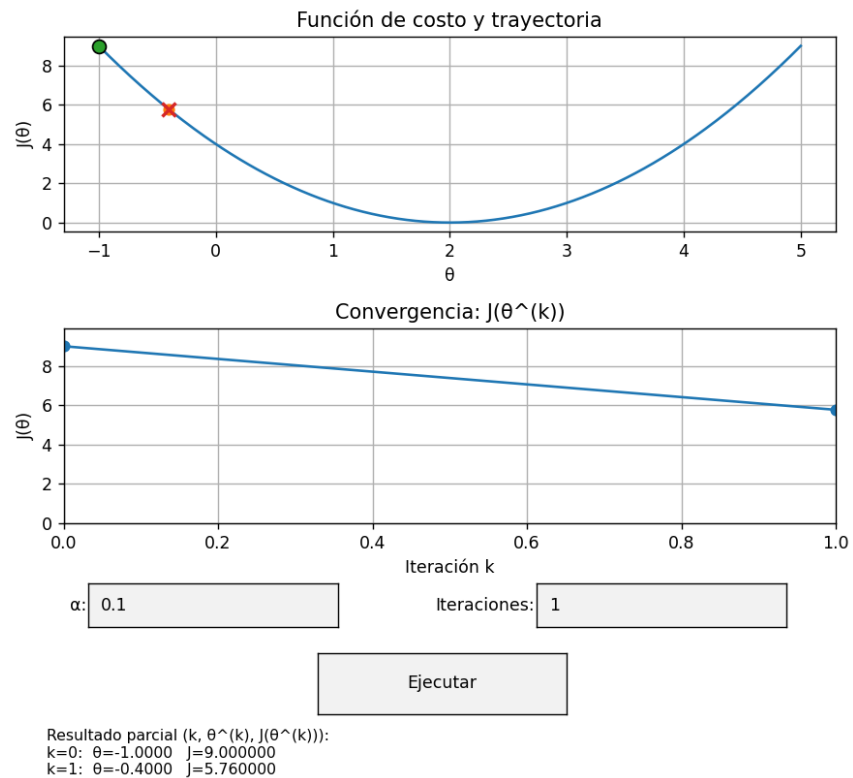


Gráfica 9 - A medida que avanzan las iteraciones, $\theta^{(k)}$ se aproxima a 2 y el valor de $J(\theta^{(k)})$ disminuye
 Desarrollado en Python – Elaboración propia



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>

› Iteración 1



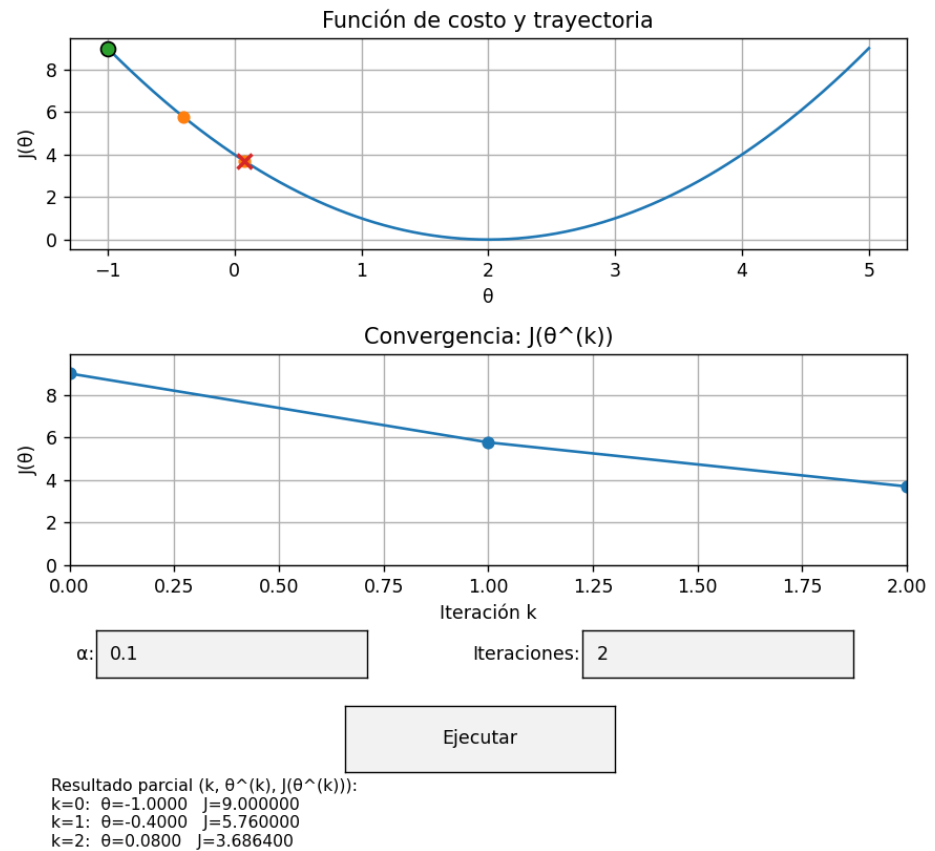
Gráfica 10- A medida que avanzan las iteraciones, $\theta^{(k)}$ se aproxima a 2 y el valor de $J(\theta^{(k)})$ disminuye Desarrollado en Python –Elaboración propia.



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>

Código

› Iteración 2

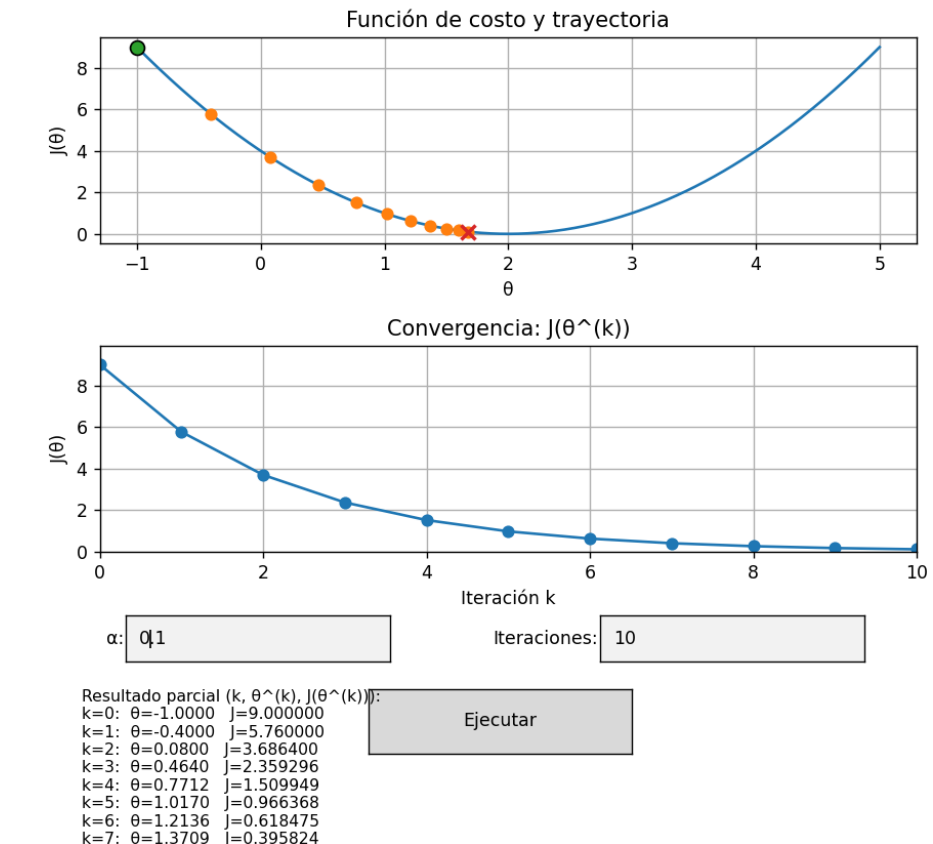


Gráfica 11 - A medida que avanzan las iteraciones, $\theta^{(k)}$ se aproxima a 2 y el valor de $J(\theta^{(k)})$ disminuye
Desarrollado en Python – Elaboración propia.



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>

› Iteración n



Gráfica 12 - A medida que avanzan las iteraciones, $\hat{\theta}^{(k)}$ se aproxima a 2 y el valor de $J(\hat{\theta}^{(k)})$ disminuye – Desarrollado en Python – Elaboración propia.



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>

› Extensión a varias variables

En muchos problemas de IA los parámetros son vectores de alta dimensión. Por ejemplo, en una regresión lineal simple con un parámetro de pendiente w y un sesgo b , el vector de parámetros es $\theta = (w, b)$. La función de costo típica (error cuadrático medio) para un conjunto de datos $\{(x_i, y_i)\}$ es:

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m (wx_i + b - y_i)^2.$$

El gradiente de J es ahora un vector:

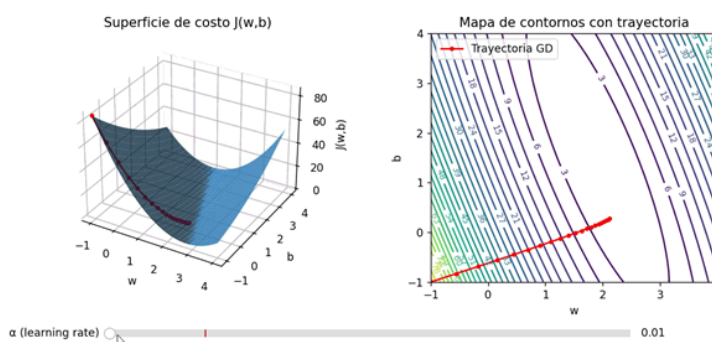
$$\nabla J(w, b) = \left(\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b} \right),$$

donde cada derivada parcial se puede calcular de manera explícita. Con estas expresiones se aplica el mismo esquema de actualización:

$$w^{(k+1)} = w^{(k)} - \alpha \frac{\partial J}{\partial w}(w^{(k)}, b^{(k)}),$$

$$b^{(k+1)} = b^{(k)} - \alpha \frac{\partial J}{\partial b}(w^{(k)}, b^{(k)}).$$

En términos geométricos, la superficie $J(w, b)$ es una especie de “valle” en dos dimensiones, y el algoritmo desciende por ese valle siguiendo la dirección del gradiente negativo, como muestra la animación.



Animación 1— Gráfica de superficie 3D de J - Desarrollado en Python—Elaboración propia.



Código

Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>



Multimedia

Para seguir aprendiendo sobre gradientes descendientes se sugiere ver Gradiente Descendente desde cero con Python

También disponible desde:
<https://tinyurl.com/3z2t5ezu>



- *Variantes: gradiente estocástico y mini-batch*

Cuando el número de ejemplos m es muy grande, calcular el gradiente exacto a partir de todos los datos en cada iteración puede resultar costoso. En los modelos de IA actuales, esto sucede con frecuencia.

Para enfrentar este problema se utilizan variantes:

- › **Descenso por gradiente estocástico (SGD):** en lugar de usar todos los ejemplos, se actualiza el parámetro utilizando un solo ejemplo (o un pequeño subconjunto) por iteración. El gradiente se vuelve ruidoso, pero el método es mucho más rápido y permite actualizar los parámetros casi en tiempo real.
- › **Descenso por gradiente mini-batch:** compromiso entre batch completo y SGD puro. En cada iteración se toma un subconjunto de tamaño moderado (por ejemplo, 32, 64 o 128 ejemplos) y se calcula el gradiente de su costo medio. Es la estrategia más habitual en el entrenamiento de redes neuronales.

En términos teóricos, estas variantes intentan aproximar el gradiente verdadero introduciendo cierto ruido que, paradójicamente, puede ayudar a escapar de mínimos locales poco interesantes en problemas no convexos.



Para profundizar lo visto hasta aquí, se le recomienda la lectura de Gradient Descent : Batch , Stochastic and Mini batch.



Asimismo, le sugerimos complementar la lectura con el siguiente video: *Gradiente Descendente: Batch vs Estocástico vs Mini-batch*

También disponible desde:
<https://tinyurl.com/hf6teafk>



Llegado a este punto se encuentra en condiciones de realizar la *Actividad 2: Análisis guiado del video sobre Descenso por Gradiente*.

› Tema 3: Programación convexa

La programación convexa es una de las bases matemáticas para el desarrollo de modelos de Inteligencia Artificial. Su importancia radica en que permite formular problemas de optimización donde la estructura misma del modelo —tanto la función objetivo como el conjunto de restricciones— garantiza que cualquier mínimo local sea también el mínimo global.

Esto brinda estabilidad, reproducibilidad y eficiencia computacional, cualidades esenciales en algoritmos modernos como la regresión regularizada, las máquinas de vectores de soporte y numerosos métodos de aprendizaje supervisado.

Este tema nos introduce a los conceptos fundamentales de convexidad, por ello analizaremos las propiedades que aseguran la unicidad del óptimo y presentaremos ejemplos típicos de programación lineal, cuadrática y regularizada, mostrando cómo estas herramientas matemáticas sustentan muchas de las soluciones prácticas utilizadas en IA.

› Convexidad y conjuntos convexos



Un conjunto convexo $C \subset \mathbb{R}^n$ es aquel que contiene por completo cualquier segmento de recta que una dos puntos del conjunto.

Formalmente, C es convexo si, para cualesquiera $x, y \in C$ y para todo $\theta \in [0, 1]$,

$$\theta x + (1 - \theta)y \in C.$$

En términos geométricos, si tomamos dos soluciones factibles de un problema de optimización que pertenecen a un conjunto convexo, cualquier combinación intermedia también es factible. Este hecho es fundamental en IA, porque muchos algoritmos exploran “camino” entre soluciones y necesitan que esos caminos permanezcan dentro de la región permitida.

Veamos un ejemplo:

Considere el conjunto de puntos en el plano que satisfacen

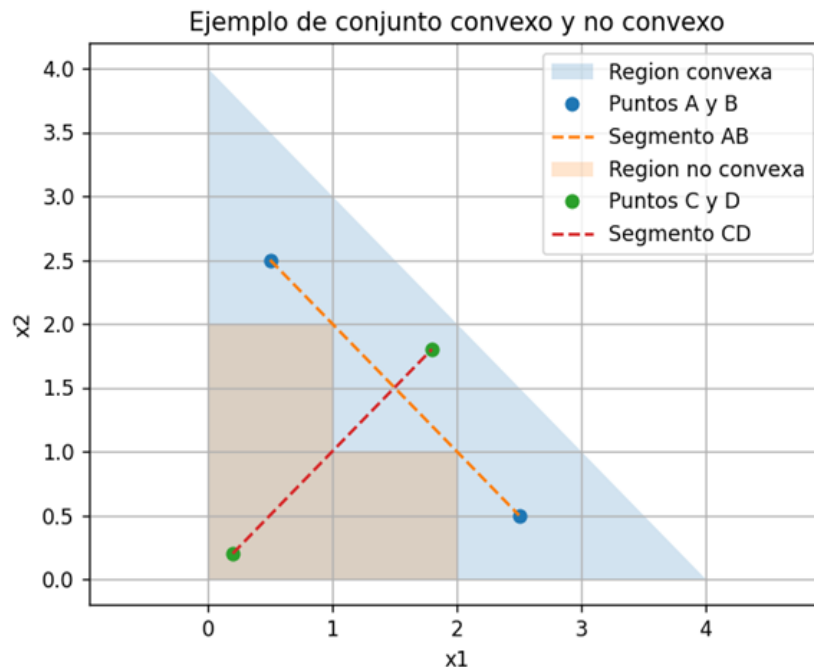
$$x_1 + x_2 \leq 4,$$

$$x_1 \geq 0,$$

$$x_2 \geq 0.$$

Esta región es un polígono (un triángulo) en el primer cuadrante. Si elegimos dos puntos cualesquiera de este triángulo, todo el segmento que los une permanece dentro de la región: el conjunto es convexo.

En cambio, el conjunto formado por dos discos separados en el plano no es convexo: el segmento que une un punto de cada disco pasa por zonas que no pertenecen al conjunto.



Gráfica 13 - gráfico en 2D que muestre en el mismo sistema de ejes una región convexa (por ejemplo, un polígono) y una región no convexa (por ejemplo, forma de "L"). Se traza el segmento entre dos puntos de cada región, resaltando cuándo el segmento queda dentro y cuándo sale. — Desarrollo Python — Elaboración propia.



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>

Código

› Funciones convexas

Una función $f: \mathbb{R}^n \rightarrow \mathbb{R}$ es convexa si su gráfica se encuentra siempre por debajo del segmento que une dos puntos cualesquiera de la gráfica. Formalmente, para todo $x, y \in \text{dom}(f)$, $\theta \in [0,1]$,

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y).$$

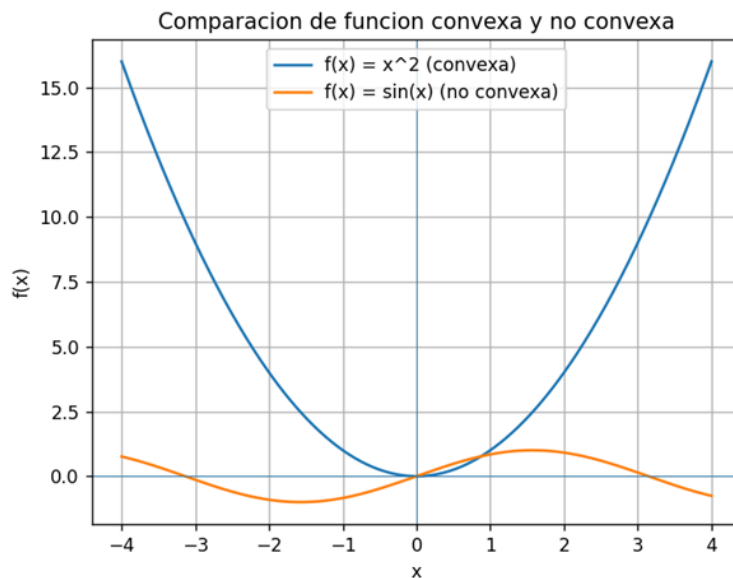
En $\mathbb{R}$ (una sola variable), una condición suficiente muy utilizada es que la segunda derivada sea no negativa en todo punto: si $f''(x) \geq 0$ para todo x , entonces f es convexa.

En $\mathbb{R}^n$, la extensión es que la matriz Hessiana $H_f(x)$ (matriz de segundas derivadas parciales) sea semidefinida positiva en todo punto del dominio.

Ejemplos:

- › $f(x) = x^2$ es convexa, porque $f''(x) = 2 > 0$.
- › $f(x) = |x|$ también es convexa, aunque no es diferenciable en $x = 0$.
- › $f(x) = \sin(x)$ no es convexa en todo $\mathbb{R}$, porque alterna curvaturas.

En IA, muchas funciones de costo (loss functions) se diseñan para ser convexas, ya que esta propiedad facilita la búsqueda del mínimo global.



Gráfica 14 - Gráfico 2D con dos funciones: x^2 (convexa) y $\sin(x)$ (no convexa), mostrando la diferencia en la forma de "tazón" frente a una forma ondulada con múltiples mínimos locales.
Desarrollo Python – Elaboración propia.



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>

Código

- **Programación convexa y unicidad del óptimo**

Un problema de optimización convexa tiene la forma general $\min_{x \in C} f(x)$, donde f es una función convexa y C es un conjunto convexo (conjunto de restricciones).

Dos propiedades clave:

1. **Cualquier mínimo local es también mínimo global.** Esto elimina el problema de quedar atrapado en mínimos locales “malos”, tan frecuente en problemas no convexos (como algunas redes neuronales profundas).

2. **Unicidad del óptimo:**

- › Si f es estrictamente convexa y el conjunto factible C es convexo, entonces el problema tiene a lo sumo una solución óptima.
- › En aplicaciones de IA, esto se traduce en modelos más estables: pequeños cambios en las condiciones iniciales o en el algoritmo no deberían producir soluciones radicalmente distintas.

Esta unicidad resulta muy deseable cuando se entrenan modelos de regresión o clasificación que deben ser reproducibles y explicables.

Veamos ahora un ejemplo de regresión lineal con regularización L2:

Supongamos que deseamos ajustar un modelo de regresión lineal para predecir el precio de una vivienda a partir de una única característica: la superficie en metros cuadrados.

Disponemos de datos (x_i, y_i) , donde:

- › x_i es la superficie de la vivienda i ,
- › y_i es el precio observado.

El modelo lineal sencillo es

$$\hat{y}_i = wx_i + b,$$

donde w es la pendiente y b es el término independiente.

La función de costo sin regularización (error cuadrático medio) puede escribirse, para simplicidad, como

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (y_i - (wx_i + b))^2.$$

Esta función es *convexa* en (w, b) .

Para mejorar la estabilidad del modelo y evitar sobreajuste, se añade un término de regularización L2:

$$J_{\lambda}(w, b) = \frac{1}{2n} \sum_{i=1}^n (y_i - (wx_i + b))^2 + \frac{\lambda}{2} w^2,$$

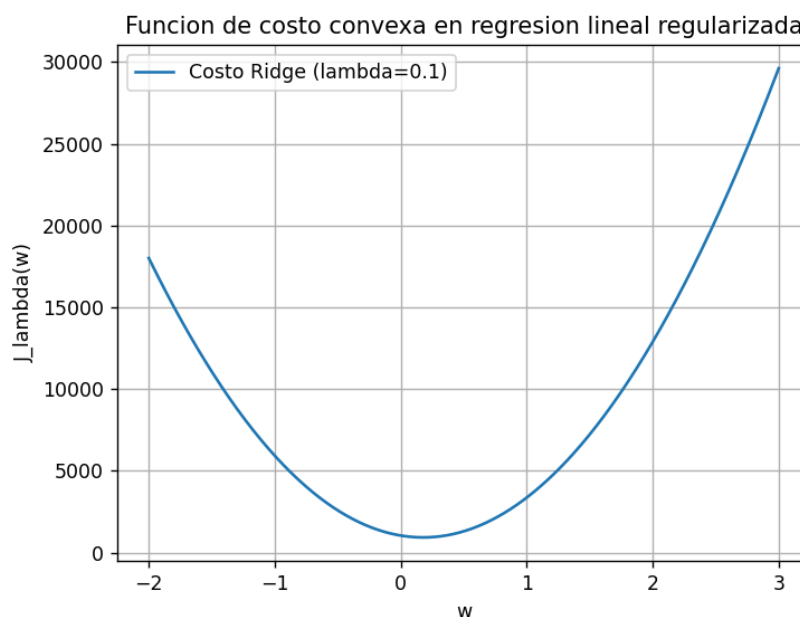
donde $\lambda \geq 0$ controla la fuerza de la regularización.

Ahora comenzaremos con el análisis de convexidad, paso a paso:

1. El término de error cuadrático $\frac{1}{2n} \sum (y_i - (wx_i + b))^2$ es una suma de funciones cuadráticas en w y b , luego es convexa.
2. El término $\frac{\lambda}{2} w^2$ es una función cuadrática en w con coeficiente positivo, por lo que también es convexa.
3. La suma de funciones convexas es convexa. Por lo tanto, J_{λ} es convexa en (w, b) .
4. Además, si $\lambda > 0$, el término w^2 introduce estricta convexidad en la dirección de w , favoreciendo la unicidad del óptimo.

En términos de IA, este problema es un caso típico de optimización convexa regularizada, que aparece en numerosos algoritmos (*regresión ridge*, *regresión logística regularizada*, etc.).

Para simplificar, fijemos el parámetro b (por ejemplo, igual al promedio de los y_i) y estudiemos la función $J_{\lambda}(w)$ en una dimensión. De este modo, podemos visualizar la “forma de tazón” del costo.



Gráfica 15—“tazón” del costo - Desarrollo Python –Elaboración propia.



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>

Código

- **Otros problemas convexos relevantes en IA**

Debemos aclarar que, aunque el ejemplo anterior se centra en regresión lineal, la programación convexa abarca una amplia familia de problemas que aparecen de forma recurrente:

- **Programación lineal (LP)**

La programación lineal (LP) es un tipo de optimización donde tanto la función que queremos minimizar o maximizar como las restricciones que limitan el problema son lineales. Su importancia radica en que estos problemas siempre forman una región factible convexa (un poliedro), lo que garantiza que cualquier solución óptima se encuentre en los vértices de esa región y pueda hallarse mediante métodos eficientes.

En el contexto de la Inteligencia Artificial, la programación lineal aparece cuando se necesita asignar recursos de manera óptima. Por ejemplo, si un sistema debe distribuir capacidad de cómputo entre distintos modelos —como un modelo de lenguaje, uno de visión y uno de recomendación— puede formularse un LP que:

- › maximice el rendimiento total del sistema,
- › asegurándose de no exceder el presupuesto de energía, memoria o tiempo.

Como todo en programación convexa, este tipo de problema proporciona soluciones estables y reproducibles, lo que es esencial en entornos donde el rendimiento y la eficiencia son críticos.

Veamos con un ejemplo sencillo, supongamos que tiene 10 horas para estudiar dos materias:

- › x : horas de Matemática
- › y : horas de Programación

Cada hora de Matemática suma 3 “*puntos de aprendizaje*” y cada hora de Programación suma 5 puntos.

Queremos repartir las 10 horas para maximizar los puntos totales, con estas condiciones:

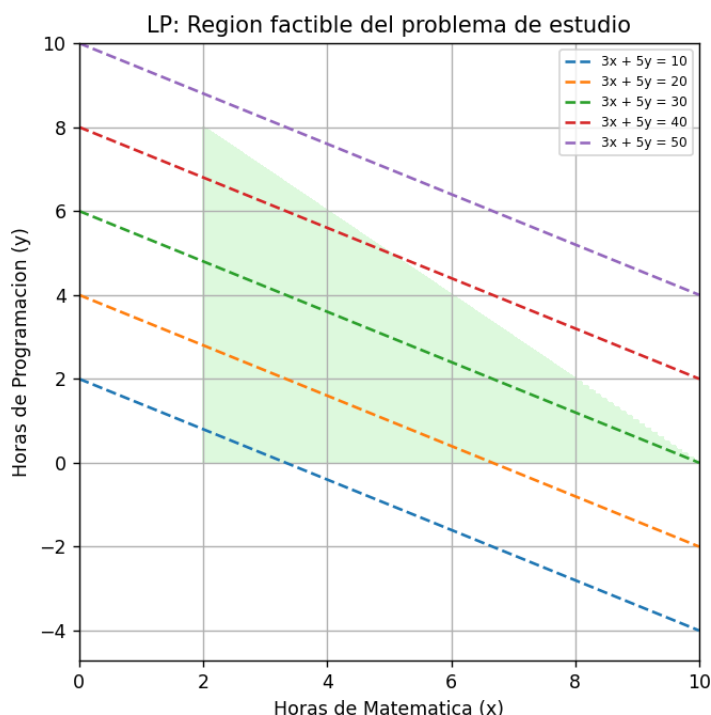
- › No podemos estudiar más de 8 horas de Programación: $y \leq 8$.
- › Queremos dedicar al menos 2 horas a Matemática: $x \geq 2$.
- › El total de horas no puede superar 10: $x + y \leq 10$

$$x \geq 0, y \geq 0$$

- › **Función objetivo (lo que maximizamos):**

$$\text{Puntos} = 3x + 5y$$

Es un problema lineal y convexo: las restricciones forman un polígono convexo en el plano x, y .



Gráfica 16—LP - Desarrollo Python—Elaboración propia.



Código

Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>



Lectura
complementaria

Para profundizar lo visto se recomienda la lectura del siguiente sitio: *Formulación de modelos de programación lineal*

► Programación cuadrática (QP)

La programación cuadrática (QP) es un tipo de optimización donde la función objetivo es cuadrática y convexa, y las restricciones son lineales. Esto permite resolver problemas en los que se busca equilibrar varios factores con una penalización suave, manteniendo al mismo tiempo un conjunto factible sencillo.

Un ejemplo muy conocido es la formulación clásica de las Máquinas de Vectores de Soporte (SVM): el modelo busca el hiperplano que separe dos clases con el mayor margen posible. Esa búsqueda se expresa como la minimización de una función cuadrática que depende de los parámetros del hiperplano, sujetando la solución a restricciones lineales que garantizan que los datos queden correctamente clasificados.

La razón por la cual este enfoque es tan valioso es que la convexidad del problema asegura que exista un único óptimo global, lo que hace que las SVM sean modelos estables, reproducibles y matemáticamente bien fundamentados.

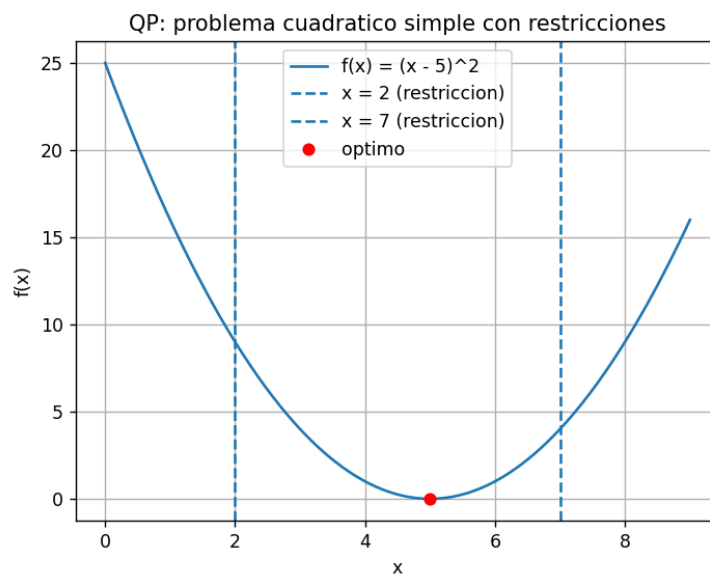
Veamos un ejemplo sencillo para analizar este comportamiento, queremos elegir un valor x que esté lo más cerca posible de 5, pero con una penalización:

$$\min_x f(x) = (x - 5)^2$$

Si agregamos una restricción lineal, por ejemplo:

$$x \geq 2, x \leq 7$$

La solución sigue siendo el punto donde la parábola alcanza su mínimo, siempre que esté dentro del intervalo. Aquí el mínimo está en $x = 5$, que cumple las restricciones, así que el óptimo es $x^* = 5$.



Gráfica 17—GP - Desarrollo Python—Elaboración propia.



Código

Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>



Multimedia

Además, para ampliar el tema que hemos abordado le recomendamos ver: *Máquinas de Soporte Vectorial | Cómo funcionan las SVM | Código en python*

También disponible desde:
<https://tinyurl.com/mv2ahn98>



- **Optimización regularizada L1 (LASSO)**

La regularización L1 (LASSO) es una técnica de optimización convexa que agrega al modelo una penalización basada en la suma de los valores absolutos de los coeficientes. Esta penalización “empuja” a muchos de esos coeficientes a ser exactamente cero, lo que produce modelos dispersos y, por lo tanto, más simples e interpretables.

En Inteligencia Artificial esto es particularmente útil cuando se trabaja con conjuntos de datos que tienen muchísimas variables: LASSO actúa como un mecanismo automático de selección de características, conservando solo las más relevantes y descartando el ruido. Al ser un problema convexo, garantiza soluciones estables y un óptimo global.

En otras palabras, en muchos modelos queremos que algunos parámetros sean exactamente cero para simplificar el modelo (usar menos características). La regularización L1 agrega una penalización basada en el valor absoluto de los coeficientes:

$$\min_w \underbrace{(w - 3)^2}_{\text{ajuste a los datos}} + \lambda |w|_{\text{penalización L1}}$$

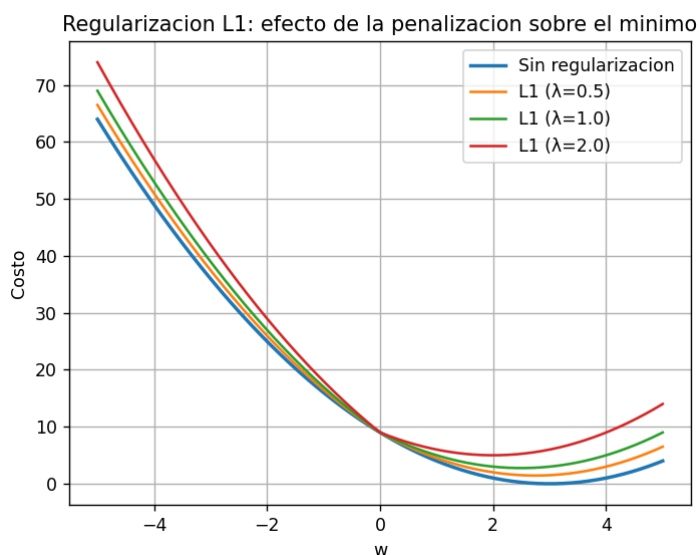
›

El primer término intenta que w sea cercano a 3.

› El segundo término “empuja” a w hacia 0.

› El parámetro λ controla cuánta presión hacia 0 hay.

Es un problema convexo (parábola + V) pero no derivable en 0; aun así, se resuelve bien numéricamente.



Gráfica 18—L1 - Desarrollo Python—Elaboración propia



Puede encontrar el enlace de descarga en <https://tinyurl.com/yd4tk44x>



Multimedia

Le recomendamos acceder al enlace “*Entrenando redes neuronales profundas*”. El mismo corresponde a un interesante curso de Deep Learning que puede ser de utilidad.



Actividad

Habiendo llegado hasta aquí, usted está en condiciones de realizar la *Actividad 3: Análisis práctico de un problema de regresión lineal regularizada como problema de programación convexa*.



Multimedia

Finalmente, le proporcionamos un recurso audiovisual que relaciona todos los temas tratados en el presente módulo: *Optimización Matemática en IA*



También disponible desde:

<https://tinyurl.com/u9jhjtb>

El trabajo durante este módulo nos permitió consolidar los fundamentos teóricos y prácticos de la optimización matemática aplicada a la inteligencia artificial. A través del estudio de funciones objetivo, restricciones, criterios de optimalidad, gradientes y programación convexa, hemos incorporado las herramientas esenciales que sustentan los procesos de entrenamiento de modelos modernos. Comprender cómo se formulan y resuelven problemas de optimización, tanto en su versión lineal como no lineal, es indispensable para interpretar el comportamiento de algoritmos de aprendizaje, evaluar su desempeño y tomar decisiones fundamentadas sobre su ajuste y estabilidad. Por lo tanto, un puente entre la teoría matemática rigurosa y las aplicaciones concretas de la IA, permite fortalecer la capacidad analítica necesaria para abordar modelos más complejos en los módulos posteriores.

¡Nos vemos en el próximo y último módulo!

MÓDULO 4 | GLOSARIO

Criterio de optimalidad: Condición matemática que permite verificar si un punto candidato es un mínimo o máximo de una función. En funciones diferenciables, suele basarse en el análisis de la primera y segunda derivada, o del gradiente y la matriz Hessiana en espacios multidimensionales.

Dominio: Conjunto de valores permitidos para las variables de una función u objeto matemático. En optimización, determina la región donde se busca la solución. Puede ser todo el espacio real o estar delimitado por restricciones.

Función convexa: Función cuya curva se encuentra siempre por debajo de cualquier segmento que una dos puntos de su gráfica. Este tipo de funciones garantiza la existencia de un único mínimo global y facilita el análisis de convergencia de métodos de optimización.

Función objetivo: Expresión matemática que representa aquello que se desea minimizar o maximizar en un problema de optimización. En inteligencia artificial suele corresponder a una medida de error, pérdida o desempeño del modelo.

Gradiente: Vector que contiene las derivadas parciales de una función en cada una de sus variables. Indica la dirección de mayor crecimiento de la función y es fundamental en algoritmos de optimización basados en derivadas, como el descenso por gradiente.

Mínimo global: Punto del dominio de una función donde esta alcanza su menor valor absoluto dentro de toda la región considerada. Es el objetivo ideal en la mayoría de los problemas de optimización para entrenamiento de modelos.

Mínimo local: Punto donde la función toma un valor menor que en los puntos cercanos, pero no necesariamente el más bajo de todo el dominio. Es frecuente en funciones no convexas, como las asociadas al entrenamiento de redes neuronales profundas.

Optimización lineal (Programación lineal): Rama de la optimización que aborda problemas cuya función objetivo y restricciones son lineales. Permite resolver de manera eficiente modelos con múltiples variables y restricciones, y posee aplicaciones en asignación, logística y toma de decisiones.

Optimización no lineal: Área que estudia problemas donde la función objetivo o las restricciones presentan componentes no lineales. Suele ser más compleja que la lineal e incluye métodos iterativos basados en derivadas y aproximaciones numéricas.

Programación convexa: Conjunto de técnicas y modelos de optimización donde tanto el dominio como la función objetivo son convexos. Se caracteriza por garantizar soluciones únicas y estables, siendo central en métodos de regularización y modelos como SVM.

Punto crítico: Valor de una variable donde la derivada (o gradiente) se anula. Puede corresponder a un mínimo, un máximo o un punto de inflexión, dependiendo del comportamiento local de la función.

Región factible: Conjunto de puntos del dominio que cumplen con todas las restricciones del problema de optimización. Representa el espacio donde la solución es válida.

Restricción: Condición que limita los valores permitidos para las variables de un problema. Puede expresarse como ecuaciones o desigualdades y define la forma y tamaño del dominio factible.

Solución óptima: Valor o conjunto de valores que permiten alcanzar el mínimo o máximo de la función objetivo dentro del dominio factible. Representa el resultado final del proceso de optimización.

MÓDULO 4 | ACTIVIDADES



Actividad

ACTIVIDAD N° 1: Problema 1

Resuelva el siguiente problema y explique cada paso, tal como se hizo en el ejemplo presentado en el desarrollo del módulo:

$$h(x)=(x+1)^2+4$$

1. Encuentre el mínimo mediante derivadas.
2. Verifique si es realmente un mínimo.
3. Calcule el valor mínimo.
4. Interprete el resultado como si $h(x)$ fuera una función de pérdida.
5. Genere en Python la gráfica de $h(x)$ marcando el punto óptimo.

Extensión sugerida: entre media página y una página.



Actividad

ACTIVIDAD N° 2: Análisis guiado del video sobre Descenso por Gradiente

El objetivo de la actividad radica en relacionar los conceptos teóricos de la unidad con un ejemplo práctico visual en video, reforzando así la comprensión del gradiente, su cálculo y su aplicación en la optimización de modelos de IA.



Multimedia

1. Para comenzar —y si aún no lo hizo— vea el video titulado “*Gradiente Descendente desde cero con Python*” (presentado en los contenidos del módulo):

También disponible desde:
<https://tinyurl.com/3z2t5ezu>



2. Durante la visualización, tome nota de los siguientes puntos clave:
 - › ¿Cuál es la función de costo que se presenta o asume?

- › ¿Cómo se describe la dirección del gradiente y su signo (positivo/negativo) para la actualización de los parámetros?
 - › ¿Qué ejemplos de actualización de parámetros se muestran (iteraciones, tasa de aprendizaje, escollo de convergencia)?
 - › ¿Se menciona alguna variante del método clásico de descenso por gradiente (por ejemplo, mini-batch, estocástico, adaptativo)?
3. A continuación, responda las siguientes preguntas en un informe breve (máximo 2 páginas de extensión):
- a) Resuma con sus propias palabras lo que el video muestra respecto a cómo el gradiente guía la búsqueda del mínimo.
 - b) Elija un parámetro (real o hipotético) y describa qué pasaría si la tasa de aprendizaje fuera demasiado alta o baja, basándose en lo que vio en el video.
 - c) Aplique el ejercicio unidimensional del ejemplo del tema (por ejemplo $J(\theta) = (\theta - 2)^2$), pero cambie la tasa de aprendizaje a un valor diferente al que se usó como ejemplo en clase (por ejemplo, $\alpha = 0,05$ o $\alpha = 0,2$).
- › Ejecute al menos 5 iteraciones y muestre la evolución de $\theta^{(k)}$ y de $J(\theta^{(k)})$.

Utilice el código que se compartió.

- d) Reflexione: Según el video, ¿cuál es la ventaja de usar variaciones de descenso por gradiente (mini-batch, adaptativo) en problemas reales de IA? ¿Cómo cambiaría lo visto si el dataset fuera muy grande?

¿Cómo entregar esta actividad?

Suba el informe junto con capturas de pantalla (o export del script) que muestren la evolución de θ y de J . También incluya en el encabezado del archivo: nombre, fecha, módulo, tema (“Tema 2: Gradientes y técnicas basadas en derivadas”).



ACTIVIDAD N° 3:

Análisis práctico de un problema de regresión lineal regularizada como problema de programación convexa.

La siguiente consigna puede resolverse de forma individual o en pequeños grupos:

1. En Visual Studio Community, cree un proyecto de Python y elabore un script que:
 - › Genere un conjunto sintético de datos (x_i, y_i) similar al del ejemplo (superficie vs precio), con algo de ruido.

- › Implemente la función de costo de regresión lineal con y sin regularización L2.
 - › Calcule el valor del costo para distintos valores de w , manteniendo b fijo (por ejemplo, el promedio de los y_i), y grafique $J(w)$ para varios valores de w .
2. A partir de las gráficas generadas:
- › Identifique visualmente el mínimo global en cada caso.
 - › Analice cómo cambia la posición del mínimo y la curvatura del “tazón” al modificar λ .
 - › Explique por qué, desde la teoría de programación convexa, el problema sigue teniendo un único óptimo global para cada $\lambda > 0$.
3. Redacte un breve informe (1–2 páginas) donde:
- › Formule explícitamente el problema de optimización convexa (*función objetivo y restricciones*).
 - › Justifique la convexidad de la función de costo empleando argumentos matemáticos.
 - › Relacione los resultados numéricos con la importancia de la convexidad para garantizar soluciones estables en modelos de IA.

MÓDULO 5

Microobjetivos

- › Identificar las estructuras básicas de un grafo para reconocer los tipos de relaciones presentes en modelos aplicados a inteligencia artificial en contextos reales o simulados.
- › Representar grafos mediante matrices y listas de adyacencia para formalizar relaciones y dependencias en sistemas y flujos de información utilizados en IA.
- › Aplicar los recorridos DFS y BFS para analizar conectividad, niveles de propagación y componentes dentro de modelos basados en grafos.
- › Determinar la accesibilidad de un grafo mediante el cierre transitivo para comprender dependencias directas e indirectas en estructuras de decisión y sistemas inteligentes.



VIDEO DOCENTE

Contenidos

- **Teoría de Grafos para la Inteligencia Artificial**

La teoría de grafos constituye otro de los fundamentos matemáticos que permite comprender el funcionamiento interno de numerosos modelos y algoritmos de la inteligencia artificial moderna. Desde redes neuronales y sistemas de recomendación hasta análisis de redes sociales, grafos semánticos y modelos probabilísticos, gran parte de las estructuras que sustentan estos sistemas pueden describirse como conjuntos de nodos interrelacionados mediante conexiones de distinto tipo. Por ello, el estudio formal de los grafos permite desarrollar una base rigurosa para interpretar y analizar problemas donde la conectividad, la accesibilidad y la propagación de información son elementos esenciales.

En este módulo abordaremos los fundamentos matemáticos necesarios para trabajar con grafos: su definición formal, los tipos más relevantes, las representaciones mediante matrices y listas de adyacencia; también los algoritmos básicos de recorrido y búsqueda, como BFS y DFS. Además, incorporaremos el concepto de cierre transitivo, una herramienta algebraica clave que permite determinar la accesibilidad entre nodos a partir de operaciones matriciales.

El propósito de esta unidad es que se adquiera una comprensión conceptual profunda de los grafos como estructuras matemáticas y relacione estas ideas con casos reales en inteligencia artificial. Aunque en ocasiones los grafos se estudian desde la programación o la ingeniería, en este módulo los abordaremos desde su dimensión matemática, enfatizando propiedades formales, operaciones y comportamientos que pueden generalizarse y aplicarse en múltiples áreas de la IA.

El dominio de estos conceptos permitirá, en instancias posteriores, comprender cómo se modelan las redes complejas, cómo se calculan trayectorias, cómo se detectan patrones de conectividad y cómo se fundamentan matemáticamente diversas técnicas empleadas en sistemas inteligentes.

¿Qué es un grafo?



Un grafo es una estructura que modela relaciones entre pares de elementos. Está compuesto por un conjunto de nodos (también llamados vértices) y un conjunto de aristas (o arcos) que conectan pares de nodos.

En matemáticas, un grafo se define como:

$G = (V, E)$ donde:

- › V es el conjunto de vértices o nodos
- › $E \subseteq V \times V$ es el conjunto de aristas o relaciones entre nodos

Esta estructura permite modelar procesos donde intervienen relaciones binarias: conectividad, dependencias, propagación, accesibilidad o tránsito de información.

Por ejemplo, en un sistema de recomendación, cada usuario puede representarse como un nodo, y existe una arista entre dos usuarios si comparten un patrón de comportamiento relevante (por ejemplo, más del 80 % de coincidencia en preferencias). Este grafo permite estudiar comunidades y comportamientos colectivos.

• Tipos de grafos relevantes en IA

› Grafo dirigido - Digrafo

Modelan relaciones asimétricas:

$$(i, j) \neq (j, i)$$

Las aristas tienen un sentido (de un nodo a otro). En estos, cada arista tiene una dirección que indica una relación unidireccional entre dos nodos. Este tipo de grafos se utiliza para modelar situaciones en las que las relaciones no son bidireccionales, por ejemplo, un grafo donde cada nodo representa una página web y las aristas representan enlaces. Es pertinente para introducir modelos matemáticos de análisis en motores de búsqueda (por ejemplo, PageRank).

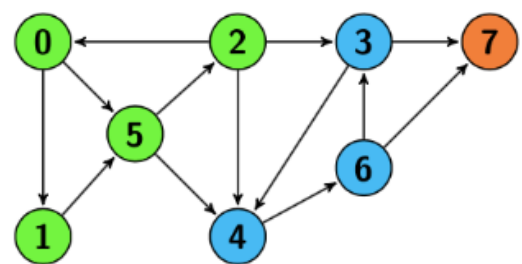


Ilustración 1 - Grafos dirigidos - Fuente: <https://tinyurl.com/yntk8bs8>

› **Grafo no dirigido**

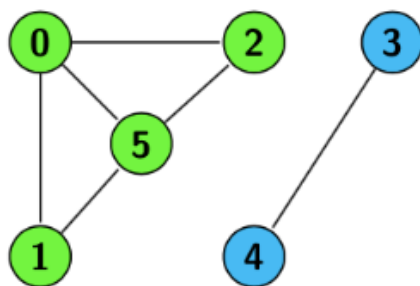


Ilustración 2 - Grafo no dirigido - Fuente: <https://tinyurl.com/mr38pf9b>

Representan relaciones simétricas. Las aristas no tienen dirección. En otras palabras, si hay una relación entre dos nodos, esta relación es bidireccional; ambos nodos están conectados de manera equivalente. Este tipo de grafos se utiliza comúnmente en situaciones donde las conexiones son recíprocas, como en las redes sociales donde la amistad es mutua o red de similitud entre imágenes donde la arista indica similitud perceptual basada en embeddings visuales.

› **Grafo ponderado**

Cada arista tiene un peso:

$$w: E \rightarrow \mathbb{R}^+$$

En este tipo de grafo, cada arista tiene un valor asociado, conocido como peso o costo. Este valor puede representar diversas magnitudes, como distancia, tiempo, capacidad, costo, entre otros. Por ejemplo, en un grafo de recomendación musical, las aristas pueden llevar un peso que represente la distancia entre vectores de características (embedding) de dos canciones.

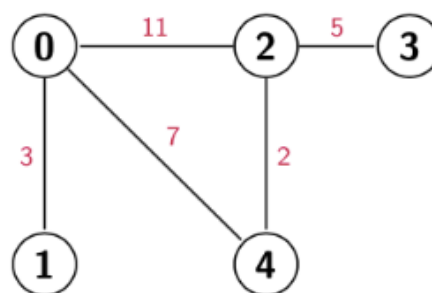


Ilustración 3 - Grafo ponderado
Fuente: <https://tinyurl.com/mr2hc3ph>

› **Grafo bipartito**

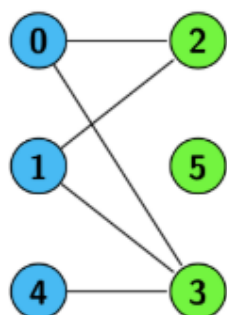


Ilustración 4 - Grafo bipartito - Fuente: <https://tinyurl.com/5adt5ezn>

Conjuntos disjuntos U y W :

$$E \subseteq U \times W$$

Un grafo bipartito es aquel cuyos nodos pueden ser divididos en dos conjuntos disjuntos de tal manera que todas las aristas conectan nodos de conjuntos diferentes.

Este tipo de grafo es útil para modelar problemas de emparejamiento, como la asignación de tareas a recursos o la relación entre dos grupos de entidades, por ejemplo, estudiantes y proyectos.

› Grafo cíclico

Un grafo cíclico contiene al menos un ciclo, es decir, una secuencia de aristas que comienza y termina en el mismo nodo, formando un camino cerrado. Los grafos cíclicos son esenciales en la representación de problemas que implican repetición y bucles, como los circuitos eléctricos y ciertos algoritmos de programación o red de sensores donde un paquete de datos puede regresar al nodo inicial por múltiples rutas

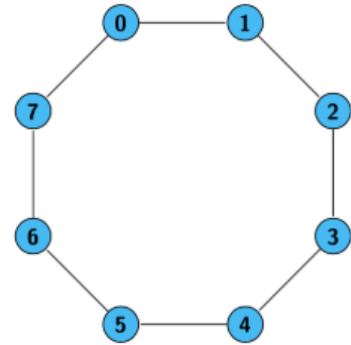


Ilustración 5 - Ciclo

Fuente: <https://tinyurl.com/4xnhat47>

› Grafo acíclico

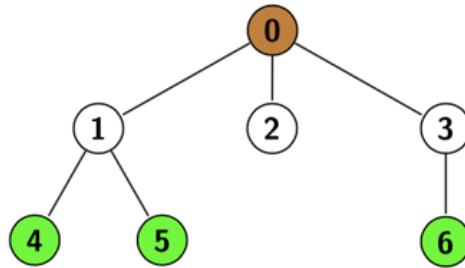


Ilustración 6 - Grafo acíclico

Fuente: <https://tinyurl.com/3w624fm9>

Por el contrario, un grafo acíclico no contiene ningún ciclo. Los grafos acíclicos son fundamentales en el modelado de estructuras jerárquicas y dependencias, como las redes de precedencia de tareas, los árboles de decisión, y las representaciones de flujos de trabajo.

La diversidad de tipos de grafos permite abordar una amplia gama de problemas y aplicaciones en informática, operativa y otras disciplinas como la IA. Cada tipo de grafo tiene sus propias características y ventajas, lo que los hace adecuados para diferentes contextos y necesidades.

› Representaciones matemáticas

Las formas más usuales de representar grafos son:

• Matriz de Adyacencia

Sea $n = |V|$.

La matriz A se define como:

$$A_{ij} = \begin{cases} 1 & \text{si } (i,j) \in E \\ 0 & \text{si no existe arista} \end{cases}$$

La matriz de adyacencia es una representación en forma de tabla bidimensional (o matriz) de tamaño $n \times n$, donde n es el número de nodos en el grafo. Cada posición $A[i][j]$ de la matriz indica si existe una arista entre el nodo i y el nodo j . Si existe tal conexión, el valor

de $A[i][j]$ será 1 (o un valor diferente de cero en algunos casos), mientras que, si no hay conexión, el valor será 0.

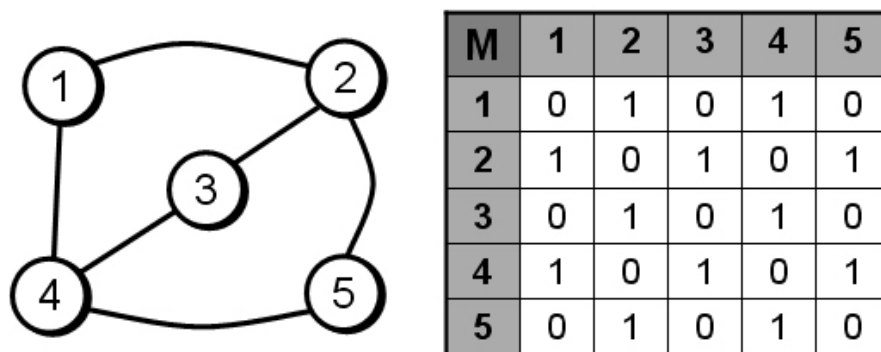


Ilustración 7 - Matriz de Adyacencia - Fuente: <https://tinyurl.com/ajpbx3z8>

Esta técnica es especialmente útil para grafos densos, donde el número de aristas es cercano al número máximo posible en el grafo, ya que permite consultar rápidamente si dos nodos están conectados. Sin embargo, en grafos dispersos, donde el número de aristas es mucho menor que el número posible, el uso de una matriz de adyacencia puede ser ineficiente en términos de memoria, ya que muchas posiciones de la matriz permanecerán en cero.

Un beneficio clave de esta representación es su simplicidad para realizar ciertas operaciones, como encontrar aristas o calcular el grafo transpuesto. Sin embargo, para grafos con muchos nodos y pocas conexiones, otras representaciones como la lista de adyacencia pueden ser más adecuadas.

- **Lista de Adyacencia**

Una función:

$$L(v) = \{u \in V : (v, u) \in E\}$$

Cada nodo tiene una lista con los nodos adyacentes. La lista de adyacencia es una representación alternativa para los grafos que resulta particularmente eficiente en el manejo de grafos dispersos, es decir, aquellos que tienen un número de aristas significativamente menor en comparación con el número máximo posible de conexiones.

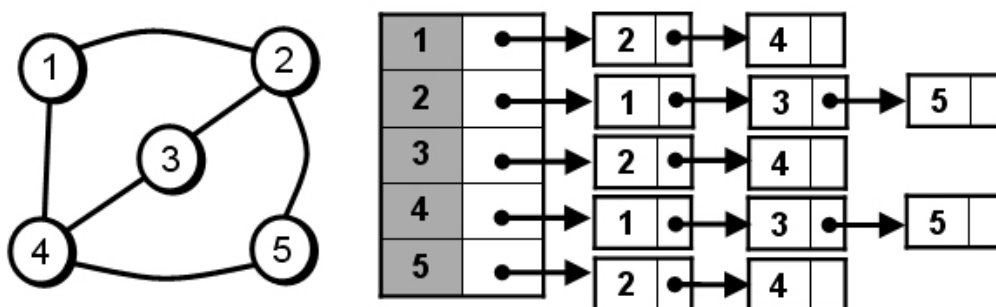


Ilustración 8 - Lista de Adyacencia - Fuente: <https://tinyurl.com/32vbsj5d>

En este método, cada nodo del grafo está asociado con una lista que contiene los nodos a los que está directamente conectado mediante aristas. Estas listas pueden implementarse utilizando estructuras como arrays dinámicos o listas enlazadas, dependiendo de las necesidades específicas del sistema.

Una de las ventajas clave de la lista de adyacencia es su eficiencia en el uso de memoria, ya que solo se almacenan las conexiones existentes en lugar de ocupar espacio para todas las posibles combinaciones de nodos, como ocurre en la matriz de adyacencia. Esto la convierte en una opción ideal para grafos grandes y dispersos. Además, esta representación facilita ciertas operaciones como la enumeración de vecinos de un nodo, lo que es útil en algoritmos de recorrido como el DFS (*Depth-First Search*) o BFS (*Breadth-First Search*).

Sin embargo, esta técnica también tiene sus limitaciones. Por ejemplo, determinar si dos nodos están directamente conectados puede ser más lento que en una matriz de adyacencia, ya que requiere recorrer la lista asociada a uno de los nodos. Por lo tanto, la elección entre lista de adyacencia y matriz de adyacencia debe hacerse considerando el tipo de grafo y las operaciones que se realizarán con mayor frecuencia.

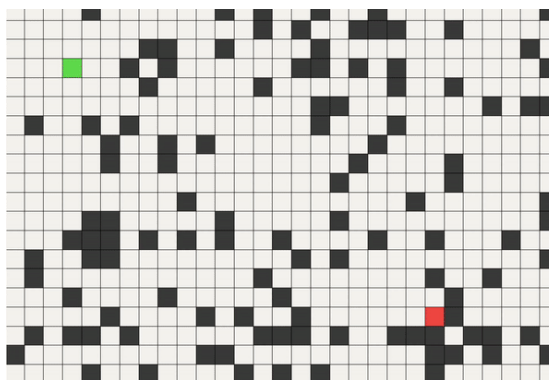
- **Recorridos y búsquedas en grafos**

- › **Recorrido en profundidad (DFS):**

El recorrido en profundidad, conocido como Depth-First Search (DFS), es un procedimiento matemático-computacional que explora un grafo siguiendo un camino todo lo posible antes de retroceder.

Esto significa que, comenzando en un nodo inicial, el algoritmo avanza hacia uno de sus vecinos no visitados, luego hacia un vecino de ese vecino, y así sucesivamente. Solo cuando no quedan más nodos nuevos por explorar en esa rama, el algoritmo regresa al nodo anterior y continúa por otra ruta disponible.

Esta estrategia genera un patrón de exploración profundo y secuencial que permite revelar la estructura interna del grafo.



Animación 1 - Búsqueda en profundidad – Fuente: <https://tinyurl.com/44cyztc8>



Le recomendamos leer el artículo del enlace <https://tinyurl.com/bc2jap28> para comprender el funcionamiento interno de este tipo de recorridos en un proceso de ejecución de un desarrollo Python.



Por otro lado, a continuación, encontrará una animación del recorrido en profundidad

También disponible desde:
<https://tinyurl.com/bcadwu68>



- **Propiedades matemáticas del DFS**

Detección de ciclos	Si durante el recorrido se encuentra un nodo que ya ha sido visitado y que no corresponde al nodo anterior en la recursión, entonces existe un ciclo. <i>Matemáticamente, esto significa que hay un conjunto de nodos:</i> $V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_k \rightarrow V_1$ lo que indica un camino cerrado. DFS puede identificar estos patrones debido a su registro de nodos visitados y rutas activas.
Producción de árboles de profundidad	El recorrido DFS genera un árbol de expansión en profundidad, donde cada arista elegida para avanzar forma parte de un árbol que representa la estructura de exploración. <i>Ese árbol:</i> › es un subgrafo del grafo original, › contiene todos los nodos alcanzables desde el nodo inicial, › y refleja la jerarquía de decisiones de exploración. Este árbol es útil para representar dependencias

Conectividad y análisis de regiones

DFS permite identificar componentes conexas:

- › En grafos no dirigidos, cada vez que DFS se inicia desde un nodo no visitado, se obtiene una componente.
- › En grafos dirigidos, ayuda a determinar regiones fuerte o débilmente conectadas, según cómo se interpreten las direcciones de las aristas.

En términos matemáticos, permite responder preguntas como:

- › ¿Qué nodos son accesibles desde un cierto nodo?
- › ¿Cuántas regiones independientes existen en el grafo?
- › ¿Qué subestructuras se forman a partir de las conexiones existentes?

Veamos un ejemplo donde se usa DFS en análisis de imágenes: segmentación en componentes conexas

En visión por computadora, una imagen puede modelarse como un grafo no dirigido, donde:

- › cada píxel es un nodo,
- › las conexiones se establecen con sus vecinos inmediatos (*según la vecindad elegida*),
- › la vecindad de 4 considera arriba, abajo, izquierda y derecha,
- › la vecindad de 8 añade las diagonales.

El objetivo de segmentar una imagen en componentes conexas consiste en identificar regiones donde los píxeles comparten una característica (*por ejemplo, color o intensidad similar*).

DFS funciona así:

1. Se toma un píxel inicial que cumpla cierta condición (por ejemplo, “es blanco”).
2. DFS explora recursivamente todos los píxeles vecinos que también sean blancos.
3. Cuando no encuentra más, devuelve el control.
4. Se elige otro píxel no visitado que también sea blanco y se repite el proceso.

El resultado es un conjunto de regiones aisladas, cada una representada por un árbol DFS. Esto es fundamental en IA para:

- › preprocesamiento de datos,
- › segmentación automática,
- › identificación de objetos,
- › análisis de patrones espaciales.

Veamos:

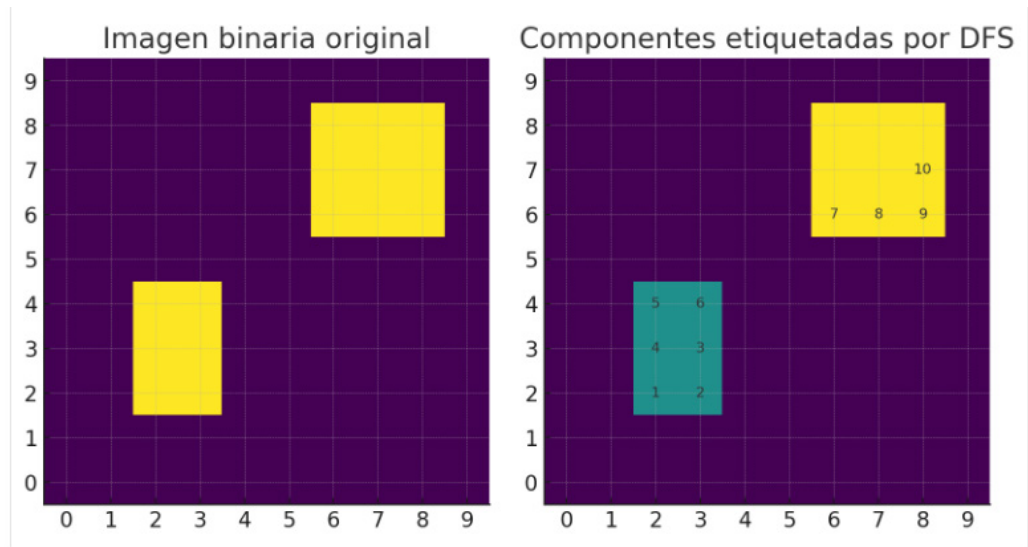


Ilustración 9 - Componentes etiquetadas por DFS—Figura generada con Python: DFS sobre una “imagen”—Elaboración propia



Puede descargar el código aquí: <https://tinyurl.com/mrxhm8c4>

Código

La figura que se generó con el código muestra:

- › **Izquierda:** una imagen binaria 10×10
 - › Fondo = 0 (negro)
 - › Dos regiones “blancas” (valor 1) que simulan objetos en la imagen.
- › **Derecha:** resultado de aplicar *DFS por vecindad 4*
 - › Cada región recibe una etiqueta distinta (1, 2, ...).
 - › Sobre la segunda imagen se escriben los números 1, 2, 3, ... 10 indicando el orden de visita de los primeros píxeles recorridos por DFS.

Esta figura ilustra visualmente cómo DFS:

1. Entra en una región (primer objeto).
2. Recorre todos sus píxeles conectados antes de pasar al siguiente objeto.
3. Luego inicia una nueva DFS sobre la otra región.

- **Pseudocódigo matemático de DFS para segmentar una imagen**

Consideremos:

- › I : matriz binaria de tamaño $m \times n$, con valores 0 (fondo) o 1 (objeto).
- › L : matriz de etiquetas, inicialmente en 0.
- › V : matriz booleana de visitados, inicialmente en FALSO.

Usaremos vecindad 4 (arriba, abajo, izquierda, derecha).

DFS	Algoritmo principal de segmentación
<pre> Subprocedimiento DFS(r, c, etiqueta) Entrada: r, c : coordenadas del píxel actual etiqueta : etiqueta actual de la componente conexa Si V[r, c] = VERDADERO entonces retornar FinSi V[r, c] ← VERDADERO L[r, c] ← etiqueta Para cada (dr, dc) en {(-1,0), (1,0), (0,-1), (0,1)} hacer r2 ← r + dr c2 ← c + dc Si 0 ≤ r2 < m y 0 ≤ c2 < n entonces Si I[r2, c2] = 1 y V[r2, c2] = FALSO entonces DFS(r2, c2, etiqueta) FinSi FinSi FinPara FinSubprocedimiento </pre>	<pre> Algoritmo Segmentar_Imagen_DFS(I) Crear matrices V[m, n] llenas de FALSO Crear matriz L[m, n] llena de 0 etiqueta ← 0 Para r desde 0 hasta m - 1 Para c desde 0 hasta n - 1 Si I[r, c] = 1 y V[r, c] = FALSO entonces etiqueta ← etiqueta + 1 DFS(r, c, etiqueta) FinSi FinPara FinPara Devolver L FinAlgoritmo </pre>



Multimedia

A continuación, ponemos a disposición un podcast explicativo: *DFS Segmentación de Imágenes La Magia Oculta*.

También disponible desde:
<https://tinyurl.com/2tvvztjk>



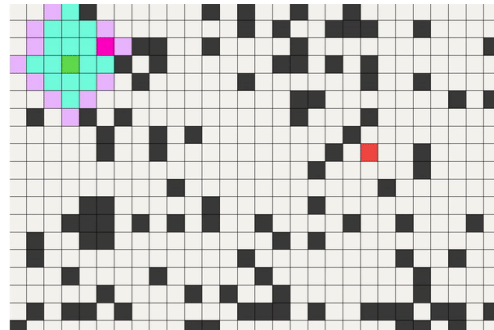
- **Recorrido en amplitud (BFS)**

El recorrido en amplitud, conocido como Breadth-First Search (BFS), es un método de exploración de grafos que avanza nivel por nivel. A diferencia de DFS, que sigue un camino profundo hasta agotar nodos, BFS explora primero todos los vecinos directos del nodo inicial, luego los vecinos de esos vecinos, y así sucesivamente.

Desde el punto de vista matemático, BFS construye capas sucesivas de nodos alcanzables:

$\ell(v)$ = mínima cantidad de aristas necesarias para llegar al nodo v .

Es decir, la función $\ell(v)$ asigna a cada nodo su *distancia mínima* respecto del origen, medida en número de pasos. Esta distancia no depende de pesos solo de la estructura del grafo— lo que hace a BFS la herramienta adecuada para grafos *no ponderados* o para problemas donde todos los pasos tienen el mismo costo.



Animación 2 - Búsqueda en amplitud—Fuente: <https://tinyurl.com/mvdxp2nn>

- **Propiedades matemáticas del BFS**

Cálculo de distancias mínimas en grafos no ponderados	BFS garantiza que la primera vez que un nodo v es visitado: $\ell(v)$ = "la longitud mínima corta de cualquier camino entre el origen y v". Esto se debe a que BFS expande la frontera de exploración como ondas concéntricas: primero todos los nodos a distancia 1, luego los de distancia 2, etc. Es un resultado clave en teoría de grafos, pues asegura que BFS genera un árbol donde cada arista conduce al nodo más cercano posible según la métrica natural del grafo.
Propagación de fenómenos en redes	El comportamiento por capas hace que BFS modele de manera natural: › difusión de información, › propagación de señales, › contagios epidémicos, › expansión de influencia, › crecimiento de comunidades o clusters. Esto se debe a que BFS identifica oleadas de nodos alcanzados después de cierto número de pasos, que es análogo a iteraciones temporales en modelos dinámicos.

Ahora veamos este ejemplo: Difusión de una noticia en una red social donde se aplica este recorrido. Supongamos que tenemos un grafo donde:

- › cada nodo representa un usuario,
- › una arista indica interacción directa (amistad, seguimiento, canal de contacto).

Si un usuario u_0 publica una noticia:

- › los nodos en $\ell = 1$ son los usuarios que la ven inmediatamente,
- › los nodos en $\ell = 2$ la reciben a través de quienes ya la vieron,
- › los nodos en $\ell = 3$ corresponden a la siguiente “capa” de difusión, y así sucesivamente.

BFS modela exactamente estas “ondas” de transmisión:

$u_0 \rightarrow \{u_1, u_2, u_3\} \rightarrow \{u_4, u_5, u_6\} \rightarrow \dots$

Cada nivel del BFS equivale a un “salto social” una barrera de información que refleja cómo se expande la noticia temporalmente en la red.

Este tipo de modelado es fundamental en:

- › análisis de viralidad,
- › detección de influenciadores,
- › simulaciones de propaganda o fake news,
- › estudios de resiliencia en redes sociales.

Veamos gráficamente como es:

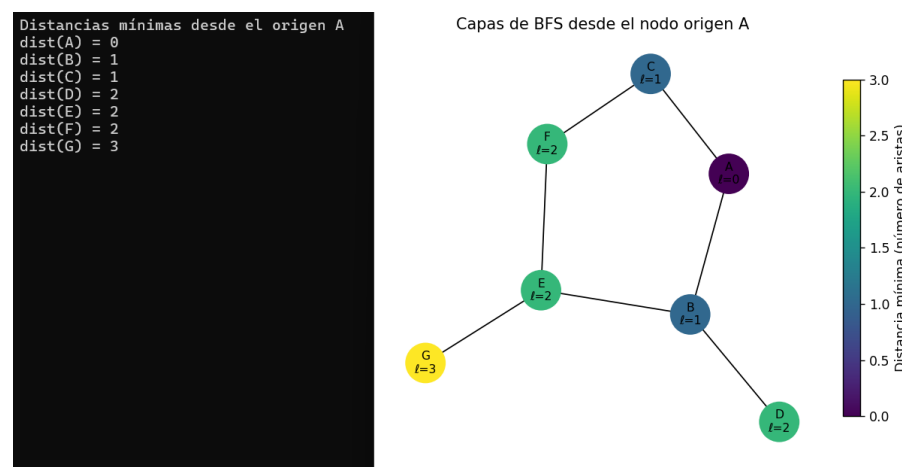


Ilustración 10 – BFS – Desarrollo Python – Propio
Descargar de: <https://tinyurl.com/mrxhm8c4>

La idea de la figura es mostrar:

- › Un grafo no ponderado que representa una pequeña red social.
- › Un nodo origen (el que “lanza la noticia”).
- › Los demás nodos coloreados por nivel de BFS:
 - › *nivel 0*: el origen,
 - › *nivel 1*: vecinos directos,
 - › *nivel 2*: amigos de amigos, etc.

Visualmente se ve como “anillos” o “capas” que salen del nodo origen, lo que refuerza la interpretación:

$\ell(v)$ = mínimo número de aristas para llegar a v

La figura que generará el código del punto 3 muestra justamente las capas BFS con distintos colores y etiqueta en cada nodo su distancia $\ell(v)$ desde el origen.

- **Pseudocódigo matemático de BFS (distancias mínimas)**

Supongamos un grafo no ponderado $G = (V, E)$ representado por una lista de adyacencia

$L(v)$ = vecinos de v . Sea $s \in V$ el nodo origen.

Al finalizar:

- › $\text{dist}[v]$ contiene la distancia mínima en número de aristas desde s hasta v (0 si v es inalcanzable).
- › La función $\ell(v)$ del texto se identifica con $\text{dist}[v]$.

A continuación, dejamos el pseudocódigo matemático:

```
Subprocedimiento BFS( $G, s$ )
  Entrada:
     $G = (V, E)$  grafo no ponderado
     $s$  nodo origen

  Para cada  $v$  en  $V$  hacer
    visitado[ $v$ ]  $\leftarrow$  FALSO
    dist[ $v$ ]  $\leftarrow \infty$ 
  FinPara

  Crear una cola  $Q$  vacía
  visitado[ $s$ ]  $\leftarrow$  VERDADERO
  dist[ $s$ ]  $\leftarrow 0$ 
  Encolar( $Q, s$ )

  Mientras  $Q$  no esté vacía hacer
     $u \leftarrow$  Desencolar( $Q$ )

    Para cada  $w$  en  $L(u)$  hacer
      Si visitado[ $w$ ] = FALSO entonces
        visitado[ $w$ ]  $\leftarrow$  VERDADERO
        dist[ $w$ ]  $\leftarrow$  dist[ $u$ ] + 1
        Encolar( $Q, w$ )
      FinSi
    FinPara
  FinMientras
FinSubprocedimiento
```

Captura 3 - Pseudo BFS – Propio



Multimedia

A continuación, encontrará un podcast con una explicación del tema
BFS: *BFS El Secreto del Camino Más Corto.m4a*



También disponible desde:
<https://tinyurl.com/4nyftf8v>



Énfasis

BFS, al basarse en una exploración nivel a nivel desde el nodo inicial, garantiza que el primer camino identificado hacia un nodo sea el más corto en términos de cantidad de aristas. Esto lo hace particularmente eficiente para la identificación de rutas óptimas en grafos no ponderados.

Por su parte, DFS se caracteriza por su enfoque de exploración profunda, priorizando la expansión de nodos tan lejos como sea posible antes de retroceder. Este método es especialmente útil para el análisis de estructuras jerárquicas, detección de ciclos y ordenamiento topológico en grafos dirigidos acíclicos. Su capacidad para recorrer exhaustivamente todas las posibles rutas en un grafo lo convierte en una herramienta versátil para problemas complejos de conectividad y optimización.

En términos prácticos, el orden de visita generado por BFS y DFS puede influir significativamente en la resolución de problemas como la construcción de árboles de expansión, la identificación de componentes conexas o la optimización de rutas en redes complejas. La selección del algoritmo adecuado dependerá de los requerimientos específicos del problema y de las características del grafo en cuestión.

- **Algoritmo de Warshall. Cierre transitivo**

El cierre transitivo de un grafo dirigido es una construcción matemática que permite determinar, para cada par de nodos, si existe algún camino, directo o indirecto, que conecte uno con otro.

En otras palabras:

El cierre transitivo responde a la pregunta:

- › “¿Es posible llegar del nodo i al nodo j siguiendo aristas del grafo, aunque debamos pasar por otros nodos?”

Esto convierte la estructura del grafo en una relación *cerrada por transitividad*, lo que permite analizar dependencias, accesibilidad y alcance global dentro de la red.

- **Representación matricial**

Partimos de la *matriz de adyacencia* A , donde:

$$A[i][j] = \begin{cases} 1 & \text{si existe una arista } i \rightarrow j, \\ 0 & \text{en caso contrario.} \end{cases}$$

El **algoritmo de Warshall** construye una matriz T (del inglés *transitive closure*) tal que:

$T[i][j] = 1$ si existe algún camino de i a j .

Esto incluye:

- › caminos de longitud 1 (los originales del grafo),
- › caminos de longitud 2,
- › caminos de longitud 3,
- › y así sucesivamente.

Por eso, el cierre transitivo revela la *estructura completa de accesibilidad* del grafo.

- **Regla algebraica de actualización**

Warshall utiliza un proceso sistemático basado en operaciones lógicas entre filas y columnas de la matriz:

$$T[i][j] = T[i][j] \vee (T[i][k] \wedge T[k][j])$$

donde:

- › $\vee$ es **OR lógico** (verdadero si alguna condición es verdadera),
- › $\wedge$ es **AND lógico**,
- › k recorre todos los nodos del grafo.
- › ¿Qué significa esta fórmula?

Se evalúa si el camino $i \rightarrow j$ se puede mejorar o completar pasando por un nodo intermedio k :

1. Si ya existía un camino de i a j , entonces $T[i][j]$ se mantiene en 1.
2. Si no existía, se activa $T[i][j]$ en 1 si existe un camino de i a k y otro de k a j .

Es decir:



Énfasis

Si puedo ir de i a k , y también puedo ir de k a j , entonces puedo ir de i a j .

Esta es la esencia de la transitividad.

El algoritmo explora iterativamente si vale la pena usar cada nodo como intermediario.

- › En la primera iteración se pregunta:
 - › “Si usamos el nodo 1 como punto intermedio, ¿aparecen nuevos caminos?”

- › En la segunda iteración:
 - › “¿Qué caminos pueden obtenerse pasando por el nodo 2?”
- › Y así sucesivamente.

Al finalizar, T es la matriz definitiva de accesibilidad total.

Esta matriz tiene un gran valor matemático porque convierte el grafo en una estructura completamente cerrada respecto de la relación “llegar a”.

Veamos un ejemplo de Dependencias de módulos en software, supongamos un grafo donde:

- › Cada nodo representa un módulo del sistema.
- › Una arista $A \rightarrow B$ significa que A depende de B para funcionar.

Directamente, podemos ver dependencias inmediatas (solo un salto). Pero muchas veces necesitamos saber:

- › ¿qué módulos dependen indirectamente de otros?,
- › ¿qué módulos afectan a una cadena completa de dependencias?,
- › ¿qué módulos deben actualizarse juntos?

El cierre transitivo responde precisamente eso:

- › Si $T[A][D] = 1$, entonces A depende de D, aunque la ruta sea:

$$A \rightarrow B \rightarrow C \rightarrow D$$

- › Si una celda queda en 0, significa que *no hay* forma de llegar.
- › *En ingeniería de software:*
 - › permite detectar módulos críticos,
 - › identificar cadenas de impacto,
 - › evitar ciclos peligrosos en dependencias,
 - › validar que no haya rutas ocultas que compliquen el mantenimiento del sistema.
- › *Pseudocódigo matemático completo del algoritmo de Warshall*

Dado un grafo dirigido $G = (V, E)$ con $|V| = n$, inicializamos:

- › $T^{(0)} = A$, la matriz de adyacencia.
- › Cada iteración k considera si conviene usar el nodo k como intermediario.

El algoritmo construye sucesivamente:

$$T^{(k)}[i][j] = T^{(k-1)}[i][j] \vee (T^{(k-1)}[i][k] \wedge T^{(k-1)}[k][j])$$

```
Algoritmo Warshall(A)
  Entrada:
    A : matriz de adyacencia n x n
  Salida:
    T : matriz de cierre transitivo

  T ← A

  Para k desde 1 hasta n hacer
    Para i desde 1 hasta n hacer
      Para j desde 1 hasta n hacer
        T[i][j] ← T[i][j] OR ( T[i][k] AND T[k][j] )
      FinPara
    FinPara
  FinPara

  Devolver T
FinAlgoritmo
```

Captura 4 –Cierre transitivo–FW–Elaboración Propia



Le sugerimos escuchar la explicación en formato podcast: *El Algoritmo de Warshall y el Cierre Transitivo.m4a*

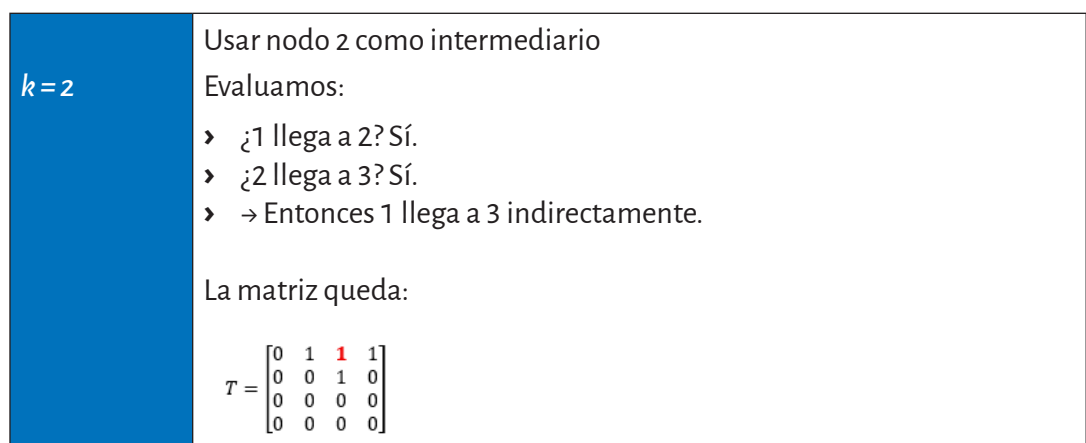
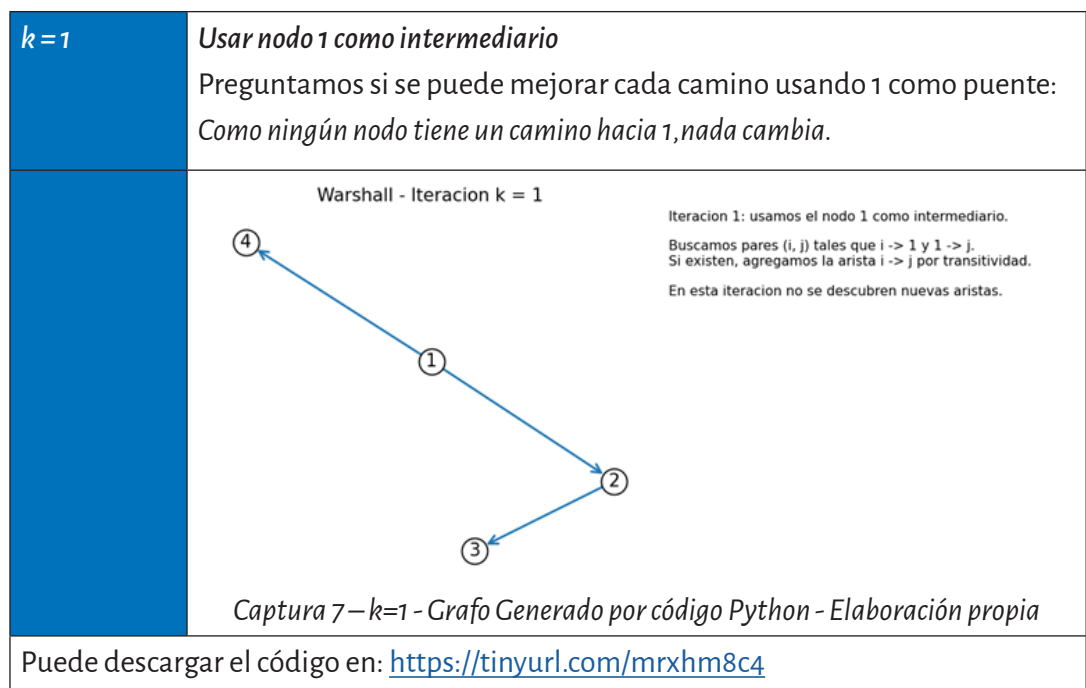
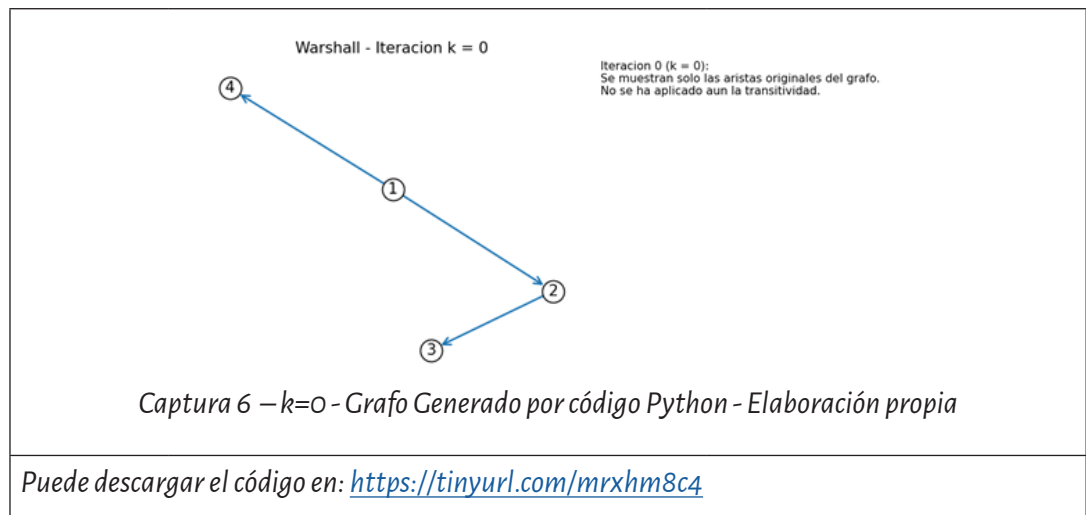


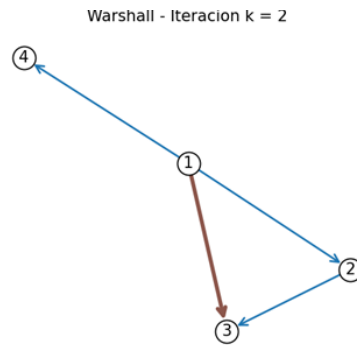
También disponible desde:
<https://tinyurl.com/25d4d443>

Y ahora, toca el turno de ver números.

Consideremos el siguiente grafo dirigido:

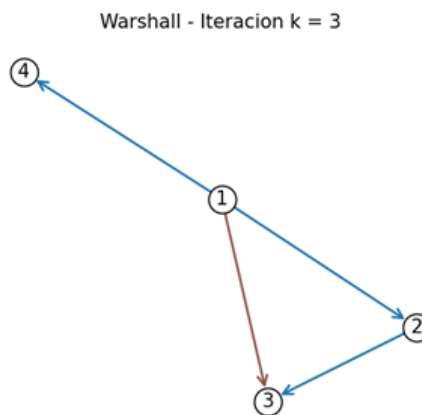
1→2 2→3 1→4	 Captura 5 -Elaboración Propia - Graphonline	La matriz de adyacencia es: $A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
-------------------	--	---





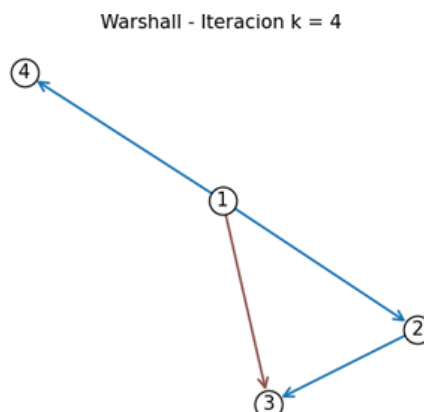
Captura 8 – $k=2$ - Grafo Generado por código Python - Elaboración propia

Puede descargar el código en: <https://tinyurl.com/mrxhm8c4>



Captura 9 – $k=3$ - Grafo Generado por código Python - Elaboración propia

Puede descargar el código en: <https://tinyurl.com/mrxhm8c4>



Captura 10— $k=4$ - Grafo Generado por código Python - Elaboración propia

Puede descargar el código en: <https://tinyurl.com/mrxhm8c4>

Resultado
final del
cierre
transitivo

$$T = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Interpretación:

- › Desde 1 se puede llegar a 2, 3 y 4.
- › Desde 2 se puede llegar a 3.
- › Los nodos 3 y 4 no alcanzan a nadie.



Actividad

Llegado a este punto estamos en condiciones de realizar la Actividad 1: Estructuras, recorridos y accesibilidad: un análisis completo de un grafo aplicado a IA. La misma puede realizarse de manera grupal.



Lectura
complementaria

Para seguir profundizando sobre cierre transitivo y la forma en que se arman las matrices, aquí le dejamos un video: *Algoritmo de Warshall*

› Conclusión del Módulo

El estudio que usted fue desarrollando en este módulo le permitió comprender los fundamentos matemáticos que sustentan la teoría de grafos y su relevancia en los modelos y algoritmos de la inteligencia artificial contemporánea. A partir del análisis de representaciones formales, propiedades estructurales, recorridos y técnicas de accesibilidad, quedó en evidencia que los grafos constituyen una herramienta esencial para modelar relaciones, flujos de información, procesos de propagación y estructuras complejas que aparecen en aplicaciones reales de IA.

Los recorridos DFS y BFS, junto con el cierre transitivo, nos ofrecieron una primera aproximación al análisis sistemático de redes y dependencias, nos mostraron cómo las ideas matemáticas pueden traducirse directamente en comportamientos observables dentro de sistemas inteligentes. Esta capacidad de vincular teoría y práctica prepara el terreno para futuras unidades en las que estos conceptos serán profundizados y aplicados en contextos más avanzados.

Nuestra materia “Matemáticas para la Inteligencia Artificial” constituyó así un primer acercamiento formal a los principios matemáticos que sostienen el desarrollo de modelos, técnicas y métodos utilizados en IA. A lo largo del cursado hemos explorado pilares fundamentales como álgebra lineal, cálculo, probabilidad, estadística, optimización y teoría de grafos, cada uno con un rol específico y complementario dentro del ecosistema de la inteligencia artificial moderna.

Esperamos que este recorrido haya permitido no solo adquirir herramientas teóricas, sino también reconocer cómo cada una de ellas se articula con problemas reales de clasificación, predicción, análisis de datos, toma de decisiones y modelado de sistemas complejos. Sin embargo, es importante destacar que este trayecto constituye tan solo el comienzo. Muchos de los temas introducidos aquí se ampliarán, profundizarán y especializarán en diversas asignaturas del Ciclo de Licenciatura, donde se trabajará con modelos más

avanzados, técnicas de aprendizaje profundo, algoritmos de optimización compleja y análisis estructural de redes a gran escala.

La intención de esta materia es, por lo tanto, sentar las bases conceptuales que permitan comprender y apropiarse de los fundamentos matemáticos de la IA, y al mismo tiempo motivar la exploración continua de nuevas herramientas que serán clave en su formación académica y profesional. Lo estudiado aquí es un punto de partida: el verdadero desarrollo se construirá a lo largo del ciclo, donde la matemática se vuelve un instrumento poderoso para pensar, diseñar e implementar soluciones inteligentes en un mundo cada vez más orientado por los datos y la automatización.



Evaluación

En este momento, se encuentra Ud. en condiciones de realizar la segunda y última parte de la evaluación integradora. ¡Éxitos!



VIDEO DOCENTE

MÓDULO 5 | GLOSARIO

Accesibilidad: Propiedad que indica si existe un camino, directo o indirecto, entre dos nodos de un grafo. Se determina aplicando recorridos o el cierre transitivo.

Arista: Relación que conecta dos nodos de un grafo. Puede ser dirigida (con sentido) o no dirigida.

Cierre transitivo: Operación que determina todas las conexiones posibles entre nodos considerando caminos de cualquier longitud. Su cálculo sistemático se realiza mediante el algoritmo de Warshall.

Componente conexa: Conjunto de nodos que están comunicados entre sí por algún camino. En grafos no dirigidos indica conectividad; en dirigidos, se relaciona con regiones alcanzables.

DFS (Depth-First Search): Recorrido en profundidad. Explora un grafo siguiendo un camino lo más lejos posible antes de retroceder. Útil para detectar ciclos, componentes y estructuras internas.

Distancia mínima: Cantidad mínima de aristas que deben atravesarse para ir de un nodo a otro en un grafo no ponderado. Se obtiene mediante BFS.

Grafo: Estructura matemática formada por un conjunto de nodos y un conjunto de aristas que representan relaciones entre ellos.

Grafo bipartito: Grafo cuyo conjunto de nodos puede dividirse en dos grupos disjuntos, donde todas las aristas conectan un nodo de un grupo con uno del otro.

Grafo dirigido (dígrafo): Grafo en el que las aristas tienen dirección, indicando una relación asimétrica entre los nodos.

Grafo no dirigido: Grafo en el que las aristas no tienen sentido. La relación entre nodos es simétrica.

Grafo ponderado: Grafo en el que cada arista tiene un peso asociado que representa la intensidad, costo o distancia entre dos nodos.

Lista de adyacencia: Representación de un grafo que asocia a cada nodo el conjunto de sus vecinos directos. Es eficiente en grafos dispersos.

Matriz de adyacencia: Matriz cuadrada que indica, mediante valores 0 o 1, si existe o no una arista entre pares de nodos.

Nivel BFS (capa): Conjunto de nodos alcanzados con la misma distancia mínima desde el nodo origen durante un recorrido BFS. Representa “saltos” o etapas de propagación.

Nodo (vértice): Elemento fundamental del grafo que representa una entidad, estado, módulo, usuario, concepto o componente del sistema modelado.

Recorrido BFS (Breadth-First Search): Recorrido en amplitud. Explora primero los vecinos más cercanos y luego los de niveles posteriores. Produce distancias mínimas y capas de expansión.

Recorrido en grafo: Proceso sistemático de visitar nodos en un orden específico (DFS, BFS) para analizar estructura, accesibilidad y conectividad.

Transitividad: Propiedad según la cual si un nodo A llega a B y B llega a C, entonces A llega a C. Es la base del cierre transitivo.

MÓDULO 5 | ACTIVIDADES



ACTIVIDAD N° 1:

Estructuras, recorridos y accesibilidad: un análisis completo de un grafo aplicado a IA

Una empresa desarrolla un sistema inteligente para analizar cómo se propaga información interna entre equipos y módulos de software. Para ello, modela la estructura de dependencias y comunicación como un grafo dirigido. Cada nodo representa un módulo del sistema, y una arista $A \rightarrow B$ significa que A envía datos directamente a B.

El grafo obtenido es el siguiente:

- › Módulo A envía información a B y C
- › Módulo B envía información a D
- › Módulo C envía información a D y E
- › Módulo D envía información a F
- › Módulo E envía información a F
- › Módulo F no envía información a ningún módulo

Representación formal:

$A \rightarrow B, A \rightarrow C, B \rightarrow D, C \rightarrow D, C \rightarrow E, D \rightarrow F, E \rightarrow F$

Resuelva las siguientes consignas:

1. Tipos de grafos

a) Justifique la razón por la cual este grafo presenta las características mencionadas:

- › dirigido,
- › no ponderado,
- › no bipartito.

b) ¿Qué característica del sistema haría que este grafo fuera ponderado?

2. Matriz de Adyacencia

Construya la matriz de adyacencia usando el siguiente orden de nodos:

$A = 1, B = 2, C = 3, D = 4, E = 5, F = 6$

3. Lista de Adyacencia

Elabore la lista de adyacencia correspondiente. A continuación, proceda a responder lo solicitado.

- › ¿Qué nodo tiene mayor grado de salida?
- › ¿Qué interpretación tiene esto en el sistema?

4. Recorrido DFS

Ejecute el algoritmo DFS iniciando en el nodo A y siguiendo la secuencia alfabética para recorrer los nodos vecinos.

- › Indique el orden de visita.
- › Explique qué tipo de información revela el árbol sobre el sistema.
- › Represente gráficamente el árbol DFS obtenido.

5. Recorrido BFS

Ejecute el algoritmo de BFS a partir del nodo A.

- a) Establezca, la distancia mínima desde A hacia cada nodo.
- b) Identifique:
 - › los nodos pertenecientes al nivel 1,
 - › los nodos correspondientes al nivel 2,
 - › los nodos que conforman el nivel 3.
- c. Analice e interprete estos niveles como intervalos de propagación de un mensaje interno.

6. Cierre Transitivo - Accesibilidad total

Utilice la matriz de adyacencia para calcular el cierre transitivo mediante el algoritmo de Warshall.

- a) Realice la primera iteración ($k = 1$).
- b) Identifique las nuevas aristas que se incorporan al grafo.
- c) Explique el significado, dentro del contexto del sistema, de la aparición de una nueva arista.

7. Análisis integrado

Considerando la información expuesta previamente, responda lo siguiente:

- a) ¿Cuáles son los módulos esenciales para la propagación de datos?
- b) ¿Qué módulos presentan dependencia indirecta respecto de otros componentes?
- c) ¿En qué áreas del sistema podrían surgir cuellos de botella?
- d) ¿De qué manera podrían emplearse estos resultados para optimizar un sistema de inteligencia artificial basado en flujos de información?

8. Entrega

La actividad deberá contener:

- › explicaciones detalladas y precisas,
- › representación gráfica de los recorridos DFS y BFS (*realizadas manualmente o mediante Python*),
- › presentación de matrices y listas elaboradas,
- › conclusiones integradoras que sintetizen los resultados obtenidos.